

O'REILLY®

7. Auflage

C# 10

kurz & gut

O'Reillys
Taschenbibliothek



Joseph Albahari
Ben Albahari

Übersetzung von Thomas Demmig



Zu diesem Buch – sowie zu vielen weiteren O'Reilly-Büchern – können Sie auch das entsprechende E-Book im PDF-Format herunterladen. Werden Sie dazu einfach Mitglied bei oreilly.plus⁺:

www.oreilly.plus

7. AUFLAGE

C# 10

kurz & gut

Joseph Albahari & Ben Albahari

*Deutsche Übersetzung von
Thomas Demmig*

O'REILLY®

Joseph Albahari, Ben Albahari

Lektorat: Alexandra Follenius

Übersetzung: Thomas Demmig

Copy-Editing: Sibylle Feldmann, www.richtiger-text.de

Satz: Ill-Satz, www.drei-satz.de

Herstellung: Stefanie Weidner

Umschlaggestaltung: Karen Montgomery, Michael Oréal, www.oreal.de

Druck und Bindung: mediaprint solutions GmbH, 33100 Paderborn

Bibliografische Information der Deutschen Nationalbibliothek

Die Deutsche Nationalbibliothek verzeichnet diese Publikation in der Deutschen Nationalbibliografie; detaillierte bibliografische Daten sind im Internet über <http://dnb.d-nb.de> abrufbar.

ISBN:

Print 978-3-96009-200-1

PDF 978-3-96010-683-8

ePub 978-3-96010-684-5

mobi 978-3-96010-685-2

7., aktualisierte Auflage 2022

Translation Copyright für die deutschsprachige Ausgabe © 2022 dpunkt.verlag GmbH

Wiebling Weg 17

69123 Heidelberg

Authorized German translation of the English edition of *C# 10 Pocket Reference*, ISBN 9781098122041 © 2022 Joseph Albahari and Ben Albahari. This translation is published and sold by permission of O'Reilly Media, Inc., which owns or controls all rights to publish and sell the same.

Dieses Buch erscheint in Kooperation mit O'Reilly Media, Inc. unter dem Imprint »O'REILLY«. O'REILLY ist ein Markenzeichen und eine eingetragene Marke von O'Reilly Media, Inc. und wird mit Einwilligung des Eigentümers verwendet.

Hinweis:

Dieses Buch wurde auf PEFC-zertifiziertem Papier aus nachhaltiger Waldwirtschaft gedruckt. Der Umwelt zuliebe verzichten wir auf die Einschweißfolie.



Schreiben Sie uns:

Falls Sie Anregungen, Wünsche und Kommentare haben, lassen Sie es uns wissen: komentar@oreilly.de.

Die vorliegende Publikation ist urheberrechtlich geschützt. Alle Rechte vorbehalten. Die Verwendung der Texte und Abbildungen, auch auszugsweise, ist ohne die schriftliche Zustimmung des Verlags urheberrechtswidrig und daher strafbar. Dies gilt insbesondere für die Vervielfältigung, Übersetzung oder die Verwendung in elektronischen Systemen. Es wird darauf hingewiesen, dass die im Buch verwendeten Soft- und Hardware-Bezeichnungen sowie Markennamen und Produktbezeichnungen der jeweiligen Firmen im Allgemeinen warenzeichen-, marken- oder patentrechtlichem Schutz unterliegen. Alle Angaben und Programme in diesem Buch wurden mit größter Sorgfalt kontrolliert. Weder Autor noch Verlag noch Übersetzer können jedoch für Schäden haftbar gemacht werden, die in Zusammenhang mit der Verwendung dieses Buches stehen.

Inhalt

C# 10 – kurz & gut	7
Ein erstes C#-Programm	7
Syntax	11
Typgrundlagen	14
Numerische Typen	25
Der Typ bool und die booleschen Operatoren	32
Strings und Zeichen	34
Arrays	38
Variablen und Parameter	44
Ausdrücke und Operatoren	53
Null-Operatoren	59
Anweisungen	61
Namensräume	71
Klassen	76
Vererbung	94
Der Typ object	104
Structs	108
Zugriffsmodifikatoren	111
Interfaces	113
Enums	118
Eingebettete Typen	121
Generics	121
Delegates	130
Events	136
Lambda-Ausdrücke	142
Anonyme Methoden	148

try-Anweisungen und Exceptions.....	149
Enumeration und Iteratoren.....	157
Nullbare Werttypen.....	162
Nullbare Referenztypen.....	167
Erweiterungsmethoden.....	169
Anonyme Typen.....	171
Tupel.....	172
Datensätze.....	174
Muster.....	181
LINQ.....	185
Die dynamische Bindung.....	211
Überladen von Operatoren.....	220
Attribute.....	223
Aufrufer-Info-Attribute.....	227
Asynchrone Funktionen.....	229
Unsicherer Code und Zeiger.....	240
Präprozessordirektiven.....	245
XML-Dokumentation.....	248
Index.....	253

C# 10 – kurz & gut

C# ist eine allgemein anwendbare, typsichere, vorwiegend objektorientierte Programmiersprache, die die Produktivität des Programmiers erhöhen soll. Zu diesem Zweck versucht die Sprache, die Balance zwischen Einfachheit, Ausdrucksfähigkeit und Performance zu finden. Die Sprache C# ist plattformneutral, wurde aber geschrieben, um gut mit dem *.NET Framework* von Microsoft zusammenzuarbeiten. C# 10 ist darauf ausgerichtet, mit der *.NET Core 6*-Runtime von Microsoft zusammenzuarbeiten (während C# 9 für *.NET 5*, C# 8 für *.NET Core 3* und C# 7 für *.NET Core 2* sowie Microsoft *.NET Framework 4.6/4.7/4.8* gedacht war).



Die Programme und Codefragmente in diesem Buch entsprechen denen aus den Kapiteln 2 und 4 von *C# 10 in a Nutshell* und sind alle als interaktive Beispiele in LINQPad verfügbar. Das Durcharbeiten der Beispiele im Zusammenhang mit diesem Buch fördert den Lernvorgang, da Sie bei der Bearbeitung der Beispiele unmittelbar die Ergebnisse sehen können, ohne dass Sie in Visual Studio dazu Projekte und Projektmappen einrichten müssten.

Um die Beispiele herunterzuladen, klicken Sie in LINQPad auf den Samples-Tab und wählen dort *Download more samples*. LINQPad ist kostenlos – Sie finden es unter <http://www.linqpad.net>.

Ein erstes C#-Programm

Das hier ist ein Programm, das 12 mit 30 multipliziert und das Ergebnis ausgibt (360). Der doppelte Schrägstrich (Slash) gibt an, dass der Rest einer Zeile ein *Kommentar* ist.

```
int x = 12 * 30;           // Anweisung 1
System.Console.WriteLine (x); // Anweisung 2
```

Unser Programm besteht aus zwei *Anweisungen*. Anweisungen werden in C# sequenziell ausgeführt und durch ein Semikolon abgeschlossen. Die erste Anweisung berechnet den *Ausdruck* $12 * 30$ und speichert das Ergebnis in einer *Variablen* namens *x*, deren Typ ein 32-Bit-Integer ist (*int*). Die zweite Anweisung ruft die *Methode* *WriteLine* einer *Klasse* namens *Console* auf, die in einem *Namensraum* *System* definiert ist. Damit wird die Variable *x* in einem Textfenster auf dem Bildschirm ausgegeben.

Eine Methode führt eine Funktion aus – eine Klasse führt Funktions-Member und Daten-Member zusammen, um einen objektorientierten Baustein zu bilden. Die Klasse *Console* fasst Member zusammen, die sich um die Ein- und Ausgabe (*Input/Output*, *I/O*) an der Befehlszeile kümmern, wie zum Beispiel die Methode *WriteLine*. Eine Klasse ist die Art eines Typs, den wir im Abschnitt »Typgrundlagen« auf Seite 14 behandeln.

Auf der obersten Ebene sind Typen in Namensräumen organisiert. Viele häufig genutzte Typen – einschließlich der Klasse *Console* – befinden sich im Namensraum *System*. Die .NET-Bibliotheken sind in verschachtelten Namensräumen organisiert. So enthält beispielsweise der Namensraum *System.Text* Typen für den Umgang mit Text, und *System.IO* enthält Typen für die Ein- und Ausgabe.

Muss man die Klasse *Console* bei jeder Verwendung durch den Namensraum *System* qualifizieren, wird das schnell unübersichtlich. Die *using*-Direktive ermöglicht Ihnen, das Ganze besser lesbar zu machen, indem ein Namensraum *importiert* wird:

```
using System;           // Namensraum System importieren

int x = 12 * 30;
Console.WriteLine (x);  // System muss nicht angegeben werden
```

Eine einfache Form der Wiederverwendung von Code ist das Schreiben von High-Level-Funktionen, die wiederum Low-Level-Funktionen aufrufen. Wir können unser Programm mit einer wiederverwendbaren Methode namens *FeetToInches* *refaktorisieren*, die eine Ganzzahl mit 12 multipliziert:

```
using System;

Console.WriteLine (FeetToInches (30));      // 360
Console.WriteLine (FeetToInches (100));     // 1200

int FeetToInches (int feet)
{
    int inches = feet * 12;
    return inches;
}
```

Unsere Methode enthält eine Folge von Anweisungen, die durch ein Paar geschweiffter Klammern umschlossen sind. Das wird als *Anweisungsblock* bezeichnet.

Eine Methode kann *Eingabedaten* vom Aufrufer erhalten, indem sie *Parameter* spezifiziert, und Daten an den Aufrufer zurückgeben, indem sie einen *Rückgabetyt* festlegt. Unsere Methode FeetToInches besitzt einen Parameter für die Übergabe von Feet und einen Rückgabetyt für die Ausgabe der Inches:

```
int FeetToInches (int feet)
...
```

Die *Literale* 30 und 100 sind die *Argumente*, die an die Methode FeetToInches übergeben werden.

Erhält eine Methode keine Eingabewerte, nutzen Sie ein leeres Klammernpaar. Gibt sie nichts zurück, verwenden Sie das Schlüsselwort `void`:

```
using System;
SayHello();

void SayHello()
{
    Console.WriteLine ("Hello, world");
}
```

Methoden sind eine von vielen Arten von Funktionen in C#. Eine andere Art von Funktionen, die wir in unserem Beispielprogramm genutzt haben, war der *Operator* *, der eine Multiplikation durchführt. Es gibt zudem *Konstrukturen*, *Eigenschaften*, *Events*, *Indexer* und *Finalizer*.

Kompilation

Der C#-Compiler führt Quellcode, der in einer Reihe von Dateien mit der Endung *.cs* untergebracht ist, in einer *Assembly* zusammen. Eine *Assembly* ist die Verpackungs- und Auslieferungseinheit in .NET und kann entweder eine *Anwendung* oder eine *Bibliothek* sein, wobei eine Konsolen- oder Windows-Anwendung einen *Einsprungpunkt* besitzt, während das bei einer Bibliothek nicht der Fall ist. Der Zweck einer Bibliothek ist es, von einer Anwendung oder anderen Bibliotheken aufgerufen (*referenziert*) zu werden. .NET selbst ist ein Satz von Bibliotheken (und eine Laufzeitumgebung).

Jedes der Programme im vorherigen Abschnitt begann direkt mit einer Folge von Anweisungen (auch als *Anweisungen auf oberster Ebene* bezeichnet). Gibt es solche Anweisungen, wird implizit ein Einstiegspunkt für eine Konsolen- oder Windows-Anwendung erzeugt. (Ohne Anweisungen auf oberster Ebene stellt eine *Main-Methode* den Einstiegspunkt dar – siehe den Abschnitt »Symmetrie vordefinierter und benutzerdefinierter Typen« auf Seite 16.)

Um den Compiler aufzurufen, können Sie entweder eine integrierte Entwicklungsumgebung (*Integrated Development Environment*, IDE) wie Visual Studio oder Visual Studio Code nutzen oder ihn selbst per Hand über die Befehlszeile aufrufen. Um eine Konsolenanwendung manuell mit .NET zu kompilieren, laden Sie zunächst das .NET 6 SDK herunter und erzeugen dann ein neues Projekt:

```
dotnet new console -o MyFirstProgram
cd MyFirstProgram
```

Das erstellt einen Ordner namens *MyFirstProgram* mit der C#-Datei *Program.cs*, die Sie dann bearbeiten können. Zum Aufruf des Compilers rufen Sie *dotnet build* (oder *dotnet run*) auf, wodurch das Programm ausgeführt und dann gestartet wird. Die Ausgabe wird in ein Unterverzeichnis unter *bin\debug* geschrieben. Dort finden sich *MyFirstProgram.dll* (die Ausgabe-Assembly) und *MyFirstProgram.exe* (was das kompilierte Programm direkt aufruft).

Syntax

Die Syntax von C# ist von der Syntax von C und C++ inspiriert. In diesem Abschnitt beschreiben wir die C#-Elemente der Syntax anhand des folgenden Programms:

```
using System;

int x = 12 * 30;
Console.WriteLine (x);
```

Bezeichner und Schlüsselwörter

Bezeichner sind Namen, die Programmierer für ihre Klassen, Methoden, Variablen und so weiter wählen. Das hier sind die Bezeichner in unserem Beispielprogramm in der Reihenfolge ihres Auftretens:

```
System  x  Console  WriteLine
```

Ein Bezeichner muss ein ganzes Wort sein und aus Unicode-Zeichen bestehen, wobei den Anfang ein Buchstabe oder der Unterstrich bildet. C#-Bezeichner unterscheiden Groß- und Kleinschreibung. Es ist üblich, Argumente, lokale Variablen und private Felder in *Camel-Case* zu schreiben (zum Beispiel `myVariable`) und alle anderen Bezeichner in *Pascal-Schreibweise* (zum Beispiel `MyMethod`).

Schlüsselwörter sind Namen, die für den Compiler eine bestimmte Bedeutung haben. Die Schlüsselwörter in unserem Beispielprogramm sind `using` und `int`.

Die meisten Schlüsselwörter sind für den Compiler *reserviert*, Sie können sie nicht als Bezeichner verwenden. Hier ist eine vollständige Liste aller C#-Schlüsselwörter:

abstract	catch	default	explicit
as	char	delegate	extern
base	checked	do	false
bool	class	double	finally
break	const	else	fixed
byte	continue	enum	float
case	decimal	event	for

foreach	null	sbyte	typeof
goto	object	sealed	uint
if	operator	short	ulong
implicit	out	sizeof	unchecked
in	override	stackalloc	unsafe
int	params	static	ushort
interface	private	string	using
internal	protected	struct	virtual
is	public	switch	volatile
lock	readonly	this	void
long	record	throw	while
namespace	ref	true	
new	return	try	

Konflikte vermeiden

Wenn Sie wirklich einen Bezeichner nutzen wollen, der mit einem reservierten Schlüsselwort in Konflikt geraten würde, müssen Sie ihn mit dem Präfix @ auszeichnen:

```
class class {...}    // illegal
class @class {...}  // legal
```

Das Zeichen @ gehört nicht zum Bezeichner selbst, daher ist @myVariable das Gleiche wie myVariable.

Kontextuelle Schlüsselwörter

Einige Schlüsselwörter sind *kontextbezogen*. Das heißt, sie können – auch ohne ein vorangestelltes @-Zeichen – als Bezeichner eingesetzt werden, und zwar folgende:

add	from	nameof	set
alias	get	nint	unmanaged
alias	global	not	value
and	group	notnull	var
async	into	nuint	when
await	init	or	where
by	join	orderby	with
descending	let	partial	yield
dynamic	managed	remove	
equals	nameof	select	

Bei den kontextabhängigen Schlüsselwörtern kann es innerhalb des verwendeten Kontexts keine Mehrdeutigkeit geben.

Literale, Satzzeichen und Operatoren

Literale sind einfache Daten, die statisch im Programm verwendet werden. Die Literale in unserem Beispielprogramm sind 12 und 30. *Satzzeichen* helfen dabei, die Struktur des Programms abzugrenzen. Ein Beispiel ist das Semikolon, das eine Anweisung beendet. Anweisungen können mehrere Zeilen übergreifen:

```
Console.WriteLine  
    (1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10);
```

Ein *Operator* verwandelt und kombiniert Ausdrücke. In C# werden die meisten Operatoren mithilfe von Symbolen angezeigt, beispielsweise dem Multiplikationsoperator *. Die Operatoren in unserem Programm sind folgende:

= * . ()

Ein Punkt zeigt ein Member von etwas an (oder, in numerischen Literalen, den Dezimaltrenner). Die Klammern werden genutzt, wenn eine Methode aufgerufen oder deklariert wird; leere Klammern werden verwendet, wenn eine Methode keine Argumente akzeptiert. Das Gleichheitszeichen führt eine *Zuweisung* aus (ein doppeltes Gleichheitszeichen, ==, führt einen Vergleich auf Gleichheit durch).

Kommentare

C# bietet zwei verschiedene Arten von Quellcodekommentaren: *einzeilige* und *mehrzeilige Kommentare*. Ein einzeiliger Kommentar beginnt mit zwei Schrägstrichen und geht bis zum Ende der aktuellen Zeile, zum Beispiel so:

```
int x = 3;   // Kommentar zur Zuweisung von 3 an x
```

Ein mehrzeiliger Kommentar beginnt mit /* und endet mit */, zum Beispiel so:

```
int x = 3;   /* Das ist ein Kommentar, der  
                zwei Zeilen umspannt. */
```

Kommentare können in XML-Dokumentations-Tags (siehe »XML-Dokumentation« auf Seite 248) eingebettet sein.

Typgrundlagen

Ein *Typ* definiert die Blaupause für einen Wert. In unserem Beispiel haben wir zwei Literale des Typs `int` mit den Werten 12 und 30 genutzt. Wir haben außerdem eine *Variable* des Typs `int` deklariert, deren Name `x` lautete.

Eine *Variable* zeigt einen Speicherort an, der mit der Zeit unterschiedliche Werte annehmen kann. Im Unterschied dazu repräsentiert eine *Konstante* immer den gleichen Wert (mehr dazu später).

Alle Werte sind in C# *Instanzen* eines spezifischen Typs. Die Bedeutung eines Werts und die Menge der möglichen Werte, die eine Variable aufnehmen kann, wird durch seinen bzw. ihren Typ bestimmt.

Vordefinierte Typen

Vordefinierte Typen (die auch als »eingebaute Typen« bezeichnet werden), sind solche, die besonders vom Compiler unterstützt werden. Der Typ `int` ist ein vordefinierter Typ, der die Menge der Ganzzahlen darstellen kann, die in einen 32-Bit-Speicher passen – von -2^{31} bis $2^{31}-1$. Wir können zum Beispiel arithmetische Funktionen mit Instanzen des Typs `int` durchführen:

```
int x = 12 * 30;
```

Ein weiterer vordefinierter Typ in C# ist `string`. Der Typ `string` repräsentiert eine Folge von Zeichen, zum Beispiel »`.NET`« oder »`http://oreilly.com`«. Wir können Strings bearbeiten, indem wir ihre Funktionen aufrufen:

```
string message = "Hallo Welt";  
string upperMessage = message.ToUpper();  
Console.WriteLine (upperMessage);    // HALLO WELT  
  
int x = 2022;
```

```
message = message + x.ToString();  
Console.WriteLine (message);           // Hallo Welt2022
```

Der vordefinierte Typ `bool` hat genau zwei mögliche Werte: `true` und `false`. `bool` wird häufig verwendet, um zusammen mit der `if`-Anweisung Befehle nur bedingt ausführen zu lassen:

```
bool simpleVar = false;  
if (simpleVar)  
    Console.WriteLine ("Das wird nicht ausgegeben");  
  
int x = 5000;  
bool lessThanAMile = x < 5280;  
if (lessThanAMile)  
    Console.WriteLine ("Das wird ausgegeben");
```

Der Namensraum `System` in `.NET Core` enthält viele wichtige Typen, die `C#` nicht vordefiniert (zum Beispiel `DateTime`).

Benutzerdefinierte Typen

So, wie wir komplexe Funktionen aus einfachen Funktionen aufbauen können, können wir auch komplexe Typen aus primitiven Typen aufbauen. In diesem Beispiel werden wir einen eigenen Typ namens `UnitConverter` definieren – eine Klasse, die als Vorlage für die Umwandlung von Einheiten dient:

```
UnitConverter feetToInches = new UnitConverter (12);  
UnitConverter milesToFeet = new UnitConverter (5280);  
  
Console.WriteLine (feetToInches.Convert(30)); // 360  
Console.WriteLine (feetToInches.Convert(100)); // 1200  
Console.WriteLine (feetToInches.Convert  
    (milesToFeet.Convert(1))); // 63360
```

```
public class UnitConverter  
{  
    int ratio; // Feld  
  
    public UnitConverter (int unitRatio) // Konstruktor  
    {  
        ratio = unitRatio;  
    }  
}
```

```

    public int Convert (int unit)           // Methode
    {
        return unit * ratio;
    }
}

```

Member eines Typs

Ein Typ enthält *Daten-Member* und *Funktions-Member*. Das Daten-Member von `UnitConverter` ist das *Feld* mit dem Namen `ratio`. Die Funktions-Member von `UnitConverter` sind die Methode `Convert` und der *Konstruktor* von `UnitConverter`.

Symmetrie vordefinierter und benutzerdefinierter Typen

Das Schöne an C# ist, dass vordefinierte und selbst definierte Typen nur wenige Unterschiede aufweisen. Der primitive Typ `int` dient als Vorlage für Ganzzahlen (Integer). Er speichert Daten – 32 Bit – und stellt Funktions-Member bereit, die diese Daten verwenden, zum Beispiel `ToString`. Genauso dient unser selbst definierter Typ `UnitConverter` als Vorlage für die Einheitenumrechnung. Er enthält Daten – das Verhältnis zwischen den Einheiten – und stellt Funktions-Member bereit, die diese Daten nutzen.

Konstrukturen und Instanziierung

Daten werden erstellt, indem ein Typ *instanziiert* wird. Vordefinierte Typen können einfach mit einem Literal wie `12` oder `"Hallo Welt"` definiert werden.

Der `new`-Operator erstellt Instanzen von benutzerdefinierten Typen. Wir haben unser Programm damit begonnen, dass wir zwei Instanzen des Typs `UnitConverter` erstellten. Unmittelbar nachdem der `new`-Operator ein Objekt instanziiert hat, wird der *Konstruktor* des Objekts aufgerufen, um die Initialisierung durchzuführen. Ein Konstruktor wird wie eine Methode definiert, aber der Methodenname und der Rückgabotyp werden auf den Namen des einschließenden Typen reduziert:

```

    public UnitConverter (int unitRatio)    // Konstruktor
    {
        ratio = unitRatio;
    }

```

Instanz-Member versus statische Member

Die Daten-Member und die Funktions-Member, die mit der *Instanz* des Typs arbeiten, werden als Instanz-Member bezeichnet. Die Methode `Convert` von `UnitConverter` und die Methode `ToString` von `int` sind Beispiele für solche Instanz-Member. Standardmäßig sind Member Instanz-Member.

Daten-Member und Funktions-Member, die nicht mit der Instanz des Typs arbeiten, können als `static` gekennzeichnet werden. Um sich außerhalb eines Typs auf einen statischen Member von ihm zu beziehen, geben Sie statt einer *Instanz seinen Typnamen* an. Ein Beispiel ist die Methode `WriteLine` der Klasse `Console`. Da sie statisch ist, rufen wir `Console.WriteLine()` und nicht `new Console().WriteLine()` auf.

Der Unterschied zwischen Instanz- und statischen Members ist dieser: Im folgenden Beispielcode gehört das Instanzfeld `Name` zu einer Instanz eines bestimmten `Panda`, während `Population` zur Menge aller `Panda`-Instanzen gehört. Wir erzeugen zwei Instanzen von `Panda`, geben ihre Namen und dann die gesamte `Population` aus:

```
Panda p1 = new Panda ("Pan Dee");
Panda p2 = new Panda ("Pan Dah");

Console.WriteLine (p1.Name);      // Pan Dee
Console.WriteLine (p2.Name);      // Pan Dah

Console.WriteLine (Panda.Population);    // 2

public class Panda
{
    public string Name;              // Instanzfeld
    public static int Population;    // statisches Feld

    public Panda (string n)         // Konstruktor
    {
        Name = n;                  // Instanzfeld
        Population = Population + 1 // statisches Feld
    }
}
```

Versuchen Sie, `p1.Population` oder `Panda.Name` auszuwerten, wird das zu einem Fehler beim Kompilieren führen.

Das Schlüsselwort `public`

Das Schlüsselwort `public` macht Member für andere Klassen zugänglich. Wenn in diesem Beispiel das Feld `Name` in `Panda` nicht als öffentlich markiert gewesen wäre, würde es sich um ein `private` Feld handeln und könnte nicht von außerhalb der Klasse angesprochen werden. Das »Öffentlichmachen« eines Members mit `public` lässt einen Typ sagen: »Das hier will ich andere Typen sehen lassen – alles andere sind meine privaten Implementierungsdetails.« In objektorientierten Begriffen sagen wir, dass die öffentlichen Member die privaten Member der Klasse *kapseln*.

Einen Namensraum erstellen

Insbesondere bei größeren Programmen ist es sinnvoll, Typen in Namensräumen zu organisieren. So definieren Sie die Klasse `Panda` innerhalb eines Namensraums `Animals`:

```
namespace Animals
{
    public class Panda
    {
        ...
    }
}
```

Wir behandeln Namensräume detaillierter im Abschnitt »Namensräume« auf Seite 71.

Eine Main-Methode definieren

Alle unsere Beispiele haben bisher Anweisungen auf oberster Ebene genutzt – ein Feature, das in C# 9 eingeführt wurde. Ohne solche Anweisungen sieht eine einfache Konsolen- oder Windows-Anwendung wie folgt aus:

```
using System;

class Program
{
    static void Main()    // Programm-Einstiegspunkt
    {
        int x = 12 * 30;
    }
}
```

```

        Console.WriteLine (x);
    }
}

```

Gibt es keine Anweisungen auf oberster Ebene, sucht C# nach einer statischen Methode namens Main, die dann zum Einstiegspunkt wird. Die Methode Main kann innerhalb einer beliebigen Klasse definiert sein (und es kann nur eine Methode Main geben). Muss Ihre Main-Methode auf private Member einer bestimmten Klasse zugreifen können, kann es einfacher sein, eine Main-Methode innerhalb dieser Klasse zu definieren, statt Anweisungen auf oberster Ebene zu verwenden.

Die Main-Methode kann optional einen Integer-Wert zurückgeben (statt void), um der ausführenden Umgebung einen Wert mitzuteilen (wobei ein Wert ungleich null normalerweise für einen Fehler steht). Sie kann optional auch ein Array mit Strings als Parameter übernehmen (das mit allen Argumenten gefüllt wird, die an die ausführbare Datei übergeben werden) – zum Beispiel:

```
static int Main (string[] args) {...}
```



Ein Array (wie `string[]`) steht für eine feste Zahl an Elementen eines bestimmten Typs. Arrays werden spezifiziert, indem man eckige Klammern hinter den Typ des Elements schreibt. Wir beschreiben sie im Abschnitt »Arrays« auf Seite 38.

(Die Methode Main kann auch `async` deklariert werden und einen Task oder `Task<int>` zurückgeben, um asynchrone Programmierung zu unterstützen – siehe den Abschnitt »Asynchrone Funktionen« auf Seite 229.)

Anweisungen auf oberster Ebene

Anweisungen auf oberster Ebene ermöglichen Ihnen (seit C# 9), den Aufwand mit einer statischen Main-Methode und einer Klasse, die diese Methode enthält, zu vermeiden. Eine Datei mit Anweisungen auf oberster Ebene besteht aus drei Abschnitten in dieser Reihenfolge:

1. (optional) `using`-Direktiven
2. eine Folge von Anweisungen, optional vermischt mit Methodendeklarationen
3. (optional) Typ- und Namensraumdeklarationen

Alles in Abschnitt 2 landet in einer vom Compiler generierten `Main`-Methode innerhalb einer ebenso generierten Klasse. Das heißt, dass die Methoden in Ihren Anweisungen auf oberster Ebene zu *lokalen Methoden* werden (deren Feinheiten wir im Abschnitt »Lokale Methoden« auf Seite 78 beschreiben). Anweisungen auf oberster Ebene können optional einen Integer-Wert an den Aufrufenden zurückgeben und auf eine »magische« Variable namens `args` vom Typ `string[]` zugreifen, in der die vom Aufrufenden übergebenen Befehlszeilenargumente enthalten sind.

Da ein Programm nur einen Einstiegspunkt haben kann, kann es in einem C#-Projekt auch nur höchstens eine Datei mit Anweisungen auf oberster Ebene geben.

Typen und Umwandlungen

C# kann Instanzen kompatibler Typen umwandeln. Eine Umwandlung erstellt immer einen neuen Wert für einen bestehenden Wert. Umwandlungen können entweder *implizit* oder *explizit* sein. Implizite Umwandlungen erfolgen automatisch, während explizite Umwandlungen einen `Cast` erfordern. Im folgenden Beispiel konvertieren wir *implizit* einen `int` in einen `long` (der doppelt so viel Kapazität an Bits wie ein `int` bietet) und casten *explizit* einen `int` auf einen `short` (der nur die halbe Bit-Kapazität eines `int` bietet):

```
int x = 12345;           // int ist ein 32-Bit-Integer
long y = x;              // implizite Umwandlung in einen 64-Bit-int
short z = (short)x;      // explizite Umwandlung in einen 16-Bit-int
```

In der Regel sind implizite Umwandlungen dann zulässig, wenn der Compiler garantieren kann, dass sie immer gelingen werden, ohne dass dabei Informationen verloren gehen. Andernfalls müssen Sie einen expliziten `Cast` nutzen, um die Umwandlung zwischen kompatiblen Typen durchzuführen.

Werttypen vs. Referenztypen

C#-Typen können in *Werttypen* und *Referenztypen* eingeteilt werden.

Werttypen enthalten die meisten eingebauten Typen (genauer gesagt, alle numerischen Typen sowie die Typen `char` und `bool`), aber auch selbst definierte `struct`- und `enum`-Typen. *Referenztypen* enthalten alle Klassen-, Array-, Delegate- und Interface-Typen.

Der prinzipielle Unterschied zwischen Werttypen und Referenztypen ist ihre Behandlung im Arbeitsspeicher.

Werttypen

Der Inhalt einer *Werttyp*-Variablen oder -Konstanten ist einfach ein Wert. So besteht zum Beispiel der Inhalt des eingebauten Werttyps `int` aus 32 Bit mit Daten.

Sie können einen selbst definierten Werttyp mithilfe des Schlüsselworts `struct` definieren (siehe Abbildung 1):

```
public struct Point { public int X, Y; }
```

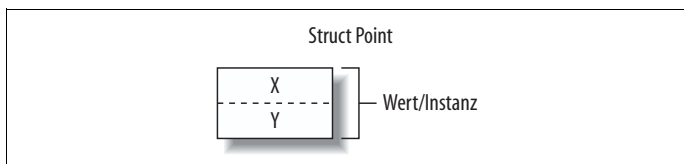


Abbildung 1: Eine Werttyp-Instanz im Speicher

Das Zuweisen einer Werttyp-Instanz *kopiert* immer die Instanz:

```
Point p1 = new Point();  
p1.X = 7;  
  
Point p2 = p1;           // Zuweisung führt zum Kopieren  
  
Console.WriteLine (p1.X); // 7  
Console.WriteLine (p2.X); // 7  
  
p1.X = 9;                // ändert p1.X  
Console.WriteLine (p1.X); // 9  
Console.WriteLine (p2.X); // 7
```

Abbildung 2 zeigt, dass p1 und p2 unabhängig voneinander gespeichert werden.

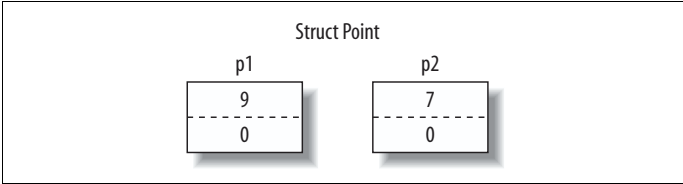


Abbildung 2: Eine Zuweisung kopiert eine Werttyp-Instanz.

Referenztypen

Ein *Referenztyp* ist komplexer als ein Werttyp. Er besteht aus zwei Teilen: einem *Objekt* und der *Referenz* auf dieses Objekt. Der Inhalt einer Referenztyp-Variablen oder -Konstanten ist eine Referenz auf ein Objekt, das den Wert enthält. Hier ist der Typ Point aus unserem vorigen Beispiel als Klasse umgeschrieben worden (siehe Abbildung 3):

```
public class Point { public int X, Y; }
```

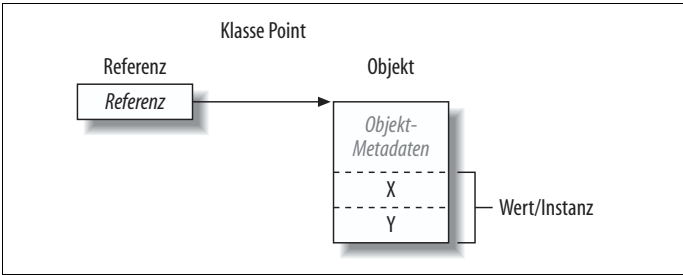


Abbildung 3: Ein Referenztyp im Speicher

Durch das Zuweisen einer Referenztyp-Variablen wird die Referenz kopiert, nicht die Objektinstanz. Damit ist es möglich, mit mehreren Variablen auf dasselbe Objekt zu verweisen – etwas, das mit Werttypen normalerweise nicht geht. Wenn wir das vorige Beispiel wiederholen, diesmal aber mit Point als Klasse, beeinflusst eine Operation auf p1 auch p2:

```

Point p1 = new Point();
p1.X = 7;

Point p2 = p1;           // kopiert Referenz von p1

Console.WriteLine (p1.X); // 7
Console.WriteLine (p2.X); // 7

p1.X = 9;                // ändert p1.X
Console.WriteLine (p1.X); // 9
Console.WriteLine (p2.X); // 9

```

Abbildung 4 zeigt, dass p1 und p2 zwei Referenzen sind, die auf dasselbe Objekt verweisen.

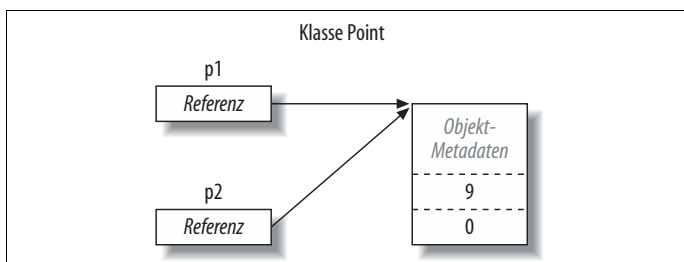


Abbildung 4: Eine Zuweisung kopiert eine Referenz.

Null

Einer Referenz kann das Literal `null` zugewiesen werden, wodurch ausgesagt wird, dass die Referenz auf kein Objekt zeigt – vorausgesetzt, `Point` ist eine Klasse:

```

Point p = null;
Console.WriteLine (p == null); // True

```

Der Versuch, auf ein Member einer Null-Referenz zuzugreifen, führt zu einem Laufzeitfehler:

```

Console.WriteLine (p.X); // NullReferenceException

```



Im Abschnitt »Nullbare Referenztypen« auf Seite 167 beschreiben wir ein Feature von C#, mit dem sich ungewollte `NullReferenceException`-Fehler verhindern lassen.

Im Gegensatz dazu kann einem Werttyp auf normalem Weg kein Null-Wert zugewiesen werden:

```
struct Point {...}  
...  
Point p = null; // Compilerfehler  
int x = null;   // Compilerfehler
```



C# bietet *nullbare Typen* an, mit denen Werttypen auch Null-Werte repräsentieren können (siehe den Abschnitt »Nullbare Werttypen« auf Seite 162).

Die Einteilung der vordefinierten Typen

Die vordefinierten Typen in C# sind folgende:

Werttypen

- Numerisch
 - Ganzzahl mit Vorzeichen (sbyte, short, int, long)
 - Ganzzahl ohne Vorzeichen (byte, ushort, uint, ulong)
 - Reelle Zahl (float, double, decimal)
- Logisch (bool)
- Zeichen (char)

Referenztypen

- String (string)
- Objekt (object)

Die vordefinierten Typen in C# sind Aliase für .NET-Typen aus dem Namensraum `System`. Zwischen den beiden folgenden Anweisungen gibt es nur syntaktische Unterschiede:

```
int i = 5;  
System.Int32 i = 5;
```

Die vordefinierten Werttypen (mit Ausnahme von `decimal`) werden in der *Common Language Runtime* (CLR) als *elementare Typen* bezeichnet. Sie heißen so, weil sie im kompilierten Code direkt über Anweisungen unterstützt werden, die üblicherweise auf eine unmittel-

telbare Unterstützung durch den zugrunde liegenden Prozessor zurückgehen.

Numerische Typen

C# bietet die folgenden vordefinierten numerischen Typen:

C#-Typ	Systemtyp	Suffix	Breite	Bereich
Ganzzahlig mit Vorzeichen				
sbyte	SByte		8 Bit	-2^7 bis $2^7 - 1$
short	Int16		16 Bit	-2^{15} bis $2^{15} - 1$
int	Int32		32 Bit	-2^{31} bis $2^{31} - 1$
long	Int64	L	64 Bit	-2^{63} bis $2^{63} - 1$
nint	IntPtr		32/64 Bit	
Ganzzahlig ohne Vorzeichen				
byte	Byte		8 Bit	0 bis $2^8 - 1$
ushort	UInt16		16 Bit	0 bis $2^{16} - 1$
uint	UInt32	U	32 Bit	0 bis $2^{32} - 1$
ulong	UInt64	UL	64 Bit	0 bis $2^{64} - 1$
uint	UIntPtr		32/64 Bit	
Reell				
float	Single	F	32 Bit	$\pm (\sim 10^{-45}$ bis $10^{38})$
double	Double	D	64 Bit	$\pm (\sim 10^{-324}$ bis $10^{308})$
decimal	Decimal	M	128 Bit	$\pm (\sim 10^{-28}$ bis $10^{28})$

Von den *ganzzahligen* Typen sind `int` und `long` Bürger erster Klasse und werden von C# und der Runtime bevorzugt. Die anderen ganzzahligen Typen werden üblicherweise im Dienste der Interoperabilität eingesetzt oder wenn eine effiziente Speicherplatznutzung wichtig ist. Die ganzzahligen Typen `nint` und `nuint` (eingeführt in C# 9) haben eine Größe, die zum Adressraum des Laufzeitprozesses passt. Sie können bei der Arbeit mit Zeiger-Arithmetik hilfreich sein. Wir beschreiben sie detailliert in Kapitel 4 von *C# 10 in a Nutshell*.

Von den *reellen* Zahltypen werden `float` und `double` auch als *Gleitkommatypen* bezeichnet und üblicherweise für wissenschaftliche Berechnungen sowie im Grafikumfeld genutzt. Der Typ `decimal` wird in der Regel für finanzmathematische Berechnungen verwendet, bei denen eine exakte Basis-10-Arithmetik und hohe Genauigkeit erforderlich sind. (Technisch betrachtet, ist `decimal` ebenfalls ein Gleitkommatyp, wird normalerweise aber nicht als solcher bezeichnet.)

Numerische Literale

*Ganzzahl*litterale können mit der Dezimal-, Hexadezimal- oder Binärnotation dargestellt werden; die Hexadezimalnotation wird mit dem Präfix `0x` angezeigt (z. B. entspricht `0x7f` dem Dezimalwert 127), die Binärnotation mit dem Präfix `0b`. *Reelle* *Literale* können die Dezimal- oder die Exponentialnotation nutzen, z. B. `1E06`. Numerische Literale können für eine bessere Lesbarkeit durch Unterstriche ergänzt werden (zum Beispiel `1_000_000`).

Typableitung bei numerischen Literalen

Standardmäßig geht der Compiler davon aus, dass ein numerisches Literal entweder einen `double`-Wert oder einen ganzzahligen Wert darstellt:

- Wenn das Literal einen Dezimaltrenner oder das Exponentialsymbol (E) enthält, ist der Typ `double`.
- Andernfalls ist der Typ des Literals der erste Typ aus der folgenden Liste, in den der Wert des Literals passt: `int`, `uint`, `long` und `ulong`.

Hier sehen Sie Beispiele dafür:

```
Console.Write (    1.0.GetType()); // Double (double)
Console.Write (    1E06.GetType()); // Double (double)
Console.Write (    1.GetType());   // Int32 (int)
Console.Write ( 0xF0000000.GetType()); // UInt32 (uint)
Console.Write (0x100000000.GetType()); // Int64 (long)
```

Numerische Suffixe

Die *numerischen Suffixe*, die in der vorangegangenen Tabelle aufgeführt sind, definieren den Typ eines Literals:

```
decimal d = 3.5M;    // M = decimal (ignoriert Groß-/Kleinschrei-  
                     // bung)
```

Die Suffixe U und L werden nur selten benötigt, weil die Typen `uint`, `long` und `ulong` fast immer *erschlossen* werden können oder `int` *implicit* in sie umgewandelt werden kann.

```
long i = 5;          // implizite Umwandlung von int in long
```

Das Suffix D ist technisch gesehen redundant, da bei allen Literalen, die einen Dezimaltrenner enthalten, geschlossen wird, dass es `double`-Werte sind (und man einem numerischen Literal immer einen Dezimaltrenner anhängen kann). Die Suffixe F und M sind am nützlichsten und außerdem unumgänglich, wenn man `float`- oder `decimal`-Literele angeben will. Ohne Suffixe ließen sich die folgenden Anweisungen nicht kompilieren, da geschlossen würde, dass 4.5 den Typ `double` hat, der sich nicht *implicit* in `float` oder `decimal` umwandeln lässt.

```
float f = 4.5F;      // kompiliert ohne Suffix nicht  
decimal d = 4.5M;    // kompiliert ohne Suffix nicht
```

Numerische Umwandlung

Ganzzahlige Umwandlungen

Ganzzahlige Umwandlungen sind *implicit*, wenn der Zieltyp jeden möglichen Wert des Ausgangstyps darstellen kann. Andernfalls ist eine *explizite* Umwandlung erforderlich, zum Beispiel so:

```
int x = 12345;        // int ist ein 32-Bit-Wert  
long y = x;           // implizite Umwandlung in einen 64-Bit-int  
short z = (short)x;   // explizite Umwandlung in einen 16-Bit-int
```

Reelle Umwandlungen

Ein `float` kann *implicit* in einen `double` umgewandelt werden, weil ein `double` jeden möglichen `float`-Wert darstellen kann. Die umgekehrte Umwandlung muss *explizit* sein.

Die Umwandlung zwischen `decimal` und den anderen reellen Typen muss *explizit* sein.

Reelle Typen in ganzzahlige Typen umwandeln

Die Umwandlung eines ganzzahligen Typs in einen reellen Typ ist immer implizit, während die umgekehrte Umwandlung immer explizit angegeben werden muss. Bei der Umwandlung eines Gleitkommatyps in einen ganzzahligen Typ wird immer der möglicherweise vorhandene Nachkommateil der Zahl abgeschnitten. Rundende Umwandlungen können Sie mit der statischen Klasse `System.Convert` durchführen.

Ein Nachteil der impliziten Umwandlung ist, dass bei der Umwandlung großer ganzzahlige Werte in Gleitkommawerte zwar die *Größenordnung* erhalten bleibt, gelegentlich aber *Genauigkeit* verloren geht.

```
int i1 = 1000000001;
float f = i1;        // Größe bleibt erhalten, Genauigkeit ver-
                     // schwindet
int i2 = (int)f;     // 1000000000
```

Arithmetische Operatoren

Die arithmetischen Operatoren (+, -, *, /, %) sind für alle numerischen Typen außer den 8- und 16-Bit-Ganzzahltypen definiert. Der %-Operator wird zum Rest nach der Division ausgewertet.

Inkrement- und Dekrementoperatoren

Die Inkrement- und Dekrementoperatoren (++ , --) erhöhen bzw. verringern numerische Typen um eins. Der Operator kann entweder vor oder hinter der Variablen stehen – je nachdem, ob Sie die Variable *vor* oder *nach* dem Auswerten des Ausdrucks verändern wollen. Hier sehen Sie ein Beispiel:

```
int x = 0;
Console.WriteLine (x++); // Ausgabe 0; x ist jetzt 1
Console.WriteLine (++x);  // Ausgabe 2; x ist jetzt 2
Console.WriteLine (--x);  // Ausgabe 1; x ist jetzt 1
```

Besondere integrale Operationen

Division

Divisionsoperationen mit ganzzahligen Operanden schneiden immer den Rest ab (runden auf null). Eine Division durch eine Variable, deren Wert null ist, führt zu einem Laufzeitfehler (einer `DivideByZeroException`). Eine Division durch das *Literal* 0 oder eine *Konstante* mit dem Wert 0 führt zu einem Kompilationsfehler.

Überlauf

Arithmetische Operationen auf Integer-Typen können zur Laufzeit zu einem Überlauf führen. Standardmäßig geschieht das still und leise – es wird keine Exception ausgelöst, und das Ergebnis weist ein Umschlagverhalten auf, als ob die Berechnung mit einem größeren Ganzzahltypen durchgeführt und die zusätzlichen signifikanten Bits verworfen worden wären. Verringert man zum Beispiel den minimalen möglichen `int`-Wert um eins, erhält man den maximal möglichen `int`-Wert:

```
int a = int.MinValue; a--;  
Console.WriteLine (a == int.MaxValue); // True
```

Die Operatoren `checked` und `unchecked`

Der Operator `checked` teilt der Laufzeitumgebung mit, eine `OverflowException` zu erzeugen, statt still und heimlich fehlzuschlagen, wenn ein ganzzahliger Ausdruck oder eine Anweisung die arithmetischen Grenzen dieses Typs überschreitet. Der Operator `checked` ist für Ausdrücke mit `++`, `-` (unär), `+`, `-`, `*`, `/` und expliziten Konvertierungsoperatoren zwischen ganzzahligen Typen nutzbar. Die Überlaufprüfung bringt ein wenig Performanceverlust mit sich.

Sie können `checked` um einen Ausdruck oder einen Anweisungsblock herum verwenden, zum Beispiel so:

```
int a = 1000000, b = 1000000;  
  
int c = checked (a * b);    // Prüft nur den Ausdruck
```

```
checked                // Prüft alle Ausdrücke
{                      // im Anweisungsblock
    c = a * b;
    ...
}
```

Sie können die Prüfung auf arithmetische Überläufe zum Standard für alle Ausdrücke in einem Programm machen, indem Sie sie mit dem Schalter `/checked+` an der Befehlszeile kompilieren (in Visual Studio gehen Sie zu den Advanced Build Settings). Wenn Sie die Überlaufprüfung nur für bestimmte Ausdrücke oder Anweisungen abschalten wollen, können Sie das mit dem Operator `unchecked` erreichen.

Bitoperatoren

C# unterstützt die folgenden Bitoperatoren :

Operator	Bedeutung	Beispiel	Ergebnis
~	Komplement	~0xfU	0xffffffff0U
&	Und	0xf0 & 0x33	0x30
	Oder	0xf0 0x33	0xf3
^	Exklusives Oder	0xff00 ^ 0x0ff0	0xf0f0
<<	Verschiebung nach links	0x20 << 2	0x80
>>	Verschiebung nach rechts	0x20 >> 1	0x10

8- und 16-Bit-Ganzzahlwerte

Die 8- und 16-Bit-Integer-Typen sind `byte`, `sbyte`, `short` und `ushort`. Sie haben keine eigenen arithmetischen Operatoren, daher konvertiert C# sie bei Bedarf implizit in größere Typen. Das kann zu einem Kompilierungsfehler führen, wenn man versucht, das Ergebnis wieder einem kleinen Integer-Typ zuzuweisen:

```
short x = 1, y = 1;
short z = x + y;           // Compilerfehler
```

In diesem Fall werden `x` und `y` implizit nach `int` konvertiert, damit die Addition durchgeführt werden kann. Das bedeutet aber, dass das Ergebnis ebenfalls ein `int` ist, der nicht implizit in einen `short`

konvertiert werden kann (da es dabei zu Datenverlust kommen könnte). Damit sich diese Anweisung kompilieren lässt, müssen wir einen expliziten Cast hinzufügen:

```
short z = (short) (x + y);    // okay
```

Spezielle Float- und Double-Werte

Im Unterschied zu Integer-Typen haben Gleitkommatypen Werte, die bestimmte Operationen besonders behandeln. Diese speziellen Werte sind NaN (*Not a Number*, keine Zahl), $+\infty$, $-\infty$ und -0 . Die Float- und Double-Klassen haben Konstanten für NaN, $+\infty$ und $-\infty$ und auch für andere Werte (MaxValue, MinValue und Epsilon). Hier sehen Sie ein Beispiel:

```
Console.Write (double.NegativeInfinity);    // -Infinity
```

Die Division einer Zahl ungleich null durch null ergibt einen der Unendlich-Werte:

```
Console.WriteLine ( 1.0 / 0.0);    // Infinity
Console.WriteLine (-1.0 / 0.0);    // -Infinity
Console.WriteLine ( 1.0 / -0.0);    // -Infinity
Console.WriteLine (-1.0 / -0.0);    // Infinity
```

Null geteilt durch null oder unendlich minus unendlich ergibt NaN:

```
Console.Write ( 0.0 / 0.0);          // NaN
Console.Write ((1.0 / 0.0) - (1.0 / 0.0));    // NaN
```

Wenn `==` genutzt wird, ist ein NaN-Wert niemals gleich einem anderen Wert, auch nicht einem anderen NaN-Wert. Um zu prüfen, ob ein Wert NaN ist, müssen Sie die Methoden `float.IsNaN` oder `double.IsNaN` einsetzen:

```
Console.WriteLine (0.0 / 0.0 == double.NaN);    // False
Console.WriteLine (double.IsNaN (0.0 / 0.0));    // True
```

Verwenden Sie allerdings `object.Equals`, sind zwei NaN-Werte gleich:

```
bool isTrue = object.Equals (0.0/0.0, double.NaN);
```

double vs. decimal

Für wissenschaftliche Berechnungen (wie die Berechnung von Raumkoordinaten) eignet sich `double`. `decimal` ist für finanzmathematische Berechnungen geeignet und für Werte, die »menschengeschaffen« sind und nicht das Ergebnis echter Messungen. Hier sehen Sie eine Zusammenfassung der Unterschiede:

Eigenschaft	double	decimal
Interne Darstellung	Basis 2	Basis 10
Genauigkeit	15 bis 16 signifikante Ziffern	28 bis 29 signifikante Ziffern
Wertbereich	$\pm(\sim 10^{-324} \text{ bis } \sim 10^{308})$	$\pm(\sim 10^{-28} \text{ bis } \sim 10^{28})$
Besondere Werte	+0, -0, +∞, -∞ und NaN	keine
Geschwindigkeit	native CPU-Verarbeitung	keine native Verarbeitung auf der CPU (ungefähr 10-mal langsamer als double)

Rundungsfehler bei reellen Zahlen

`float` und `double` werden intern durch Zahlen zur Basis 2 repräsentiert. Aus diesem Grund werden die meisten Literale mit einem Nachkommateil (die zur Basis 10 sind) nicht genau abgebildet, was der Grund dafür ist, dass `float` und `double` für finanzmathematische Berechnungen nicht geeignet sind. Im Gegensatz dazu nutzt `decimal` die Basis 10 und kann Nachkommaanteile wie 0.1 (deren Basis-10-Darstellung nicht unendlich periodisch ist) deswegen genau darstellen.

Der Typ bool und die booleschen Operatoren

Der C#-Typ `bool` (der ein Alias für den Typ `System.Boolean` ist) ist ein logischer Wert, dem eines der Literale `true` und `false` zugewiesen werden kann.

Auch wenn ein boolescher Wert beim Speichern nur ein Bit benötigt, nutzt die Laufzeitumgebung ein Byte im Speicher, da dies die kleinste Einheit ist, mit der die Laufzeitumgebung und der Prozes-

sor sinnvoll umgehen können. Um bei Arrays den Speicherplatz nicht ineffizient zu nutzen, stellt .NET im Namensraum `System.Collections` eine Klasse namens `BitArray` bereit, die nur ein Bit pro booleschen Wert verwendet.

Gleichheits- und Vergleichsoperatoren

`==` und `!=` prüfen auf Gleichheit bzw. Ungleichheit zwischen beliebigen Typen, liefern aber immer einen `bool`-Wert zurück. Werttypen haben normalerweise eine sehr einfache Vorstellung von Gleichheit:

```
int x = 1, y = 2, z = 1;
Console.WriteLine (x == y);      // False
Console.WriteLine (x == z);      // True
```

Bei Referenztypen basiert die Gleichheit standardmäßig auf der *Referenz* und nicht auf dem eigentlichen *Wert* des zugrunde liegenden Objekts. Zwei Instanzen eines Objekts mit identischen Daten werden nur dann als gleich betrachtet, wenn der `==`-Operator für diesen Typ explizit entsprechend überladen wurde (mehr Informationen finden Sie in den Abschnitten »Der Typ `object`« auf Seite 104 und »Überladen von Operatoren« auf Seite 220).

Die Gleichheits- und Vergleichsoperatoren `==`, `!=`, `<`, `>`, `>=` und `<=` funktionieren bei allen numerischen Typen, sollten bei reellen Zahlen aber mit Vorsicht eingesetzt werden (siehe den Abschnitt »Rundungsfehler bei reellen Zahlen« auf Seite 32). Die Vergleichsoperatoren funktionieren auch bei Mitgliedern von `enum`-Typen, indem sie die zugrunde liegenden ganzzahligen Werte vergleichen.

Bedingungsoperatoren

Die Operatoren `&&` und `||` testen auf die Bedingungen *und* und *oder*. Sie werden häufig zusammen mit dem Operator `!` verwendet, der für *nicht* steht. Im folgenden Beispiel liefert die Methode `UseUmbrella` den Wert `true` zurück, wenn es regnet oder die Sonne scheint (um uns vor dem Regen oder der Sonne zu schützen), solange es nicht auch noch windig ist (da Regenschirme bei Wind nicht viel helfen):

```
static bool UseUmbrella (bool rainy, bool sunny,
                        bool windy)
{
    return !windy && (rainy || sunny);
}
```

Die Operatoren && und || *kürzen* die Auswertung wenn möglich *ab*. Im vorigen Beispiel wird der Ausdruck (rainy || sunny) gar nicht ausgewertet, wenn es nicht windig ist. Das Abkürzen spielt eine wesentliche Rolle dabei, dass Ausdrücke wie der folgende ausgewertet werden können, ohne dass eine `NullReferenceException` ausgelöst wird:

```
if (sb != null && sb.Length > 0) ...
```

Die Operatoren & und | können ähnlich verwendet werden:

```
return !windy & (rainy | sunny);
```

Der Unterschied liegt darin, dass sie *keine abgekürzte Auswertung* vornehmen. Aus diesem Grund werden sie nur selten als bedingte Operatoren eingesetzt.

Der ternäre Bedingungsoperator (der einfach als *Bedingungsoperator* bezeichnet wird) hat die Form `q ? a : b`. Dabei wird `a` ausgewertet, falls die Bedingung `q` wahr ist, ansonsten `b`. Hier sehen Sie ein Beispiel:

```
static int Max (int a, int b)
{
    return (a > b) ? a : b;
}
```

Der Bedingungsoperator ist bei LINQ-Abfragen besonders nützlich.

Strings und Zeichen

Der C#-Typ `char` (ein Alias des Typs `System.Char`) repräsentiert ein Unicode-Zeichen und nimmt zwei Bytes ein (UTF-16). Ein `char`-Literal wird innerhalb von einfachen Anführungszeichen angegeben:

```
char c = 'A';           // einzelnes Zeichen
```

Escape-Sequenzen geben Zeichen an, die nicht direkt ausgedrückt oder interpretiert werden können. Eine Escape-Sequenz ist ein Backslash, gefolgt von einem Zeichen mit einer bestimmten Bedeutung:

```
char newLine = '\n';  
char backSlash = '\\';
```

Die Escape-Sequenz-Zeichen sind folgende:

Char	Bedeutung	Wert
\'	einfache Anführungszeichen	0x0027
\"	doppelte Anführungszeichen	0x0022
\\	Backslash	0x005C
\0	Null	0x0000
\a	Alert	0x0007
\b	Backspace	0x0008
\f	Seitenvorschub	0x000C
\n	Zeilenumbruch	0x000A
\r	Carriage-Return	0x000D
\t	horizontaler Tabulator	0x0009
\v	vertikaler Tabulator	0x000B

Die Escape-Sequenz `\u` (bzw. `\x`) ermöglicht Ihnen, ein beliebiges Unicode-Zeichen über seinen vierstelligen Hexadezimalcode anzugeben:

```
char copyrightSymbol = '\u00A9';  
char omegaSymbol    = '\u03A9';  
char newLine        = '\u000A';
```

Eine implizite Konvertierung von einem `char` in einen numerischen Typ funktioniert für die numerischen Typen, die ein vorzeichenloses `short` aufnehmen können. Bei anderen numerischen Typen ist eine explizite Konvertierung notwendig.

String-Typ

Der `string`-Typ von C# (ein Alias für den Typ `System.String`) repräsentiert eine nicht veränderbare Kette von Unicode-Zeichen.

Ein String-Literal wird innerhalb von doppelten Anführungszeichen angegeben:

```
string a = "Heat";
```



string ist ein Referenztyp, kein Werttyp. Seine Vergleichsoperatoren folgen jedoch der Werttyp-Semantik.

```
string a = "test", b = "test";  
Console.Write (a == b); // True
```

Die für char-Literale möglichen Escape-Sequenzen lassen sich auch bei Strings nutzen:

```
string a = "Hier ist ein Tabulator:\t";
```

Wenn Sie allerdings einen Backslash als Zeichen nutzen wollen, müssen Sie ihn doppelt schreiben:

```
string a1 = "\\server\\fileshare\\helloworld.cs";
```

Um dieses Problem zu vermeiden, bietet C# *Verbatim-Strings* an. Ein Verbatim-string-Literal wird durch ein @ eingeleitet und unterstützt keine Escape-Sequenzen. Der folgende Verbatim-String ist mit dem String oben identisch:

```
string a2 = @"\\server\fileshare\helloworld.cs";
```

Ein Verbatim-String-Literal kann auch über mehrere Zeilen gehen. Sie können das doppelte Anführungszeichen in einem Verbatim-Literal nutzen, indem Sie es doppelt schreiben.

String-Verkettung

Der +-Operator verkettet zwei Strings:

```
string s = "a" + "b";
```

Einer der Operanden kann ein Nicht-String-Wert sein. Dann wird ToString auf diesem Wert aufgerufen, zum Beispiel so:

```
string s = "a" + 5; // a5
```

Es kann ineffizient sein, einen String unter wiederholter Verwendung des +-Operators aufzubauen. Eine bessere Lösung ist der Ein-

satz des Typs `System.Text.StringBuilder`, der einen veränderlichen (bearbeitbaren) String repräsentiert und Methoden bietet, über die Substrings effizient *angehängt*, *eingefügt*, *entfernt* oder *ersetzt* werden können.

String-Interpolation

Ein String, dem ein `$` (Dollarzeichen) vorangestellt ist, wird als *interpolierter String* bezeichnet. Interpolierte Strings können in geschweiften Klammern Ausdrücke enthalten:

```
int x = 4;
Console.WriteLine($"Ein Quadrat hat {x} Seiten");
// Gibt aus: Ein Quadrat hat 4 Seiten
```

Jeder gültige C#-Ausdruck beliebigen Typs kann in den geschweiften Klammern stehen, und C# wird ihn durch den Aufruf seiner `ToString`-Methode oder eines Äquivalents umwandeln. Sie können die Formatierung anpassen, indem Sie auf den Ausdruck einen Doppelpunkt und einen *Formatstring* folgen lassen:

```
string s = $"15 in Hex ist {15:X2}";
// Wird zu "15 in Hex ist 0F"
```

Interpolierte Strings dürfen nicht über eine Zeile hinausgehen, sofern Sie nicht auch noch den Verbatim-String-Operator verwenden:

```
int x = 2;
string s = @$"Dies geht über {
x} Zeilen.";
```

Um in einem interpolierten String eine geschweifte öffnende oder schließende Klammer auszugeben, geben Sie sie doppelt an.

String-Vergleiche

`string` unterstützt nicht die Vergleichsoperatoren `<` und `>`. Sie müssen stattdessen die Methode `CompareTo` von `string` nutzen, die eine positive Zahl, eine negative Zahl oder null zurückgibt – abhängig davon, ob der erste Wert nach oder vor dem zweiten Wert kommt bzw. ob er gleich ist:

```
Console.WriteLine("Boston".CompareTo("Austin")); // 1
Console.WriteLine("Boston".CompareTo("Boston")); // 0
Console.WriteLine("Boston".CompareTo("Chicago")); // -1
```

Konstante interpolierte Strings (C# 10)

Seit C# 10 kann es sich bei interpolierten Strings um Konstanten handeln, solange die interpolierten Werte ebenfalls Konstanten sind:

```
const string greeting = "Hallo";  
const string message = $"{greeting} Welt";
```

In Strings suchen

Der Indexer von `string` liefert ein Zeichen an einer bestimmten Position zurück:

```
Console.Write ("Wort"[2]); // r
```

Die Methoden `IndexOf` und `LastIndexOf` suchen nach einem Zeichen innerhalb eines String. Die Methoden `Contains`, `StartsWith` und `EndsWith` suchen nach einem Substring in einem String.

Strings verändern

Da `string` nicht veränderbar ist, liefern alle Methoden, die einen String »bearbeiten«, einen neuen zurück und lassen das Original unangetastet:

- `Substring` liefert einen Teil eines Strings zurück.
- `Insert` und `Remove` fügen Zeichen an einer bestimmten Position ein bzw. entfernen sie.
- `PadLeft` und `PadRight` fügen Leerzeichen ein.
- `TrimStart`, `TrimEnd` und `Trim` entfernen Leerraum.

Die Klasse `string` definiert zudem die Methoden `ToUpper` und `ToLower`, um die Groß- und Kleinschreibung zu ändern, eine Methode `Split`, um einen String in Substrings aufzuteilen (basierend auf einem angegebenen Trennzeichen), und eine statische Methode `Join`, um Substrings wieder in einen String zusammenzuführen.

Arrays

Ein *Array* repräsentiert eine feste Zahl von Elementen eines bestimmten Typs. Die Elemente in einem Array werden immer in einem zusammenhängenden Speicherabschnitt gespeichert, wodurch man sehr schnell auf sie zugreifen kann.

Ein Array wird durch eckige Klammern nach dem Typ des Elements definiert. Die folgende Zeile deklariert ein Array mit fünf Zeichen:

```
char[] vowels = new char[5];
```

Eckige Klammern *indexieren* das Array auch, wodurch man auf ein bestimmtes Element über die Position zugreifen kann:

```
vowels[0] = 'a'; vowels[1] = 'e'; vowels[2] = 'i';  
vowels[3] = 'o'; vowels[4] = 'u';
```

```
Console.WriteLine (vowels [1]); // e
```

Damit wird »e« ausgegeben, weil Array-Indizes bei null beginnen. Wir können eine *for*-Schleife nutzen, um über jedes Element im Array zu iterieren. Die *for*-Schleife lässt in diesem Beispiel den Integer-Wert *i* von 0 bis 4 laufen:

```
for (int i = 0; i < vowels.Length; i++)  
    Console.Write (vowels [i]); // aeiou
```

Arrays implementieren zudem *IEnumerable<T>* (siehe »Enumeration und Iteratoren« auf Seite 157), sodass Sie Elemente über die Anweisung *foreach* aufzählen können:

```
foreach (char c in vowels) Console.Write (c); // aeiou
```

Bei allen Array-Indizierungen prüft die Laufzeit die Grenzen. Wenn Sie einen ungültigen Index nutzen, wird eine *IndexOutOfRangeException* ausgelöst:

```
vowels[5] = 'y'; // Laufzeitfehler
```

Die Eigenschaft *Length* eines Arrays liefert die Anzahl der Elemente im Array zurück. Ist ein Array einmal angelegt, kann seine Länge nicht mehr geändert werden. Der Namensraum *System.Collection* und die darunterliegenden Namensräume stellen weiterentwickelte Datenstrukturen bereit, zum Beispiel dynamisch in der Größe veränderbare Arrays und Dictionaries.

Mit einem *Array-Initialisierungsausdruck* können Sie ein Array in einem Schritt deklarieren und füllen, zum Beispiel so:

```
char[] vowels = new char[] { 'a', 'e', 'i', 'o', 'u' };
```

Oder einfach so:

```
char[] vowels = { 'a', 'e', 'i', 'o', 'u' };
```

Alle Arrays erben von der Klasse `System.Array`, in der Methoden und Eigenschaften für alle Arrays definiert sind. Dazu gehören Instanzeigenschaften wie `Length` und `Rank`, aber auch statische Methoden, um

- ein Array dynamisch anzulegen (`CreateInstance`),
- Elemente unabhängig vom Array-Typ zu lesen und zu schreiben (`GetValue/SetValue`),
- ein sortiertes Array (`BinarySearch`) oder ein unsortiertes Array (`IndexOf`, `LastIndexOf`, `Find`, `FindIndex`, `FindLastIndex`) zu durchsuchen,
- ein Array zu sortieren (`Sort`) oder
- ein Array zu kopieren (`Copy`).

Standard-Elementinitialisierung

Beim Erstellen eines Arrays werden dessen Elemente immer durch Standardwerte vorinitialisiert. Der Standardwert eines Typs ist das Ergebnis eines bitweisen Löschsens seines Speicherbereichs. Stellen Sie sich zum Beispiel das Erstellen eines Arrays mit Integer-Werten vor. Da `int` ein Werttyp ist, werden 1.000 Integer-Zahlen in einem zusammenhängenden Speicherabschnitt zugewiesen. Der Standardwert für jedes Element ist 0:

```
int[] a = new int[1000];  
Console.Write (a[123]);           // 0
```

Bei Referenztyp-Elementen ist der Vorgabewert `null`.

Ein Array *selbst* ist immer ein Referenztyp-Objekt – unabhängig davon, welchen Typ seine Elemente haben. Beispielsweise ist Folgendes zulässig:

```
int[] a = null;
```

Indizes und Bereiche

Seit C# 8 gibt es *Indizes* und *Bereiche*, mit denen sich die Arbeit mit Elementen oder Teilen eines Arrays vereinfachen lässt.



Indizes und Bereiche funktionieren ebenfalls für die CLR-Typen `Span<T>` und `ReadOnlySpan<T>`, mit denen Sie einen effizienten Low-Level-Zugriff auf gemanagten oder nicht gemanagten Speicher erhalten.

Sie können auch Ihre eigenen Typen mit Indizes und Bereichen zusammenarbeiten lassen, indem Sie einen Indexer des Typs `Index` oder `Range` definieren (siehe Abschnitt »Indexer« auf Seite 88).

Indizes

Mit Indizes können Sie Elemente relativ vom Ende eines Arrays aus ansprechen, indem Sie den Operator `^` verwenden. `^1` bezieht sich auf das letzte Element, `^2` auf das vorletzte und so weiter:

```
char[] vowels = new char[] { 'a', 'e', 'i', 'o', 'u' };
char lastElement = vowels[<strong>^1</strong>]; // 'u'
char secondToLast = vowels[<strong>^2</strong>]; // 'o'
```

(`^0` gibt die Länge des Arrays zurück, daher erzeugt `vowels[^0]` einen Fehler.)

C# implementiert Indizes mithilfe des Typs `Index`, daher können Sie Folgendes tun:

```
Index first = 0;
Index last = ^1;
char firstElement = vowels[first]; // 'a'
char lastElement = vowels[last]; // 'u'
```

Bereiche

Mit Bereichen können Sie ein Array mit dem Operator `..` »zerschneiden«:

```
char[] firstTwo = vowels[..2]; // 'a', 'e'
char[] lastThree = vowels[2..]; // 'i', 'o', 'u'
char[] middleOne = vowels[2..3]; // 'i'
```

Die zweite Zahl im Bereich ist ausschließend, das heißt, `..2` liefert die Elemente vor `vowels[2]` zurück.

Sie können in Bereichen auch das Symbol `^` nutzen. Mit dem folgenden Code erhalten Sie die letzten beiden Zeichen:

```
char[] lastTwo = vowels [^2..^0];    // 'o', 'u'
```

(`^0` ist hier gültig, weil die zweite Zahl im Bereich ausschließend ist.)

C# implementiert Bereiche mithilfe des Typs `Range`, daher können Sie Folgendes schreiben:

```
Range firstTwoRange = 0..2;  
char[] firstTwo = vowels [firstTwoRange];    // 'a', 'e'
```

Mehrdimensionale Arrays

Mehrdimensionale Arrays gibt es in zwei Varianten: *rechteckig* und *ungleichförmig*. Rechteckige Arrays stellen einen n -dimensionalen Speicherblock dar, während es sich bei ungleichförmigen Arrays um Arrays aus Arrays handelt.

Rechteckige Arrays

Rechteckige Arrays werden deklariert, indem die einzelnen Dimensionen durch Kommata getrennt werden. Mit der folgenden Anweisung wird ein rechteckiges, zweidimensionales Array deklariert, bei dem die Dimensionen 3×3 sind:

```
int[,] matrix = new int [3, 3];
```

Die Methode `GetLength` eines Arrays liefert die Länge für eine gegebene Dimension zurück (beginnend bei 0):

```
for (int i = 0; i < matrix.GetLength(0); i++)  
    for (int j = 0; j < matrix.GetLength(1); j++)  
        matrix [i, j] = i * 3 + j;
```

Ein rechteckiges Array kann wie folgt initialisiert werden (jedes Element wird in diesem Beispiel so initialisiert wie im vorigen Beispiel):

```
int[,] matrix = new int[,]  
{  
    {0,1,2},  
    {3,4,5},  
    {6,7,8}  
};
```

(Der fett gedruckte Code kann in Deklarationsanweisungen wie dieser weggelassen werden.)

Ungleichförmige Arrays

Ungleichförmige Arrays werden mit aufeinanderfolgenden eckigen Klammern deklariert, mit denen jede Dimension repräsentiert wird. Hier sehen Sie ein Beispiel für das Deklarieren eines unregelmäßigen zweidimensionalen Arrays, bei dem die äußerste Dimension 3 ist:

```
int[][] matrix = new int[3][];
```

Die inneren Dimensionen werden in der Deklaration nicht spezifiziert, da anders als bei einem rechteckigen Array jedes innere Array eine eigene Länge haben kann. Jedes innere Array wird implizit nicht auf ein leeres Array initialisiert, sondern auf null. Jedes innere Array muss per Hand angelegt werden:

```
for (int i = 0; i < matrix.Length; i++)
{
    matrix[i] = new int [3];          // inneres Array erzeugen
    for (int j = 0; j < matrix[i].Length; j++)
        matrix[i][j] = i * 3 + j;
}
```

Ein ungleichförmiges Array kann folgendermaßen initialisiert werden (um ein Array zu erstellen, das dem aus dem letzten Beispiel entspricht und nur am Ende ein zusätzliches Element enthält):

```
int[][] matrix = new int[][]
{
    new int[] {0,1,2},
    new int[] {3,4,5},
    new int[] {6,7,8,9}
};
```

(Der fett gedruckte Code kann in Deklarationsanweisungen wie dieser weggelassen werden.)

Vereinfachter Array-Initialisierungsausdruck

Wir haben bereits gesehen, dass man den Array-Initialisierungsausdruck vereinfachen kann, indem man das Schlüsselwort **new** und die Typdeklaration weglässt:

```
char[] vowels = new char[] { 'a','e','i','o','u' };
char[] vowels =           { 'a','e','i','o','u' };
```

Ein weiterer Ansatz ist, den Typnamen nach dem Schlüsselwort `new` wegzulassen und den Compiler den Array-Typ *erschließen* zu lassen. Das ist eine praktische Kurzform, wenn man Arrays als Argumente übergibt. Betrachten Sie beispielsweise die folgende Methode:

```
void Foo (char[] data) { ... }
```

Wir können diese Methode mit einem Array aufrufen, das wir im Vorübergehen erstellen:

```
Foo ( new char[] { 'a','e','i','o','u' } );    // Langform
Foo ( new[] { 'a','e','i','o','u' } );        // Kurzform
```

Diese Kurzform ist notwendig, wenn Sie Arrays *anonymer Typen* erstellen, wie Sie später noch sehen werden .

Variablen und Parameter

Eine *Variable* repräsentiert einen Speicherbereich, der einen veränderbaren Wert enthält. Eine Variable kann eine *lokale Variable*, ein *Parameter* (value, ref, out oder in), ein *Feld* (*Instanz* oder *statisch*) oder ein *Array-Element* sein.

Der Stack und der Heap

Der *Stack* und der *Heap* sind die Orte, an denen Variablen abgelegt werden. Jeder hat bezüglich der Lebensspanne der Objekte eine andere Semantik.

Stack

Der Stack ist der Speicher, in dem lokale Variablen und Parameter gespeichert werden. Der Stack wächst und schrumpft, sobald Methoden oder Funktionen betreten und wieder verlassen werden. Schauen Sie sich die folgende Methode an (um es nicht zu kompliziert zu machen, wird die Prüfung des übergebenen Arguments ignoriert):

```
static int Factorial (int x)
{
    if (x == 0) return 1;
    return x * Factorial (x-1);
}
```

Diese Methode ist rekursiv, ruft sich also selbst auf. Bei jedem Aufruf der Methode wird Platz für ein neues `int` auf dem Stack reserviert, und bei jedem Verlassen der Methode wird `int` wieder freigegeben.

Heap

Der *Heap* ist der Speicher, in dem *Objekte* (also Instanzen von Referenztypen) liegen. Immer dann, wenn ein neues Objekt erzeugt wird, wird auf dem Heap dafür Platz reserviert und eine Referenz auf dieses Objekt zurückgegeben. Während der Ausführung eines Programms füllt sich der Heap, während neue Objekte angelegt werden. Die Laufzeitumgebung hat einen Garbage Collector, der regelmäßig Objekte vom Heap entfernt, sodass Ihrem Programm nicht der Speicher ausgeht. Ein Objekt kann dann vom Heap entfernt werden, wenn es von keinem Objekt mehr referenziert wird, das selbst noch lebt.

Werttyp-Instanzen (und Objektreferenzen) leben dort, wo die Variable deklariert wurde. Wenn die Instanz als Feld innerhalb eines Klassentyps oder als Array-Element deklariert wurde, lebt diese Instanz auf dem Heap.



Sie können Objekte in C# nicht explizit löschen, so wie es in C++ möglich ist. Ein nicht referenziertes Objekt wird irgendwann vom Garbage Collector abgeräumt.

Der Heap wird auch genutzt, um statische Felder und Konstanten zu speichern. Anders als Objekte, die auf dem Heap untergebracht sind (und von der Garbage Collection gelöscht werden können), leben diese Daten, bis die Anwendungsdomäne ihrem Ende entgegengeht.

Sichere Zuweisung

C# nutzt ausschließlich sichere Zuweisungen. In der Praxis bedeutet dies, dass es außerhalb eines `unsafe`-Kontexts unmöglich ist, auf

nicht initialisierten Speicher zuzugreifen. Die sichere Zuweisung hat drei Folgen:

- Lokalen Variablen muss ein Wert zugewiesen werden, bevor man lesend auf sie zugreifen kann.
- Beim Aufruf einer Methode müssen die Funktionsargumente gefüllt sein (es sei denn, sie sind als optional markiert – mehr Informationen finden Sie unter »Optionale Parameter« auf Seite 50).
- Alle anderen Variablen (wie Felder und Array-Elemente) werden zur Laufzeit automatisch initialisiert.

Der folgende Code führt zum Beispiel zu einem Kompilierungsfehler:

```
int x;                // x ist eine lokale Variable
Console.WriteLine (x); // Kompilierungsfehler
```

Der folgende Code gibt hingegen 0 aus, weil Feldern implizit ein Vorgabewert zugewiesen wird (egal ob Instanzfeld oder statisches Feld):

```
Console.WriteLine (Test.X); // 0
class Test { public static int X; } // Feld
```

Vorgabewerte

Alle Typinstanzen haben einen Vorgabewert. Dieser Vorgabewert vordefinierter Typen ist das Ergebnis der Nullsetzung der Speicherbits und ist bei Referenztypen null, bei numerischen Typen und Enums 0, beim Typ char '\0' und beim Typ bool false.

Sie können den Vorgabewert für jeden Typ mit dem Schlüsselwort `default` anfordern (in der Praxis ist das, wie wir später sehen werden, bei Generics nützlich). Der Vorgabewert für einen selbst definierten Werttyp (also ein Struct) entspricht dem Vorgabewert für jedes Feld, das vom selbst definierten Typ angegeben wird:

```
Console.WriteLine (default (decimal)); // 0
decimal d = default;
```

Parameter

Eine Methode kann eine Reihe von Parametern haben. Parameter definieren die Argumente, die für diese Methode angegeben werden

müssen. In diesem Beispiel hat die Methode Foo einen einzelnen Parameter p vom Typ int:

```
Foo (8); // 8 ist ein Argument  
static void Foo (int p) {...} // p ist ein Parameter
```

Sie können mit den Modifikatoren ref, in und out beeinflussen, wie Parameter übergeben werden.

Parameter-Modifikator	Übergeben als	Variable muss zugewiesen sein
keiner	Wert	beim <i>Betreten</i>
ref	Referenz	beim <i>Betreten</i>
in	Referenz (schreibgeschützt)	beim <i>Betreten</i>
out	Referenz	beim <i>Verlassen</i>

Argumente als Wert übergeben

Standardmäßig werden Argumente in C# als Wert übergeben (*by Value*), was mit Abstand der am häufigsten vorkommende Fall ist. Das bedeutet, dass eine Kopie des Werts erstellt wird, wenn man ihn an die Methode übergibt:

```
int x = 8;  
Foo (x); // eine Kopie von x erstellen  
Console.WriteLine (x); // x ist immer noch 8  
  
static void Foo (int p)  
{  
    p = p + 1; // p um 1 erhöhen  
    Console.WriteLine (p); // p auf dem Bildschirm ausgeben  
}
```

Weist man p einen neuen Wert zu, ändert das nicht den Inhalt von x, weil p und x an unterschiedlichen Stellen im Speicher liegen.

Wenn ein Referenztyp-Argument als Wert übergeben wird, wird die Referenz kopiert, nicht das Objekt. Foo sieht dasselbe StringBuilder-Objekt, das wir instanziiert haben (sb), besitzt aber eine eigene Referenz darauf. Das bedeutet also, dass sb und fooSB unterschiedliche Variablen sind, die auf dasselbe StringBuilder-Objekt verweisen:

```

StringBuilder sb = new StringBuilder();
Foo (sb);
Console.WriteLine (sb.ToString());    // Test

static void Foo (StringBuilder fooSB)
{
    fooSB.Append ("Test");
    fooSB = null;
}

```

Da fooSB eine Kopie einer Referenz ist, wird sb nicht zu null, wenn man fooSB auf null setzt. (Wenn allerdings fooSB mit dem Modifikator ref deklariert und aufgerufen wurde, würde sb null werden.)

Der Modifikator ref

Um eine Referenz zu übergeben (*by Reference*), stellt C# den Modifikator ref bereit. Im folgenden Beispiel verweisen p und x auf denselben Speicherbereich:

```

int x = 8;
Foo (ref x);                // Foo direkt mit
                             // x arbeiten lassen
Console.WriteLine (x);      // x ist jetzt 9

static void Foo (ref int p)
{
    p = p + 1;               // p um 1 erhöhen
    Console.WriteLine (p);   // p am Bildschirm ausgeben
}

```

Weist man p nun einen neuen Wert zu, ändert sich auch der Inhalt von x. Beachten Sie, dass der Modifikator ref sowohl beim Schreiben als auch beim Aufrufen der Methode notwendig ist. Damit wird sehr deutlich, was hier passiert.



Ein Parameter kann als Referenz oder als Wert übergeben werden – unabhängig davon, ob der Parametertyp ein Referenztyp oder ein Werttyp ist.

Der Modifikator out

Ein out-Argument ist wie ein ref-Argument, jedoch mit folgenden Ausnahmen:

- Es muss nicht zugewiesen sein, bevor es der Funktion *übergeben* wird.
- Es muss zugewiesen sein, bevor die Funktion *verlassen* wird.

Der Modifikator out wird meist genutzt, um mehrere Rückgabewerte in einer Methode zu haben.

out-Variablen und Discards

Seit C# 7 können Sie Variablen beim Aufrufen von Methoden mit out-Parametern spontan deklarieren:

```
int.TryParse ("123", out int x);  
Console.WriteLine (x);
```

Das entspricht:

```
int x;  
int.TryParse ("123", out x);  
Console.WriteLine (x);
```

Bei Aufruf von Methoden mit mehreren out-Parametern können Sie alle für Sie uninteressanten Parameter durch einen Unterstrich »verwerfen«. Angenommen, SomeBigMethod wurde mit fünf out-Parametern definiert. Dann können alle bis auf den dritten wie folgt ignoriert werden:

```
SomeBigMethod (out _, out _, out int x, out _, out _);  
Console.WriteLine (x);
```

Der Modifikator in

Seit C# 7.2 können Sie einem Parameter den Modifikator in voranstellen, um zu verhindern, dass er in der Methode verändert wird. Damit spart sich der Compiler den Overhead, der beim Kopieren des Arguments vor der Übergabe entsteht – was gerade bei großen, eigenen Werttypen ins Gewicht fallen kann (siehe Abschnitt »Structs« auf Seite 108).

Der Modifikator `params`

Der Modifikator `params` kann für den letzten Parameter einer Methode angegeben werden. Dann akzeptiert die Methode eine beliebige Zahl an Argumenten eines bestimmten Typs. Der Parametertyp muss als (eindimensionales) Array deklariert werden:

```
int Sum (params int[] ints)
{
    int sum = 0;
    for (int i = 0; i < ints.Length; i++) sum += ints[i];
    return sum;
}
```

Diese Methode können wir folgendermaßen aufrufen:

```
Console.WriteLine (Sum (1, 2, 3, 4));    // 10
```

Gibt es null Argumente an der `params`-Position, wird ein Array der Länge null erstellt.

Sie können ein `params`-Argument auch als normales Array angeben. Der vorangegangene Aufruf entspricht dem folgenden:

```
Console.WriteLine (Sum(new int[] { 1, 2, 3, 4 } ));
```

Optionale Parameter

Methoden, Konstruktoren und Indexer können *optionale Parameter* deklarieren. Ein Parameter ist optional, wenn er in seiner Deklaration einen *Vorgabewert* definiert:

```
void Foo (int x = 23) { Console.WriteLine (x); }
```

Optionale Parameter können ausgspart werden, wenn die Methode aufgerufen wird:

```
Foo();    // 23
```

Das Vorgabeargument 23 wird tatsächlich an den optionalen Parameter `x` übergeben – der Compiler schreibt den Wert 23 auf der aufrufenden Seite in den kompilierten Code. Der vorangegangene Aufruf von `Foo` entspricht semantisch dem Aufruf

```
Foo (23);
```

weil der Compiler einfach den Vorgabewert einsetzt, wenn ein optionaler Parameter genutzt wird.



Wird einer öffentlichen Methode, die von einer anderen Assembly aufgerufen wird, ein optionaler Parameter hinzugefügt, müssen beide Assemblies neu kompiliert werden – also genau wie bei einem obligatorischen Parameter.

Der Vorgabewert für einen optionalen Parameter muss durch einen konstanten Ausdruck, den parameterlosen Konstruktor eines Werttyps oder einen default-Ausdruck angegeben werden. Optionale Parameter dürfen nicht mit den Modifikatoren *ref* oder *out* markiert werden.

Notwendige Parameter müssen in der Methodendeklaration und im Methodenaufruf *vor* optionalen Parametern stehen (ausgenommen *params*-Argumente, die immer noch stets an letzter Stelle kommen). Im folgenden Beispiel wird *x* der explizit angegebene Wert 1 übergeben und *y* der Vorgabewert 0:

```
Foo(1);    // 1, 0

void Foo (int x = 0, int y = 0)
{
    Console.WriteLine (x + ", " + y);
}
```

Wenn Sie das Gegenteil tun wollen (*x* den Vorgabewert und *y* einen expliziten Wert übergeben), müssen Sie optionale Parameter mit benannten Argumenten kombinieren.

Benannte Argumente

Statt über die Position können Sie Argumente auch über den Namen identifizieren, zum Beispiel so:

```
Foo (x:1, y:2); // 1, 2

void Foo (int x, int y)
{
    Console.WriteLine (x + ", " + y);
}
```

Benannte Argumente können in beliebiger Abfolge auftreten. Die beiden folgenden Aufrufe von *Foo* haben die gleiche Bedeutung:

```
Foo (x:1, y:2);  
Foo (y:2, x:1);
```

Sie können benannte und positionelle Argumente mischen, die benannten Parameter müssen aber an letzter Stelle erscheinen:

```
Foo (1, y:2);
```

Benannte Parameter sind insbesondere in Kombination mit optionalen Parametern nützlich. Betrachten Sie beispielsweise die folgende Methode:

```
void Bar (int a=0, int b=0, int c=0, int d=0) { ... }
```

Diese Methode können wir folgendermaßen nur mit einem Wert für *d* aufrufen:

```
Bar (d:3);
```

Das ist besonders hilfreich, wenn Sie COM-APIs aufrufen.

var – implizit typisierte lokale Variablen

Häufig deklarieren und initialisieren Sie eine Variable in einem Schritt. Wenn der Compiler den Typ aus dem Initialisierungsausdruck ermitteln kann, können Sie das Wort *var* statt der Typdeklaration nutzen, zum Beispiel so:

```
var x = "Hallo";  
var y = new System.Text.StringBuilder();  
var z = (float)Math.PI;
```

Das ist vollkommen äquivalent zu Folgendem:

```
string x = "Hallo";  
System.Text.StringBuilder y =  
    new System.Text.StringBuilder();  
float z = (float)Math.PI;
```

Aufgrund dieser direkten Äquivalenz sind implizit typisierte Variablen statisch typisiert. Der folgende Code würde zum Beispiel zu einem Kompilierungsfehler führen:

```
var x = 5;  
x = "Hallo";    // Kompilierungsfehler; x hat den Typ int.
```

Im Abschnitt »Anonyme Typen« auf Seite 171 beschreiben wir ein Szenario, bei dem die Verwendung von *var* unumgänglich ist.

Mit dem Ziel typisierte new-Ausdrücke

Eine andere Möglichkeit, Wiederholungen im Code zu vermeiden, ist der Einsatz von mit dem Ziel typisierten new-Ausdrücken (seit C# 9):

```
StringBuilder sb1 = new();  
StringBuilder sb2 = new ("Test");
```

Das entspricht exakt diesen Zeilen:

```
StringBuilder sb1 = new StringBuilder();  
StringBuilder sb2 = new StringBuilder ("Test");
```

Das Prinzip ist, dass Sie new aufrufen können, ohne einen Typnamen angeben zu müssen, wenn der Compiler ihn eindeutig ableiten kann. Mit dem Ziel typisierte new-Ausdrücke sind insbesondere dann nützlich, wenn sich die Deklaration und die Initialisierung einer Variablen an unterschiedlichen Stellen Ihres Codes befinden. Ein häufig vorkommendes Beispiel ist das Initialisieren eines Felds in einem Konstruktor:

```
class Foo  
{  
    System.Text.StringBuilder sb;  
  
    public Foo (string initialValue)  
    {  
        sb = new (initialValue);  
    }  
}
```

Mit dem Ziel typisierte new-Ausdrücke sind auch in folgendem Szenario hilfreich:

```
MyMethod (new ("Test"));  
void MyMethod (System.Text.StringBuilder sb) { ... }
```

Ausdrücke und Operatoren

Ein *Ausdruck* entspricht prinzipiell einem Wert. Die einfachste Form eines Ausdrucks ist eine Konstante (wie 123) oder eine Variable (wie x). Ausdrücke können mithilfe von Operatoren umgewandelt

und kombiniert werden. Ein *Operator* erwartet einen oder mehrere *Operanden* als Eingabe, um einen neuen Ausdruck auszugeben.

```
12 * 30    // * ist ein Operator; 12 und 30 sind Operanden.
```

Komplexe Ausdrücke können gebaut werden, da ein Operand selbst ein Ausdruck sein kann, zum Beispiel der Operand (12 * 30) im folgenden Beispiel:

```
1 + (12 * 30)
```

Operatoren sind in C# als *unär*, *binär* oder *ternär* klassifiziert, abhängig von der Zahl der Operanden, mit denen sie arbeiten (einem, zwei oder drei). Die binären Operatoren verwenden immer die *Infix*-Notation, bei der der Operator zwischen den beiden Operanden platziert wird.

Operatoren, die wesentliche Bestandteile des Grundgerüsts der Sprache sind, werden als *primäre Operatoren* bezeichnet; ein Beispiel dafür ist der Methodenaufrufoperator. Ein Ausdruck, der keinen Wert hat, wird als *leerer Ausdruck* bezeichnet:

```
Console.WriteLine (1)
```

Da ein leerer Ausdruck keinen Wert hat, kann er nicht als Operand genutzt werden, um einen komplexeren Ausdruck aufzubauen:

```
1 + Console.WriteLine (1)    // Kompilierungsfehler
```

Zuweisungsausdrücke

Ein *Zuweisungsausdruck* nutzt den Operator =, um einer Variablen das Ergebnis eines anderen Ausdrucks zuzuweisen:

```
x = x * 5
```

Ein Zuweisungsausdruck ist kein *void*-Ausdruck. Er gibt den zugewiesenen Wert aus und kann daher in einem anderen Ausdruck verwendet werden. Im folgenden Beispiel wird x der Wert 2 und y der Wert 10 zugewiesen:

```
y = 5 * (x = 2)
```

Auf diese Weise können mehrere Variablen initialisiert werden:

```
a = b = c = d = 0
```

Die *zusammengesetzten Zuweisungsoperatoren* sind syntaktische Abkürzungen, die eine Zuweisung mit einem anderen Operator kombinieren:

```
x *= 2    // entspricht x = x * 2  
x <<= 1   // entspricht x = x << 1
```

(Eine feine Ausnahme von dieser Regel finden Sie bei *Events*, die wir später beschreiben werden: Die Operatoren `+=` und `-=` werden bei diesen besonders behandelt und auf die Zugriffsmethoden `add` und `remove` abgebildet.)

Priorität und Assoziativität von Operatoren

Wenn ein Ausdruck mehrere Operatoren enthält, bestimmen *Priorität* und *Assoziativität* die Auswertungsreihenfolge. Operatoren mit einer höheren Priorität werden vor Operatoren mit einer niedrigeren Priorität ausgeführt. Wenn die Operatoren die gleiche Priorität haben, bestimmt die Assoziativität die Reihenfolge der Auswertung.

Priorität

Der Ausdruck `1 + 2 * 3` wird als `1 + (2 * 3)` ausgewertet, weil `*` eine höhere Priorität hat als `+`.

Linksassoziative Operatoren

Binäre Operatoren (mit Ausnahme von Zuweisungen, dem Lambda-Operator und dem Null-Verbindungsoperator) sind *linksassoziativ*. Anders gesagt, werden sie von links nach rechts ausgewertet. Beispielsweise wird der Ausdruck `8/4/2` aufgrund der Linksassoziativität als `(8/4)/2` ausgewertet. Natürlich können Sie Ihre eigenen Klammern einsetzen, um die Auswertungsreihenfolge zu ändern.

Rechtsassoziative Operatoren

Die Zuweisungs- und Lambda-Operatoren, der Null-Verbindungsoperator und der (ternäre) Bedingungsoperator sind *rechtsassoziativ*. Anders gesagt, werden sie von rechts nach links ausgewertet. Die Rechtsassoziativität ermöglicht die Kompilation von Mehrfachzuweisungen wie `x=y=3`: Das funktioniert, weil erst `y 3` zugewiesen wird und dann das Ergebnis dieses Ausdrucks `(3) x`.

Operatorentabelle

Die folgende Tabelle führt die Operatoren von C# der Priorität nach auf. Operatoren, die unter ein und derselben Unterüberschrift aufgeführt werden, haben die gleiche Priorität. Von Benutzern überladbare Operatoren erläutern wir im Abschnitt »Überladen von Operatoren« auf Seite 220.

Operatorsymbol	Operatorenname	Beispiel	Überladbar
Primär (höchste Priorität)			
.	Member-Zugriff	x.y	Nein
?.	Null-bedingt	y?.y	Nein
! (Postfix)	Null-tolerant	x!.y	Nein
->	Zeiger auf Struct (unsicher)	x->y	Nein
()	Funktionsaufruf	x()	Nein
[]	Array/Index	a[x]	Via Indexer
++	Post-Inkrement	x++	Ja
--	Post-Dekrement	x--	Ja
new	Instanz erstellen	new Foo()	Nein
stackalloc	Stack-Allozierung	stackalloc(10)	Nein
typeof	Typ eines Bezeichners ermitteln	typeof(int)	Nein
nameof	Namen eines Bezeichners ermitteln	nameof(x)	Nein
checked	Ganzzahlüberlaufprüfung an	checked(x)	Nein
unchecked	Ganzzahlüberlaufprüfung aus	unchecked(x)	Nein
default	Vorgabewert	default(char)	Nein
sizeof	Struct-Größe ermitteln	sizeof(int)	Nein
Unär			
await	Warten auf	await mytask	Nein
+	Positiver Wert von	+x	Ja
-	Negativer Wert von	-x	Ja
!	Nicht	!x	Ja

Operatorsymbol	Operatorenname	Beispiel	Überladbar
~	Bit-Komplement	~x	Ja
++	Prä-Inkrement	++x	Ja
--	Prä-Dekrement	--x	Ja
()	Cast	(int)x	Nein
*	Wert an Adresse (unsicher)	*x	Nein
&	Adresse von Wert (unsicher)	&x	Nein
Bereich			
..	Bereich von Indizes	x..y	Nein
.. [^]		x.. [^] y	
switch und with			
switch	switch-Ausdruck	<pre>num switch { 1 => true, - => false }</pre>	Nein
with	with-Ausdruck	<pre>rec with { X = 123 }</pre>	Nein
Multiplikativ			
*	Multiplikation	x * y	Ja
/	Division	x / y	Ja
%	Rest	x % y	Ja
Additiv			
+	Addition	x + y	Ja
-	Subtraktion	x - y	Ja
Verschiebung			
<<	Verschiebung nach links	x << 1	Ja
>>	Verschiebung nach rechts	x >> 1	Ja
Relational			
<	Kleiner	x < y	Ja
>	Größer	x > y	Ja
<=	Kleiner-gleich	x <= y	Ja
>=	Größer-gleich	x >= y	Ja

Operatorsymbol	Operatorenname	Beispiel	Überladbar
is	Typ entspricht Klasse oder Subklasse von	x is y	Nein
as	Typumwandlung	x as y	Nein
Gleichheit			
==	Gleich	x == y	Ja
!=	Ungleich	x != y	Ja
Logisches Und			
&	UND	x & y	Ja
Logisches Xoder			
^	Exklusives ODER	x ^ y	Ja
Logisches Oder			
	ODER	x y	Ja
Bedingungs-Und			
&&	Bedingungs-UND	x && y	Via &
Null-Verbindung			
??	Null-Verbindung	x??y	Nein
Bedingungs-Oder			
	Bedingungs-ODER	x y	Via
Bedingung (ternär)			
? :	Bedingung	isTrue ? thenThis : elseThis	Nein
Zuweisung und Lambda (geringste Priorität)			
=	Zuweisung	x = y	Nein
*=	Multiplikation und Zuweisung	x *= 2	Via *
/=	Division und Zuweisung	x /= 2	Via /
+=	Addition und Zuweisung	x += 2	Via +
-=	Subtraktion und Zuweisung	x -= 2	Via -
<<=	Linksverschiebung und Zuweisung	x <<= 2	Via <<

Operatorsymbol	Operatorenname	Beispiel	Überladbar
>>=	Rechtsverschiebung und Zuweisung	x >>= 2	Via >>
&=	UND und Zuweisung	x &= 2	Via &
^=	XOR und Zuweisung	x ^= 2	Via ^
=	ODER und Zuweisung	x = 2	Via
=>	Lambda	x => x + 1	Nein

Null-Operatoren

C# bietet drei Operatoren, die die Arbeit mit Null-Werten vereinfachen: den Null-Verbindungsoperator, den Null-Bedingungsoperator und den Null-Verbindungs-Zuweisungsoperator.

Null-Verbindungs-zuweisungsoperator

Der `??=`-Operator (neu in C# 8) ist der *Null-Verbindungs-zuweisungsoperator*. Er sagt: »Wenn der linke Operand null ist, weise den rechten Operanden dem linken Operanden zu.« Schauen Sie sich diese Anweisung an:

```
myVariable ??= someDefault;
```

Das entspricht:

```
if (myVariable == null) myVariable = someDefault;
```

Null-Verbindungs-zuweisungsoperator

Der `??=`-Operator (neu in C# 8) ist der *Null-Verbindungs-zuweisungsoperator*. Er sagt: »Wenn der linke Operand null ist, weise den rechten Operanden dem linken Operanden zu.« Schauen Sie sich diese Anweisung an:

```
myVariable ??= someDefault;
```

Das entspricht:

```
if (myVariable == null) myVariable = someDefault;
```

Null-Verbindungsoperator

Der Operator `??` ist der *Null-Verbindungsoperator*. Er sagt: »Wenn der linke Operand nicht null ist, gib ihn mir – ansonsten gib mir einen anderen Wert.« Hier sehen Sie ein Beispiel:

```
string s1 = null;  
string s2 = s1 ?? "nichts"; // s2 enthält "nichts"
```

Ist der linksseitige Ausdruck nicht null, wird der rechtsseitige Ausdruck nie ausgewertet. Der Null-Verbindungsoperator funktioniert auch mit nullbaren Typen (siehe »Nullbare Werttypen« auf Seite 162).

Null-Bedingungsoperator

Der Operator `?.` ist der *Null-Bedingungsoperator* (oder *Elvis-Operator*). Er ermöglicht Ihnen, wie mit dem normalen Punktoperator Methoden aufzurufen und Member anzusprechen, jedoch wird mit diesem Operator bei einem linksseitigen Operanden der Ausdruck zu null, es wird also keine `NullReferenceException` geworfen:

```
System.Text.StringBuilder sb = null;  
string s = sb?.ToString(); // kein Fehler, s ist null
```

Die zweite Zeile entspricht:

```
string s = (sb == null ? null : sb.ToString());
```

Null-Bedingungsausdrücke funktionieren auch mit Indexern:

```
string foo = null;  
char? c = foo[1]; // c ist null
```

Trifft der Elvis-Operator auf null, verwirft er den Rest des Ausdrucks. Im folgenden Beispiel wird `s` zu null ausgewertet, obwohl zwischen `ToString()` und `ToUpper()` ein Standard-Punktoperator genutzt wird:

```
System.Text.StringBuilder sb = null;  
string s = sb?.ToString().ToUpper(); // kein Fehler
```

Ein wiederholter Einsatz des Elvis-Operators ist nur dann notwendig, wenn der direkt links danebenstehende Operand null sein kann. Der folgende Ausdruck reagiert robust, wenn `x` oder `x.y` null ist:

```
x?.y?.z
```

Er entspricht dem folgenden Code (abgesehen davon, dass `x.y` nur einmal ausgewertet wird):

```
x == null ? null
      : (x.y == null ? null : x.y.z)
```

Der endgültige Ausdruck muss eine Nullreferenz aufnehmen können. Folgender Code ist ungültig, weil `int` keine Null akzeptiert:

```
System.Text.StringBuilder sb = null;
int length = sb?.ToString().Length; // ungültig
```

Wir können das beheben, indem wir einen nullbaren Werttyp einsetzen (siehe »Nullbare Werttypen« auf Seite 162):

```
int? length = sb?.ToString().Length; // okay:
                                     // int? kann null sein
```

Sie können den Null-Bedingungsoperator auch einsetzen, um eine `void`-Methode aufzurufen:

```
someObject?.SomeVoidMethod();
```

Ist `someObject` null, wird dies zu einer »Nicht-Operation«, statt eine `NullReferenceException` zu werfen.

Der Null-Bedingungsoperator kann mit den üblichen Typ-Membren genutzt werden, die wir im Abschnitt »Klassen« auf Seite 76 beschreiben – einschließlich Methoden, Feldern, Eigenschaften und Indexern. Er lässt sich auch gut mit dem Null-Verbindungsoperator kombinieren:

```
System.Text.StringBuilder sb = null;
string s = sb?.ToString() ?? "nichts"; // s wird zu "nichts"
```

Anweisungen

Funktionen enthalten *Anweisungen*, die in der Reihenfolge abgearbeitet werden, in der sie im Quelltext erscheinen. Ein *Anweisungsblock* ist eine Abfolge von Anweisungen, die zwischen geschweiften Klammern stehen (den Token `{}`).

Deklarationsanweisungen

Eine Variablendeklaration führt eine neue Variable ein und initialisiert sie optional mit einem Ausdruck. Eine Deklarationsanweisung

endet mit einem Semikolon. Sie können mehrere Variablen desselben Typs in einer kommaseparierten Liste angeben:

```
bool rich = true, famous = false;
```

Eine *Deklaration einer Konstanten* ähnelt der Variablendeklaration, nur dass sie nach der Deklaration nicht mehr geändert werden kann und die Initialisierung zusammen mit der Deklaration geschehen muss (siehe »Konstanten« auf Seite 77):

```
const double c = 2.99792458E08;
```

Variablen mit lokaler Geltung

Die Geltung lokaler Variablen und lokaler Konstanten erstreckt sich über den aktuellen Block. Sie können keine weitere lokale Variable mit dem gleichen Namen im aktuellen Block oder eingebetteten Blöcken deklarieren.

Ausdrucksanweisungen

Ausdrucksanweisungen sind Ausdrücke, die auch gültige Anweisungen sind. In der Praxis sind das Ausdrücke, die etwas »tun«; anders gesagt, sind es Ausdrücke, die

- eine Variable zuweisen oder verändern,
- ein Objekt instanziiieren oder
- eine Methode aufrufen.

Ausdrücke, die nichts davon tun, sind keine gültigen Anweisungen:

```
string s = "foo";  
s.Length;           // Illegale Anweisung: Tut nichts!
```

Wenn Sie einen Konstruktor oder eine Methode aufrufen, der/die einen Wert zurückgibt, sind Sie nicht gezwungen, das Ergebnis zu nutzen. Wenn allerdings der Konstruktor oder die Methode keinen Zustand ändert, ist die Anweisung komplett nutzlos:

```
new StringBuilder(); // Legal, aber nutzlos.  
x.Equals(y);          // Legal, aber nutzlos.
```

Auswahlanweisungen

Auswahlanweisungen steuern den Fluss der Programmausführung mithilfe von Bedingungen.

Die if-Anweisung

Eine if-Anweisung führt eine Anweisung aus, wenn ein bool-Ausdruck wahr ist, zum Beispiel so:

```
if (5 < 2 * 3)
    Console.WriteLine ("true");      // true
```

Die Anweisung kann ein Anweisungsblock sein:

```
if (5 < 2 * 3)
{
    Console.WriteLine ("true");      // true
    Console.WriteLine ("...")
}
```

Die else-Klausel

Eine if-Anweisung kann optional eine else-Klausel haben:

```
if (2 + 2 == 5)
    Console.WriteLine ("Wird nicht ausgeführt");
else
    Console.WriteLine ("False");      // False
```

In eine else-Klausel können Sie weitere if-Anweisungen einbetten:

```
if (2 + 2 == 5)
    Console.WriteLine ("Wird nicht ausgeführt");
else
    if (2 + 2 == 4)
        Console.WriteLine ("Wird ausgeführt"); // Wird ausgeführt.
```

Änderung des Programmflusses durch geschweifte Klammern

Eine else-Klausel gehört immer zur direkt davor befindlichen if-Anweisung im Anweisungsblock, zum Beispiel hier:

```
if (true)
    if (false)
        Console.WriteLine();
else
    Console.WriteLine ("Wird ausgeführt");
```

Das ist semantisch identisch mit dem hier:

```
if (true)
{
    if (false)
        Console.WriteLine();
    else
        Console.WriteLine ("Wird ausgeführt");
}
```

Wir können den Programmablauf ändern, indem wir die Klammern verschieben:

```
if (true)
{
    if (false)
        Console.WriteLine();
}
else
    Console.WriteLine ("Wird nicht ausgeführt");
```

C# hat kein »elseif«-Schlüsselwort, aber mit dem folgenden Muster erzielt man das gleiche Ergebnis :

```
if (age >= 40)
    Console.WriteLine ("Sie können Bundespräsident werden!");
else if (age >= 18)
    Console.WriteLine ("Sie dürfen wählen!");
else if (age >= 16)
    Console.WriteLine ("Sie dürfen Bier trinken!");
else
    Console.WriteLine ("Sie müssen warten!");
```

Die switch-Anweisung

switch-Anweisungen ermöglichen Ihnen, den Programmablauf basierend auf einer Auswahl möglicher Werte einer Variablen zu verzweigen. switch-Anweisungen können zu saubererem Code als mit mehreren aufeinanderfolgenden if-Anweisungen führen, da ein Ausdruck bei switch-Anweisungen nur einmal ausgewertet werden muss:

```
static void ShowCard (int cardNumber)
{
    switch (cardNumber)
    {
        case 13:
            Console.WriteLine ("König");
```

```

        break;
    case 12:
        Console.WriteLine ("Königin");
        break;
    case 11:
        Console.WriteLine ("Bauer");
        break;
    default:    // ein anderer Kartenwert
        Console.WriteLine (cardNumber);
        break;
    }
}

```

Die Werte in jedem case-Ausdruck müssen Konstanten sein, was die erlaubten Typen auf die eingebauten Integraltypen, auf bool, char, enum-Typen und string beschränkt. Am Ende jeder case-Klausel müssen Sie explizit mit einer Art Sprunganweisung angeben, wo die Ausführung fortgesetzt werden soll. Hier sind die Optionen:

- break (springt an das Ende der switch-Anweisung)
- goto case x (springt zu einer anderen case-Klausel)
- goto default (springt zur default-Klausel)
- jede beliebige andere Sprunganweisung – also return, throw, continue oder goto *label*

Wenn für mehr als einen Wert der gleiche Code ausgeführt werden soll, können Sie die gemeinsamen case-Klauseln hintereinander auflisten:

```

switch (cardNumber)
{
    case 13:
    case 12:
    case 11:
        Console.WriteLine ("Bildkarte");
        break;
    default:
        Console.WriteLine ("Gewöhnliche Karte");
        break;
}

```

Dieses Feature einer switch-Anweisung kann in Bezug auf das Schreiben saubereren Codes im Vergleich zu mehreren if-else-Anweisungen entscheidend sein.

Die switch-Anweisung abhängig vom Typ

Seit C# 7 können Sie bei switch abhängig vom *Typ* vorgehen:

```
static void TellMeTheType (object x)
{
    switch (x)
    {
        case int i:
            Console.WriteLine ("Es ist ein int!");
            break;
        case string s:
            Console.WriteLine (s.Length);           // Wir können s nutzen
            break;
        case bool b when b == true:                // Wenn b true ist
            Console.WriteLine ("True");
            break;
        case null:    // In C# 7 können Sie auch auf null prüfen.
            Console.WriteLine ("null");
            break;
    }
}
```

(Der object-Typ erlaubt eine Variable beliebigen Typs – siehe »Vererbung« auf Seite 95 und »Der Typ object« auf Seite 104.)

Jeder case-Fall gibt einen Typ an, der passen muss, und eine Variable, der der typisierte Wert zugewiesen werden soll, wenn die Prüfung erfolgreich ist. Anders als bei Konstanten gibt es keine Beschränkung der möglichen Typen. Die optionale when-Klausel legt eine Bedingung fest, die erfüllt sein muss.

Die Reihenfolge der case-Klauseln hat bei der Auswahl nach dem Typ eine Bedeutung – anders als bei der Auswahl nach Konstanten. Eine Ausnahme zu dieser Regel bildet die default-Klausel, die unabhängig von ihrer Position in der switch-Anweisung als Letztes ausgeführt wird.

Sie können mehrere case-Klauseln aufeinanderfolgen lassen. Im nächsten Code wird Console.WriteLine für einen beliebigen Gleitkommatyp ausgeführt, dessen Wert größer als 1000 ist:

```
switch (x)
{
    case float f when f > 1000:
    case double d when d > 1000:
    case decimal m when m > 1000:
        Console.WriteLine ("f, d und m sind außerhalb des Scopes");
        break;
}
```

In diesem Beispiel können wir die Variablen `f`, `d` und `m` nur innerhalb der `when`-Klauseln einsetzen. Rufen wir `Console.WriteLine` auf, weiß dieses nicht, welche der drei Variablen zugewiesen sein werden, daher sorgt der Compiler dafür, dass keine im Scope ist.

switch-Ausdrücke

Seit C# 8 können Sie `switch` auch im Rahmen eines *Ausdrucks* verwenden. Mit `cardNumber` vom Typ `int` zeigt der folgende Code seinen Einsatz:

```
string cardName = cardNumber switch
{
    13 => "King",
    12 => "Queen",
    11 => "Jack",
    _ => "Pip card"    // entspricht 'default'
};
```

Beachten Sie, dass das Schlüsselwort `switch` hinter dem Variablennamen steht und dass es sich bei der `case`-Klausel um Ausdrücke handelt (die durch Kommata beendet werden) und nicht um Anweisungen. Sie können auch abhängig von mehreren Werten (Tupeln) vorgehen:

```
int cardNumber = 12; string suite = "spades";
string cardName = (cardNumber, suite) switch
{
    (13, "spades") => "King of spades",
    (13, "clubs") => "King of clubs",
    ...
};
```

Iterationsanweisungen

C# ermöglicht es, eine Abfolge von Anweisungen mit den Anweisungen `while`, `do-while`, `for` und `foreach` wiederholt auszuführen.

while- und do-while-Schleifen

while-Schleifen wiederholen Code, solange ein bool-Ausdruck wahr ist. Der Ausdruck wird geprüft, *bevor* die Schleife ausgeführt wird. Beispielsweise gibt Folgendes 012 aus:

```
int i = 0;
while (i < 3)
{
    Console.Write (i++);
}
```

// Die geschweiften Klammern
// sind hier optional.

do-while-Schleifen unterscheiden sich von while-Schleifen dadurch, dass sie den Ausdruck prüfen, *nachdem* der Anweisungsblock durchlaufen wurde (was sicherstellt, dass der Block immer zumindest einmal ausgeführt wird). So sieht das obige Beispiel mit einer do-while-Schleife aus:

```
int i = 0;
do
{
    Console.WriteLine (i++);
}
while (i < 3);
```

for-Schleifen

for-Schleifen sind wie while-Schleifen mit besonderen Klauseln für die *Initialisierung* und das *Iterieren* einer Schleifenvariablen. Eine for-Schleife besteht aus drei Klauseln:

```
for (Initialisierung; Bedingung; Iteration)  
    Anweisung-oder-Anweisungsblock
```

Die *Initialisierungsklausel* wird vor dem Eintritt in die Schleife ausgeführt und initialisiert eine oder mehrere *Iterationsvariablen*.

Die *Bedingungsklausel* ist ein bool-Ausdruck, der *vor* jedem Schleifendurchlauf ausgeführt wird. Der Schleifeninhalt wird ausgeführt, wenn die Bedingung wahr ist.

Die *Iterationsklausel* wird *nach* jeder Ausführung des Schleifeninhalts ausgeführt. Sie wird üblicherweise genutzt, um die Iterationsvariable zu aktualisieren.

Folgendes gibt beispielsweise die Zahlen 0 bis 2 aus:

```
for (int i = 0; i < 3; i++)  
    Console.WriteLine (i);
```

Die folgende Schleife gibt die ersten zehn Fibonacci-Zahlen aus (bei denen jede Zahl die Summe der beiden vorangegangenen Zahlen ist):

```
for (int i = 0, prevFib = 1, curFib = 1; i < 10; i++)  
{  
    Console.WriteLine (prevFib);  
    int newFib = prevFib + curFib;  
    prevFib = curFib; curFib = newFib;  
}
```

Von den drei Teilen der for-Anweisung können beliebige weggelassen werden. Sie können eine unendliche Schleife wie folgt erzeugen (auch wenn man stattdessen `while(true)` nutzen kann):

```
for (;;) Console.WriteLine ("Unterbrich mich");
```

foreach-Schleifen

Die `foreach`-Schleife iteriert über jedes Element eines aufzählbaren Objekts. Die meisten .NET-Typen, die eine Menge oder Liste von Elementen repräsentieren, sind aufzählbar (*enumerable*). So sind zum Beispiel sowohl ein Array als auch ein String aufzählbar. Hier ein Beispiel für das Enumerieren der Zeichen eines Strings – vom ersten bis zum letzten Zeichen:

```
foreach (char c in "Bier")  
    Console.WriteLine (c + " "); // B i e r
```

Enumerierbare Objekte definieren wir im Abschnitt »Enumeration und Iteratoren« auf Seite 157.

Sprunganweisungen

Die C#-Sprunganweisungen sind `break`, `continue`, `goto`, `return` und `throw`. Das Schlüsselwort `throw` behandeln wir im Abschnitt »try-Anweisungen und Exceptions« auf Seite 149.

Die break-Anweisung

Die `break`-Anweisung beendet die Ausführung des Bodys einer Iterations- oder `switch`-Anweisung:

```
int x = 0;
while (true)
{
    if (x++ > 5) break;          // Aus der Schleife ausbrechen.
}
// Hier wird die Ausführung nach dem break fortgesetzt
...
```

Die continue-Anweisung

Die continue-Anweisung übergeht die restlichen Anweisungen einer Schleife und geht zur nächsten Iteration weiter. Die folgende Schleife überspringt alle geraden Zahlen:

```
for (int i = 0; i < 10; i++)
{
    if ((i % 2) == 0) continue;
    Console.Write (i + " ");      // 1 3 5 7 9
}
```

Die goto-Anweisung

Die goto-Anweisung übergibt die Ausführung an eine Marke (die mit einem vorangestellten Doppelpunkt angezeigt wird) in einem Anweisungsblock. Der folgende Code iteriert über die Zahlen 1 bis 5 und imitiert eine for-Schleife:

```
int i = 1;
startLoop:
if (i <= 5)
{
    Console.Write (i + " ");      // 1 2 3 4 5
    i++;
    goto startLoop;
}
```

Die return-Anweisung

Die return-Anweisung beendet die Methode und muss einen Ausdruck liefern, dessen Typ dem Rückgabetyt der Methode entspricht, wenn die Methode nicht void ist:

```
static decimal AsPercentage (decimal d)
{
    decimal p = d * 100m;
```

```

        return p;        // Folgenden Wert an die aufrufende
                          // Methode liefern.
    }

```

Eine `return`-Anweisung kann überall in einer Methode vorkommen (außer in einem `finally`-Block) und mehr als einmal eingesetzt werden.

Namensräume

Ein *Namensraum* ist eine Domäne, in der Namen von Typen eindeutig sein müssen. Typen werden normalerweise innerhalb von hierarchischen Namensräumen organisiert – um Namenskonflikte zu vermeiden und die Typen leichter zu finden. So ist zum Beispiel der `RSA`-Typ, der die Verschlüsselung mit öffentlichen Schlüsseln behandelt, in folgendem Namensraum definiert:

```
System.Security.Cryptography
```

Ein Namensraum gehört direkt zum Namen eines Typs. Dieser Code ruft die Methode `Create` von `RSA` auf:

```

System.Security.Cryptography.RSA rsa =
    System.Security.Cryptography.RSA.Create();

```



Namensräume sind unabhängig von Assemblies, bei denen es sich um Auslieferungseinheiten wie `.exe` oder `.dll` handelt. Namensräume haben auch keinen Einfluss auf die Sichtbarkeit von Mitgliedern – `public`, `internal`, `private` und so weiter.

Das Schlüsselwort `namespace` definiert einen Namensraum für die Typen in diesem Block, zum Beispiel so:

```

namespace Outer.Middle.Inner
{
    class Class1 {}
    class Class2 {}
}

```

Die Punkte im Namensraum stehen für eine Hierarchie verschachtelter Namensräume. Der folgende Code ist mit dem vorigen Beispiel semantisch identisch:

```

namespace Outer
{
    namespace Middle
    {
        namespace Inner
        {
            class Class1 {}
            class Class2 {}
        }
    }
}

```

Sie können einen Typ über seinen *vollständig qualifizierten Namen* ansprechen, zu dem alle Namensräume gehören – vom äußersten bis zum innersten. So könnten wir zum Beispiel auf `Class1` aus dem vorigen Beispiel über `Outer.Middle.Inner.Class1` zugreifen.

Typen, die in keinem Namensraum definiert sind, liegen im *globalen Namensraum*. Zum globalen Namensraum gehören auch die obersten Namensräume wie `Outer` in unserem Beispiel.

Dateibezogene Namensräume (C# 10)

Oft wollen Sie, dass alle Typen in einer Datei in einem Namensraum definiert sind:

```

namespace MyNamespace
{
    class Class1 {}
    class Class2 {}
}

```

Mit C# 10 können Sie das mit einem *dateibezogenen Namensraum* erreichen:

```

namespace MyNamespace; // Gilt für alles Folgende

class Class1 {}          // innerhalb von MyNamespace
class Class2 {}          // innerhalb von MyNamespace

```

Dateibezogene Namensräume verringern überflüssigen Code und eliminieren eine unnötige Einrückungsebene.

Die using-Direktive

Die using-Direktive importiert einen Namensraum und ist eine praktische Möglichkeit, auf Typen zuzugreifen, ohne ihre vollständig qualifizierten Namen angeben zu müssen:

```
using Outer.Middle.Inner;

class Test    // Test ist im globalen Namensraum
{
    static void Main()
    {
        Class1 c;    // vollständig qualifizierter Name
                     // nicht erforderlich ...
    }
}
```

Eine using-Direktive kann innerhalb eines Namensraums selbst angegeben werden, um den Geltungsbereich der Direktive einzuschränken.

Die global using-Direktive (C# 10)

Mit C# 10 können Sie einer using-Direktive das Schlüsselwort `global` voranstellen, damit die Direktive für alle Dateien im Projekt oder in der Kompilierungseinheit wirksam wird:

```
global using System;
global using System.Collection.Generic;
```

Damit können Sie die gebräuchlichsten Importe zentralisieren und vermeiden das Wiederholen der gleichen Direktiven in jeder Datei.

`global using`-Direktiven müssen vor den nicht globalen Anweisungen stehen, und sie können nicht innerhalb von Namensraumdeklarationen auftauchen. Die `global`-Direktive kann zusammen mit `using static` verwendet werden.

using static

Die using-Direktive importiert einen *Typ* anstelle eines Namensraums. Alle statischen Member dieses Typs können dann genutzt werden, ohne sie über den Typnamen qualifizieren zu müssen. Im folgen-

den Beispiel rufen wir die statische Methode `Writeln` der Klasse `Console` auf:

```
using static System.Console;
```

```
Writeln ("Hallo");
```

Die `using static`-Direktive importiert alle erreichbaren statischen Member des Typs – einschließlich Feldern, Eigenschaften und eingebetteten Typen. Sie können diese Direktive auch auf Enum-Typen anwenden (siehe »Enums« auf Seite 118), bei denen dann deren Member importiert werden. Sollte es bei mehreren statischen Importen Mehrdeutigkeiten geben, kann der C#-Compiler den richtigen Typ nicht aus dem Kontext ermitteln und meldet einen Fehler.

Regeln in einem Namensraum

Geltung von Namen

Namen, die in einem äußeren Namensraum deklariert sind, können unqualifiziert in inneren Namensräumen verwendet werden. In diesem Beispiel muss `Class1` in `Inner` nicht qualifiziert werden:

```
namespace Outer
{
    class Class1 {}

    namespace Inner
    {
        class Class2 : Class1 {}
    }
}
```

Wenn Sie auf einen Typ in einem anderen Zweig Ihrer Namensraumhierarchie zugreifen wollen, können Sie einen teilweise qualifizierten Namen nutzen. Im folgenden Beispiel lassen wir `SalesReport` auf `Common.ReportBase` basieren:

```
namespace MyTradingCompany
{
    namespace Common
    {
        class ReportBase {}
    }
    namespace ManagementReporting
```

```
{
    class SalesReport : Common.ReportBase {}
}
```

Namen verdecken

Wenn der gleiche Typname sowohl in einem inneren als auch in einem äußeren Namensraum auftritt, gewinnt der innere Name. Um auf den äußeren Namen zugreifen zu können, müssen Sie seinen vollständig qualifizierten Namen verwenden.



Alle Typnamen werden zur Kompilierungszeit in vollständig qualifizierte Namen konvertiert. Der Intermediate-Language-(IL-)Code enthält keine unqualifizierten oder teilweise qualifizierten Namen.

Wiederholte Namensräume

Sie können eine Namensraumdeklaration wiederholen, solange die Typnamen im Namensraum nicht für Konflikte sorgen:

```
namespace Outer.Middle.Inner { class Class1 {} }
namespace Outer.Middle.Inner { class Class2 {} }
```

Die Klassen können sogar Quellcodedateien und Assemblies übergreifen.

Der Qualifizierer global::

Gelegentlich kann ein vollständig qualifizierter Typname mit einem inneren Namen in Konflikt geraten. Sie können C# dazu zwingen, den vollständig qualifizierten Typnamen zu nutzen, indem Sie ihm das Präfix `global::` voranstellen:

```
global::System.Text.StringBuilder sb;
```

Aliase für Typen und Namensräume

Das Importieren eines Namensraums kann zu Kollisionen bei Typnamen führen. Anstatt den gesamten Namensraum zu importieren, können Sie nur die für Sie notwendigen Typen importieren und jedem davon einen Aliasnamen geben:

```
using PropertyInfo2 = System.Reflection.PropertyInfo;
class Program { PropertyInfo2 p; }
```

Es kann auch ein gesamter Namensraum mit einem Alias versehen werden:

```
using R = System.Reflection;
class Program { R.PropertyInfo p; }
```

Klassen

Die *Klasse* ist die am häufigsten verwendete Form eines Referenztyps. Die einfachste mögliche Klassendeklaration sieht so aus:

```
class Foo
{
}
```

Eine komplexere Klassendeklaration kann folgende Elemente enthalten:

Vor dem Schlüsselwort <code>class</code>	<i>Attribute und Klassenmodifikatoren.</i> Die nicht geschachtelten Klassenmodifikatoren sind <code>public</code> , <code>internal</code> , <code>abstract</code> , <code>sealed</code> , <code>static</code> , <code>unsafe</code> und <code>partial</code> .
Hinter <i>Foo</i>	<i>Generische Typparameter und Constraints, eine Basisklasse und Schnittstellen.</i>
Innerhalb der geschweiften Klammern	<i>Klassen-Member (das sind Methoden, Eigenschaften, Indexer, Events, Felder, Konstruktoren, überladene Operatoren, eingebettete Typen und ein Finalizer).</i>

Felder

Ein *Feld* ist eine Variable, die ein Member einer Klasse oder eines Struct ist:

```
class Octopus
{
    string name;
    public int age = 10;
}
```

Ein Feld kann den Modifikator `readonly` haben, um zu verhindern, dass es nach dem Erstellen verändert wird. Einem nur lesbaren Feld

kann lediglich während seiner Deklaration oder im Konstruktor des Typs, der es enthält, ein Wert zugewiesen werden.

Die Initialisierung eines Felds ist optional. Ein nicht initialisiertes Feld hat einen Standardwert (0, '\0', null, false). Feldinitialisierer werden vor den Konstruktoren in der Reihenfolge ausgeführt, in der sie erscheinen.

Aus Gründen der Bequemlichkeit können Sie mehrere Felder desselben Typs in einer kommaseparierten Liste deklarieren. Das ist praktisch für Felder, die die gleichen Attribute und Feldmodifikatoren haben:

```
static readonly int legs = 8, eyes = 2;
```

Konstanten

Eine *Konstante* ist ein Feld, dessen Wert sich nicht ändern kann. Eine Konstante wird statisch beim Kompilieren ausgewertet, und ihr Wert wird bei jeder Verwendung vom Compiler direkt eingesetzt (so ähnlich wie ein Makro in C++). Eine Konstante kann einen der eingebauten numerischen Typen, bool, char und string oder einen enum-Typ nutzen.

Eine Konstante wird mit dem Schlüsselwort `const` deklariert und muss mit einem Wert initialisiert werden:

```
public class Test
{
    public const string Message = "Hello World";
}
```

Eine Konstante ist deutlich restriktiver als ein `static readonly`-Feld, und zwar sowohl bezüglich der nutzbaren Typen als auch bei der Semantik der Feldinitialisierung. Eine Konstante unterscheidet sich von einem `static readonly`-Feld zudem darin, dass die Auswertung der Konstanten schon beim Kompilieren stattfindet. Konstanten können auch lokal in einer Methode deklariert werden:

```
static void Main()
{
    const double twoPI = 2 * System.Math.PI;
    ...
}
```

Methoden

Eine *Methode* führt eine Aktivität als Abfolge von Anweisungen aus. Sie kann *Eingabedaten* vom aufrufenden Code bekommen, indem *Parameter* spezifiziert werden, und *Ausgabedaten* an den Aufrufenden zurückgeben, indem ein *Rückgabotyp* angegeben wird. Eine Methode kann als Rückgabotyp `void` spezifizieren und damit zeigen, dass sie keinen Wert zurückgeben wird. Sie kann zudem über *ref*- und *out*-Parameter Daten an den Aufrufenden übermitteln.

Die *Signatur* einer Methode muss innerhalb des Typs eindeutig sein. Sie besteht aus dem Namen der Methode und der Reihenfolge der Parametertypen (aber nicht den Parameternamen oder dem Rückgabotyp).

Expression-bodied Methoden

Eine Methode, die aus einem einzelnen Ausdruck besteht, zum Beispiel:

```
int Foo (int x) { return x * 2; }
```

kann kompakter als *Expression-bodied Methode* geschrieben werden. Ein »dicker Pfeil« ersetzt die geschweiften Klammern und das Schlüsselwort `return`:

```
int Foo (int x) => x * 2;
```

Expression-bodied Methoden können auch einen `void`-Rückgabotyp besitzen:

```
void Foo (int x) => Console.WriteLine (x);
```

Lokale Methoden

Sie können eine Methode innerhalb einer anderen Methode definieren:

```
void WriteCubes()  
{  
    Console.WriteLine (Cube (3));  
  
    int Cube (int value) => value * value * value;  
}
```

Die lokale Methode (in diesem Fall `Cube`) ist nur für die umschließende Methode (`WriteCubes`) sichtbar. Das vereinfacht den enthaltenen Typ und macht jedem, der sich den Code anschaut, direkt deutlich, dass `Cube` nirgendwo anders verwendet wird. Lokale Methoden können auf die lokalen Variablen und Parameter der umschließenden Methode zugreifen. Das hat eine Reihe von Konsequenzen, die wir im Abschnitt »Äußere Variablen übernehmen« auf Seite 144 beschreiben.

Lokale Methoden können innerhalb anderer Funktionsarten eingesetzt werden, wie Eigenschafts-Accessoren, Konstruktoren usw., sogar innerhalb anderer lokaler Methoden. Lokale Methoden können Iteratoren oder asynchron sein.

Methoden, die in Anweisungen auf oberster Ebene deklariert wurden, sind implizit lokal – wir können das wie folgt demonstrieren:

```
int x = 3; Foo();  
void Foo() => Console.WriteLine (x); // Wir können auf x  
                                     // zugreifen
```

Statische lokale Methoden

Durch Hinzufügen des Modifikators `static` zu einer lokalen Methode wird (seit C# 8) verhindert, dass sie die lokalen Variablen und Parameter der umschließenden Methode sieht. Das hilft dabei, eine Kopplung geringer zu gestalten, und verhindert, dass sich die lokale Methode unabsichtlich auf Variablen in der umschließenden Methode bezieht.

Methoden überladen



Lokale Methoden können nicht überladen werden. Daher können auch Methoden, die in Anweisungen auf oberster Ebene deklariert wurden (die als lokale Methoden behandelt werden), nicht überladen werden.

Ein Typ kann Methoden überladen (also mehrere Methoden mit demselben Namen haben), solange sich die Parametertypen unterscheiden. So können zum Beispiel die folgenden Methoden alle gemeinsam im selben Typ existieren:

```
void Foo (int x);
void Foo (double x);
void Foo (int x, float y);
void Foo (float x, int y);
```

Instanzkonstruktoren

Konstruktoren führen Initialisierungscode für eine Klasse oder ein Struct aus. Ein Konstruktor wird wie eine Methode definiert, nur dass der Methodenname und der Rückgabewert auf den Namen des Typs reduziert werden, in dem sich der Konstruktor befindet:

```
Panda p = new Panda ("Petey"); // Konstruktor aufrufen
public class Panda
{
    string name; // Feld definieren
    public Panda (string n) // Konstruktor definieren
    {
        name = n; // Initialisierungscode
    }
}
...
```

Sie können aus einer Anweisung bestehende Konstruktoren als Expression-bodied Methode schreiben:

```
public Panda (string n) => name = n;
```

Klassen und Structs können Konstruktoren überladen. Eine Überladung ruft über das Schlüsselwort **this** eine andere auf:

```
public class Wine
{
    public Wine (decimal price) {...}

    public Wine (decimal price, int year)
        : this (price) {...}
}
```

Wenn ein Konstruktor einen anderen aufruft, wird der aufgerufene Konstruktor zuerst ausgeführt.

Sie können einem anderen Konstruktor einen *Ausdruck* wie folgt übermitteln:

```
public Wine (decimal price, DateTime year)
    : this (price, year.Year) {...}
```

Der Ausdruck selbst kann nicht auf die Referenz `this` zurückgreifen, um zum Beispiel eine Instanzmethode aufzurufen. Allerdings können dabei statische Methoden genutzt werden.

Implizite parameterlose Konstruktoren

Für Klassen generiert der C#-Compiler automatisch einen parameterlosen Konstruktor, wenn (und nur wenn) Sie keinerlei eigene Konstruktoren definiert haben. Sobald Sie aber mindestens einen Konstruktor definiert haben, wird der parameterlose Konstruktor nicht mehr automatisch generiert.

Nicht öffentliche Konstruktoren

Konstruktoren müssen nicht öffentlich sein. Ein guter Grund für einen nicht öffentlichen Konstruktor ist, die Instanzerzeugung über einen statischen Methodenaufruf zu kontrollieren. Mit diesem Aufruf kann dann zum Beispiel ein Objekt aus einem Pool zurückgegeben werden, anstatt ein neues Objekt zu erzeugen, oder es wird abhängig von den Eingabewerten eine Instanz einer Subklasse zurückgegeben.

Dekonstruktoren

Während ein Konstruktor im Allgemeinen (als Parameter) einen Satz Werte mitbekommt und diese Feldern zuweist, geht ein Dekonstruktor (ab C# 7) umgekehrt vor – die Feldwerte werden an Variablen übertragen. Eine Dekonstruktormethode muss den Namen `Deconstruct` tragen und einen oder mehrere `out`-Parameter besitzen:

```
class Rectangle
{
    public readonly float Width, Height;

    public Rectangle (float width, float height)
    {
        Width = width; Height = height;
    }

    public void Deconstruct (out float width,
                            out float height)
    { width = Width; height = Height; }
}
```

Um den Dekonstruktor aufzurufen, verwenden wir die folgende spezielle Syntax:

```
var rect = new Rectangle (3, 4);  
(float width, float height) = rect;  
Console.WriteLine (width + " " + height) // 3 4
```

Die zweite Zeile enthält den Dekonstruktoraufruf. Sie erstellt zwei lokale Variablen und ruft dann die Deconstruct-Methode auf. Dies ist äquivalent zu:

```
rect.Deconstruct (out var width, out var height);
```

Dekonstruktoraufrufe erlauben eine implizite Typisierung, daher können wir unseren Aufruf wie folgt kürzer fassen:

```
(var width, var height) = rect;
```

oder noch einfacher:

```
var (width, height) = rect;
```

Sie können dem Aufrufer mehrere Dekonstruktionsoptionen bieten, indem Sie die Methode Deconstruct überladen.



Bei der Deconstruct-Methode kann es sich um eine Erweiterungsmethode handeln (siehe »Erweiterungsmethoden« auf Seite 169). Das ist ein nützlicher Trick, wenn Sie Typen dekonstruieren wollen, von denen Sie nicht der Autor sind.

Seit C# 10 können Sie bestehende und neue Variablen beim Dekonstruieren beliebig kombinieren:

```
double x1 = 0;  
(x1, double y2) = rect;
```

Objektinitialisierer

Um die Objektinitialisierung zu vereinfachen, können die von außen erreichbaren Felder bzw. Eigenschaften eines Objekts über einen *Objektinitialisierer* direkt nach dem Erzeugen initialisiert werden. Schauen Sie sich zum Beispiel diese Klasse an:

```
public class Bunny
{
    public string Name;
    public bool LikesCarrots, LikesHumans;

    public Bunny () {}
    public Bunny (string n) { Name = n; }
}
```

Mithilfe von Objektinitialisierern können Sie Bunny-Objekte wie folgt initialisieren:

```
Bunny b1 = new Bunny {
    Name="Bo",
    LikesCarrots = true,
    LikesHumans = false
};

Bunny b2 = new Bunny ("Bo") {
    LikesCarrots = true,
    LikesHumans = false
};
```

Die Referenz this

Die Referenz `this` verweist auf die Instanz selbst. Im nächsten Beispiel nutzt die Methode `Marry` dieses `this`, um das Feld `Mate` von `partner` zu setzen:

```
public class Panda
{
    public Panda Mate;

    public void Marry (Panda partner)
    {
        Mate = partner;
        partner.Mate = this;
    }
}
```

Die Referenz `this` sorgt zudem für das Auflösen möglicher Mehrdeutigkeiten zwischen lokalen Variablen bzw. Parametern und einem Feld:

```
public class Test
{
    string name;
    public Test (string name) { this.name = name; }
}
```

Die Referenz `this` ist nur bei nicht statischen Mitgliedern einer Klasse oder eines Struct gültig.

Eigenschaften

Eigenschaften sehen von außen wie Felder aus, enthalten intern aber wie Methoden eine Logik. Beispielsweise können Sie mit einem Blick auf den folgenden Code nicht erkennen, ob `CurrentPrice` ein Feld oder eine Eigenschaft ist:

```
Stock msft = new Stock();
msft.CurrentPrice = 30;
msft.CurrentPrice -= 3;
Console.WriteLine (msft.CurrentPrice);
```

Eine Eigenschaft wird wie ein Feld deklariert, besitzt aber zusätzlich einen `get/set`-Block. So wird `CurrentPrice` als Eigenschaft implementiert:

```
public class Stock
{
    decimal currentPrice; // das private "Hintergrundfeld"

    public decimal CurrentPrice // die öffentliche Eigenschaft
    {
        get { return currentPrice; }
        set { currentPrice = value; }
    }
}
```

`get` und `set` sind *Eigenschaften-Accessors*. Der `get`-Accessor wird ausgeführt, wenn die Eigenschaft gelesen wird. Sie müssen einen Wert mit dem Typ der Eigenschaft zurückgeben. Der `set`-Accessor wird ausgeführt, wenn der Eigenschaft ein Wert zugewiesen wird. Er hat einen impliziten Parameter namens `value` mit dem Typ der Eigenschaft, den Sie meist einem privaten Feld zuweisen (in diesem Fall `currentPrice`).

Auch wenn Eigenschaften auf die gleiche Art und Weise angesprochen werden wie Felder, unterscheiden sie sich von ihnen darin, dass der Implementierer bei ihnen die vollständige Kontrolle über das Lesen und Schreiben der Werte erhält. Diese Kontrolle ermöglicht ihm, eine für ihn bevorzugte interne Repräsentation zu wählen, ohne die internen Details dem Anwender der Eigenschaft preisgeben zu müssen. In diesem Beispiel könnte die `set`-Methode eine Exception werfen, wenn sich `value` außerhalb eines gewissen Wertebereichs befände.



In diesem Buch nutzen wir öffentliche Felder, um die Beispiele einfach zu halten. In einer echten Anwendung werden Sie meist öffentliche Eigenschaften bevorzugen, um die Kapselung zu fördern.

Auf eine Eigenschaft lässt sich nur lesend zugreifen, wenn lediglich ein `get`-Accessor angegeben ist, während sie nur beschreibbar ist, wenn es lediglich einen `set`-Accessor gibt. Allerdings werden Eigenschaften, die nur beschreibbar sind, selten verwendet.

Eine Eigenschaft hat meist ein eindeutig zugehöriges privates Feld »im Hintergrund«, in dem die zugrunde liegenden Daten abgelegt werden. Das muss allerdings nicht der Fall sein – eine Eigenschaft kann auch aus anderen Daten berechnet werden.

```
decimal currentPrice, sharesOwned;

public decimal Worth
{
    get { return currentPrice * sharesOwned; }
}
```

Expression-bodied Eigenschaften

Sie können eine schreibgeschützte Eigenschaft wie die im vorigen Abschnitt kompakter als *Expression-bodied Eigenschaft* deklarieren. Ein »dicker Pfeil« ersetzt alle geschweiften Klammern sowie die Schlüsselwörter `get` und `return`:

```
public decimal Worth => currentPrice * sharesOwned;
```

Seit C# 7 können auch set-Accessoren Expression-bodied sein:

```
public decimal Worth
{
    get => currentPrice * sharesOwned;
    set => sharesOwned = value / currentPrice;
}
```

Automatische Eigenschaften

Die häufigste Implementierung einer Eigenschaft ist ein Getter und/oder Setter, der einfach ein `private` Feld des gleichen Typs wie die Eigenschaft liest und schreibt. Eine Deklaration einer *automatischen Eigenschaft* weist den Compiler an, diese Implementierung bereitzustellen. Wir können das erste Beispiel dieses Abschnitts verbessern, indem wir `CurrentPrice` als automatische Eigenschaft deklarieren:

```
public class Stock
{
    public decimal CurrentPrice { get; set; }
}
```

Der Compiler generiert automatisch ein `private` Feld mit vom Compiler generierten Namen im Hintergrund, auf das nicht zugegriffen werden kann. Der set-Accessor kann als `private` oder als `protected` gekennzeichnet werden, wenn Sie die Eigenschaft als für andere Typen nur lesbar bereitstellen wollen.

Eigenschaftsinitialisierer

Sie können automatische Eigenschaften wie schon Felder mit einem *Eigenschaftsinitialisierer* ausstatten:

```
public decimal CurrentPrice { get; set; } = 123;
```

Damit erhält `CurrentPrice` den Initialwert 123. Eigenschaften mit einem Initialisierer können schreibgeschützt sein:

```
public int Maximum { get; } = 999;
```

Wie schreibgeschützte Felder können auch schreibgeschützte automatische Eigenschaften im Konstruktor des Typs zugewiesen werden. Das ist nützlich, wenn man *unveränderliche* (schreibgeschützte) Typen erstellt.

Sichtbarkeit von get und set

Die get- und set-Accessors können unterschiedliche Sichtbarkeit haben. Oft hat man eine public Eigenschaft mit einem internal oder private Zugriffsmodifikator für den Setter :

```
private decimal x;  
public decimal X  
{  
    get { return x; }  
    private set { x = Math.Round (value, 2); }  
}
```

Beachten Sie, dass Sie die Eigenschaft selbst mit der weiteren Sichtbarkeit (hier public) deklarieren und den Modifikator dem Accessor hinzufügen, der *weniger* sichtbar sein soll.

Init-Only-Setter

Seit C# 9 können Sie einen Eigenschafts-Accessor statt mit set auch mit init deklarieren:

```
public class Note  
{  
    public int Pitch { get; init; } = 20;  
    public int Duration { get; init; } = 100;  
}
```

Diese *Init-Only*-Eigenschaften verhalten sich wie schreibgeschützte Eigenschaften, nur dass sie auch über einen Objektinitialisierer gesetzt werden können:

```
var note = new Note { Pitch = 50 };
```

Danach lässt sich die Eigenschaft nicht mehr verändern:

```
note.Pitch = 200; // Fehler - Init-Only-Setter!
```

Init-Only-Eigenschaften lassen sich nicht einmal innerhalb ihrer Klasse setzen – außer über ihren Eigenschaftsinitialisierer, den Konstruktor oder einen anderen Init-Only-Accessor.

Die Alternative zu Init-Only-Eigenschaften sind schreibgeschützte Eigenschaften, die Sie über einen Konstruktor befüllen:

```
public Note (int pitch = 20, int duration = 100)
{
    Pitch = pitch; Duration = duration;
}
```

Sollte die Klasse Teil einer öffentlichen Bibliothek sein, wird das Versionieren durch diesen Ansatz schwieriger, weil das spätere Hinzufügen eines optionalen Parameters die Binärkompatibilität für den anwendenden Code aufhebt (während das Hinzufügen einer neuen Init-Only-Eigenschaft nichts kaputt macht).



Init-Only-Eigenschaften haben einen weiteren wichtigen Vorteil: Sie ermöglichen nicht destruktive Mutationen, wenn sie zusammen mit Datensätzen genutzt werden (siehe den Abschnitt »Datensätze« auf Seite 174).

Wie bei einem normalen set-Accessor können Init-Only-Accessors eine Implementierung bereitstellen:

```
public class Point
{
    readonly int _x;
    public int X { get => _x; init => _x = value; }
    ...
}
```

Beachten Sie, dass das Feld `_x` schreibgeschützt ist: Init-Only-Setter dürfen `readonly`-Felder in ihrer eigenen Klasse verändern. (Ohne dieses Feature müsste `_x` schreibbar sein, und die Klasse würde intern nicht immutabel sein.)

Indexer

Indexer stellen eine natürliche Syntax für den Zugriff auf die Elemente einer Klasse oder eines Struct bereit, die eine Liste oder ein Dictionary kapselt. Indexer ähneln Eigenschaften, werden aber über ein Indexargument statt über einen Eigenschaftsnamen angesprochen. Die Klasse `string` hat einen Indexer, mit dem Sie auf jeden einzelnen `char`-Wert über einen `int`-Index zugreifen können:

```
string s = "hello";
Console.WriteLine (s[0]); // 'h'
Console.WriteLine (s[3]); // 'l'
```

Die Syntax für die Verwendung von Indexern entspricht der für die Verwendung von Arrays, aber die Indexargumente können einen beliebigen Typ haben. Indexer lassen sich null-bedingt aufrufen, indem Sie vor den eckigen Klammern ein Fragezeichen einfügen (siehe »Null-Operatoren« auf Seite 59):

```
string s = null;
Console.WriteLine (s?[0]); // Schreibt nichts; kein Fehler.
```

Implementieren eines Indexers

Um einen Indexer zu schreiben, definieren Sie eine Eigenschaft mit dem Namen `this` und geben die Argumente in eckigen Klammern an:

```
class Sentence
{
    string[] words = "The quick brown fox".Split( );

    public string this [int wordNum] // Indexer
    {
        get { return words [wordNum]; }
        set { words [wordNum] = value; }
    }
}
```

Und so könnten wir den Indexer nutzen:

```
Sentence s = new Sentence( );
Console.WriteLine (s[3]); // fox
s[3] = "kangaroo";
Console.WriteLine (s[3]); // kangaroo
```

Ein Typ kann mehrere Indexer deklarieren, bei denen jeder Parameter einen anderen Typ hat. Ein Indexer kann auch mehr als einen Parameter erwarten:

```
public string this [int arg1, string arg2]
{
    get { ... } set { ... }
}
```

Wenn Sie den `set-Accessor` weglassen, wird ein Indexer schreibgeschützt. Mit der Expression-bodied Syntax lässt sich die Definition kompakter halten:

```
public string this [int wordNum] => words [wordNum];
```

Indizes und Bereiche mit Indexern verwenden

Sie können Indizes und Bereiche (siehe Abschnitt »Indizes und Bereiche« auf Seite 40) in Ihren eigenen Klassen unterstützen, indem Sie einen Indexer mit einem Parametertyp `Index` oder `Range` definieren. Wir könnten beispielsweise unser voriges Beispiel erweitern, indem wir die Klasse `Sentence` um die folgenden Indexer ergänzen:

```
public string this [Index index] => words [index];  
public string[] this [Range range] => words [range];
```

Das ermöglicht das Folgende:

```
Sentence s = new Sentence();  
Console.WriteLine (s [^1]);           // fox  
string[] firstTwoWords = s [..2];     // (The, quick)
```

Statische Konstruktoren

Ein statischer Konstruktor wird einmal pro *Typ* statt einmal pro *Instanz* ausgeführt. Ein Typ kann nur einen statischen Konstruktor definieren; dieser muss parameterlos sein und den gleichen Namen wie der Typ haben:

```
class Test  
{  
    static Test() { Console.Write ("Typ initialisiert"); }  
}
```

Die Laufzeitumgebung ruft einen statischen Konstruktor automatisch auf, und zwar unmittelbar bevor der Typ genutzt wird. Das wird von zwei Dingen ausgelöst: einer Instanziierung des Typs und dem Zugriff auf einen der statischen Member des Typs.



Wenn ein statischer Konstruktor eine unbehandelte Ausnahme auslöst, wird dieser Typ für die gesamte Lebensdauer der Anwendung unbrauchbar.



Seit C# 9 können Sie auch *Modulinitialisierer* definieren, die einmal pro Assembly ausgeführt werden (wenn sie erstmalig geladen wird). Um einen Modulinitialisierer zu definieren, schreiben Sie eine statische void-Methode und wenden dann das Attribut `[ModuleInitializer]` darauf an:

```
[System.Runtime.CompilerServices.ModuleInitializer]
internal static void InitAssembly()
{
    ...
}
```

Statische Feldinitialisierer werden unmittelbar vor dem Aufruf des statischen Konstruktors ausgeführt. Wenn ein Typ keinen statischen Konstruktor hat, werden statische Feldinitialisierer direkt vor der Verwendung des Typs ausgeführt – oder irgendwann vorher, wenn es der Laufzeit beliebt.

Statische Klassen

Eine Klasse, die als `static` markiert ist, kann nicht instanziiert werden, es lässt sich nicht von ihr ableiten, und sie muss vollständig aus statischen Mitgliedern bestehen. Die Klassen `System.Console` und `System.Math` sind gute Beispiele für statische Klassen.

Finalizer

Finalizer sind Klassenmethoden, die ausgeführt werden, bevor der Garbage Collector den Speicher eines nicht mehr referenzierten Objekts freigibt. Die Syntax für einen Finalizer ist der Name der Klasse, dem das Zeichen `~` vorangestellt ist:

```
class Class1
{
    ~Class1() { ... }
}
```

C# übersetzt einen Finalizer in eine Methode, die die `Finalize`-Methode der Klasse `object` überschreibt.

Finalizer, die aus einer einzelnen Anweisung bestehen, können in der Expression-bodied Syntax geschrieben werden.

Partielle Typen und Methoden

Partielle Typen ermöglichen es, eine Typdefinition aufzuteilen – meist auf mehrere Dateien. Häufig wird das für automatisch generierte Klassen aus anderen Quellen (zum Beispiel aus einer Visual-Studio-Vorlage) genutzt, bei denen die Klasse dann um eigene Methoden ergänzt werden kann, zum Beispiel so:

```
// PaymentFormGen.cs - auto-generated
partial class PaymentForm { ... }

// PaymentForm.cs - handgeschrieben
partial class PaymentForm { ... }
```

Jeder Teil muss die Deklaration `partial` einschließen.

Die Teilnehmer dürfen keine Member haben, die zu Konflikten führen. So kann zum Beispiel ein Konstruktor mit den gleichen Parametern nicht wiederholt werden. Partielle Typen werden komplett vom Compiler aufgelöst, daher muss jeder Teilnehmer beim Kompilieren verfügbar sein und sich in der gleichen Assembly befinden.

Eine Basisklasse kann auf einem oder mehreren Teilen angegeben werden (solange die spezifizierte Basisklasse die gleiche ist). Außerdem kann jeder Teilnehmer unabhängig zu implementierende Schnittstellen angeben. Basisklassen und Schnittstellen behandeln wir in den Abschnitten »Vererbung« auf Seite 95 und »Interfaces« auf Seite 113 ausführlicher.

Partielle Methoden

Ein partieller Typ kann *partielle Methoden* enthalten. Über diese kann ein automatisch generierter partieller Typ konfigurierbare Hooks für manuell geschriebene Teile anbieten, zum Beispiel so:

```

partial class PaymentForm    // in automatisch generierter
                             // Datei
{
    partial void ValidatePayment (decimal amount);
}

partial class PaymentForm    // in handgeschriebener Datei
{
    partial void ValidatePayment (decimal amount)
    {
        if (amount > 100) Console.Write ("Teuer!");
    }
}

```

Eine partielle Methode besteht aus zwei Teilen: einer *Definition* und einer *Implementierung*. Die Definition wird typischerweise von einem Codegenerator geschrieben, während die Implementierung meist per Hand erstellt wird. Wird keine Implementierung bereitgestellt, wird die Definition der partiellen Methode beim Kompilieren entfernt. Damit kann automatisch generierter Code beim Bereitstellen von Hooks großzügig sein, ohne dass man sich um explodierende Codemengen sorgen müsste. Partielle Methoden müssen void sein und sind implizit private.

Erweiterte partielle Methoden

Erweiterte partielle Methoden sind (seit C# 9) für das Reverse-Code-Generation-Szenario entworfen worden, bei dem ein Programmierer Hooks definiert, die ein Codegenerator implementiert. Ein Beispiel dafür sind *Source Generators* – ein Roslyn-Feature, bei dem Sie dem Compiler eine Assembly geben können, die automatisch einen Teil Ihres Codes generiert.

Eine partielle Methodendeklaration ist *erweitert*, wenn sie mit einem Zugriffsmodifikator beginnt:

```

public partial class Test
{
    public partial void M1();    // erweiterte partielle Methode
    private partial void M2(); // erweiterte partielle Methode
}

```

Die Anwesenheit des Zugriffsmodifikators beeinflusst nicht nur den Zugriff: Er weist den Compiler an, die Deklaration anders zu behandeln.

Erweiterte partielle Methoden *müssen* Implementierungen besitzen – sie verschwinden nicht einfach, wenn sie nicht implementiert sind. In diesem Beispiel müssen sowohl M1 als auch M2 Implementierungen haben, weil beide Zugriffsmodifikatoren spezifizieren (public und private).

Da sie nicht verschwinden können, können erweiterte partielle Methoden einen beliebigen Typ zurückgeben und auch out-Parameter enthalten.

Der Operator nameof

Der Operator nameof gibt den Namen eines beliebigen Symbols (Typ, Member, Variable und so weiter) als String zurück:

```
int count = 123;
string name = nameof (count);    // name ist "count"
```

Sein Vorteil gegenüber dem schlichten Definieren eines Strings ist die statische Typprüfung. Tools wie Visual Studio können die Symbolreferenz verstehen, und wenn Sie das fragliche Symbol umbenennen, werden alle Referenzen ebenfalls umbenannt.

Um den Namen eines Typ-Members wie zum Beispiel eines Felds oder einer Eigenschaft zu erhalten, geben Sie auch den Typ selbst an. Das funktioniert sowohl mit statischen als auch mit Instanz-Membnern:

```
string name = nameof (StringBuilder.Length);
```

Das wird zu Length ausgewertet. Um StringBuilder.Length zu erhalten, würden Sie so vorgehen müssen:

```
nameof(StringBuilder)+"."+nameof(StringBuilder.Length);
```

Vererbung

Eine Klasse kann von einer Klasse *erben*, damit die ursprüngliche Klasse erweitert oder angepasst wird. Durch das Erben von einer Klasse können Sie die Funktionalität in dieser Klasse nutzen, statt alles von Grund auf wieder selbst aufbauen zu müssen. Eine Klasse kann nur von einer einzigen Klasse erben, kann aber selbst an viele andere Klassen weitervererben, wodurch eine Klassenhierarchie entsteht. In diesem Beispiel wollen wir damit beginnen, eine Klasse namens `Asset` zu erstellen:

```
public class Asset { public string Name; }
```

Als Nächstes definieren wir Klassen mit den Namen `Stock` und `House`, die von `Asset` erben werden. `Stock` und `House` erhalten alles, was `Asset` hat, und dazu noch alle Member, die sie selbst definieren:

```
public class Stock : Asset    // erbt von Asset
{
    public long SharesOwned;
}

public class House : Asset    // erbt von Asset
{
    public decimal Mortgage;
}
```

Und so können wir diese Klassen nutzen:

```
Stock msft = new Stock { Name="MSFT",
                        SharesOwned=1000 };

Console.WriteLine (msft.Name);           // MSFT
Console.WriteLine (msft.SharesOwned);    // 1000

House mansion = new House { Name="Mansion",
                           Mortgage=250000 };

Console.WriteLine (mansion.Name);        // Mansion
Console.WriteLine (mansion.Mortgage);    // 250000
```

Die *Subklassen* `Stock` und `House` erben das Feld `Name` von der *Basis-klass* `Asset`.

Subklassen werden auch als *abgeleitete Klassen* bezeichnet.

Polymorphie

Referenzen sind *polymorph*. Das bedeutet, dass eine Variable des Typs x auf Instanzen von Typen verweisen kann, die Subklassen von x sind. Betrachten Sie beispielsweise die folgende Methode:

```
public static void Display (Asset asset)
{
    System.Console.WriteLine (asset.Name);
}
```

Diese Methode kann ein Stock- und ein House-Objekt anzeigen, da beides Assets sind. Polymorphie basiert darauf, dass die Subklassen (Stock und House) alle Features ihrer Basisklasse (Asset) besitzen. Das Gegenteil ist aber nicht der Fall. Wenn Display so geändert würde, dass es ein House akzeptiert, könnten Sie kein Asset übergeben.

Casting und Referenzumwandlungen

Eine Objektreferenz kann

- implizit zu einer Basisklassenreferenz nach oben gecastet werden (*Upcast*) oder
- explizit zu einer Subklassenreferenz nach unten gecastet werden (*Downcast*).

Ein Upcasting oder Downcasting zwischen kompatiblen Referenztypen führt zu einer *Referenzumwandlung*: Es wird eine neue Referenz erstellt, die auf dasselbe Objekt zeigt. Ein Upcast ist immer erfolgreich; ein Downcast ist nur erfolgreich, wenn das Objekt den richtigen Typ hat.

Upcasting

Eine Upcast-Operation erzeugt aus einer Subklassenreferenz eine Basisklassenreferenz:

```
Stock msft = new Stock();    // aus letztem Beispiel
Asset a = msft;           // Upcast
```

Nach dem Upcast verweist Variable `a` immer noch auf dasselbe Stock-Objekt wie die Variable `msft`. Das referenzierte Objekt ist nicht verändert oder umgewandelt worden:

```
Console.WriteLine (a == msft);           // True
```

Auch wenn `a` und `msft` auf dasselbe Objekt verweisen, hat `a` eine eingeschränktere Sicht darauf:

```
Console.WriteLine (a.Name);               // Okay  
Console.WriteLine (a.SharesOwned);       // Kompilierungsfehler
```

Die letzte Zeile führt zu einem Kompilierungsfehler, da die Variable `a` vom Typ `Asset` ist, obwohl sie auf ein Objekt vom Typ `Stock` verweist. Um an dessen Feld `SharesOwned` zu gelangen, müssen Sie das `Asset` zu einem `Stock` »downcasten«.

Downcasting

Eine Downcast-Operation erzeugt eine Subklassenreferenz aus einer Basisklassenreferenz:

```
Stock msft = new Stock();  
Asset a = msft;                       // Upcast  
Stock s = (Stock)a;                 // Downcast  
Console.WriteLine (s.SharesOwned);    // <Kein Fehler>  
Console.WriteLine (s == a);           // True  
Console.WriteLine (s == msft);        // True
```

Wie bei einem Upcast sind nur die Referenzen betroffen, nicht das zugrunde liegende Objekt. Ein Downcast benötigt einen expliziten Cast, da er zur Laufzeit potenziell fehlschlagen kann:

```
House h = new House();  
Asset a = h;           // Upcast ist immer erfolgreich  
Stock s = (Stock)a;    // Downcast schlägt fehl: a ist kein Stock
```

Wenn ein Downcast fehlschlägt, wird eine `InvalidCastException` geworfen. Dies ist ein Beispiel für *Typprüfung zur Laufzeit* (siehe »Statische und Laufzeit-Typprüfung« auf Seite 106).

Der as-Operator

Der Operator `as` führt einen Downcast durch, der `null` als Ergebnis hat, wenn der Downcast fehlschlägt (anstatt eine Exception zu werfen):

```
Asset a = new Asset();  
Stock s = a as Stock; // s ist null; keine Ausnahme
```

Das ist praktisch, wenn Sie später prüfen werden, ob das Ergebnis null ist:

```
if (s != null) Console.WriteLine (s.SharesOwned);
```

Der as-Operator kann keine angepassten Umwandlungen (siehe »Überladen von Operatoren« auf Seite 220) und keine numerischen Umwandlungen durchführen.

Der is-Operator

Der is-Operator prüft, ob eine Referenzumwandlung erfolgreich wäre – mit anderen Worten, ob ein Objekt von einer bestimmten Klasse abgeleitet ist (oder ein Interface implementiert). Er wird häufig genutzt, um vor einem Downcast eine Prüfung vorzunehmen:

```
if (a is Stock) Console.Write (((Stock)a).SharesOwned);
```

Der is-Operator gibt auch true zurück, wenn eine Unboxing-Umwandlung erfolgreich wäre (siehe »Der Typ object« auf Seite 104). Benutzerdefinierte und numerische Umwandlungen werden allerdings nicht berücksichtigt. Seit C# 7 können Sie im Rahmen des is-Operators eine Variable einführen:

```
if (a is Stock s)  
    Console.WriteLine (s.SharesOwned);
```

Die so geschaffene Variable steht zur sofortigen Verwendung zur Verfügung und bleibt auch außerhalb des is-Ausdrucks im Scope:

```
f (a is Stock s && s.SharesOwned > 100000)  
    Console.WriteLine ("Reich");  
else  
    s = new Stock(); // s ist im Scope
```

```
Console.WriteLine (s.SharesOwned); // immer noch im Scope
```

Der is-Operator funktioniert mit anderen Mustern, die in früheren Versionen von C# eingeführt wurden. Eine vollständige Behandlung finden Sie im Abschnitt »Muster« auf Seite 181.

Virtuelle Funktions-Member

Eine Funktion, die als virtual gekennzeichnet ist, kann von Subklassen *überschrieben* werden, die eine speziellere Implementierung

bereitstellen wollen. Methoden, Eigenschaften, Indexer und Events können als `virtual` deklariert werden:

```
public class Asset
{
    public string Name;
    public virtual decimal Liability => 0; }
}
```

`Liability => 0` ist eine Kurzform für `{ get { return 0; } }` (siehe dazu »Expression-bodied Eigenschaften« auf Seite 85). Eine Subklasse überschreibt eine virtuelle Methode, indem sie den Modifikator `override` angibt:

```
public class House : Asset
{
    public decimal Mortgage;

    public override decimal Liability => Mortgage;
}
```

Standardmäßig liefert die `Liability` eines `Asset` den Wert 0. Ein `Stock` muss hier nicht spezieller tätig werden. Aber das `House` spezialisiert die Eigenschaft `Liability` so, dass der Wert der `Mortgage` zurückgegeben wird:

```
House mansion = new House { Name="Mansion",
                             Mortgage=250000 };

Asset a = mansion;
Console.WriteLine (mansion.Liability); // 250000
Console.WriteLine (a.Liability);       // 250000
```

Signaturen, Rückgabewerte und Sichtbarkeit der virtuellen und überschriebenen Methoden müssen identisch sein. Eine überschriebene Methode kann ihre Implementierung in der Basisklasse mithilfe des Schlüsselworts `base` ansprechen (siehe »Das Schlüsselwort `base`« auf Seite 101).

Kovariante Rückgabetypen

Seit C# 9 können Sie eine Methode (oder einen Eigenschafts-get-Accessor) so überschreiben, dass sie einen *weiter abgeleiteten* Typ (eine Subklasse) zurückgibt. Sie können beispielsweise eine `Clone`-Methode in der Klasse `Asset` schreiben, die eine `Asset` zurückgibt,

und diese Methode in der Klasse House überschreiben, sodass diese ein House zurückgibt.

Dies ist erlaubt, weil es nicht die Vereinbarung bricht, dass Clone ein Asset zurückgeben muss: Es gibt ein House zurück, das ein Asset (und mehr) ist (im Sinne von is).

Abstrakte Klassen und abstrakte Member

Eine Klasse, die als abstrakt deklariert wurde, kann man niemals instanziiieren. Stattdessen können nur ihre konkreten Subklassen instanziiert werden.

Abstrakte Klassen können *abstrakte Member* deklarieren, die sich wie virtuelle Member verhalten, aber keine Standardimplementierung bereitstellen. Diese Implementierung muss in der Subklasse vorgenommen werden, sofern die Subklasse nicht auch als abstrakt deklariert wurde:

```
public abstract class Asset
{
    // Beachten Sie die leere Implementierung.
    public abstract decimal NetValue { get; }
}
```

Subklassen überschreiben abstrakte Member ebenso, wie sie virtuelle Member überschreiben.

Verbergen geerbter Member

Eine Basisklasse und eine Subklasse können identische Member definieren:

```
public class A      { public int Counter = 1; }
public class B : A  { public int Counter = 2; }
```

Das Feld Counter in Klasse B verbirgt das Feld Counter aus Klasse A. Normalerweise geschieht das unabsichtlich, wenn der Basisklasse ein Member hinzugefügt wird, *nachdem* dem Subtyp ein identisches Member hinzugefügt wurde. Aus diesem Grund erzeugt der Compiler eine Warnung und löst die Mehrdeutigkeit wie folgt auf:

- Referenzen auf A (beim Kompilieren) werden mit A.Counter verbunden.
- Referenzen auf B (beim Kompilieren) werden mit B.Counter verbunden.

Gelegentlich wollen Sie ein Member mit Absicht verbergen. In solchen Fällen können Sie dem Member in der Subklasse den Modifikator `new` mitgeben. Dieser sorgt lediglich dafür, dass die Compilerwarnung unterdrückt wird, die es sonst hier gäbe:

```
public class A    { public    int Counter = 1; }
public class B : A { public new int Counter = 2; }
```

Der Modifikator `new` teilt dem Compiler (und den weiteren Programmierern) mit, dass das doppelte Member kein Versehen von Ihnen ist.

Funktionen und Klassen versiegeln

Ein überschriebenes Funktions-Member kann seine Implementierung mithilfe des Schlüsselworts `sealed` *versiegeln*, um zu verhindern, dass es durch weitere Subklassen überschrieben wird. In unserem Beispiel für virtuelle Funktions-Member könnten wir die Implementierung von `Liability` in `House` versiegeln und damit verhindern, dass eine Klasse, die von `House` abgeleitet wird, `Liability` überschreibt:

```
public sealed override decimal Liability { get { ... } }
```

Sie können den Modifikator `sealed` auch selbst auf die Klasse anwenden, um das Erstellen von abgeleiteten Klassen zu verhindern.

Das Schlüsselwort `base`

Das Schlüsselwort `base` entspricht dem Schlüsselwort `this`. Dabei dient es zwei Zwecken: dem Zugriff auf ein überschriebenes Funktions-Member aus der Subklasse und dem Aufruf eines Basisklassenkonstruktors (siehe nächsten Abschnitt).

In diesem Beispiel nutzt `House` das Schlüsselwort `base`, um auf die Asset-Implementierung von `Liability` zuzugreifen:

```
public class House : Asset
{
    ...
    public override decimal Liability
        => base.Liability + Mortgage;
}
```

Wir greifen mit dem Schlüsselwort *base* *nichtvirtuell* auf die Liability-Eigenschaft von Asset zu. Das bedeutet, dass wir immer auf die Asset-Version dieser Eigenschaft zugreifen – unabhängig davon, was der tatsächliche Laufzeittyp der Instanz ist.

Der gleiche Ansatz funktioniert, wenn Liability *verborgen* ist und nicht *überschrieben*. (Sie können auch auf verborgene Member zugreifen, indem Sie auf die Basisklasse casten, bevor Sie die Funktion aufrufen.)

Konstrukturen und Vererbung

Eine Subklasse muss ihre eigenen Konstrukturen definieren. Würden wir zum Beispiel Baseclass und Subclass so definieren:

```
public class Baseclass
{
    public int X;
    public Baseclass () { }
    public Baseclass (int x) { this.X = x; }
}
public class Subclass : Baseclass { }
```

wäre das Folgende ungültig:

```
Subclass s = new Subclass (123);
```

Subclass muss daher alle Konstrukturen neu deklarieren, die sie bereitstellen will. Dabei kann sie beliebige Basisklassenkonstrukturen mit dem Schlüsselwort *base* aufrufen:

```
public class Subclass : Baseclass
{
    public Subclass (int x) : base (x) { ... }
}
```

Das Schlüsselwort *base* funktioniert wie das Schlüsselwort *this*, nur dass es einen Konstruktor in der Basisklasse aufruft. Basisklassen-

konstruktoren werden immer zuerst ausgeführt – damit ist sichergestellt, dass die *Basisinitialisierung* vor der *spezialisierten Initialisierung* ausgeführt wird.

Verwendet ein Konstruktor in einer Subklasse das Schlüsselwort *base* nicht, wird der *parameterlose* Konstruktor der Basisklasse implizit aufgerufen (wenn die Basisklasse keinen erreichbaren parameterlosen Konstruktor hat, meldet der Compiler einen Fehler).

Reihenfolge von Konstruktor- und Feldinitialisierung

Wenn ein Objekt instanziiert wird, geschieht die Initialisierung in dieser Reihenfolge:

1. Von der Subklasse zur Basisklasse:
 - a. Felder werden initialisiert.
 - b. Argumente für den Basisklassenkonstruktor werden ausgewertet.
2. Von der Basisklasse zur Subklasse:
 - c. Konstruktorcode wird ausgeführt.

Auflösen beim Überladen

Die Vererbung hat einen interessanten Einfluss auf das Überladen von Methoden. Schauen Sie sich die folgende überladene Methode an:

```
static void Foo (Asset a) { }  
static void Foo (House h) { }
```

Wird eine überladene Methode aufgerufen, hat der spezifischste Typ Vorrang:

```
House h = new House (...);  
Foo(h); // Ruft Foo(House) auf.
```

Die aufzurufende Version der überladenen Methode wird statisch bestimmt (beim Kompilieren) und nicht zur Laufzeit. Der folgende Code ruft `Foo(Asset)` auf, auch wenn der Typ zur Laufzeit ein `House` ist:

```
Asset a = new House (...);  
Foo(a); // Ruft Foo(Asset) auf.
```



Wenn Sie `Asset` auf `dynamic` casten (siehe »Die dynamische Bindung« auf Seite 211), wird die Entscheidung darüber, welche Überladung aufgerufen wird, bis zur Laufzeit aufgeschoben und basiert dann auf dem tatsächlichen Typ des Objekts.

Der Typ `object`

`object` (`System.Object`) ist die Ausgangsbasis für alle Typen. Jeder Typ kann per Upcast in ein `object` umgewandelt werden.

Um zu zeigen, wie nützlich das sein kann, stellen Sie sich einen allgemeinen *Stack* vor. Ein Stack ist eine Datenstruktur, die auf dem Prinzip »Last in, first out« (kurz LIFO) basiert. Ein Stack hat zwei Operationen: per Push ein Objekt auf den Stack schieben und per Pop eines wieder herunterholen. So sieht eine einfache Implementierung aus, die bis zu zehn Objekte verwalten kann:

```
public class Stack
{
    int position;
    object[] data = new object[10];
    public void Push (object o) { data[position++] = o; }
    public object Pop() { return data[--position]; }
}
```

Da Stack mit dem Typ `object` arbeitet, können wir Instanzen von *jedem beliebigen Typ* per Push und Pop auf den Stack bringen und wieder von ihm herunterbringen:

```
Stack stack = new Stack();
stack.Push ("Würstchen");
string s = (string) stack.Pop();    // Cast nötig
Console.WriteLine (s);              // Würstchen
```

`object` ist ein Referenztyp, da es eine Klasse ist. Trotzdem können Werttypen wie `int` auf `object` gecastet werden. Das ermöglicht die CLR mit einigen speziellen Operationen, die die grundlegenden Unterschiede zwischen Wert- und Referenztypen überbrücken. Man bezeichnet diese Schritte als *Boxing* und *Unboxing*.



Im Abschnitt »Generics« auf Seite 121 werden wir beschreiben, wie wir unsere Stack-Klasse verbessern können, damit sie besser mit Stacks von Elementen desselben Typs umgeht.

Boxing und Unboxing

Boxing ist das Casten einer Instanz eines Werttyps auf eine Instanz eines Referenztyps. Der Referenztyp kann entweder von der Klasse `Object` oder ein Interface sein (siehe »Interfaces« auf Seite 113). In diesem Beispiel *boxen* wir ein `int` in ein `Object`:

```
int x = 9;  
Object obj = x;           // int verpacken
```

Beim Unboxing wird die Operation umgekehrt, indem das Objekt zurück in den ursprünglichen Werttyp gewandelt wird:

```
int y = (int)obj;         // int auspacken
```

Unboxing erfordert einen expliziten Cast. Die Laufzeitumgebung prüft, ob der angegebene Werttyp dem tatsächlichen Objekttyp entspricht, und wirft bei Nichtübereinstimmung eine `InvalidCastException`. So wird zum Beispiel hier eine Exception geworfen, weil `long` nicht genau zu `int` passt:

```
Object obj = 9;           // 9 wird als int erkannt  
long x = (long) obj;      // InvalidCastException
```

Der folgende Code wird aber erfolgreich sein:

```
Object obj = 9;  
long x = (int) obj;
```

Und dieser auch:

```
Object obj = 3.5;         // erschlossener Typ von 3.5 ist double  
int x = (int) (double) obj; // x ist jetzt 3
```

Im letzten Beispiel führt `(double)` ein Unboxing durch und `(int)` eine numerische Umwandlung.

Beim Boxing wird die Werttyp-Instanz in das neue Objekt kopiert, und beim Unboxing wird der Inhalt des Objekts wieder in eine Werttyp-Instanz kopiert:

```
int i = 3;
object boxed = i;
i = 5;
Console.WriteLine (boxed);    // 3
```

Statische und Laufzeit-Typprüfung

C# prüft Typen sowohl statisch (beim Kompilieren) als auch zur Laufzeit.

Die statische Typprüfung ermöglicht dem Compiler, die Korrektheit Ihres Programms zu prüfen, ohne es auszuführen. Der folgende Code wird fehlschlagen, da der Compiler die statische Typprüfung durchführt:

```
int x = "5";
```

Die Laufzeit-Typprüfung wird von der CLR durchgeführt, wenn Sie über eine Referenzumwandlung oder Unboxing einen Downcast vornehmen:

```
object y = "5";
int z = (int) y;    // Laufzeitfehler, Downcast
                  // fehlgeschlagen
```

Die Laufzeit-Typprüfung ist möglich, weil jedes Objekt auf dem Heap intern ein kleines Typ-Token mit abspeichert. Dieses Token kann durch den Aufruf der Methode `GetType` von `object` genutzt werden.

Die `GetType`-Methode und der `typeof`-Operator

Alle Typen in C# werden zur Laufzeit durch eine Instanz von `System.Type` repräsentiert. Ein Objekt vom Typ `System.Type` kann man auf zweierlei Weise erhalten: durch einen Aufruf von `GetType` für die Instanz oder durch den Einsatz des Operators `typeof` für einen Typnamen. `GetType` wird zur Laufzeit ausgewertet, während `typeof` statisch beim Kompilieren ermittelt wird.

`System.Type` hat Eigenschaften für den Namen des Typs, die Assembly, den Basistyp und so weiter:

```
int x = 3;
```

```

Console.Write (x.GetType().Name);           // Int32
Console.Write (typeof(int).Name);           // Int32
Console.Write (x.GetType().FullName);       // System.Int32
Console.Write (x.GetType() == typeof(int)); // True

```

System.Type stellt auch Methoden zur Verfügung, die als Brücke zum Reflection-Modell der Laufzeitumgebung dienen.

Die Member von object

Das hier sind alle Member von object:

```

public extern Type GetType();
public virtual bool Equals (object obj);
public static bool Equals (object objA, object objB);
public static bool ReferenceEquals (object objA,
                                     object objB);

public virtual int GetHashCode();
public virtual string ToString();
protected virtual void Finalize();
protected extern object MemberwiseClone();

```

Equals, ReferenceEquals und GetHashCode

Die Methode Equals entspricht dem Operator ==, nur dass Equals virtuell, == aber statisch ist. Das folgende Beispiel zeigt den Unterschied:

```

object x = 3;
object y = 3;
Console.WriteLine (x == y);           // False
Console.WriteLine (x.Equals (y));     // True

```

Da x und y auf den Typ object gecastet wurden, bindet der Compiler statisch gegen den Operator == von object, der die *Referenztyp*-Semantik nutzt, um zwei Instanzen zu vergleichen. (Da x und y geboxt sind, handelt es sich um zwei verschiedene Speicherplätze, die damit nicht gleich sind.) Die virtuelle Methode Equals greift auf Equals des Typs Int32 zurück, das eine *Werttyp*-Semantik beim Vergleich zweier Werte nutzt.

Die statische Methode object.Equals ruft einfach die virtuelle Methode Equals auf – nachdem sie sichergestellt hat, dass die Argumente nicht null sind.

```
object x = null, y = 3;  
bool error = x.Equals(y);           // Laufzeitfehler!  
bool ok = object.Equals(x, y);      // Okay (false)
```

ReferenceEquals erzwingt einen Test auf Referenztyp-Gleichheit (das ist gelegentlich bei Referenztypen nützlich, wenn der Operator == überladen wurde, um etwas anderes zu bewirken).

GetHashCode gibt einen Hashcode aus, der für den Einsatz in Hash-table-basierten Dictionaries geeignet ist, d. h. System.Collections.Generic.Dictionary und System.Collections.Hashtable.

Um die Gleichheitssemantik eines Typs anzupassen, müssen Sie mindestens Equals und GetHashCode überschreiben. Sie werden meist aber auch die Operatoren == und != überladen. Ein Beispiel dazu finden Sie im Abschnitt »Überladen von Operatoren« auf Seite 220.

Die ToString-Methode

Die Methode ToString liefert die Standardtextdarstellung einer Typinstanz zurück. Die Methode wird von allen eingebauten Typen überschrieben. Hier sehen Sie ein Beispiel für die Verwendung der Methode ToString des Typs int:

```
string s1 = 1.ToString();           // s1 ist "1"  
string s2 = true.ToString();        // s2 ist "True"
```

Sie können die Methode ToString für selbst definierte Typen wie folgt überschreiben:

```
public override string ToString() => "Foo";
```

Structs

Ein *Struct* ähnelt einer Klasse, es gibt aber auch entscheidende Unterschiede:

- Ein Struct ist ein Werttyp, während eine Klasse ein Referenztyp ist.
- Ein Struct unterstützt keine Vererbung (außer dass es implizit von object oder genauer von System.ValueType abgeleitet ist).

Ein Struct kann alle Member nutzen, die eine Klasse haben kann, mit Ausnahme von Finalizern und virtuellen oder geschützten (protected) Membern.



Vor C# 10 durften Structs zudem keine Feldinitialisierer und parameterlose Konstruktoren definieren. Auch wenn dieses Verbot nun gelockert wurde – vor allem zum Vorteil von Datensatz-Structs (siehe den Abschnitt »Datensätze« auf Seite 174) –, ist es sinnvoll, sich sorgfältig Gedanken zu machen, bevor Sie diese Konstrukte definieren, da sie zu verwirrendem Verhalten führen können, das wir im Abschnitt »Semantik beim Erzeugen eines Struct« auf Seite 110 beschreiben.

Ein Struct ist passend, wenn die Werttyp-Semantik gewünscht wird. Gute Beispiele für Structs sind numerische Typen, bei denen es natürlicher ist, in einer Zuweisung einen Wert zu kopieren als eine Referenz. Da ein Struct ein Werttyp ist, muss nicht bei jeder Instanziierung ein Objekt auf dem Heap erstellt werden. Das kann sich zu einer nützlichen Einsparung summieren, wenn man viele Instanzen eines Typs erstellen muss.

Wie jeder Werttyp kann ein Struct indirekt auf dem Heap landen – entweder per Boxing oder wenn es als Feld in einer Klasse auftaucht. Instanzieren wir `SomeClass` im folgenden Beispiel, bezieht sich das Feld `Y` auf ein Struct auf dem Heap:

```
struct SomeStruct { public int X;          }  
class SomeClass  { public SomeStruct Y; }
```

Deklariieren Sie ein Array mit `SomeStruct`, findet sich die Instanz auch auf dem Heap wieder (weil Arrays Referenztypen sind), auch wenn für das gesamte Array nur eine einzelne Speicheranforderung erforderlich wäre.

Seit C# 7.2 können Sie mit dem Modifikator `ref` für ein Struct sicherstellen, dass es nur so eingesetzt werden kann, dass es auf dem Stack landet. Das ermöglicht dem Compiler weitere Optimierungsmöglichkeiten für den Typ `Span<T>` (siehe <https://bit.ly/2LR2ctm>).

Semantik beim Erzeugen eines Struct

Anders als bei Klassen muss in einem Struct jedes Feld explizit im Konstruktor (oder Feldinitialisierer) zugewiesen werden.

Neben jedem Konstruktor, den Sie definieren, besitzt ein Struct immer auch einen implizit parameterlosen Konstruktor, der ein bitweises Löschen seiner Felder vornimmt (und sie auf ihre Standardwerte setzt):

```
Point p = new Point();    // p.x und p.y sind 0
struct Point { int x, y; }
```

Auch wenn Sie selbst einen parameterlosen Konstruktor definieren, wird es den implizit parameterlosen Konstruktor weiterhin geben, der sich über das Schlüsselwort `default` ansprechen lässt:

```
Point p1 = new Point();    // p1.x und p1.y sind 1
Point p2 = default;        // p2.x und p2.y sind 0

struct Point
{
    int x = 1; int y;
    public Point() => y = 1;
}
```

In diesem Beispiel haben wir `x` über einen Feldinitialisierer mit 1 initialisiert, während wir `y` über den parameterlosen Konstruktor auf 1 gesetzt haben. Und trotzdem sind wir mit dem Schlüsselwort `default` dazu in der Lage, ein `Point` zu erzeugen, das beide Initialisierungsschritte umgeht. Der Default-Konstruktor lässt sich aber auch auf anderen Wegen erreichen, wie das folgende Beispiel zeigt:

```
var points = new Point[10];    // Jedes Point ist (0,0)
var test = new Test();         // test.p ist (0,0)
class Test { Point p; }
```

Bei Structs ist es eine gute Strategie, sie so zu entwerfen, dass ihr `default`-Wert ein gültiger Status ist, sodass Initialisierungen redundant sind.

Structs und Funktionen mit `readonly`

Sie können den Modifikator `readonly` auf ein Struct anwenden, um sicherzustellen, dass alle Felder `readonly` sind – das erleichtert das

Aufzeigen der Absicht hinter einem Struct und ermöglicht dem Compiler weitere Optimierungen:

```
readonly struct Point
{
    public readonly int X, Y;    // X und Y müssen readonly sein
}
```

Müssen Sie `readonly` auf einer detaillierteren Ebene anwenden, können Sie (seit C# 8) `readonly` auf die Funktionen eines Struct anwenden. Damit ist sichergestellt, dass es zu einem Kompilierungsfehler kommt, wenn die Funktion versucht, ein Feld zu verändern:

```
struct Point
{
    public int X, Y;
    public readonly void ResetX() => X = 0; // Fehler!
}
```

Ruft eine `readonly`-Funktion eine Funktion ohne `readonly` auf, erzeugt der Compiler eine Warnung (und kopiert vorsichtshalber das Struct, um die Gefahr einer Veränderung zu vermeiden).

Zugriffsmodifikatoren

Um die Kapselung zu unterstützen, kann ein Typ oder Typ-Member seine *Sichtbarkeit* gegenüber anderen Typen und Assemblies einschränken, indem er seiner Deklaration einen von fünf *Zugriffsmodifikatoren* hinzufügt:

`public`
Vollständig sichtbar. Der implizite Wert für Member eines Enum oder eines Interface.

`internal`
Sichtbar nur innerhalb der Assembly, in der man sich befindet, und in befreundeten Assemblies. Standardwert für nicht verschachtelte Typen.

`private`
Sichtbar nur für den umschließenden Typ. Standardwert für Member einer Klasse oder eines Struct.

`protected`

Sichtbar nur für enthaltende Typen oder Subklassen.

`protected internal`

Die *Vereinigung* von `protected` und `internal`. (Das ist weniger restriktiv als `protected` oder `internal` allein, da es ein Member auf zweierlei Weise besser sichtbar macht).

`private protected` (ab C# 7.2)

Die Schnittmenge von `protected` und `internal` (das ist restriktiver als nur `protected` oder `internal`).

Im folgenden Beispiel ist `Class2` auch außerhalb seiner Assembly erreichbar, `Class1` nicht:

```
class Class1 {}           // Class1 ist internal (default)
public class Class2 {}
```

`ClassB` stellt das Feld `x` anderen Typen in der Assembly bereit, `ClassA` nicht:

```
class ClassA { int x;      } // x ist private
class ClassB { internal int x; }
```

Wenn Sie eine Basisklassenfunktion überschreiben, muss die Sichtbarkeit derjenigen der überschriebenen Funktion entsprechen. Der Compiler verhindert jegliche inkonsistente Verwendung von Zugriffsmodifikatoren – beispielsweise kann eine Subklasse eine geringere Sichtbarkeit haben als eine Elternklasse, eine bessere Sichtbarkeit hingegen nicht.

Friend Assemblies

Sie können `internal`-Member für andere *befreundete* Assemblies sichtbar machen, indem Sie das Attribut `System.Runtime.CompilerServices.InternalsVisibleTo` setzen und dabei folgendermaßen den Namen der Friend Assembly angeben:

```
[assembly: InternalsVisibleTo ("Friend")]
```

Wenn die Friend Assembly mit einem Strong Name signiert ist, müssen Sie ihren vollständigen öffentlichen 160-Byte-Schlüssel angeben. Sie können diesen Schlüssel über eine LINQ-Abfrage (*Language Integrated Query*) heranziehen – ein interaktives Beispiel dafür finden

Sie in LINQ Pads freier Beispielbibliothek für *C# 10 in a Nutshell* in Kapitel 3.

Beschneiden der Sichtbarkeit

Ein Typ beschneidet die Sichtbarkeit seiner deklarierten Member. Der häufigste Fall ist ein `internal` Typ mit `public` Membern:

```
class C { public void Foo() {} }
```

Die (Standard-)Sichtbarkeit `internal` von `C` beschneidet die Sichtbarkeit von `Foo`, wodurch `Foo` im Endeffekt `internal` wird. Ein Grund dafür, dass man `Foo` als `public` kennzeichnet, könnte im einfacheren Refactoring in dem Fall liegen, dass man `C` später in `public` ändern möchte.

Interfaces

Ein *Interface* ähnelt einer Klasse, stellt aber für ihre Member nur eine Spezifikation und keine Implementierung bereit (ab C# 8.0 kann ein Interface auch eine *Default*-Implementierung bereitstellen – siehe Abschnitt »Default-Interface-Member« auf Seite 117). Ein Interface hat folgende Besonderheiten:

- Interface-Member sind *alle implizit abstrakt*. Im Gegensatz dazu kann eine Klasse sowohl abstrakte als auch konkrete Member mit Implementierungen bereitstellen.
- Eine Klasse kann *mehrere* Interfaces implementieren. Im Gegensatz dazu kann sie nur von einer *einzigen* Klasse erben. Structs können Interfaces implementieren, aber nicht von einer Klasse erben (davon abgesehen, dass sie von `System.ValueType` abgeleitet sind).

Eine Deklaration eines Interface ist wie eine Klassendeklaration, stellt aber für seine Member keine Implementierungen bereit, da alle seine Member implizit abstrakt sind. Diese Member werden von den Klassen und Structs implementiert, die das Interface implementieren. Ein Interface kann nur Methoden, Eigenschaften, Events und Indexer enthalten, was durchaus nicht ohne Absicht genau die Member einer Klasse sind, die abstrakt sein können.

Hier sehen Sie eine etwas vereinfachte Version des Interface `IEnumerator`, das in `System.Collections` definiert ist:

```
public interface IEnumerator
{
    bool MoveNext();
    object Current { get; }
    void Reset();
}
```

Interface-Member sind implizit `public` und dürfen keine Zugriffsmodifikatoren deklarieren. Implementiert man ein Interface, muss man eine `public` Implementierung für alle seine Member bereitstellen:

```
internal class Countdown : IEnumerator
{
    int count = 6;
    public bool MoveNext() => count-- > 0 ;
    public object Current => count;
    public void Reset() => count = 6;
}
```

Sie können ein Objekt implizit auf jedes Interface casten, das es implementiert:

```
IEnumerator e = new Countdown();
while (e.MoveNext())
    Console.Write (e.Current + " "); // 5 4 3 2 1 0
```

Erweitern eines Interface

Interfaces können von anderen Interfaces abgeleitet werden:

```
public interface IUndoable { void Undo(); }
public interface IRedoable : IUndoable { void Redo(); }
```

`IRedoable` »erbt« alle Member von `IUndoable`.

Explizite Implementierung eines Interface

Das Implementieren mehrerer Interfaces kann manchmal zu einem Konflikt zwischen den Member-Signaturen führen. Sie können solche Konflikte durch die *explizite Implementierung* eines Interface-Members auflösen, zum Beispiel so:

```

interface I1 { void Foo(); }
interface I2 { int Foo(); }

public class Widget : I1, I2
{
    public void Foo()    // implizite Implementierung
    {
        Console.Write ("Widget-Implementierung von I1.Foo");
    }

    int I2.Foo()    // explizite Implementierung von I2.Foo
    {
        Console.Write ("Widget-Implementierung von I2.Foo");
        return 42;
    }
}

```

Da sowohl I1 als auch I2 sich einander ausschließende Foo-Signaturen besitzen, implementiert Widget explizit die Methode Foo von I2. Damit können die beiden Methoden in einer Klasse gemeinsam existieren. Die einzige Möglichkeit, ein explizit implementiertes Member aufzurufen, ist ein Cast auf sein Interface:

```

Widget w = new Widget();
w.Foo();           // Widget-Implementierung von I1.Foo
((I1)w).Foo();     // Widget-Implementierung von I1.Foo
((I2)w).Foo();     // Widget-Implementierung von I2.Foo

```

Ein anderer Grund für explizit implementierte Interface-Member ist das Verbergen von Mitgliedern, die sehr spezialisiert sind und den normalen Anwender eher ablenken. So würde zum Beispiel ein Typ, der ISerializable implementiert, normalerweise vermeiden wollen, seine ISerializable-Member zu präsentieren, sofern nicht explizit auf dieses Interface gecastet wurde.

Virtuelles Implementieren von Interface-Mitgliedern

Ein implizit implementiertes Interface-Member ist standardmäßig versiegelt. Es muss in der Basisklasse als virtual oder abstract gekennzeichnet werden, damit man es überschreiben kann. Ein Aufruf des Interface-Members sowohl über die Basisklasse als auch über das Interface ruft die Subklassenimplementierung auf.

Ein explizit implementiertes Interface-Member kann nicht als `virtual` gekennzeichnet und auch nicht auf die übliche Art und Weise überschrieben werden. Es kann aber *reimplementiert* werden.

Reimplementieren eines Interface in einer Subklasse

Eine Subklasse kann jedes Interface-Member *reimplementieren*, das schon von einer Basisklasse implementiert wurde. Durch das Reimplementieren wird eine Member-Implementierung gekapert (wenn sie über das Interface aufgerufen wurde), was unabhängig davon funktioniert, ob das Member in der Basisklasse `virtual` ist oder nicht.

Im folgenden Beispiel implementiert `TextBox` das Member `IUndo`. `Undo` explizit, daher kann es nicht als `virtual` gekennzeichnet werden. Um es zu »überschreiben«, muss `RichTextBox` die Methode `Undo` von `IUndoable` reimplementieren:

```
public interface IUndoable { void Undo(); }

public class TextBox : IUndoable
{
    void IUndoable.Undo()
        => Console.WriteLine ("TextBox.Undo");
}

public class RichTextBox : TextBox, IUndoable
{
    public new void Undo()
        => Console.WriteLine ("RichTextBox.Undo");
}
```

Ein Aufruf des reimplementierten Members über das Interface sorgt für einen Aufruf der Implementierung in der Subklasse:

```
RichTextBox r = new RichTextBox();
r.Undo();           // RichTextBox.Undo
((IUndoable)r).Undo(); // RichTextBox.Undo
```

In diesem Fall wird `Undo` explizit implementiert. Implizit implementierte Member können auch reimplementiert werden, aber das wirkt sich nicht durchgängig aus, da ein Aufruf des Members über eine Basisklassenreferenz die Basisklassenimplementierung aufruft.

Default-Interface-Member

Ab C# 8 können Sie Interface-Member mit einer Default-Implementierung versehen und damit eine eigene Implementierung optional machen:

```
interface ILogger
{
    void Log (string text) => Console.WriteLine (text);
}
```

Das ist sehr hilfreich, wenn Sie ein Member einem Interface in einer oft eingesetzten Bibliothek hinzufügen wollen, ohne (möglicherweise tausendfach) vorhandene Implementierungen kaputtzumachen.

Default-Implementierungen sind immer explizit – definiert also eine Klasse, die ILogger implementiert, keine Log-Methode, kann diese nur über das Interface aufgerufen werden:

```
class Logger : ILogger { }
...
((ILogger)new Logger()).Log ("message");
```

Damit wird ein Problem bei mehrfacher Vererbung vermieden: Wird das gleiche Default-Member zwei Interfaces hinzugefügt, die eine Klasse implementiert, ist so immer klar, welches Member aufgerufen wird.

Interfaces können jetzt auch statische Member (einschließlich Feldern) definieren, auf die aus Code innerhalb von Default-Implementierungen zugegriffen werden kann:

```
interface ILogger
{
    void Log (string text) =>
        Console.WriteLine (Prefix + text);

    static string Prefix = "";
}
```

Weil Interface-Member implizit öffentlich sind, können Sie von außerhalb ebenfalls auf statische Member zugreifen:

```
ILogger.Prefix = "File log: ";
```

Das können Sie einschränken, indem Sie dem statischen Interface-Member einen Zugriffsmodifikator hinzufügen (wie `private`, `protected` oder `internal`).

Instanzfelder sind (immer noch) nicht erreichbar. Das passt zu dem Prinzip von Interfaces, *Verhalten* zu definieren, aber keinen *Status*.

Enums

Ein *Enum* ist ein spezieller Werttyp, der es Ihnen ermöglicht, eine Gruppe von benannten numerischen Konstanten zu definieren:

```
public enum BorderSide { Left, Right, Top, Bottom }
```

Sie können diesen Enum-Typ wie folgt verwenden:

```
BorderSide topSide = BorderSide.Top;  
bool isTop = (topSide == BorderSide.Top);    // true
```

Jedes Enum-Member hat einen zugrunde liegenden ganzzahligen Wert. Standardmäßig haben die zugrunde liegenden Werte den Typ `int`, und den Enum-Membnern werden die Konstanten 0, 1, 2 ... (in der Deklarationsabfolge) zugewiesen. Folgendermaßen können Sie einen anderen Ganzzahltypen angeben:

```
public enum BorderSide : byte { Left, Right, Top, Bottom }
```

Sie können auch für jedes Member einen expliziten ganzzahligen Wert festlegen:

```
public enum BorderSide : byte  
{ Left=1, Right=2, Top=10, Bottom=11 }
```

Der Compiler ermöglicht Ihnen ebenfalls, nur ein paar der Enum-Member explizit mit einer Zahl zu versehen. Die so nicht ausgezeichneten Member werden weiterhin ausgehend vom letzten expliziten Wert durchnummeriert. Das vorige Beispiel ist also äquivalent zu dem hier:

```
public enum BorderSide : byte  
{ Left=1, Right, Top=10, Bottom }
```

Enum-Konvertierungen

Sie können eine enum-Instanz mit einem expliziten Cast auf ihren zugrunde liegenden ganzzahligen Wert und wieder zurück umwandeln:

```
int i = (int) BorderSide.Left;
BorderSide side = (BorderSide) i;
bool leftOrRight = (int) side <= 2;
```

Auch ist es möglich, explizit einen Enum-Typ in einen anderen zu casten. Die Übersetzung nutzt dann die zugrunde liegenden ganzzahligen Werte für das Member.

Das numerische Literal 0 erhält eine spezielle Behandlung und erfordert keinen expliziten Cast:

```
BorderSide b = 0;    // kein Cast erforderlich
if (b == 0) ...
```

In diesem speziellen Beispiel hat BorderSide kein Member mit dem ganzzahligen Wert 0. Das führt nicht zu einem Fehler: Einer der Haken an Enums ist, dass der Compiler und die CLR die Zuweisung von Werten, die außerhalb des Bereichs der Member liegen, nicht verhindern:

```
BorderSide b = (BorderSide) 12345;
Console.WriteLine (b);           // 12345
```

Flag-Enums

Sie können Enum-Member kombinieren. Um Mehrdeutigkeiten zu vermeiden, müssen den Members eines kombinierbaren enum explizit Werte zugewiesen werden, üblicherweise Potenzen von zwei:

```
[Flags]
public enum BorderSides
{ None=0, Left=1, Right=2, Top=4, Bottom=8 }
```

Konventionsgemäß erhalten kombinierbare Enums einen Namen im Plural statt im Singular. Um mit kombinierten Enum-Werten zu arbeiten, verwenden Sie Operatoren, die bitweise arbeiten, zum Beispiel | und &. Diese Operatoren arbeiten mit den zugrunde liegenden ganzzahligen Werten:

```

BorderSides leftRight =
    BorderSides.Left | BorderSides.Right;

if ((leftRight & BorderSides.Left) != 0)
    Console.WriteLine ("Enthält Left");    // Enthält Left

string formatted = leftRight.ToString(); // "Left, Right"

BorderSides s = BorderSides.Left;
s |= BorderSides.Right;
Console.WriteLine (s == leftRight);      // True

```

Das Attribut `Flags` sollte auf kombinierbare Enum-Typen angewendet werden. Tun Sie das nicht, liefert der Aufruf von `ToString` auf einer enum-Instanz eine Zahl statt einer Folge von Namen.

Praktisch ist es, in eine Enum-Deklaration auch Kombinations-Member aufzunehmen:

```

[Flags] public enum BorderSides
{
    None=0,
    Left=1, Right=2, Top=4, Bottom=8,
    LeftRight = Left | Right,
    TopBottom = Top | Bottom,
    All = LeftRight | TopBottom
}

```

Enum-Operatoren

Folgende Operatoren arbeiten mit Enums:

```

=  ==  !=  <  >  <=  >=  +  -  ^  &  |  ~
+=  -=  ++  -  sizeof

```

Die bitweise arbeitenden, arithmetischen und Vergleichsoperatoren liefern das Ergebnis einer Verarbeitung der zugrunde liegenden ganzzahligen Werte zurück. Addition und Subtraktion sind zwischen einem enum und einem Integer-Typ erlaubt, aber nicht zwischen zwei enums.

Eingebettete Typen

Ein *eingebetteter Typ* wird im Geltungsbereich eines anderen Typs deklariert:

```
public class TopLevel
{
    public class Nested { }           // eingebettete Klasse
    public enum Color { Red, Blue, Tan } // eingebettetes Enum
}
```

Ein eingebetteter Typ hat folgende Merkmale:

- Er kann auf die privaten Member des umhüllenden Typs und alles andere zugreifen, auf das der umhüllende Typ Zugriff hat.
- Er kann aus allen möglichen Zugriffsmodifikatoren auswählen und muss sich nicht auf `public` und `internal` beschränken.
- Die Standardsichtbarkeit für einen eingebetteten Typ ist `private` und nicht `internal`.
- Der Zugriff auf einen eingebetteten Typ von außerhalb des umhüllenden Typs erfordert eine Qualifikation durch den Namen des umhüllenden Typs (wie beim Zugriff auf statische Member).

Um zum Beispiel von außerhalb unserer Klasse `TopLevel` auf `Color.Red` zugreifen zu können, müssten wir Folgendes schreiben:

```
TopLevel.Color color = TopLevel.Color.Red;
```

Alle Typen können eingebettet werden, allerdings kann nur in Klassen und Structs eingebettet werden.

Generics

C# verfügt über zwei separate Mechanismen, um Code zu schreiben, der mit verschiedenen Typen verwendbar ist: *Vererbung* und *Generics*. Während bei der Vererbung die Wiederverwendbarkeit durch einen Basistyp ausgedrückt wird, geschieht das bei Generics durch ein »Template«, das Typen als »Platzhalter« enthält. Generics können im Vergleich zur Vererbung die *Typsicherheit erhöhen* und *für weniger Casting und Boxing sorgen*.

Generische Typen

Ein *generischer Typ* deklariert *Typparameter* – Platzhaltertypen, die vom Anwender des generischen Typs gefüllt werden, indem er die *Typargumente* bereitstellt. Hier wurde ein generischer Typ `Stack<T>` entworfen, der Instanzen vom Typ `T` auf einem Stack verwalten soll. `Stack<T>` deklariert einen einzelnen Typparameter `T`:

```
public class Stack<T>
{
    int position;
    T[] data = new T[100];
    public void Push (T obj) => data[position++] = obj;
    public T Pop()          => data[--position];
}
```

Wir können `Stack<T>` so nutzen:

```
var stack = new Stack<int>();
stack.Push(5);
stack.Push(10);
int x = stack.Pop();          // x ist 10
int y = stack.Pop();          // y ist 5
```



Beachten Sie, dass in den beiden letzten Zeilen keine Downcasts erforderlich sind, was das Risiko eines Laufzeitfehlers und den Overhead der Boxing/Unboxing-Operationen vermeidet. Das macht unseren generischen Stack einem nichtgenerischen Stack überlegen, der `object` anstelle von `T` nutzt (ein Beispiel finden Sie unter »Der Typ `object`« auf Seite 104).

`Stack<int>` gibt für den Typparameter `T` das Typargument `int` vor, wodurch implizit ein Typ erstellt wird (die Synthese geschieht zur Laufzeit). `Stack<int>` hat im Endeffekt folgende Definition (die Ersetzungen sind hervorgehoben, und der Klassenname wurde durch `###` ersetzt, um Verwirrung zu vermeiden):

```
public class ###
{
    int position;
    int[] data = new int[100];
}
```

```

    public void Push (int obj) => data[position++] = obj;
    public int Pop()           => data[--position];
}

```

Technisch ausgedrückt, ist `Stack<T>` ein *offener Typ*, `Stack<int>` dagegen ein *geschlossener Typ*. Zur Laufzeit sind alle Instanzen generischer Typen geschlossen – ihre Typplatzhalter sind gefüllt.

Generische Methoden

Eine *generische Methode* deklariert Typparameter innerhalb ihrer Signatur. Mit generischen Methoden können viele grundlegende Algorithmen sehr allgemein implementiert werden. Hier sehen Sie eine generische Methode, die zwei Werte eines beliebigen Typs `T` vertauscht:

```

static void Swap<T> (ref T a, ref T b)
{
    T temp = a; a = b; b = temp;
}

```

`Swap<T>` kann wie folgt verwendet werden:

```

int x = 5, y = 10;
Swap (ref x, ref y);

```

Im Allgemeinen ist es nicht notwendig, Typargumente an eine generische Methode zu übergeben, da der Compiler den Typ implizit ermitteln kann. Würde das zu einer Mehrdeutigkeit führen, können generische Methoden mit dem Typargument aufgerufen werden:

```

Swap<int> (ref x, ref y);

```

Innerhalb eines generischen Typs ist eine Methode so lange nicht-generisch, wie sie nicht selbst Typparameter *einführt* (mit den spitzen Klammern). In unserem generischen `Stack` nutzt die Methode `Pop` nur den schon im Typ bestehenden Typparameter `T` und ist daher nicht als generische Methode klassifiziert.

Methoden und Typen sind die einzigen Konstrukte, die Typparameter einführen können. Eigenschaften, Indexer, Events, Felder, Operatoren und so weiter können keine Typparameter deklarieren, auch wenn sie die Typparameter verwenden können, die vom um-

hüllenden Typ deklariert wurden. In unserem Beispiel mit dem generischen Stack könnten wir zum Beispiel einen Indexer schreiben, der ein generisches Element zurückgibt:

```
public T this [int index] { get { return data[index]; } }
```

Auch Konstruktoren können nur die vorhandenen Typparameter nutzen, aber nicht selbst welche einführen.

Deklariieren generischer Parameter

Typparameter können bei der Deklaration von Klassen, Structs, Interfaces, Delegates (siehe »Delegates« auf Seite 130) und Methoden eingeführt werden. Ein generischer Typ bzw. eine generische Methode kann mehrere durch Komma getrennte Typparameter haben:

```
class Dictionary<TKey, TValue> {...}
```

Die Instanziierung erfolgt folgendermaßen:

```
var myDict = new Dictionary<int,string>();
```

Generische Typnamen und Methodennamen können überladen werden, solange sich die Anzahl der Typparameter unterscheidet. So kommen zum Beispiel die folgenden drei Typnamen nicht miteinander in Konflikt:

```
class A {}  
class A<T> {}  
class A<T1,T2> {}
```



Es ist üblich, bei generischen Typen und Methoden mit einem einzelnen Typparameter diesen Parameter *T* zu nennen, solange der Sinn des Parameters klar ist. Bei mehreren Typparametern beginnt jeder Parameter mit *T*, hat aber einen stärker beschreibenden Namen.

typeof und ungebundene generische Typen

Zur Laufzeit gibt es keine offenen generischen Typen: Offene generische Typen werden bei der Kompilation geschlossen. Aber es kann zur Laufzeit *ungebundene* generische Typen geben – nur als Type-

Objekt. Einen ungebundenen generischen Typ können Sie in C# nur mit dem `typeof`-Operator angeben:

```
class A<T> {}  
class A<T1,T2> {}  
...  
  
Type a1 = typeof (A<>); // ungebundener Typ  
Type a2 = typeof (A<,>); // zeigt 2 Typargumente an  
Console.Write (a2.GetGenericArguments().Count()); // 2
```

Sie können den `typeof`-Operator nutzen, um einen geschlossenen Typ anzugeben:

```
Type a3 = typeof (A<int,int>);
```

oder einen offenen Typ (der zur Laufzeit geschlossen ist):

```
class B<T> { void X() { Type t = typeof (T); } }
```

Der generische Wert default

Das Schlüsselwort `default` kann genutzt werden, um den Standardwert für einen angegebenen generischen Typparameter zu erhalten. Der Standardwert für einen Referenztyp ist `null`, der für Werttypen ist das Ergebnis eines bitweisen Löschsens der Felder des Typs:

```
static void Zap<T> (T[] array)  
{  
    for (int i = 0; i < array.Length; i++)  
        array[i] = default(T);  
}
```

Ab C# 7.1 können Sie das Typargument weglassen, wenn der Compiler es sich selbst ableiten kann:

```
array[i] = default;
```

Generische Constraints

Standardmäßig kann ein Typparameter durch jeden beliebigen Typ ersetzt werden. *Constraints* können auf einen Typparameter angewandt werden, um spezifischere Typargumente zu verlangen. Es gibt acht Arten von Constraints:

```

where T : Basisklasse // Basisklassen-Constraint
where T : Interface   // Interface-Constraint
where T : class        // Referenztyp-Constraint
where T : class?       // (siehe "Nullbare Referenztypen"
                        // auf Seite 167)

where T : struct       // Werttyp-Constraint
where T : unmanaged    // unmanageder Constraint
where T : new()        // parameterloser Konstruktor-
                        // Constraint

where U : T            // Typ-Constraint
where T : notnull      // nicht nullbare Werttypen
                        // oder nicht nullbare Referenztypen

```

Im folgenden Beispiel verlangt `GenericClass<T,U>`, dass `T` von `SomeClass` abgeleitet ist (oder der Klasse selbst entspricht) und `Interface1` implementiert, und verlangt außerdem, dass `U` einen parameterlosen Konstruktor anbietet:

```

class    SomeClass {}
interface Interface1 {}

class GenericClass<T,U> where T : SomeClass, Interface1
                        where U : new()
{ ... }

```

Constraints können überall dort angewendet werden, wo generische Parameter definiert sind, sowohl in Methoden als auch in Typdefinitionen.

Ein *Basisklassen-Constraint* verlangt, dass der Typparameter eine Subklasse einer bestimmten Klasse sein muss (oder die Klasse selbst); ein *Interface-Constraint* fordert, dass der Typparameter dieses Interface implementieren muss. Diese Constraints gestatten es, dass Instanzen des Typparameters implizit in diese Klasse oder dieses Interface umgewandelt werden.

Der *class-Constraint* und der *struct-Constraint* erfordern, dass `T` ein Referenztyp bzw. ein (nicht nullbarer) Werttyp ist. Der *ungemanagte* Constraint ist eine strengere Version eines struct-Constraints: `T` muss ein einfacher Werttyp oder ein Struct sein, das (rekursiv) frei von Referenztypen ist. Der *parameterlose Konstruktor-Constraint* erfordert, dass `T` einen öffentlichen parameterlosen Konstruktor hat, und erlaubt Ihnen, `new()` auf `T` aufzurufen:

```
static void Initialize<T> (T[] array) where T : new()
{
    for (int i = 0; i < array.Length; i++)
        array[i] = new T();
}
```

Der *Typ-Constraint* verlangt, dass der eine Typparameter von einem anderen Typparameter abgeleitet ist bzw. ihm entspricht.

Erstellen abgeleiteter Klassen von generischen Typen

Von einer generischen Klasse können Klassen genauso abgeleitet werden wie von einer nichtgenerischen Klasse. Die Subklasse kann den Typparameter der Basisklasse offen lassen:

```
class Stack<T> { ... }
class SpecialStack<T> : Stack<T> { ... }
```

Es ist aber auch möglich, den generischen Typparameter durch einen konkreten Typ zu schließen:

```
class IntStack : Stack<int> { ... }
```

Ein Subtyp kann auch eigene, neue generische Argumente mitbringen:

```
class List<T> { ... }
class KeyedList<T, TKey> : List<T> { ... }
```

Selbstreferenzielle generische Deklarationen

Ein Typ kann *sich selbst* als konkreten Typ nennen, wenn er ein Typargument schließt:

```
public interface IEquatable<T> { bool Equals (T obj); }

public class Balloon : IEquatable<Balloon>
{
    public bool Equals (Balloon b) { ... }
}
```

Folgendes ist ebenfalls zulässig:

```
class Foo<T> where T : IComparable<T> { ... }
class Bar<T> where T : Bar<T> { ... }
```

Statische Daten

Statische Daten sind für jeden geschlossenen Typ getrennt vorhanden:

```
Console.WriteLine (++Bob<int>.Count);    // 1
Console.WriteLine (++Bob<int>.Count);    // 2
Console.WriteLine (++Bob<string>.Count); // 1
Console.WriteLine (++Bob<object>.Count); // 1

class Bob<T> { public static int Count; }
```

Kovarianz



Kovarianz und Kontravarianz sind fortgeschrittene Konzepte. Sie wurden in C# eingeführt, damit sich generische Interfaces und Generics (insbesondere die, die in .NET definiert werden, wie `IEnumerable<T>`) eher so verhalten, wie Sie es erwarten würden. Das können Sie nutzen, ohne dass Sie die Einzelheiten hinter Kovarianz und Kontravarianz verstehen müssen.

Angenommen, A kann in B umgewandelt werden, dann hat X einen kovarianten Typparameter, wenn `X<A>` in `X<B>` umgewandelt werden kann.

(Im Sinne dessen, was C# unter Varianz versteht, bedeutet »umwandelbar« umwandelbar über eine *implizite Referenzumwandlung* – d. h., A ist eine Subklasse von B oder A implementiert B. Numerische Konvertierungen, Boxing und benutzerdefinierte Konvertierungen sind nicht eingeschlossen.)

Beispielsweise hat Typ `IFoo<T>` eine Kovariante T, wenn Folgendes zulässig ist:

```
IFoo<string> s = ...;
IFoo<object> b = s;
```

Interfaces (und Delegates) gestatten Kovarianz für Typparameter. Nehmen Sie zu Illustrationszwecken an, dass die `Stack<T>`-Klasse, die wir am Anfang dieses Abschnitts geschrieben haben, das folgende Interface implementiert:

```
public interface IPoppable<out T> { T Pop(); }
```

Der `out`-Modifikator auf `T` zeigt an, dass `T` nur auf Ausgabeseite genutzt wird (also als Rückgabebetyp von Methoden). Er markiert das Interface als *kovariant* und gestattet uns dieses:

```
// Vorausgesetzt, Bear ist eine Subklasse von Animal:  
var bears = new Stack<Bear>();  
bears.Push (new Bear());  
  
// Da bears IPoppable<Bear> implementiert,  
// können wir es in IPoppable<Animal>: umwandeln.  
IPoppable<Animal> animals = bears; // Erlaubt  
Animal a = animals.Pop();
```

Der Cast von `bears` auf `animals` wird vom Compiler gestattet, weil der Typparameter des Interface kovariant ist.



Die Interfaces `IEnumerator<T>` und `IEnumerable<T>` (siehe »Enumeration und Iteratoren« auf Seite 157) sind mit einem kovarianten `T` markiert. Das gestattet Ihnen beispielsweise, `IEnumerable<string>` auf `IEnumerator<object>` zu casten.

Der Compiler meldet einen Fehler, wenn Sie einen kovarianten Typparameter auf Eingabeseite nutzen (z. B. als Parameter für eine Methode oder für eine schreibbare Eigenschaft). Der Grund für diese Einschränkung liegt darin, dass nur so zur Kompilierzeit die Typsicherheit garantiert werden kann. Beispielsweise hindert sie uns daran, diesem Interface eine `Push(T)`-Methode hinzuzufügen, die Benutzer missbrauchen könnten, um ein `Camel`-Objekt auf unser `IPoppable<Animal>` zu schieben (denken Sie daran, dass der zugrunde liegende Typ in unserem Beispiel ein `Stack` mit Bären ist). Wenn Sie eine `Push(T)`-Methode definieren wollen, muss `T` *kontravariant* sein.



C# unterstützt Kovarianz (und Kontravarianz) nur für Elemente mit *Referenzumwandlungen* – nicht für *Boxing-Umwandlungen*. Hätten Sie eine Methode geschrieben, die einen Parameter des Typs `IPoppable<object>` akzeptiert, könnten Sie diese mit `IPoppable<string>` aufrufen, aber nicht mit `IPoppable<int>`.

Kontravarianz

Wir haben zuvor gesehen, dass, die Möglichkeit einer impliziten Referenzumwandlung von A in B vorausgesetzt, ein Typ X einen kovarianten Typparameter hat, wenn $X\langle A \rangle$ eine Referenzumwandlung in $X\langle B \rangle$ zulässt. Ein Typ ist *kontravariant*, wenn eine Konvertierung in die entgegengesetzte Richtung möglich ist – von $X\langle B \rangle$ in $X\langle A \rangle$. Das wird auf Interfaces und Delegates unterstützt, wenn der Typparameter nur auf der Eingabeseite erscheint, was mit dem Modifikator `in` angezeigt wird. Erweitern wir unser letztes Beispiel: Wenn die Klasse `Stack<T>` das folgende Interface implementiert:

```
public interface IPushable<in T> { void Push (T obj); }
```

können wir Folgendes tun:

```
IPushable<Animal> animals = new Stack<Animal>();  
IPushable<Bear> bears = animals; // zulässig  
bears.Push (new Bear());
```

Wie bei der Kovarianz meldet der Compiler einen Fehler, wenn Sie einen kontravarianten Typparameter auf Ausgabeseite nutzen (z. B. als Rückgabetyp oder als Typ einer lesbaren Eigenschaft).

Delegates

Ein Delegate verbindet einen Aufrufer einer Methode zur Laufzeit mit seiner Zielmethode. Es gibt zwei Ausprägungen bei einem Delegate: *Typ* und *Instanz*. Ein *Delegate-Typ* definiert ein Protokoll, an das sich Aufrufender und Ziel halten und das aus einer Liste von Parametertypen und einem Rückgabetyp besteht. Eine *Delegate-Instanz* ist ein Objekt, das sich auf eine oder mehrere Zielmethoden bezieht, die diesem Protokoll entsprechen.

Eine Delegate-Instanz funktioniert als Delegierter für den Aufrufen: Der Aufrufende wendet sich an das Delegate, und das Delegate ruft die Zielmethode auf. Diese Indirektion entkoppelt den Aufrufen von der Zielmethode.

Die Deklaration eines Delegate-Typs beginnt mit dem Schlüsselwort `delegate`, sieht aber ansonsten wie eine (abstrakte) Methodendeklaration aus:

```
delegate int Transformer (int x);
```

Um eine Delegate-Instanz zu erzeugen, können Sie einer Delegate-Variablen eine Methode zuweisen:

```
Transformer t = Square; // erstellt die Delegate-Instanz  
int result = t(3);      // ruft Delegate auf  
Console.Write (result); // 9  
  
int Square (int x) => x * x;
```

Das Aufrufen eines Delegate ist wie das Aufrufen einer Methode (da der Zweck des Delegate ist, nur eine Indirektionsschicht einzuschieben):

```
t(3);
```

Die Anweisung `Transformer t = Square` ist eine Kurzform für:

```
Transformer t = new Transformer (Square);
```

Und `t(3)` ist eine Kurzform für:

```
t.Invoke (3);
```

Ein Delegate entspricht einem *Callback*, einem allgemeinen Begriff, der solche Konstrukte wie C-Funktionszeiger beschreibt.

Schreiben von Plug-in-Methoden mit Delegates

Einer Delegate-Variablen wird eine Methode *zur Laufzeit* zugewiesen. Das ist nützlich, wenn man Plug-in-Methoden schreiben will. In diesem Beispiel haben wir eine Hilfsmethode namens `Transform`, die eine Transformation auf jedes Element in einem Integer-Array anwendet. Die Methode `Transform` bietet einen Delegate-Parameter, um eine Plug-in-Transformation anzugeben.

```
int[] values = { 1, 2, 3 };  
Transform (values, Square); // Hook in die Square-Methode  
  
foreach (int i in values)  
    Console.Write (i + " "); // 1 4 9  
  
void Transform (int[] values, Transformer t)  
{  
    for (int i = 0; i < values.Length; i++)  
        values[i] = t (values[i]);  
}
```

```
}  
  
int Square (int x) => x * x;  
  
delegate int Transformer (int x);
```

Instanzmethoden versus statische Methoden als Ziele

Wenn einem Delegate-Objekt eine *Instanzmethode* zugewiesen wird, muss es nicht nur die Referenz auf die *Methode*, sondern auch die auf die *Instanz* festhalten, zu der die Methode gehört. Die Target-Eigenschaft der Klasse `System.Delegate` repräsentiert diese Instanz (und ist null, wenn das Delegate eine statische Methode referenziert).

Multicast-Delegates

Alle Delegate-Instanzen haben eine *Multicast-Fähigkeit*. Das bedeutet, dass eine Delegate-Instanz nicht nur auf eine einzelne Zielmethode verweisen kann, sondern auf eine ganze Liste davon. Die Operatoren `+` und `+=` verbinden Delegate-Instanzen:

```
SomeDelegate d = SomeMethod1;  
d += SomeMethod2;
```

Funktionell entspricht diese Zeile Folgendem:

```
d = d + SomeMethod2;
```

Ruft man `d` auf, werden nun sowohl `SomeMethod1` als auch `SomeMethod2` aufgerufen. Delegates werden in der Reihenfolge ausgeführt, in der sie hinzugefügt wurden.

Die Operatoren `-` und `-=` entfernen den rechten Delegate-Operanden vom linken Delegate-Operanden:

```
d -= SomeMethod1;
```

Ruft man `d` erneut auf, wird nur noch `SomeMethod2` ausgeführt.

Ein Aufruf von `+` oder `+=` auf einer Delegate-Variablen mit dem Wert `null` ist ebenso zulässig wie der Aufruf von `-=` auf einer Delegate-Variablen mit nur einem Ziel (was dazu führt, dass die Delegate-Instanz `null` wird).



Delegates sind *unveränderlich*. Wenn Sie += oder -= nutzen, erstellen Sie eigentlich also eine *neue* Delegate-Instanz und weisen diese der vorhandenen Variablen zu.

Wenn ein Multicast-Delegate einen Rückgabewert hat, der nicht void ist, erhält der Aufrufende den Rückgabewert der letzten ausgeführten Methode. Die vorigen Methoden werden zwar auch aufgerufen, ihre Rückgabewerte aber verworfen. In den meisten Szenarien, in denen Multicast-Delegates verwendet werden, haben diese den Rückgabotyp void, daher spielt dieses Detail dort keine Rolle.

Alle Delegate-Typen erben implizit von `System.MulticastDelegate`, das von `System.Delegate` erbt. C# wandelt die Operationen +, -, += und -= für ein Delegate in die statischen Methoden `Combine` und `Remove` der Klasse `System.Delegate` um.

Generische Delegate-Typen

Ein Delegate-Typ kann generische Typparameter enthalten:

```
public delegate T Transformer<T> (T arg);
```

So könnten wir diesen Delegate-Typ verwenden:

```
Transformer<double> s = Square;  
Console.WriteLine (s (3.3));           // 10.89
```

```
double Square (double x) => x * x;
```

Die Func- und Action-Delegates

Generische Delegates ermöglichen es, einen kleinen Satz von Delegates zu schreiben, die so allgemein sind, dass sie mit Methoden beliebigen Rückgabetyps und einer bliebig Anzahl (einer vernünftigen Anzahl) von Argumenten funktionieren. Diese Delegates sind die Func- und Action-Delegates, die im Namensraum `System` definiert werden (die in- und out-Modifikatoren zeigen die Varianz an, die wir weiter oben betrachtet haben):

```

delegate TResult Func <out TResult> ();
delegate TResult Func <in T, out TResult> (T arg);
delegate TResult Func <in T1, in T2, out TResult>
    (T1 arg1, T2 arg2);
... und so weiter bis T16

delegate void Action ();
delegate void Action <in T> (T arg);
delegate void Action <in T1, in T2> (T1 arg1, T2 arg2);
... und so weiter bis T16

```

Diese Delegates sind äußerst allgemein. Das Transformer-Delegate in unserem letzten Beispiel kann durch ein Func-Delegate ersetzt werden, das ein Argument des Typs T akzeptiert und einen Wert des gleichen Typs liefert:

```

public static void Transform<T> (
    T[] values, Func<T,T> transformer)
{
    for (int i = 0; i < values.Length; i++)
        values[i] = transformer (values[i]);
}

```

Die einzigen praktischen Szenarien, die von diesen Delegates nicht abgedeckt werden, sind ref/out und Zeigerparameter.

Delegate-Kompatibilität

Delegate -Typen sind untereinander alle inkompatibel, selbst wenn ihre Signaturen gleich sind:

```

delegate void D1(); delegate void D2();
...
D1 d1 = Method1;
D2 d2 = d1;           // Kompilierungsfehler

```

Folgendes ist jedoch zulässig:

```

D2 d2 = new D2 (d1);

```

Delegate-Instanzen werden als gleich betrachtet, wenn sie den gleichen Typ und die gleichen Methodenziele haben. Bei Multicast-Delegates ist die Abfolge der Methodenziele relevant.

Rückgabotypvarianz

Wenn Sie eine Methode aufrufen, erhalten Sie eventuell einen Rückgabotyp, der spezifischer ist als das, wonach Sie gefragt haben. Dementsprechend kann eine Delegate-Zielmethode einen spezifischeren Typ liefern als den, der vom Delegate beschrieben wird. Das ist eine Form der *Kovarianz*:

```
ObjectRetriever o = new ObjectRetriever (RetriveString);  
object result = o();  
Console.WriteLine (result);      // Hallo  
  
string RetrieveString() => "Hallo";  
  
delegate object ObjectRetriever();
```

Der ObjectRetriever erwartet als Rückgabewert object, aber eine object-*Subklasse* tut es auch, weil Delegate-Rückgabotypen kovariant sind.

Parametervarianz

Wenn Sie eine Methode aufrufen, können Sie Argumente mitgeben, die spezifischere Typen haben als die Parameter dieser Methode. Das ist ganz normales polymorphes Verhalten. Dementsprechend kann eine Delegate-Zielmethode weniger spezifische Parameter haben, als vom Delegate beschrieben werden. Das nennt man *Kontravarianz*:

```
StringAction sa = new StringAction (ActOnObject);  
sa ("Hallo");  
  
void ActOnObject (object o) => Console.WriteLine (o);  
  
delegate void StringAction (string s);
```



Das Standard-Event-Muster ist dazu gedacht, Ihnen beim Hervorheben der Kontravarianz durch die Verwendung der gebräuchlichen Basisklasse EventArgs zu helfen. Sie können zum Beispiel eine einzelne Methode durch zwei verschiedene Delegates aufrufen lassen, wobei das eine ein MouseEventArgs und das andere ein KeyEventArgs übergibt.

Typparametervarianz bei generischen Delegates

Sie haben im Abschnitt »Generics« auf Seite 121 gesehen, dass Typparameter für generische Interfaces kovariant und kontravariant sein können. Generische Delegates haben die gleichen Eigenschaften. Wenn Sie einen generischen Delegate-Typ definieren, sollten Sie deswegen Folgendes tun:

- Markieren Sie Typparameter, die nur für Rückgabewerte genutzt werden, als kovariant (**out**).
- Markieren Sie Typparameter, die nur für Parameter genutzt werden, als kontravariant (**in**).

Diese Schritte ermöglichen es, dass sich Umwandlungen natürlich verhalten, weil Vererbungsbeziehungen zwischen Typen berücksichtigt werden. Das folgende Delegate (das im Namensraum `System` definiert wird) ist kovariant für `TResult`:

```
delegate TResult Func<out TResult>();
```

Es macht das hier möglich:

```
Func<string> x = ...;  
Func<object> y = x;
```

Das folgende Delegate (das im Namensraum `System` definiert wird) ist kontravariant für `T`:

```
delegate void Action<in T>(T arg);
```

Es ermöglicht das hier :

```
Action<object> x = ...;  
Action<string> y = x;
```

Events

Wenn man Delegates nutzt, gibt es zwei Rollen, die sehr häufig im Spiel sind: *Broadcaster* und *Subscriber*. Der Broadcaster ist ein Typ, der ein Delegate-Feld enthält. Er entscheidet, wann ein Broadcast verschickt werden soll, indem er das Delegate aufruft. Die Subscriber sind die Zielmethodenempfänger. Ein Subscriber entscheidet, von wann bis wann er zuhört, indem er `+=` und `-=` für das Delegate

des Broadcasters verwendet. Ein Subscriber weiß nichts von anderen Subscribern, kommt mit ihnen aber auch nicht in Konflikt.

Events sind ein Sprachfeature, das dieses Muster formalisiert. Ein event ist ein Konstrukt, das nur genau die Untermenge an Delegate-Features bereitstellt, die für das Broadcaster/Subscriber-Modell notwendig sind. Der Hauptzweck von Events ist, Subscriber davon abzuhalten, sich untereinander ins Gehege zu kommen.

Der einfachste Weg, ein Event zu deklarieren, ist, einem Delegate-Member das Schlüsselwort `event` voranzustellen:

```
public class Broadcaster
{
    public event ProgressReporter Progress;
}
```

Code innerhalb des Typs Broadcaster hat vollständigen Zugriff auf Progress und kann es wie ein Delegate behandeln. Code außerhalb von Broadcaster kann nur die Operationen `+=` und `-=` auf Progress anwenden.

Im folgenden Beispiel löst die Klasse Stock ihr PriceChanged-Event jedes Mal aus, wenn sich der Price von Stock ändert:

```
public delegate void PriceChangedHandler
(decimal oldPrice, decimal newPrice);

public class Stock
{
    string symbol; decimal price;

    public Stock (string symbol) => this.symbol = symbol;

    public event PriceChangedHandler PriceChanged;

    public decimal Price
    {
        get => price;
        set
        {
            if (price == value) return;
            // Event absetzen, wenn Aufrufliste nicht leer:
            if (PriceChanged != null)
```

```

        PriceChanged (price, value);
    price = value;
    }
}
}

```

Hätten wir das Schlüsselwort `event` aus unserem Beispiel entfernt, sodass `PriceChanged` ein normales Delegate-Feld geworden wäre, würde unser Beispiel zum gleichen Ergebnis führen. Aber `Stock` wäre dann weniger robust, denn `Subscriber` könnten Folgendes tun, um sich gegenseitig zu beeinflussen:

- Ersetzen anderer `Subscriber` durch erneutes Zuweisen an `PriceChanged` (statt den Operator `+=` zu nutzen)
- Entfernen aller `Subscriber` (indem `PriceChanged` auf `null` gesetzt wird)
- Absetzen eines Broadcast an andere `Subscriber`, indem das Delegate aufgerufen wird

Events können virtuell, überschrieben, abstrakt oder versiegelt sein. Sie können auch statisch sein.

Standard-Event-Muster

Bei so gut wie allen in der .NET-Bibliothek definierten Events orientiert sich deren Definition an einem Standardmuster, um in der Bibliothek und im Benutzercode für Konsistenz zu sorgen. So sieht das vorige Beispiel aus, wenn man es auf dieses Muster umbaut:

```

public class PriceChangedEventArgs : EventArgs
{
    public readonly decimal LastPrice, NewPrice;

    public PriceChangedEventArgs (decimal lastPrice,
                                  decimal newPrice)
    {
        LastPrice = lastPrice; NewPrice = newPrice;
    }
}

public class Stock
{
    string symbol; decimal price;
}

```

```

public Stock (string symbol) => this.symbol = symbol;

public event EventHandler<PriceChangedEventArgs>
    PriceChanged;

protected virtual void OnPriceChanged
    (PriceChangedEventArgs e) =>
    // Kurzform zum Aufruf von PriceChanged, wenn nicht null:
    PriceChanged?.Invoke (this, e);

public decimal Price
{
    get { return price; }
    set
    {
        if (price == value) return;
        OnPriceChanged (new PriceChangedEventArgs (price,value));
        price = value;
    }
}
}

```

Das Herzstück des Standard-Event-Musters ist `System.EventArgs`, eine vordefinierte .NET-Klasse ohne Member (abgesehen vom statischen Feld `Empty`). `EventArgs` ist eine Basisklasse für das Übermitteln von Informationen an ein Event. In diesem Beispiel bilden wir eine Subklasse von `EventArgs`, um den alten und den neuen Preis weiterzugeben, wenn das Event `PriceChanged` ausgelöst wird.

Das generische Delegate `System.EventHandler` ist ebenfalls Teil von .NET Core und wie folgt definiert:

```

public delegate void EventHandler<TEventArgs>
    (object source, TEventArgs e)

```



Vor C# 2.0 (als Generics hinzukamen) schrieb man stattdessen für jeden `EventArgs`-Typ ein eigenes Delegate zum Event-Handling:

```

delegate void PriceChangedHandler
    (object sender,
     PriceChangedEventArgs e);

```

Aus historischen Gründen verwenden die meisten Events in den .NET-Bibliotheken Delegates, die auf diese Art und Weise definiert wurden.

Eine geschützte (protected) virtuelle Methode namens *On-Event-name* ist zentral für das Auslösen des Events verantwortlich. Damit können Subklassen das Event feuern (was meist erwünscht ist) und vor oder nach dem Auslösen des Events Code einfügen.

So könnten wir unsere Klasse Stock nutzen:

```
Stock stock = new Stock ("THPW");
stock.Price = 27.10M;

stock.PriceChanged += stock_PriceChanged;
stock.Price = 31.59M;

static void stock_PriceChanged
    (object sender, PriceChangedEventArgs e)
{
    if ((e.NewPrice - e.LastPrice) / e.LastPrice > 0.1M)
        Console.WriteLine ("Achtung, Preissteigerung von 10%!");
}
```

Bei Events, die keine zusätzlichen Informationen übermitteln, stellt .NET noch ein nichtgenerisches Delegate namens *EventHandler* bereit. Wir können das demonstrieren, indem wir die Klasse Stock so umschreiben, dass das Event PriceChanged ausgelöst wird, *nachdem* sich der Preis geändert hat. Das bedeutet, dass keine zusätzlichen Informationen mit dem Event übermittelt werden müssen:

```
public class Stock
{
    string symbol; decimal price;

    public Stock (string symbol) => this.symbol = symbol;

    public event EventHandler PriceChanged;

    protected virtual void OnPriceChanged (EventArgs e) =>
PriceChanged?.Invoke (this, e);

    public decimal Price
    {
        get => price;
        set
        {
            if (price == value) return;
            price = value;
        }
    }
}
```

```

        OnPriceChanged (EventArgs.Empty);
    }
}

```

Beachten Sie, dass wir hier auch die Eigenschaft `EventArgs.Empty` verwendet haben – dadurch ersparen wir uns das Erzeugen einer Instanz von `EventArgs`.

Event-Accessors

Die *Accessors* eines Events sind die Implementierungen seiner Funktionen `+=` und `-=`. Standardmäßig werden Accessors *implizit* vom Compiler implementiert. Die Eventdeklaration

```
public event EventHandler PriceChanged;
```

wandelt der Compiler um in

- ein privates Delegate-Feld und
- ein öffentliches Paar Event-Accessor-Funktionen, deren Implementierungen die Operationen `+=` und `-=` an das private Delegate-Feld weiterleiten.

Sie können diesen Prozess selbst in die Hand nehmen, indem Sie *explizit* Event-Accessors definieren. So sieht eine manuelle Implementierung des Events `PriceChanged` aus unserem vorigen Beispiel aus:

```

EventHandler priceChanged; // privates Delegate
public event EventHandler PriceChanged
{
    add { priceChanged += value; }
    remove { priceChanged -= value; }
}

```

Funktionell entspricht dieses Beispiel der Standard-Accessor-Implementierung von C# (davon abgesehen, dass C# zusätzlich die Threadsicherheit um die Delegate-Aktualisierung herum sicherstellt). Wenn wir unsere Event-Accessors selbst definieren, weisen wir C# an, die standardmäßige Feld- und Accessor-Logik nicht zu generieren.

Bei expliziten Event-Accessors können Sie komplexere Strategien zum Speichern und Ansprechen des zugrunde liegenden Delegate anwenden. Das ist nützlich, wenn die Event-Accessors bloße Weiterleitungen für eine andere Klasse sind, die das Event ausstrahlt, oder wenn man explizit ein Interface implementiert, das ein Event deklariert :

```
public interface IFoo { event EventHandler Ev; }
class Foo : IFoo
{
    EventHandler ev;
    event EventHandler IFoo.Ev
    {
        { add { ev += value; } remove { ev -= value; }
        }
    }
}
```

Lambda-Ausdrücke

Ein *Lambda-Ausdruck* ist eine Methode ohne Namen, die anstelle einer Delegate-Instanz geschrieben wird. Der Compiler wandelt den Lambda-Ausdruck direkt um, und zwar

- in eine Delegate-Instanz oder
- in einen *Expression Tree* vom Typ `Expression<TDelegate>`, der den Code im Lambda-Ausdruck in einem traversierbaren Objektmodell repräsentiert. Damit können Lambda-Ausdrücke zur Laufzeit interpretiert werden.

Im folgenden Beispiel ist `x => x * x` ein Lambda-Ausdruck:

```
Transformer sqr = x => x * x;
Console.WriteLine (sqr(3));    // 9

delegate int Transformer (int i);
```



Intern löst der Compiler Lambda-Ausdrücke dieses Typs auf, indem er eine private Methode erstellt und den Code des Ausdrucks in diese Methode kopiert.

Ein Lambda-Ausdruck hat die folgende Form:

(Parameter) => Ausdruck-oder-Anweisungsblock

Aus Gründen der Bequemlichkeit können Sie die Klammern weglassen, wenn es genau einen Parameter eines ableitbaren Typs gibt.

In unserem Beispiel gibt es einen einzelnen Parameter `x`, und der Ausdruck ist `x * x`:

```
x => x * x;
```

Jeder Parameter des Lambda-Ausdrucks entspricht einem Delegate-Parameter, und der Typ des Ausdrucks (der `void` sein kann) entspricht dem Rückgabetyp des Delegate.

In unserem Beispiel entspricht `x` dem Parameter `i` und der Ausdruck `x * x` dem Rückgabetyp `int`, was kompatibel zum Delegate `Transformer` ist.

Der Code eines Lambda-Ausdrucks kann statt eines Ausdrucks auch einen Anweisungsblock enthalten. Wir können unser Beispiel wie folgt umschreiben:

```
x => { return x * x; };
```

Lambda-Ausdrücke werden am häufigsten mit den `Func`- und `Action`-Delegates eingesetzt – meist werden Ihnen die oben formulierten Ausdrücke also in folgender Form begegnen:

```
Func<int,int> sqr = x => x * x;
```

In der Regel kann der Compiler den Typ der Lambda-Parameter aus dem Kontext *erschließen*. Wenn das nicht der Fall ist, können Sie die Typen der Parameter explizit angeben:

```
Func<int,int> sqr = (int x) => x * x;
```

Hier ist ein Beispiel für einen Ausdruck, der zwei Parameter erwartet:

```
Func<string,string,int> totalLength =  
    (s1, s2) => s1.Length + s2.Length;  
  
int total = totalLength ("hallo", "welt"); // total=9;
```

Wenn Clicked ein Event des Typs EventHandler ist, bindet folgender Code über einen Lambda-Ausdruck einen Event-Handler daran:

```
obj.Clicked += (sender,args) => Console.Write ("Click");
```

Dies ist ein Beispiel für einen Ausdruck, der keine Argumente übernimmt:

```
Func<string> greeter = () => "Hallo Welt";
```

Mit C# 10 erlaubt der Compiler ein implizites Typisieren mit Lambda-Ausdrücken, die über die Delegates Func und Action aufgelöst werden können, daher lässt sich diese Anweisung wie folgt verkürzen:

```
var greeter = () => "Hallo Welt";
```

Besitzt der Lambda-Ausdruck Argumente, müssen Sie deren Typen angeben, um var nutzen zu können:

```
var sqr = (int x) => x * x;
```

Der Compiler ermittelt für sqr den Typ Func<int,int>.

Äußere Variablen übernehmen

Ein Lambda-Ausdruck kann beliebige Variablen referenzieren, die dort erreichbar sind, wo der Lambda-Ausdruck definiert wird. Diese werden als *äußere Variablen* bezeichnet und können lokale Variablen, Parameter und Felder enthalten:

```
int factor = 2;
Func<int, int> multiplier = n => n * factor;
Console.WriteLine (multiplier (3));           // 6
```

Äußere Variablen, die von einem Lambda-Ausdruck referenziert werden, bezeichnet man als *übernommene Variablen*. Einen Lambda-Ausdruck, der Variablen einfängt, nennt man eine *Closure*. Übernommene Variablen werden ausgewertet, wenn das Delegate tatsächlich *aufgerufen* wird, nicht wenn die Variablen *übernommen* werden:

```
int factor = 2;
Func<int, int> multiplier = n => n * factor;
factor = 10;
Console.WriteLine (multiplier (3));           // 30
```

Lambda-Ausdrücke können übernommene Variablen verändern:

```
int seed = 0;
Func<int> natural = () => seed++;
Console.WriteLine (natural());           // 0
Console.WriteLine (natural());           // 1
Console.WriteLine (seed);                 // 2
```

Die Lebenszeit übernommener Variablen wird auf die des Delegate ausgeweitet. Im folgenden Beispiel würde die lokale Variable `seed` normalerweise die Geltung verlieren, wenn die Ausführung von `Natural` beendet würde. Aber da `seed` übernommen wurde, wurde ihre Lebenszeit auf die des übernehmenden Delegate `natural` ausgedehnt:

```
static Func<int> Natural()
{
    int seed = 0;
    return () => seed++;           // liefert eine Closure
}
Func<int> natural = Natural();
Console.WriteLine (natural());     // 0
Console.WriteLine (natural());     // 1
```

Variablen können auch von anonymen Methoden oder lokalen Methoden übernommen werden. Die Regeln für die übernommenen Variablen sind gleich.

Statische Lambdas

Seit C# 9 können Sie sicherstellen, dass ein Lambda-Ausdruck, eine lokale Funktion oder eine anonyme Methode den Status nicht einfängt, indem Sie das Schlüsselwort `static` verwenden. Das kann in Mikrooptimierungsszenarien nützlich sein, um die (möglicherweise ungewollte) Speicherreservierung und das Aufräumen eines Closures zu vermeiden. Wir können beispielsweise den `static`-Modifikator wie folgt auf einen Lambda-Ausdruck anwenden:

```
Func<int, int> multiplier = static n => n * 2;
```

Würden wir später versuchen, den Lambda-Ausdruck so zu verändern, dass er eine lokale Variable einfängt, würde der Compiler einen Fehler erzeugen. Dieses Feature ist in lokalen Methoden noch

nützlicher (weil ein Lambda-Ausdruck selbst schon Speicher reserviert). Im folgenden Beispiel kann die Methode `Multiply` nicht auf die Variable `factor` zugreifen:

```
void Foo()
{
    int factor = 123;
    static int Multiply (int x) => x * 2;
}
```

Die Verwendung von `static` ist hier zudem als Dokumentationswerkzeug nützlich, da sie auf eine geringere Kopplung hinweist. Statische Lambdas können trotzdem auf statische Variablen und Konstanten zugreifen (weil für diese kein Closure erforderlich ist).



Das Schlüsselwort `static` dient eher der *Prüfung* – es hat keinen Effekt auf die IL, die der Compiler erzeugt. Ohne das Schlüsselwort `static` generiert der Compiler auch nur dann ein Closure, wenn er es muss (und selbst dann kennt er Tricks, um die Kosten gering zu halten).

Iterationsvariablen übernehmen

Wenn Sie die Iterationsvariable einer `for`-Schleife übernehmen, behandelt C# die Iterationsvariable so, als wäre sie außerhalb der Schleife deklariert worden. Das bedeutet, dass bei jeder Iteration dieselbe Variable übernommen wird. Das folgende Programm gibt 333 statt 012 aus:

```
Action[] actions = new Action[3];

for (int i = 0; i < 3; i++)
    actions [i] = () => Console.Write (i);

foreach (Action a in actions) a();    // 333
```

Alle Closures (fett gedruckt) übernehmen dieselbe Variable, nämlich `i`. (Das ist sinnvoll, wenn Sie bedenken, dass `i` eine Variable ist, deren Wert über die Schleifeniterationen erhalten bleibt; Sie können den Wert von `i` im Schleifenrumpf sogar explizit ändern, wenn

Sie wollen.) Die Folge ist, dass alle Delegates, wenn sie später aufgerufen werden, den Wert von `i` zur Zeit des Aufrufs sehen – und der ist 3. Dass stattdessen 012 ausgegeben wird, erreichen Sie, indem Sie die Iterationsvariable einer lokalen Variablen zuweisen, die nur innerhalb der Schleife gültig ist:

```
Action[] actions = new Action[3];
for (int i = 0; i < 3; i++)
{
    int loopScopedi = i;
    actions [i] = () => Console.Write (loopScopedi);
}
foreach (Action a in actions) a();    // 012
```

Das bewirkt, dass die Closure bei jeder Iteration eine andere Variable übernimmt.

Beachten Sie, dass die Iterationsvariable (seit C# 5) in einer `foreach`-Schleife implizit lokal ist, sodass Sie sie problemlos übernehmen können, ohne eine temporäre Variable zu benötigen.

Lambda-Ausdrücke versus lokale Methoden

Die Funktionalität der lokalen Methoden (siehe »Lokale Methoden« auf Seite 78) und von Lambda-Ausdrücken ist teilweise gleich. Lokale Methoden sind im Vorteil, da sie Rekursion zulassen und durch sie kein Delegate spezifiziert werden muss. Durch Letzteres wird zudem eine Indirektion vermieden, und sie werden so ein kleines bisschen effizienter. Darüber hinaus kann man in ihnen auf die lokalen Variablen der umgebenden Methode zugreifen, ohne dass der Compiler diese in einer verborgenen Klasse vorhalten muss.

Aber in vielen Fällen brauchen Sie ein Delegate, vor allem beim Aufruf einer Funktion höherer Ordnung (also einer Methode mit einem Delegate-typisierten Parameter):

```
public void Foo (Func<int,bool> predicate) { ... }
```

In solchen Fällen kommen Sie um ein Delegate nicht herum, und genau dort sind Lambda-Ausdrücke im Allgemeinen auch knapper und sauberer.

Anonyme Methoden

Anonyme Methoden sind eine C# 2.0-Einrichtung, die im Wesentlichen von Lambda-Ausdrücken ersetzt wurde. Eine anonyme Methode ähnelt einem Lambda-Ausdruck, dem implizit typisierte Parameter, die Ausdruckssyntax (anonyme Methoden müssen immer einen Anweisungsblock haben) und die Fähigkeit fehlen, zu einem Ausdrucksbaum kompiliert zu werden. Um eine anonyme Methode zu schreiben, nutzen Sie das Schlüsselwort `delegate`, gefolgt von einer (optionalen) Parameterdeklaration und einem Methodenrumpf. Zum Beispiel:

```
Transformer sqr = delegate (int x) {return x * x;};  
Console.WriteLine (sqr(3));           // 9
```

```
delegate int Transformer (int i);
```

Die erste Zeile ist semantisch äquivalent zum folgenden Lambda-Ausdruck:

```
Transformer sqr =      (int x) => {return x * x;};
```

Oder einfacher:

```
Transformer sqr =      x => x * x;
```

Eine einzigartige Eigenschaft anonymer Methoden ist, dass Sie die Parameterdeklaration vollständig weglassen können – selbst wenn das Delegate sie erwartet. Das kann bei der Deklaration von Events mit einem leeren Standard-Handler nützlich sein:

```
public event EventHandler Clicked = delegate { };
```

Das hebt die Notwendigkeit der Nullprüfung vor dem Absetzen des Events auf. Folgendes ist ebenfalls zulässig (beachten Sie das Fehlen der Parameter):

```
Clicked += delegate { Console.Write ("clicked"); };
```

Anonyme Methoden übernehmen äußere Variablen genau so, wie es Lambda-Ausdrücke tun.

try-Anweisungen und Exceptions

Eine try-Anweisung definiert einen Codeblock, der einer Fehlerbehandlung untersteht oder nach dem auf jeden Fall aufgeräumt werden soll. Auf den try-Block müssen einer oder mehrere catch-Blöcke, ein abschließender finally-Block oder beides folgen. Der finally-Block wird ausgeführt, wenn der try-Block verlassen wird (oder – falls vorhanden – der catch-Block), um Code auszuführen, der auf jeden Fall am Ende laufen soll, egal ob ein Fehler aufgetreten ist oder nicht.

Ein catch-Block hat Zugriff auf ein Exception-Objekt, das Informationen über den Fehler enthält. Sie verwenden einen catch-Block, um entweder den Fehler zu beheben oder die Exception *weiterzuwerfen*. Sie werfen dann eine Exception weiter, wenn Sie das Problem nur protokollieren oder einen neuen, übergeordneteren Exception-Typ werfen wollen.

Ein finally-Block sorgt für mehr Determinismus in Ihrem Programm, indem er immer ausgeführt wird – egal was passiert ist. Das ist nützlich, wenn Sie Aufräumarbeiten zu erledigen haben, zum Beispiel das Schließen von Netzwerkverbindungen.

Eine try-Anweisung sieht so aus:

```
try
{
    ... // Exception wird eventuell beim Ausführen dieses
        // Blocks geworfen
}
catch (ExceptionA ex)
{
    ... // kümmere dich um Exception vom Typ ExceptionA
}
catch (ExceptionB ex)
{
    ... // kümmere dich um Exception vom Typ ExceptionB
}
finally
{
    ... // Code zum Aufräumen
}
```

Betrachten Sie den folgenden Code:

```
int x = 3, y = 0;  
Console.WriteLine (x / y);
```

Da `y` null ist, löst die Runtime eine `DivideByZeroException` aus und bricht damit die Ausführung des Codes ab. Das können wir verhindern, indem wir diese Exception folgendermaßen abfangen:

```
try  
{  
    int x = 3, y = 0;  
    Console.WriteLine (x / y);  
}  
catch (DivideByZeroException)  
{  
    Console.Write ("y darf nicht null sein. ");  
}  
// Hier wird die Ausführung nach der Exception wieder  
// aufgenommen ...
```



Das ist ein einfaches Beispiel zur Erläuterung des Exception-Handling. Dieses spezielle Szenario könnten wir in der Praxis erheblich besser bewältigen, indem wir explizit den Teiler prüfen, bevor wir `Calc` aufrufen.

Die Verarbeitung von Exceptions ist recht teuer; sie nimmt Hunderte von CPU-Zyklen in Anspruch.

Wenn eine Exception innerhalb einer `try`-Anweisung geworfen wird, führt die CLR eine Prüfung durch:

Befindet sich das Programm gerade in einer `try`-Anweisung, die die Exception mit einem passenden `catch`-Block fangen kann?

- Wenn das der Fall ist, wird die Ausführung an den passenden `catch`-Block übergeben. Konnte der `catch`-Block erfolgreich abgearbeitet werden, wird mit der Ausführung der nächsten Anweisung nach der `try`-Anweisung fortgefahren (falls vorhanden, wird vorher noch der `finally`-Block ausgeführt).
- Wenn das nicht der Fall ist, springt die Ausführung zum aufrufenden Code der Funktion zurück, und die Prüfung wird wie-

derholt (nachdem eventuell vorhandene `finally`-Blöcke ausgeführt wurden, die die Anweisung umschließen).

Übernimmt keine Funktion die Verantwortung für die Ausführung, wird dem Benutzer eine Fehlermeldung angezeigt, und das Programm wird beendet.

Die catch-Klausel

Eine `catch`-Klausel legt fest, welche Typen von Exceptions gefangen werden sollen. Diese müssen entweder vom Typ `System.Exception` oder einer Subklasse davon sein. Mit dem Fangen von `System.Exception` fängt man alle möglichen Fehler. Das ist nützlich, wenn

- Ihr Programm möglicherweise weiterlaufen kann – unabhängig vom speziellen Exception-Typ –,
- Sie vorhaben, die Exception weiterzuwerfen (eventuell nach dem Protokollieren), oder
- Ihre Fehlerbehandlung der letzte Schritt vor dem Abbrechen des Programms ist.

Normalerweise aber fangen Sie *bestimmte Exception-Typen*, um zu vermeiden, sich mit Umständen herumschlagen zu müssen, für die Ihre Fehlerbehandlung nicht ausgelegt ist (zum Beispiel eine `OutOfMemoryException`).

Sie können mehrere Exception-Typen mit mehreren `catch`-Klauseln behandeln:

```
try
{
    DoSomething();
}
catch (IndexOutOfRangeException ex) { ... }
catch (FormatException ex)          { ... }
catch (OverflowException ex)        { ... }
```

Nur eine `catch`-Klausel wird für eine gegebene Exception ausgeführt. Wenn Sie ein Sicherheitsnetz spannen wollen, in dem Sie auch allgemeinere Exceptions fangen (zum Beispiel `System.Exception`), müssen Sie die spezifischeren Handler an den Anfang stellen.

Eine Exception kann gefangen werden, ohne eine Variable anzugeben, wenn Sie nicht auf ihre Eigenschaften zugreifen müssen:

```
catch (OverflowException)    // keine Variable
{ ... }
```

Zudem können Sie sowohl die Variable als auch den Typ weglassen (und damit alle Exceptions fangen):

```
catch { ... }
```

Exception-Filter

Sie können in einer catch-Klausel einen *Exception-Filter* definieren, indem Sie eine when-Klausel hinzufügen:

```
catch (WebException ex)
    when (ex.Status == WebExceptionStatus.Timeout)
{
    ...
}
```

Wird in diesem Beispiel eine WebException geworfen, wird der boolesche Ausdruck nach dem Schlüsselwort when ausgewertet. Ist das Ergebnis false, wird der fragliche catch-Block ignoriert, und die folgenden catch-Klauseln werden berücksichtigt. Mit Exception-Filtern kann es sinnvoll sein, den gleichen Exception-Typ mehrfach zu fangen:

```
catch (WebException ex) when (ex.Status == etwas)
{ ... }
catch (WebException ex) when (ex.Status == etwas_anderes)
{ ... }
```

Der boolesche Ausdruck in der when-Klausel kann Nebeneffekte haben, wie zum Beispiel eine Methode, die die Exception zu Diagnosezwecken protokolliert.

Der finally-Block

Ein finally-Block wird immer ausgeführt – egal, ob eine Exception geworfen wurde oder nicht und ob der try-Block bis zum Ende lief oder nicht. finally-Blöcke werden meist genutzt, um dort Code unterzubringen, der dem Aufräumen dienen soll.

Ein `finally`-Block wird ausgeführt,

- wenn ein `catch`-Block fertig ist,
- nachdem der `try`-Block durch einen Sprungbefehl verlassen wurde (zum Beispiel durch `return` oder `goto`) oder
- wenn der `try`-Block beendet ist.

Durch einen `finally`-Block kann ein Programm zuverlässiger werden. Im folgenden Beispiel wird die Datei, die wir geöffnet haben, *immer* geschlossen, egal ob

- der `try`-Block normal beendet wurde,
- die Ausführung vorzeitig endet, weil die Datei leer ist (`EndOfStream`), oder
- eine `IOException` beim Lesen der Datei geworfen wird.

Das kann zum Beispiel so aussehen:

```
static void ReadFile()
{
    StreamReader reader = null; // im Namensraum System.IO
    try
    {
        reader = File.OpenText ("file.txt");
        if (reader.EndOfStream) return;
        Console.WriteLine (reader.ReadToEnd());
    }
    finally
    {
        if (reader != null) reader.Dispose();
    }
}
```

In diesem Beispiel haben wir die Datei durch den Aufruf von `Dispose` für den `StreamReader` geschlossen. Der Aufruf von `Dispose` für ein Objekt in einem `finally`-Block ist eine Standardvorgehensweise in .NET und wird in C# explizit durch die `using`-Anweisung unterstützt.

Die `using`-Anweisung

Viele Klassen kapseln nicht verwaltete (unmanaged) Ressourcen, zum Beispiel Datei-Handles, Grafik-Handles oder Datenbankver-

bindungen. Diese Klassen implementieren `System.IDisposable`, das eine einzige parameterlose Methode namens `Dispose` definiert, um diese Ressourcen aufzuräumen. Die Anweisung `using` stellt eine elegante Syntax für das Aufrufen der `Dispose`-Methode eines `IDisposable`-Objekts in einem `finally`-Block bereit.

Das hier

```
using (StreamReader reader = File.OpenText ("file.txt"))
{
    ...
}
```

ist exakt äquivalent zu dem:

```
{
    StreamReader reader = File.OpenText ("file.txt");
    try
    {
        ...
    }
    finally
    {
        if (reader != null) ((IDisposable)reader).Dispose();
    }
}
```

using-Deklarationen

Lassen Sie die geschweiften Klammern und den Anweisungsblock nach einer `using`-Anweisung weg, wird diese zu einer `using`-Deklaration (seit C# 8). Die Ressource wird dann verworfen, wenn die Ausführung den umschließenden Anweisungsblock verlässt:

```
if (File.Exists ("file.txt"))
{
    using var reader = File.OpenText ("file.txt");
    Console.WriteLine (reader.ReadLine());
    ...
}
```

In diesem Beispiel wird `reader` verworfen, wenn die Ausführung nach dem `if`-Anweisungsblock weitergeht.

Werfen von Exceptions

Exceptions können entweder von der Laufzeitumgebung oder aus dem Anwendungscode heraus geworfen werden. Hier wirft Display eine `System.ArgumentNullException`:

```
static void Display (string name)
{
    if (name == null)
        throw new ArgumentNullException (nameof (name));

    Console.WriteLine (name);
}
```

Ausdrücke werfen

Seit C# 7 kann `throw` als Ausdruck in Expression-bodied Funktionen auftauchen:

```
public string Foo() => throw new NotImplementedException();
```

Ein `throw`-Ausdruck kann ebenfalls in einem ternären bedingten Ausdruck erscheinen:

```
string ProperCase (string value) =>
    value == null ? throw new ArgumentException ("value") :
    value == "" ? "" :
    char.ToUpper (value[0]) + value.Substring (1);
```

Eine Exception weiterwerfen

Sie können eine Exception fangen und sie dann weiterwerfen:

```
try { ... }
catch (Exception ex)
{
    // Fehler protokollieren
    ...
    throw; // gleiche Exception erneut werfen
}
```

Wenn Sie eine Exception auf diesem Weg weiterwerfen, können Sie einen Fehler protokollieren, ohne ihn zu »verschlucken«. Zudem können Sie sich so auch aus dem Verarbeiten einer Exception zurückziehen, wenn die Umstände anders sind, als von Ihnen erwartet.



Würden wir `throw` durch `throw ex` ersetzen, würde das Beispiel immer noch funktionieren, aber die `StackTrace`-Eigenschaft der `Exception` würde den ursprünglichen Fehler nicht mehr widerspiegeln.

Ein weiteres verbreitetes Szenario ist das Auslösen eines spezifischeren oder aussagekräftigeren `Exception`-Typs:

```
try
{
    ... // ein Geburtsdatum aus einem XML-Element parsen
}
catch (FormatException ex)
{
    throw new XmlException ("Falsches Geburtsdatum", ex);
}
```

Wenn eine andere `Exception` ausgelöst wird, können Sie die `InnerException`-Eigenschaft mit der ursprünglichen `Exception` füllen, um das Debugging zu vereinfachen. Fast alle `Exception`-Typen bieten einen Konstruktor zu diesem Zweck (wie in unserem Beispiel).

Wichtige Eigenschaften von `System.Exception`

Die wichtigsten Eigenschaften von `System.Exception` sind folgende:

`StackTrace`

Ein `String`, der alle Methoden repräsentiert, die aufgerufen sind – vom Ursprung der `Exception` bis zum `catch`-Block.

`Message`

Ein `String` mit einer Beschreibung des Fehlers.

`InnerException`

Die innere `Exception` (wenn vorhanden), die die äußere `Exception` verursacht hat. Diese innere `Exception` kann wieder eine weitere `InnerException` haben.

Enumeration und Iteratoren

Enumeration

Ein *Enumerator* ist ein schreibgeschützter Cursor über eine *Folge von Werten*, der sich nur vorwärts bewegen kann. C# behandelt einen Typ als Enumerator, wenn er eine der folgenden Bedingungen erfüllt:

- Besitzt eine öffentliche, parameterlose Methode namens `MoveNext` und eine Eigenschaft namens `Current`.
- Implementiert `System.Collections.Generic.IEnumerator<T>`.
- Implementiert `System.Collections.IEnumerator`.

Die `foreach`-Anweisung iteriert über ein *enumerierbares* Objekt. Ein enumerierbares Objekt ist eine logische Repräsentation einer Folge. Es ist nicht selbst ein Cursor, sondern ein Objekt, das einen Cursor über sich selbst stellen kann. C# behandelt einen Typ als enumerierbar, wenn er eine der folgenden Bedingungen erfüllt (die Prüfung geschieht in dieser Reihenfolge):

- Besitzt eine öffentliche, parameterlose Methode namens `GetEnumerator`, die einen Enumerator zurückliefert.
- Implementiert `System.Collections.Generic.IEnumerable<T>`.
- Implementiert `System.Collections.IEnumerable`.
- (Seit C# 9) Kann an eine *Erweiterungsmethode* namens `GetEnumerator` binden, die einen Enumerator zurückgibt (siehe Abschnitt »Erweiterungsmethoden« auf Seite 169).

Das Enumerierungsmuster sieht wie folgt aus:

```
class Enumerator    // implementiert üblicherweise
                    // IEnumerator<T>
{
    public IteratorVariableType Current { get {...} }
    public bool MoveNext() {...}
}
class Enumerable    // implementiert in der Regel IEnumerable<T>
{
    public Enumerator GetEnumerator() {...}
}
```

So sieht die High-Level-Variante des Iterierens über die Zeichen im Wort *beer* mithilfe einer foreach-Anweisung aus:

```
foreach (char c in "beer") Console.WriteLine (c);
```

So sieht die Low-Level-Variante des Iterierens über die Zeichen im Wort *beer* ohne Verwendung der foreach-Anweisung aus:

```
using (var enumerator = "beer".GetEnumerator())
while (enumerator.MoveNext())
{
    var element = enumerator.Current;
    Console.WriteLine (element);
}
```

Wenn der Enumerator IDisposable implementiert, fungiert die foreach-Anweisung auch als using-Anweisung und entsorgt implizit das Enumerator-Objekt.

Collection-Initialisierer

Sie können ein enumerierbares Objekt in einem Schritt erstellen und füllen, zum Beispiel so:

```
using System.Collections.Generic;
...

List<int> list = new List<int> {1, 2, 3};
```

Der Compiler übersetzt die letzte Zeile in Folgendes:

```
List<int> list = new List<int>();
list.Add (1); list.Add (2); list.Add (3);
```

Das verlangt, dass das enumerierbare Objekt das Interface `System.Collections.IEnumerable` implementiert und über eine `Add`-Methode verfügt, die die erforderliche Anzahl an Parametern für den Aufruf bietet. Dictionaries (also Typen, die `System.Collections.IDictionary` implementieren) können Sie ähnlich initialisieren:

```
var dict = new Dictionary<int, string>()
{
    { 5, "fünf" },
    { 10, "zehn" }
};
```

Oder neuer:

```
var dict = new Dictionary<int, string>()
{
    [5] = "fünf",
    [10] = "zehn"
};
```

Diese zweite Form ist nicht nur bei Dictionaries möglich, sondern bei jedem Typ, für den ein Indexer existiert.

Iteratoren

Während die `foreach`-Anweisung ein *Konsument* eines Enumerators ist, ist ein Iterator ein *Produzent* davon. In diesem Beispiel nutzen wir einen Iterator, um eine Folge von Fibonacci-Zahlen zurückzugeben (bei der jede Zahl der Summe ihrer beiden Vorgänger entspricht):

```
foreach (int fib in Fibs (6))
    Console.Write (fib + " ");

IEnumerable<int> Fibs (int fibCount)
{
    for (int i=0, prevFib=1, curFib=1; i<fibCount; i++)
    {
        yield return prevFib;
        int newFib = prevFib+curFib;
        prevFib = curFib;
        curFib = newFib;
    }
}

AUSGABE: 1 1 2 3 5 8
```

Während eine `return`-Anweisung »Hier hast du den Wert, den diese Methode zurückgeben soll« ausdrückt, besagt eine `yield return`-Anweisung: »Hier ist das nächste Element, das dieser Enumerator erzeugen soll.« Bei jeder `yield`-Anweisung wird die Kontrolle an den Aufrufenden zurückgegeben, der Zustand des Aufgerufenen bleibt aber bestehen, sodass die Methode an dieser Stelle fortfahren kann, sobald der Aufrufende nach dem nächsten Element fragt. Die Lebenszeit dieses Zustands ist an den Enumerator gebunden, sodass er freigegeben werden kann, wenn der Aufrufende mit dem Enumerieren fertig ist.



Der Compiler wandelt Iteratormethoden in private Klassen um, die `IEnumerable<T>` und/oder `IEnumerator<T>` implementieren. Die Logik im Iteratorblock wird »umgekehrt« und auf die Methode `MoveNext` und die Eigenschaft `Current` der vom Compiler erzeugten Enumerator-Klasse verteilt, die damit im Prinzip zu einer Zustandsmaschine wird. Das bedeutet, dass Sie beim Aufruf einer Iteratormethode nur die generierte Klasse instanziiieren – Ihr Code wird noch nicht ausgeführt! Das geschieht erst, wenn Sie damit beginnen, über die Ergebnissequenz zu enumerieren, im Allgemeinen mit einer `foreach`-Anweisung.

Iteratorsemantik

Ein Iterator ist eine Methode, eine Eigenschaft oder ein Indexer, die bzw. der eine oder mehrere `yield`-Anweisungen enthält. Ein Iterator muss eines der folgenden vier Interfaces zurückgeben (ansonsten wird der Compiler einen Fehler melden):

```
System.Collections.IEnumerable  
System.Collections.IEnumerator  
System.Collections.Generic.IEnumerable<T>  
System.Collections.Generic.IEnumerator<T>
```

Iteratoren, die ein *Enumerator*-Interface liefern, werden in der Regel nicht so oft verwendet. Sie sind nützlich, wenn man eine eigene Collection-Klasse schreibt: Im Allgemeinen nennen Sie den Iterator `GetEnumerator` und lassen Ihre Klasse `IEnumerable<T>` implementieren.

Iteratoren, die ein *Enumerable*-Interface liefern, sind verbreiteter und einfacher zu nutzen, weil man keine Collection-Klasse schreiben muss. Der Compiler erzeugt hinter den Kulissen eine private Klasse, die `IEnumerable<T>` (und `IEnumerator<T>`) implementiert.

Mehrere `yield`-Anweisungen

Ein Iterator kann mehrere `yield`-Anweisungen haben:

```
foreach (string s in Foo())  
    Console.Write (s + " ");    // Eins Zwei Drei  
  
IEnumerable<string> Foo()
```

```

{
    yield return "Eins";
    yield return "Zwei";
    yield return "Drei";
}

```

yield break

Eine return-Anweisung ist in einem Iteratorblock nicht erlaubt – stattdessen müssen Sie mit der Anweisung `yield break` festlegen, dass der Iteratorblock vorzeitig verlassen werden soll, ohne weitere Elemente zurückzugeben. Wir können die Methode `Foo` anpassen, um das zu demonstrieren:

```

static IEnumerable<string> Foo (bool breakEarly)
{
    yield return "Eins";
    yield return "Zwei";
    if (breakEarly) yield break;
    yield return "Drei";
}

```

Sequenzen kombinieren

Iteratoren sind sehr gut kombinierbar. Wir können unser Fibonacci-Beispiel erweitern, indem wir der Klasse folgende Methode hinzufügen:

```

IEnumerable<int> EvenNumbersOnly (
    IEnumerable<int> sequence)
{
    foreach (int x in sequence)
        if ((x % 2) == 0)
            yield return x;
}

```

Dann können wir gerade Fibonacci-Zahlen folgendermaßen ausgeben:

```

foreach (int fib in EvenNumbersOnly (Fibs (6)))
    Console.Write (fib + " "); // 2 8

```

Jedes Element wird erst im letzten Moment berechnet – dann, wenn es von einer `MoveNext()`-Operation angefordert wird. Abbildung 5 zeigt die Datenanforderung und die Datenausgabe über die Zeit.

Die Kombinierbarkeit des Iteratormusters ist ein notwendiger Baustein beim Erstellen von LINQ-Abfragen.

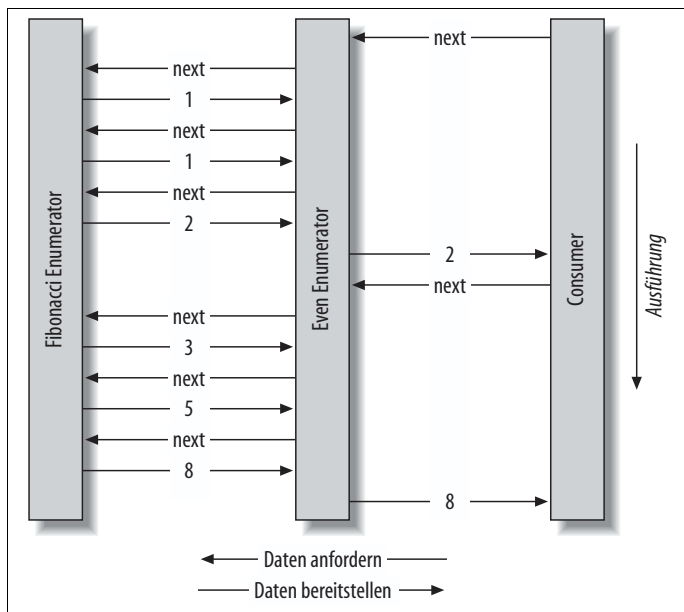


Abbildung 5: Sequenzen kombinieren

Nullbare Werttypen

Referenztypen können einen nicht existierenden Wert durch eine Nullreferenz darstellen. Bei Werttypen ist das allerdings normalerweise nicht möglich:

```
string s = null;    // Okay - Referenztyp
int i = null;       // Kompilierungsfehler -
                   // int kann nicht null sein
```

Um null (nicht 0) in einem Werttyp zu repräsentieren, müssen Sie ein spezielles Konstrukt namens *nullbarer Typ* nutzen. Ein nullba-

rer Typ wird durch einen Werttyp mit einem Fragezeichen ? dahinter kenntlich gemacht:

```
int? i = null;           // Okay - nullbarer Typ
Console.WriteLine (i == null); // True
```

Das Struct Nullable<T>

T? wird in System.Nullable<T> umgewandelt. Nullable<T> ist ein leichtgewichtiges, unveränderliches Struct, das nur zwei Felder für Value und HasValue besitzt. Leicht gekürzt, ist System.Nullable<T> sehr einfach:

```
public struct Nullable<T> where T : struct
{
    public T Value {get;}
    public bool HasValue {get;}
    public T GetValueOrDefault();
    public T GetValueOrDefault (T defaultValue);
    ...
}
```

Der Code

```
int? i = null;
Console.WriteLine (i == null);           // True
```

wird vom Compiler umgewandelt in

```
Nullable<int> i = new Nullable<int>();
Console.WriteLine (! i.HasValue);       // True
```

Wenn man versucht, Value auszulesen, wenn HasValue false ist, wird eine InvalidOperationException geworfen. GetValueOrDefault() liefert Value zurück, wenn HasValue true ist – ansonsten gibt es new T() oder einen angegebenen Standardwert zurück.

Der Standardwert von T? ist null.

Nullbare Konvertierungen

Die Konvertierung von T nach T? ist implizit, von T? nach T dagegen explizit:

```
int? x = 5;           // implizit
int y = (int)x;       // explizit
```

Der explizite Cast entspricht einem Aufruf der Eigenschaft `Value` des nullbaren Objekts. Wenn daher `HasValue` den Wert `false` hat, wird eine `InvalidOperationException` geworfen.

Boxing/Unboxing nullbarer Werte

Wenn `T?` geboxt ist, enthält der geboxte Wert auf dem Heap `T`, nicht `T?`. Diese Optimierung ist möglich, weil ein geboxter Wert ein Referenztyp ist, der schon null darstellen kann.

C# gestattet außerdem das Unboxing nullbarer Typen mit dem `as`-Operator. Das Ergebnis ist `null`, wenn der Cast fehlschlägt:

```
object o = "string";
int? x = o as int?;
Console.WriteLine (x.HasValue);    // False
```

Übernommene Operatoren

Das Struct `Nullable<T>` definiert keine Operatoren wie `<`, `>` oder sogar `==`. Trotzdem lässt sich der folgende Code kompilieren und korrekt ausführen:

```
int? x = 5;
int? y = 10;
bool b = x < y;           // True
```

Das funktioniert, weil der Compiler den Kleiner-als-Operator vom zugrunde liegenden Werttyp ausleiht oder »übernimmt«. Semantisch gesehen, wird der vorige Vergleichsausdruck in das hier umgewandelt:

```
bool b = (x.HasValue && y.HasValue)
        ? (x.Value < y.Value)
        : false;
```

Anders ausgedrückt: Wenn sowohl `x` als auch `y` Werte haben, werden sie über den Kleiner-als-Operator von `int` verglichen, ansonsten wird `false` zurückgegeben.

Das Übernehmen von Operatoren bedeutet, dass Sie implizit die Operatoren von `T` für `T?` nutzen können. Sie können Operatoren für `T?` definieren, um ein besonderes Null-Verhalten zu bieten, aber in

einem Großteil der Fälle ist es am besten, sich darauf zu verlassen, dass der Compiler automatisch eine systematische nullbare Logik für Sie anwendet.

Wie der Compiler die Null-Logik ausführt, ist von der Kategorie des Operators abhängig.

Gleichheitsoperatoren (==, !=)

Übernommene Gleichheitsoperatoren behandeln null, wie es Referenztypen tun. Das bedeutet, dass zwei Null-Werte gleich sind:

```
Console.WriteLine (      null ==      null); // True
Console.WriteLine ((bool?)null == (bool?)null); // True
```

Außerdem:

- Wenn genau ein Operand null ist, sind alle Operanden ungleich.
- Wenn beide Operanden nicht null sind, werden ihre Values verglichen.

Relationale Operatoren (<, <=, >=, >)

Die relationalen Operatoren basieren auf dem Prinzip, dass es unsinnig ist, Null-Operanden zu vergleichen. Das bedeutet, dass ein Vergleich eines Null-Werts mit einem anderen Null- oder Nicht-Null-Wert false zurückliefert.

```
bool b = x < y;    // Übersetzung:

bool b = (x == null || y == null)
    ? false
    : (x.Value < y.Value);

// b ist False (wenn x 5 und y null)
```

Alle anderen Operatoren (+, -, *, /, %, &, |, ^, <<, >>, ++, --, !, ~)

Diese Operatoren liefern null zurück, wenn einer der Operanden null ist. Das Muster sollte SQL-Anwendern vertraut sein.

```
int? c = x + y;    // Translation:

int? c = (x == null || y == null)
```

```
? null  
: (int?) (x.Value + y.Value);
```

// c ist null (wenn x 5 und y null ist)

Eine Ausnahme tritt auf, wenn die Operatoren & und | auf bool? angewandt werden. Diese werden wir uns gleich ansehen.

Mischen von nullbaren und nicht nullbaren Typen

Sie können nullbare und nicht nullbare Typen kombinieren und vergleichen (das funktioniert, weil es eine implizite Konvertierung von T nach T? gibt):

```
int? a = null;  
int b = 2;  
int? c = a + b;    // c ist null - entspricht a + (int?)b
```

bool? mit den Operatoren & und |

Wenn Operanden den Typ bool? haben, behandeln die Operatoren & und | null als unbekannten Wert. Daher ist null | true true, denn:

- Wenn der unbekannte Wert false wäre, würde das Ergebnis true sein.
- Wenn der unbekannte Wert true wäre, würde das Ergebnis true sein.

Genauso ist null & false false. Dieses Verhalten wird SQL-Anwendern vertraut vorkommen. Das folgende Beispiel zeigt andere Kombinationen:

```
bool? n = null, f = false, t = true;  
Console.WriteLine (n | n);    // (null)  
Console.WriteLine (n | f);    // (null)  
Console.WriteLine (n | t);    // True  
Console.WriteLine (n & n);    // (null)  
Console.WriteLine (n & f);    // False  
Console.WriteLine (n & t);    // (null)
```

Nullbare Typen und Null-Operatoren

Nullbare Typen arbeiten besonders gut mit dem Operator ?? zusammen (siehe »Null-Verbindungsoperator« auf Seite 60), zum Beispiel:

```
int? x = null;
int y = x ?? 5;    // y ist 5

int? a = null, b = null, c = 123;
Console.WriteLine (a ?? b ?? c);    // 123
```

Die Verwendung von ?? auf einen nullbaren Werttyp entspricht einem Aufruf von `GetValueOrDefault` mit einem expliziten Standardwert, nur dass der Ausdruck für den Standardwert niemals ausgewertet wird, wenn die Variable nicht null ist.

Nullbare Typen sind auch gut für den Einsatz mit dem Null-Bedingungsoperator (siehe »Null-Bedingungsoperator« auf Seite 60) zu gebrauchen. Im folgenden Beispiel wird `length` zu null ausgewertet:

```
System.Text.StringBuilder sb = null;
int? length = sb?.ToString().Length;
```

Darüber hinaus können wir das mit dem Null-Verbindungsoperator kombinieren, um statt null den Wert 0 zu erhalten:

```
int length = sb?.ToString().Length ?? 0;
```

Nullbare Referenztypen

Während *nullbare Typen* Werttypen den Wert null ermöglichen, tun *nullbare Referenztypen* (seit C# 8) das Gegenteil – sie stattdessen Referenztypen mit (einem gewissen Grad an) Nicht-Nullbarkeit aus. Damit soll dabei geholfen werden, `NullReferenceExceptions` zu vermeiden.

Nullbare Referenztypen bieten eine Sicherheitsebene, die nur durch den Compiler in Form von Warnungen umgesetzt wird, wenn dieser Code entdeckt, der die Gefahr mit sich bringt, dass er eine `NullReferenceException` erzeugt.

Um nullbare Referenztypen zu ermöglichen, müssen Sie in Ihre `.csproj`-Datei das `Nullable`-Element aufnehmen:

```
<Nullable>enable</Nullable>
```

und/oder Sie nutzen die folgenden Direktiven in Ihrem Code an den Stellen, an denen sie wirksam sein sollen:

```
#nullable enable // aktiviert NRT ab hier
#nullable disable // deaktiviert NRT ab hier
#nullable restore // setzt NRT auf Projekteinstellungen zurück
```

Nach dem Aktivieren macht der Compiler die Nicht-Nullbarkeit zum Standard: Wollen Sie, dass ein Referenztyp null akzeptiert, ohne dass der Compiler eine Warnung ausgibt, müssen Sie das Suffix `?` anhängen, um einen *nullbaren Referenztyp* anzuzeigen. Im folgenden Beispiel ist `s1` nicht nullbar, während `s2` nullbar ist:

```
#nullable enable // aktiviert nullbare Referenztypen

string s1 = null; // erzeugt eine Compilerwarnung!
string? s2 = null; // okay: s2 ist nullbarer Referenztyp
```



Weil es sich bei nullbaren Referenztypen um Konstrukte beim Kompilieren handelt, gibt es keine Laufzeitunterschiede zwischen `string` und `string?`. Im Gegensatz dazu sorgen nullbare (Wert-)Typen für Veränderungen im Typsystem – das `Struct Nullable<T>`.

Der folgende Code sorgt ebenfalls für eine Warnung, weil `x` nicht initialisiert wurde:

```
class Foo { string x; }
```

Die Warnung verschwindet, wenn Sie `x` initialisieren – entweder über einen Feldinitialisierer oder über Code im Konstruktor.

Der Compiler warnt Sie auch, wenn Sie einen nullbaren Referenztyp dereferenzieren, sollte er der Meinung sein, dass eine `NullReferenceException` auftreten könnte. Im folgenden Beispiel erzeugt der Zugriff auf die Eigenschaft `Length` des Strings für eine Warnung:

```
void Foo (string? s) => Console.Write (s.Length);
```

Um die Warnung loszuwerden, können Sie den *null-toleranten Operator* `!` verwenden:

```
void Foo (string? s) => Console.Write (s!.Length);
```

In diesem Beispiel ist der Einsatz des null-toleranten Operators gefährlich, denn es könnte eine `NullReferenceException` geworfen werden, die wir ja gerade vermeiden wollen. Wir könnten das Problem wie folgt beheben:

```
void Foo (string? s)
{
    if (s != null) Console.Write (s.Length);
}
```

Beachten Sie, dass wir den null-toleranten Operator jetzt nicht mehr benötigen. Denn der Compiler führt eine statische Analyse durch und ist schlau genug, selbst zu erkennen – zumindest in einfachen Fällen –, ob eine Dereferenz sicher ist und es keine Möglichkeit einer `NullReferenceException` gibt.

Die Fähigkeit des Compilers, Probleme zu erkennen und Warnungen auszugeben, ist nicht vollkommen, und sie hat auch Grenzen. So ist es beispielsweise unmöglich, zu wissen, ob die Elemente eines Arrays gefüllt wurden, daher sorgt der folgende Code für keine Warnung:

```
var strings = new string[10];
Console.WriteLine (strings[0].Length);
```

Erweiterungsmethoden

Erweiterungsmethoden ermöglichen es, bestehende Typen durch neue Methoden zu erweitern, ohne die Definition des Ursprungstyps verändern zu müssen. Eine Erweiterungsmethode ist eine statische Methode einer statischen Klasse, bei der der Modifikator `this` dem ersten Parameter mitgegeben wird. Der Typ des ersten Parameters ist dann der zu erweiternde Typ:

```
public static class StringHelper
{
    public static bool IsCapitalized (this string s)
    {
        if (string.IsNullOrEmpty (s)) return false;
        return char.IsUpper (s[0]);
    }
}
```

Die Erweiterungsmethode `IsCapitalized` kann so aufgerufen werden, als würde es sich um eine Instanzmethode eines Strings handeln:

```
Console.Write ("Perth".IsCapitalized());
```

Der Aufruf einer Erweiterungsmethode wird beim Kompilieren in einen normalen statischen Methodenaufruf umgewandelt:

```
Console.Write (StringHelper.IsCapitalized ("Perth"));
```

Interfaces können ebenfalls erweitert werden:

```
public static T First<T> (this IEnumerable<T> sequence)
{
    foreach (T element in sequence)
        return element;
    throw new InvalidOperationException ("Keine Elemente!");
}
...
Console.WriteLine ("Seattle".First());    // S
```

Verketteten von Erweiterungsmethoden

Erweiterungsmethoden können wie Instanzmethoden verkettet werden. Schauen Sie sich die folgenden beiden Funktionen an:

```
public static class StringHelper
{
    public static string Pluralize (this string s) {...}
    public static string Capitalize (this string s) {...}
}
```

x und y sind gleich und werden beide zu "Sausages", aber x verwendet Erweiterungsmethoden, während y statische Methoden nutzt:

```
string x = "sausage".Pluralize().Capitalize();

string y = StringHelper.Capitalize
    (StringHelper.Pluralize ("sausage"));
```

Mehrdeutigkeit und Auflösung

Eine kompatible Instanzmethode hat immer Vorrang vor einer Erweiterungsmethode, selbst wenn die Parameter der Erweiterungsmethode auf spezifischere Typen passen.

Wenn zwei Erweiterungsmethoden die gleiche Signatur haben, muss die Erweiterungsmethode als normale statische Methode aufgerufen werden, um eindeutig klarzustellen, welche Methode aufgerufen

werden soll. Wenn eine Erweiterungsmethode allerdings spezifischere (»passendere«) Argumente hat, bekommt diese den Vorzug.

Anonyme Typen

Ein *anonymer Typ* ist eine einfache Klasse, die bei Bedarf erzeugt wird, um ein paar Werte zu speichern. Um einen anonymen Typ zu erzeugen, verwenden Sie das Schlüsselwort `new`, gefolgt von einem Objektinitialisierer, in dem Sie die Eigenschaften und Werte angeben, die der Typ enthalten wird. Hier ist ein Beispiel:

```
var dude = new { Name = "Bob", Age = 1 };
```

Der Compiler löst das auf, indem er einen privaten eingebetteten Typ erstellt, der nur lesbare Eigenschaften für `Name` (Typ `string`) und `Age` (Typ `int`) besitzt. Sie müssen das Schlüsselwort `var` verwenden, um sich auf einen anonymen Typ zu beziehen, da der Name des Typs vom Compiler erzeugt wird.

Der Eigenschaftsname eines anonymen Typs kann aus einem Ausdruck ermittelt werden, der selbst ein Bezeichner ist:

```
int Age = 1;  
var dude = new { Name = "Bob", Age = Age };
```

Das ist äquivalent zu

```
var dude = new { Name = "Bob", Age = Age };
```

Ein Array anonymer Typen können Sie folgendermaßen erstellen:

```
var dudes = new[]  
{  
    new { Name = "Bob", Age = 30 },  
    new { Name = "Mary", Age = 40 }  
};
```

Anonyme Typen werden vor allem beim Schreiben von LINQ-Abfragen genutzt.

Anonyme Typen sind immutabel, daher können Instanzen nach ihrem Erstellen nicht mehr verändert werden. Seit C# 10 können Sie aber das Schlüsselwort `with` verwenden, um eine Kopie mit Abweichungen zu erstellen – so wie Sie es bei Datensätzen tun wür-

den. Im Abschnitt »Nicht destruktive Mutation« auf Seite 177 finden Sie mehr dazu.

Tupel

Tupel bieten wie anonyme Typen (seit C# 7) eine einfache Möglichkeit, einen Satz von Variablen abzulegen. Haupteinsatzgebiet von Tupeln ist die sichere Rückgabe mehrerer Werte aus einer Methode, ohne out-Parameter nutzen zu müssen (etwas, das mit anonymen Typen nicht möglich ist). Ein *Tupel-Literal* erzeugen Sie am einfachsten, indem Sie die gewünschten Werte in Klammern auflisten. Damit wird ein Tupel mit unbenannten Elementen geschaffen:

```
var bob = ("Bob", 23);  
Console.WriteLine (bob.Item1); // Bob  
Console.WriteLine (bob.Item2); // 23
```

Anders als bei anonymen Typen ist `var` optional, und Sie können einen *Tupel-Typ* explizit festlegen:

```
(string,int) bob = ("Bob", 23);
```

Damit können Sie ein Tupel aus einer Methode zurückgeben:

```
(string,int) person = GetPerson();  
Console.WriteLine (person.Item1); // Bob  
Console.WriteLine (person.Item2); // 23
```

```
(string,int) GetPerson() => ("Bob", 23);
```

Tupel arbeiten sehr gut mit Generics zusammen, daher sind alle folgenden Typen korrekt:

```
Task<(string,int)>  
Dictionary<(string,int),Uri>  
IEnumerable<(int ID, string Name)> // siehe unten ...
```

Tupel sind Werttypen mit *mutablen* Elementen (les- und schreibbar). Das heißt, Sie können `Item1`, `Item2` und so weiter nach dem Erstellen eines Tupels verändern.

Tupel-Elemente benennen

Sie können den Elementen beim Erstellen von Tupel-Literalen sinnvolle Namen verpassen:

```
var tuple = (Name:"Bob", Age:23);  
Console.WriteLine (tuple.Name); // Bob  
Console.WriteLine (tuple.Age);  // 23
```

Das Gleiche erreichen Sie beim Spezifizieren von Tupel-Typen:

```
static (string Name, int Age) GetPerson() => ("Bob",23);
```

Namen von Elementen werden automatisch aus Eigenschafts- oder Feldnamen *abgeleitet*:

```
var now = DateTime.Now;  
var tuple = (now.Day, now.Month, now.Year);  
Console.WriteLine (tuple.Day); // okay
```



Tupel sind eine syntaktische Vereinfachung für den Einsatz einer Familie von generischen Structs namens `ValueTuple<T1>` und `ValueTuple<T1,T2>`, die Felder mit den Namen `Item1`, `Item2` und so weiter enthalten. Daher ist `(string,int)` ein Alias für `ValueTuple<string,int>`. »Benannte Elemente« existieren daher nur im Quellcode – und in der Vorstellung des Compilers –, zur Laufzeit sind sie größtenteils wieder verschwunden.

Tupel dekonstruieren

Tupel unterstützen implizit das Dekonstruktorenmuster (siehe »Dekonstruktoren« auf Seite 81), daher können Sie ein Tupel problemlos in seine einzelnen Variablen *dekonstruieren*. Statt also diesen Code einzusetzen:

```
var bob = ("Bob", 23);  
string name = bob.Item1;  
int age = bob.Item2;
```

können Sie auch Folgendes schreiben:

```
var bob = ("Bob", 23);  
(string name, int age) = bob; // bob in name und  
                             // age auflösen.
```

```
Console.WriteLine (name);  
Console.WriteLine (age);
```

Die Syntax für das Dekonstruieren ist der zum Deklarieren eines Tupels mit benannten Elementen verwirrend ähnlich. Im Folgenden sehen Sie den Unterschied:

```
(string name, int age) = bob;           // dekonstruieren  
(string name, int age) bob2 = bob;      // Tupel deklarieren
```

Datensätze

Ein *Datensatz* oder *Record* ist (seit C# 9) eine bestimmte Form einer Klasse oder eines Struct, die dazu gedacht ist, gut mit immutablen (unveränderbaren) Daten zusammenzuarbeiten. Sein nützlichstes Feature ist die Fähigkeit der *nicht destruktiven Mutation*, bei der Sie zum »Verändern« eines immutablen Objekts ein neues erstellen und die Daten dort hineinkopieren, wobei Sie Anpassungen vornehmen können.

Datensätze sind auch hilfreich, um Typen zu erstellen, die lediglich Daten kombinieren oder vorhalten. In einfachen Fällen vermeiden Sie damit Standardcode, erhalten dabei aber eine Semantik *struktureller Gleichheit* (zwei Objekte sind gleich, wenn ihre Daten gleich sind), was normalerweise genau das ist, was Sie bei immutablen Typen haben wollen.

Einen Datensatz gibt es in C# nur beim Kompilieren. Zur Laufzeit sieht die CLR sie lediglich als Klassen oder Structs (mit einer Reihe zusätzlicher »synthetisierter« Member, die durch den Compiler ergänzt werden).

Einen Datensatz definieren

Eine Datensatzdefinition ähnelt der einer Klasse oder eines Struct, und sie kann auch die gleichen Arten von Mitgliedern enthalten – Felder, Eigenschaften, Methoden und so weiter. Datensätze können Interfaces implementieren, und (klassenbasierte) Datensätze können andere (klassenbasierte) Datensätze als Subklassen ableiten.

Standardmäßig ist der zugrunde liegende Typ eines Datensatzes eine Klasse:

```
record Point { }           // Point ist eine Klasse
```

Seit C# 10 kann der zugrunde liegende Typ auch ein Struct sein:

```
record struct Point { }    // Point ist ein Struct
```

(record class ist ebenfalls erlaubt und hat die gleiche Bedeutung wie record.)

Ein einfacher Datensatz kann aus ein paar Init-Only-Eigenschaften und vielleicht einem Konstruktor bestehen:

```
record Point
{
    public Point (double x, double y) => (X, Y) = (x, y);

    public double X { get; init; }
    public double Y { get; init; }
}
```

Beim Kompilieren wandelt C# die Datensatzdefinition in eine Klasse (oder einen Struct) um und führt die folgenden zusätzlichen Schritte aus:

- Es erzeugt einen geschützten *Copy-Konstruktor* (und eine verborgene *Clone-Methode*), um die nicht destruktive Mutation zu ermöglichen.
- Es überschreibt/überlädt die gleichheitsbezogenen Funktionen, um die strukturelle Gleichheit zu implementieren.
- Es überschreibt die Methode `ToString()` (um wie bei anonymen Typen die öffentlichen Eigenschaften des Datensatzes zu erweitern).

Die vorherige Datensatzdeklaration wird damit zu so etwas wie diesem:

```
class Point
{
    public Point (double x, double y) => (X, Y) = (x, y);

    public double X { get; init; }
    public double Y { get; init; }
}
```

```

protected Point (Point original) // "Copy-Konstruktor"
{
    this.X = original.X; this.Y = original.Y
}

// Diese Methode hat einen seltsamen
// vom Compiler erzeugten Namen:
public virtual Point <Clone>$( ) => new Point (this);

// Zusätzlicher Code zum Überschreiben von Equals,
// ==, !=, GetHashCode, ToString()...
}

```

Parameterlisten

Eine Datensatzdefinition kann auch eine *Parameterliste* enthalten:

```

record Point (double X, double Y)
{
    ...
}

```

Parameter können die Modifikatoren `in` und `params` nutzen, aber nicht `out` oder `ref`. Wird eine Parameterliste angegeben, führt der Compiler die folgenden zusätzlichen Schritte aus:

- Er schreibt eine *Init-Only-Eigenschaft* pro Parameter (oder bei Datensatz-Structs eine beschreibbare Eigenschaft).
- Er schreibt einen *primären Konstruktor*, um die Eigenschaften zu befüllen.
- Er schreibt einen Dekonstruktor.

Das heißt, dass wir unseren `Point`-Datensatz sehr einfach deklarieren können:

```

record Point (double X, double Y);

```

Der Compiler wird dann (mehr oder weniger) genau das erzeugen, was wir weiter oben alles angegeben haben. Ein kleiner Unterschied liegt darin, dass die Parameternamen im primären Konstruktor `X` und `Y` statt `x` und `y` lauten werden:

```

public Point (double X, double Y)
{
    this.X = X; this.Y = Y;
}

```



Weil es sich um einen *primären Konstruktor* handelt, sind die Parameter *X* und *Y* zudem auf magische Weise für jeden Feld- oder Eigenschafts-Initialisierer in Ihrem Datensatz verfügbar. Auf diese Details gehen wir weiter unten im Abschnitt »Primäre Konstruktoren« auf Seite 179 ein.

Zudem erzeugt der Compiler beim Definieren einer Parameterliste auch einen Dekonstruktor:

```
public void Deconstruct (out double X, out double Y)
{
    X = this.X; Y = this.Y;
}
```

Von Datensätzen mit Parameterlisten können mit folgender Syntax Subklassen erzeugt werden:

```
record Point3D (double X, double Y, double Z)
    : Point (X, Y);
```

Der Compiler generiert dann einen primären Konstruktor:

```
class Point3D : Point
{
    public double Z { get; init; }

    public Point3D (double X, double Y, double Z)
        : base (X, Y)
        => this.Z = Z;
}
```



Parameterlisten sind eine nette Vereinfachung, wenn Sie eine Klasse benötigen, die einfach eine Reihe von Werten (einen *Produkttyp* in funktionaler Programmierung) zusammenfasst, und sie können auch für das Prototyping helfen. Sie sind nicht ganz so nützlich, wenn Sie in den *init*-Accessors Logik benötigen (wie zum Beispiel zum Validieren von Argumenten).

Nicht destruktive Mutation

Der wichtigste Schritt, den der Compiler bei allen Datensätzen vornimmt, ist das Schreiben eines *Copy-Konstruktors* (und einer ver-

borgenen *Clone*-Methode). Damit lässt sich über das Schlüsselwort *with* eine nicht destruktive Mutation erreichen:

```
Point p1 = new Point (3, 3);  
Point p2 = p1 with { Y = 4 };  
Console.WriteLine (p2);           // Point { X = 3, Y = 4 }  
  
record Point (double X, double Y);
```

In diesem Beispiel ist p2 eine Kopie von p1, aber dessen Eigenschaft Y wurde auf 4 gesetzt. Der Vorteil wird größer, wenn es mehr Eigenschaften gibt.

Nicht destruktive Mutation findet in zwei Phasen statt:

1. Zuerst kopiert der Copy-Konstruktor den Datensatz. Standardmäßig kopiert er alle dem Datensatz zugrunde liegenden Felder und schafft so eine originalgetreue Replika, während er die gesamte Logik (und den Zusatzaufwand) in den *init*-Accessors umgeht. Alle Felder sind enthalten (öffentlich und privat, aber auch die verborgenen Felder, die automatische Eigenschaften abbilden).
2. Dann wird jede Eigenschaft in der *Member-Initialisierungsliste* aktualisiert (dieses Mal mit den *init*-Accessors).

Der Compiler übersetzt diesen Code:

```
Test t2 = t1 with { A = 10, C = 30 }
```

in etwas, das funktional äquivalent zu folgenden Zeilen ist:

```
Test t2 = new Test(t1); // t1 klonen  
t2.A = 10;              // Eigenschaft A anpassen  
t2.C = 30;              // Eigenschaft C anpassen
```

(Dieser Code ließe sich nicht kompilieren, wenn Sie ihn explizit schreiben würden, weil A und C *Init-Only*-Eigenschaften sind. Zudem ist der Copy-Konstruktor *geschützt* – C# umgeht das, indem es ihn über eine öffentliche verborgene Methode namens `<Clone>$` aufruft, die es in den Datensatz schreibt.)

Bei Bedarf können Sie Ihren eigenen Copy-Konstruktor definieren. C# wird dann Ihre Definition verwenden, statt selbst eine zu erstellen:

```
protected Point (Point original)
{
    this.X = original.X; this.Y = original.Y;
}
```

Beim Erstellen von Subklassen anderer Datensätze ist der Copy-Konstruktor nur für das Kopieren seiner eigenen Felder verantwortlich. Um die Felder des Basisdatensatzes zu kopieren, delegieren Sie an die Basis:

```
protected Point (Point original) : base (original)
{
    ...
}
```

Primäre Konstruktoren

Definieren Sie einen Datensatz mit einer Parameterliste, generiert der Compiler automatisch Eigenschaftsdeklarationen, aber auch einen *primären Konstruktor* (und einen Dekonstruktor). Das funktioniert in einfachen Fällen sehr gut, während Sie in komplexeren Situationen die Parameterliste weglassen und die Eigenschaftsdeklarationen und den Konstruktor manuell anlegen können. C# bietet auch den eher begrenzt nützlichen Zwischenweg an, eine Parameterliste zu definieren und gleichzeitig einige oder alle Eigenschaftsdeklarationen selbst zu schreiben:

```
record Student(int ID, string Surname, string FirstName)
{
    public int ID { get; } = ID;
}
```

In diesem Fall haben wir die Eigenschaftsdefinition der ID »übernommen« und sie als schreibgeschützt definiert (statt als Init-Only), womit wir verhindern, dass sie an der nicht destruktiven Mutation teilnimmt. Müssen Sie eine bestimmte Eigenschaft niemals nicht destruktiv mutieren, können Sie berechnete Daten im Datensatz cachen, ohne einen Refresh-Mechanismus programmieren zu müssen.

Beachten Sie, dass wir einen *Eigenschaftsinitialisierer* (hier fett gedruckt) nutzen mussten:

```
public int ID { get; } = ID;
```

»Übernehmen« Sie eine Eigenschaftsdeklaration, sind Sie auch dafür verantwortlich, deren Wert zu initialisieren – der primäre Konstruktor erledigt das nicht mehr automatisch. Beachten Sie, dass sich die fett gedruckte ID auf den *Parameter für den primären Konstruktor* bezieht, nicht auf die Eigenschaft ID.

Ein besonderes Feature des primären Konstruktors ist, dass seine Parameter (in diesem Fall ID, Surname und FirstName) automatisch für alle Feld- und Eigenschaftsinitialisierer sichtbar werden.

Sie können außerdem eine Eigenschaftsdefinition mit expliziten Accessors übernehmen:

```
int _id = ID;  
public int ID { get => _id; init => _id = value; }
```

Auch hier bezieht sich die ID in Fettdruck auf den Parameter des primären Konstruktors, nicht auf die Eigenschaft. (Der Grund dafür, dass es hier keine Mehrdeutigkeit gibt, liegt darin, dass es nicht erlaubt ist, Eigenschaften aus Initialisierern heraus anzusprechen.)

Weil wir die Eigenschaft `_id` mit ID initialisieren müssen, ist diese »Übernahme« weniger nützlich, da jegliche Logik im `init`-Accessor (wie zum Beispiel eine Validierung) vom primären Konstruktor umgangen wird.

Datensätze und Gleichheit

Wie Structs, anonyme Typen und Tupel bieten auch Datensätze ohne Zusatzaufwand eine strukturelle Gleichheit – zwei Datensätze sind also gleich, wenn ihre Felder (und automatischen Eigenschaften) gleich sind:

```
var p1 = new Point (1, 2);  
var p2 = new Point (1, 2);  
Console.WriteLine (p1.Equals (p2));    // Wahr  
  
record Point (double X, double Y);
```

Der *Gleichheitsoperator* funktioniert (wie bei Tupeln) auch für Datensätze:

```
Console.WriteLine (p1 == p2);           // Wahr
```

Anders als bei Klassen und Structs überschreiben Sie die Methode `object.Equals` nicht (was Sie auch gar nicht können), wenn Sie ein angepasstes Vergleichsverhalten erreichen wollen. Stattdessen definieren Sie eine öffentliche `Equals`-Methode mit folgender Signatur:

```
record Point (double X, double Y)
{
    public virtual bool Equals (Point other) =>
        other != null && X == other.X && Y == other.Y;
}
```

Die `Equals`-Methode muss `virtual` (nicht `override`) und *stark typisiert* sein – sie akzeptiert nur den tatsächlichen Datensatztyp (in diesem Fall `Point`, nicht `object`). Haben Sie die Signatur richtig definiert, wird der Compiler Ihre Methode automatisch einbauen.

Wie bei jedem Typ gilt: Übernehmen Sie die Prüfung auf Gleichheit, sollten Sie auch `GetHashCode()` überschreiben. Ein nettes Feature bei Datensätzen ist, dass Sie nicht `!=` oder `==` überladen und auch nicht `IEquatable<T>` implementieren – das wird alles für Sie erledigt.

Muster

Weiter oben haben wir gezeigt, wie Sie mit dem `is`-Operator prüfen können, ob eine Referenzumwandlung erfolgreich sein wird, um dann ihren umgewandelten Wert zu verwenden:

```
if (obj is string s)
    Console.WriteLine (s.Length);
```

Dabei wird ein Muster eingesetzt, dass als *Typmuster* bezeichnet wird. Der `is`-Operator unterstützt auch andere Muster, die in früheren Versionen von `C#` eingeführt wurden. Muster werden in folgenden Kontexten unterstützt:

- nach dem `is`-Operator (*variable is pattern*)
- in Switch-Anweisungen
- in Switch-Ausdrücken

Wir haben das Typmuster schon in den Abschnitten »Die switch-Anweisung abhängig vom Typ« auf Seite 66 und »Der `is`-Operator«

auf Seite 98 behandelt. In diesem Abschnitt kümmern wir uns um die fortgeschritteneren Muster, die in neueren Versionen von C# eingeführt wurden.

Manche der spezialisierteren Muster sind für den Einsatz in Switch-Anweisungen/-Ausdrücken gedacht. Hier verringern sie den Bedarf nach when-Klauseln und ermöglichen Ihnen die Verwendung von Switches dort, wo das vorher nicht möglich war.

var-Muster

Das *var-Muster* ist eine Abwandlung des Typmusters, bei dem Sie den Typnamen durch das Schlüsselwort **var** ersetzen. Die Umwandlung ist immer erfolgreich, daher liegt der Zweck vor allem darin, die folgende Variable wiederzuverwenden:

```
bool IsJanetOrJohn (string name) =>
    name.ToUpper() is var upper &&
    (upper == "JANET" || upper == "JOHN");
```

Das entspricht diesem Code:

```
bool IsJanetOrJohn (string name)
{
    string upper = name.ToUpper();
    return upper == "JANET" || upper == "JOHN";
}
```

Konstantenmuster

Mit dem *Konstantenmuster* können Sie direkt mit einer Konstanten vergleichen, was bei der Arbeit mit dem Typ `object` nützlich ist:

```
void Foo (object obj)
{
    if (obj is 3) ...
}
```

Dieser fett gedruckte Ausdruck entspricht dem folgenden:

```
obj is int && (int)obj == 3
```

Wie Sie gleich sehen werden, kann das Konstantenmuster noch nützlicher werden, wenn man es in *logischen Mustern* einsetzt.

Relationale Muster

Seit C# 9 können Sie die Operatoren `<`, `>`, `<=` und `=>` in Mustern einsetzen:

```
if (x is > 100) Console.Write ("x ist größer als 100");
```

Das wird in einem switch nützlich:

```
string GetWeightCategory (decimal bmi) => bmi switch  
{  
    < 18.5m => "untergewichtig",  
    < 25m => "normal",  
    < 30m => "übergewichtig",  
    _ => "adipös"  
};
```

Logische Muster

Seit C# 9 können Sie die Schlüsselwörter `and`, `or` und `not` verwenden, um Muster zu kombinieren:

```
bool IsJanetOrJohn (string name)  
    => name.ToUpper() is "JANET" or "JOHN";  
  
bool IsVowel (char c)  
    => c is 'a' or 'e' or 'i' or 'o' or 'u';  
  
bool Between1And9 (int n) => n is >= 1 and <= 9;  
  
bool IsLetter (char c) => c is >= 'a' and <= 'z'  
                        or >= 'A' and <= 'Z';
```

Wie bei den Operatoren `&&` und `||` hat `and` eine höhere Gewichtung als `or`. Sie können das durch Klammern verändern. Ein netter Trick ist, den `not`-Operator mit dem Typmuster zu kombinieren, um zu prüfen, ob ein Objekt (nicht) ein Typ ist:

```
if (obj is not string) ...
```

Das sieht besser aus als:

```
if (!(obj is string)) ...
```

Tupel- und Positionsmuster

Das (seit C# 8 vorhandene) *Tupelmuster* sucht nach Tupeln:

```
var p = (2, 3);
Console.WriteLine (p is (2, 3)); // Wahr
```

Das Tupelmuster kann als Spezialfall des (ebenfalls seit C# 8 angebotenen) *Positionsmusters* angesehen werden, das Typen matcht, die eine Methode Deconstruct anbieten (siehe den »Dekonstruktoren« auf Seite 81). Im folgenden Beispiel nutzen wir den vom Compiler erzeugten Dekonstruktor für den Datensatz Point:

```
var p = new Point (2, 2);
Console.WriteLine (p is (2, 2)); // Wahr
```

```
record Point (int X, int Y);
```

Sie können beim Matchen dekonstruieren, indem Sie folgende Syntax einsetzen:

```
Console.WriteLine (p is (var x, var y) && x == y);
```

Hier noch ein Switch-Ausdruck, der ein Typmuster mit einem Positionsmuster kombiniert:

```
string Print (object obj) => obj switch
{
    Point (0, 0) => "Leerer Punkt",
    Point (var x, var y) when x == y => "Diagonale"
    ...
};
```

Eigenschaftsmuster

Ein *Eigenschaftsmuster* findet (seit C# 8) einen oder mehrere Eigenschaftswerte eines Objekts:

```
if (obj is string { length:4 }) ...
```

Das bringt hier natürlich nicht viel Tippersparnis gegenüber:

```
if (obj is string s && s.Length == 4) ...
```

Bei Switch-Anweisungen und -Ausdrücken sind Eigenschaftsmuster nützlicher. Denken Sie an die Klasse `System.Uri`, die ein URI repräsentiert. Sie hat unter anderem die Eigenschaften `Scheme`, `Host`, `Port` und `IsLoopback`. Beim Schreiben einer Firewall könnten wir entscheiden, ob wir ein URI zulassen oder blockieren, indem wir einen Switch-Ausdruck nutzen, der Eigenschaftsmuster einsetzt:

```
bool ShouldAllow (Uri uri) => uri switch
{
    { Scheme: "http", Port: 80 } => true,
    { Scheme: "https", Port: 443 } => true,
    { Scheme: "ftp", Port: 21 } => true,
    { IsLoopback: true } => true,
    _ => false
};
```

Sie können Eigenschaften verschachteln, zum Beispiel so:

```
{ Scheme: { Length: 4 }, Port: 80 } => true,
```

was sich seit C# 10 vereinfachen lässt zu:

```
{ Scheme.Length: 4, Port: 80 } => true,
```

Innerhalb von Eigenschaftsmustern können Sie andere Muster verwenden, unter anderem die relationalen Muster:

```
{ Host: { Length < 1000 }, Port: > 0 } => true,
```

Sie können eine Variable am Ende einer Klausel einführen und diese dann in einer when-Klausel nutzen:

```
{ Scheme: "http", Port: 80 } httpUri
    when httpUri.Host.Length < 1000 => true,
```

Sie können auch Variablen auf der *Eigenschaftsebene* einführen:

```
{ Scheme: "http", Port: 80, Host: var host }
    when host.Length < 1000 => true,
```

In diesem Fall ist der folgende Code allerdings kürzer und einfacher:

```
{ Scheme: "http", Port: 80, Host: { Length < 1000 } }
```

LINQ

LINQ (*Language Integrated Query*) ermöglicht Ihnen, strukturierte, typsichere Abfragen für lokale Objekt-Collections und Datenquellen auf anderen Rechnern zu schreiben. LINQ lässt Sie beliebige Collections durchsuchen, die *IEnumerable<T>* implementieren – sei es ein Array, eine Liste, ein XML-DOM oder eine entfernte Datenquelle (wie eine Tabelle im SQL-Server). LINQ kombiniert die Vorteile

einer Typprüfung beim Kompilieren und einer dynamischen Abfrageerzeugung.



Eine gute Möglichkeit, mit LINQ zu experimentieren, ist das Herunterladen von LINQPad unter <http://www.linqpad.net>. LINQPad ermöglicht Ihnen, interaktiv lokale Collections und SQL-Datenbanken in LINQ ohne ein weiteres Setup abzufragen, und bringt von Haus aus zahllose Beispiele mit.

Grundlagen von LINQ

Die grundlegenden Dateneinheiten in LINQ sind *Sequenzen* und *Elemente*. Eine Sequenz ist ein beliebiges Objekt, das das generische Interface `IEnumerable` implementiert, ein Element ist jedes Teil in der Sequenz. Im folgenden Beispiel ist `names` eine Sequenz, und Tom, Dick und Harry sind Elemente:

```
string[] names = { "Tom", "Dick", "Harry" };
```

Eine Sequenz wie diese bezeichnen wir als *lokale Sequenz*, da sie eine lokale Collection mit Objekten im Speicher repräsentiert.

Ein *Abfrageoperator* ist eine Methode, die eine Sequenz transformiert. Ein typischer Abfrageoperator erwartet eine *Eingabesequenz* und gibt eine transformierte *Ausgabesequenz* zurück. In der Klasse `Enumerable` in `System.Linq` gibt es etwa 40 Abfrageoperatoren, die alle als statische Erweiterungsmethoden implementiert sind. Diese werden *Standard-Abfrageoperatoren* genannt.



LINQ unterstützt auch Sequenzen, die dynamisch aus einer Remote-Datenquelle gefüllt werden können, zum Beispiel vom SQL-Server. Diese Sequenzen implementieren zusätzlich das Interface `IQueryable<>` und erhalten Hilfe von passenden Standard-Abfrageoperatoren in der Klasse `Queryable`.

Eine einfache Abfrage

Eine *Abfrage* ist ein Ausdruck, der Sequenzen mit einem oder mehreren Abfrageoperatoren transformiert. Die einfachste Abfrage be-

steht aus einer Eingabesequenz und einem Operator. So können wir zum Beispiel den Operator `Where` auf ein einfaches Array anwenden, um die Elemente zu extrahieren, die mindestens vier Zeichen lang sind:

```
string[] names = { "Tom", "Dick", "Harry" };

IEnumerable<string> filteredNames =
    System.Linq.Enumerable.Where (
        names, n => n.Length >= 4);

foreach (string n in filteredNames)
    Console.Write (n + "|");           // Dick|Harry|
```

Da die Standard-Abfrageoperatoren als Erweiterungsmethoden implementiert sind, können wir `Where` direkt auf `names` aufrufen, als wäre es eine Instanzmethode:

```
IEnumerable<string> filteredNames =
    names.Where (n => n.Length >= 4);
```

(Damit sich dieser Code kompilieren lässt, müssen Sie den Namensraum `System.Linq` mit einer `using`-Direktive importieren.) Die Methode `Where` in `System.Linq.Enumerable` hat folgende Signatur:

```
static IEnumerable<TSource> Where<TSource> (
    this IEnumerable<TSource> source,
    Func<TSource,bool> predicate)
```

`source` ist die *Eingabesequenz*, und `predicate` ist ein Delegate, das für jedes *Eingabeelement* aufgerufen wird. Die Methode `Where` nimmt alle Elemente mit in die *Ausgabesequenz* auf, für die das Delegate `true` zurückliefert. Intern wird sie mit einem Iterator implementiert – so sieht der Quellcode aus:

```
foreach (TSource element in source)
    if (predicate (element))
        yield return element;
```

Projizieren

Ein weiterer grundlegender Abfrageoperator ist die Methode `Select`. Diese transformiert (*projiziert*) jedes Element in der Eingabesequenz mit einem angegebenen Lambda-Ausdruck:

```
string[] names = { "Tom", "Dick", "Harry" };

IEnumerable<string> upperNames =
    names.Select (n => n.ToUpper());

foreach (string n in upperNames)
    Console.Write (n + "|");    // TOM|DICK|HARRY|
```

Eine Abfrage kann in einen anonymen Typ projizieren:

```
var query = names.Select (n => new {
                                Name = n,
                                Length = n.Length
                            });

foreach (var row in query)
    Console.WriteLine (row);
```

Das Ergebnis sieht so aus:

```
{ Name = Tom, Length = 3 }
{ Name = Dick, Length = 4 }
{ Name = Harry, Length = 5 }
```

Take und Skip

Die ursprüngliche Reihenfolge der Elemente in einer Eingabesequenz ist für LINQ von Bedeutung. Manche Abfrageoperatoren verlassen sich auf dieses Verhalten, zum Beispiel Take, Skip und Reverse. Der Operator Take gibt die ersten x Elemente aus und verwirft den Rest:

```
int[] numbers = { 10, 9, 8, 7, 6 };
IEnumerable<int> firstThree = numbers.Take (3);
// firstThree ist { 10, 9, 8 }
```

Der Operator Skip ignoriert die ersten x Elemente und gibt den Rest aus:

```
IEnumerable<int> lastTwo = numbers.Skip (3);
```

Seit .NET 6 gibt es auch die Methoden TakeLast und SkipLast, die die letzten n Elemente auswählen oder überspringen. Zudem wurde die Methode Take überladen, sodass sie eine Range-Variable akzeptiert. Diese überladene Version kann die Funktionalität aller vier Methoden zusammenfassen – so ist beispielsweise Take(5..) äquivalent zu Skip(5), und Take(..^5) ist äquivalent zu SkipLast(5).

Elementoperatoren

Nicht alle Abfrageoperatoren liefern eine Sequenz zurück. Die *Elementoperatoren* extrahieren genau ein Element aus der Eingabesequenz – so zum Beispiel `First`, `Last`, `Single` und `ElementAt`:

```
int[] numbers = { 10, 9, 8, 7, 6 };
int firstNumber = numbers.First();           // 10
int lastNumber = numbers.Last();             // 6
int secondNumber = numbers.ElementAt(2);      // 8
int firstOddNum = numbers.First (n => n%2 == 1); // 9
```

Alle diese Operatoren werfen eine *Exception*, wenn keine Elemente vorhanden sind. Um die *Exception* zu vermeiden, müssen Sie `FirstOrDefault`, `LastOrDefault`, `SingleOrDefault` oder `ElementAtOrDefault` nutzen – diese geben `null` zurück (oder den Defaultwert für Werttypen), wenn kein Element gefunden wird.

Die Methoden `Single` und `SingleOrDefault` entsprechen `First` und `FirstOrDefault`, nur dass sie eine *Exception* werfen, wenn es mehr als einen Eintrag gibt. Dieses Verhalten ist nützlich, wenn man in einer Datenbankabfrage eine Zeile über einen Primärschlüssel erhalten will.

Seit .NET 6 gibt es auch die Methoden `MinBy` und `MaxBy`, die das Element mit dem niedrigsten bzw. höchsten Wert zurückgeben, der über einen Schlüsselselektor bestimmt wird:

```
string[] names = { "Tom", "Dick", "Harry" };
Console.Write (names.MaxBy (n => n.Length)); // Harry
```

Aggregationsoperatoren

Die *Aggregationsoperatoren* liefern einen skalaren Wert zurück, im Allgemeinen einen numerischen. Die am häufigsten genutzten Aggregationsoperatoren sind `Count`, `Min`, `Max` und `Average`:

```
int[] numbers = { 10, 9, 8, 7, 6 };
int count = numbers.Count();           // 5
int min = numbers.Min();               // 6
int max = numbers.Max();               // 10
double avg = numbers.Average();        // 8
```

Count kann ein optionales Prädikat übergeben werden, mit dem bestimmt wird, welche Elemente zu zählen sind. Mit der folgenden Zeile werden alle geraden Zahlen gezählt:

```
int evenNums = numbers.Count (n => n % 2 == 0);    // 3
```

Den Operatoren Min, Max und Average kann ein optionales Argument übergeben werden, das jedes Element transformiert, bevor es aggregiert wird:

```
int maxRemainderAfterDivBy5 = numbers.Max  
    (n => n % 5);          // 4
```

Mit diesem Code wird der quadratische Mittelwert von numbers berechnet:

```
double rms = Math.Sqrt (numbers.Average (n => n * n));
```

Quantifizierer

Die *Quantifizierer* liefern einen bool-Wert zurück. Quantifizierer sind Contains, Any, All und SequenceEquals (das zwei Sequenzen vergleicht):

```
int[] numbers = { 10, 9, 8, 7, 6 };  
  
bool hasTheNumberNine = numbers.Contains (9);    // true  
bool hasMoreThanZeroElements = numbers.Any();    // true  
bool hasOddNum = numbers.Any (n => n % 2 == 1);  // true  
bool allOddNums = numbers.All (n => n % 2 == 1); // false
```

Set-Operatoren

Die *Set-Operatoren* erwarten zwei Eingabesequenzen vom gleichen Typ. Concat hängt eine Sequenz an die andere an. Union tut das Gleiche, entfernt aber Duplikate:

```
int[] seq1 = { 1, 2, 3 }, seq2 = { 3, 4, 5 };  
  
IEnumerable<int>  
    concat = seq1.Concat (seq2),    // { 1, 2, 3, 3, 4, 5 }  
    union  = seq1.Union  (seq2),    // { 1, 2, 3, 4, 5 }
```

Die anderen beiden Operatoren in dieser Kategorie sind Intersect und Except:

```

IEnumerable<int>
    commonality = seq1.Intersect (seq2),    // { 3 }
    difference1 = seq1.Except  (seq2),    // { 1, 2 }
    difference2 = seq2.Except  (seq1);    // { 4, 5 }

```

Seit .NET 6 gibt es auch Set-Operatoren, die einen Schlüsselselektor übernehmen (UnionBy, ExceptBy, IntersectBy). Der Schlüsselselektor wird verwendet, um zu bestimmen, ob ein Element als Duplikat zählt:

```

string[] seq1 = { "A", "b", "C" };
string[] seq2 = { "a", "B", "c" };
var union = seq1.UnionBy (seq2, x => x.ToUpper());
// union ist { "A", "b", "C" }

```

Verzögerte Ausführung

Ein wichtiges Feature vieler Abfrageoperatoren ist, dass sie nicht beim Erstellen, sondern erst beim Enumerieren ausgeführt werden (also dann, wenn MoveNext für den Enumerator aufgerufen wird). Schauen Sie sich die folgende Abfrage an:

```

var numbers = new List<int> { 1 };

IEnumerable<int> query = numbers.Select (n => n * 10);
numbers.Add (2);    // ein zusätzliches Element einschleusen

foreach (int n in query)
    Console.Write (n + "|");    // 10|20|

```

Die zusätzliche Zahl wurde erst nach dem Erstellen der Abfrage in die Liste eingeschmuggelt, sie ist aber trotzdem im Ergebnis enthalten, weil das Filtern oder Sortieren erst bei der foreach-Anweisung durchgeführt wird. Das wird als *verzögerte* Auswertung bezeichnet. Die verzögerte Ausführung entkoppelt das *Erstellen* der Abfrage von ihrer *Ausführung* und ermöglicht Ihnen damit, eine Abfrage in mehreren Schritten aufzubauen und eine Datenbank abzufragen, ohne alle Zeilen auf den Client zu laden. Alle Standard-Abfrageoperatoren arbeiten mit der verzögerten Ausführung, jedoch mit folgenden Ausnahmen:

- Operatoren, die ein einzelnes Element oder einen skalaren Wert zurückgeben (*Elementoperatoren*, *Aggregationsoperatoren* und *Quantifizierer*)
- die *Konvertierungsoperatoren* `ToArray`, `ToList`, `ToDictionary`, `ToLookup` und `ToHashSet`

Die Konvertierungsoperatoren sind manchmal gerade deshalb nützlich, weil sie eine sofortige Auswertung anstoßen. Das kann hilfreich sein, wenn Sie das Ergebnis zu einem bestimmten Zeitpunkt »einfrieren« oder cachen wollen oder um die erneute Ausführung einer berechnungsintensiven Abfrage oder eine Abfrage auf einer entfernten Quelle wie einer Entity-Framework-Tabelle zu vermeiden. (Eine Nebenwirkung der verzögerten Ausführung ist, dass die Abfrage bei einer späteren erneuten Enumerierung neu ausgewertet wird.)

Das folgende Beispiel zeigt die Verwendung des Operators `ToList`:

```
var numbers = new List<int>() { 1, 2 };

List<int> timesTen = numbers
    .Select (n => n * 10)
    .ToList(); // wird direkt in eine List<int> herausgezogen

numbers.Clear();
Console.WriteLine (timesTen.Count);           // immer noch 2
```



Unterabfragen ermöglichen eine weitere Ebene der Indirektion. Alle Teile einer Unterabfrage werden verzögert ausgeführt, einschließlich der Aggregations- und Konvertierungsmethoden, weil die Unterabfrage selbst erst bei Bedarf ausgeführt wird. Ist `names` ein String-Array, sieht eine Unterabfrage so aus:

```
names.Where (
    n => n.Length ==
        names.Min (n2 => n2.Length))
```

Standard-Abfrageoperatoren

Man kann die Standard-Abfrageoperatoren (wie sie in der Klasse `System.Linq.Enumerable` implementiert sind) in zwölf Kategorien einteilen, die in Tabelle 1 beschrieben werden.

Tabelle 1: Abfrageoperator-Kategorien

Kategorie	Beschreibung	Verzögerte Ausführung?
Filtern	Liefert eine Untermenge der Elemente zurück, die eine bestimmte Bedingung erfüllen.	Ja
Projektion	Transformiert jedes Element (im Allgemeinen mithilfe einer Lambda-Funktion), wobei Untersequenzen optional expandiert werden.	Ja
Verknüpfung	Verbindet Elemente einer Collection mit einer anderen, wobei eine zeitsparende Lookup-Strategie gewählt wird.	Ja
Ordnen	Liefert eine umgeordnete Sequenz zurück.	Ja
Gruppieren	Gruppirt eine Sequenz in Untersequenzen.	Ja
Set	Akzeptiert zwei Sequenzen gleichen Typs und liefert ihre Gemeinsamkeiten, Summe oder Differenz zurück.	Ja
Element	Wählt ein einzelnes Element aus einer Sequenz.	Nein
Aggregation	Führt eine Berechnung für eine Sequenz durch und liefert einen skalaren Wert zurück (normalerweise eine Zahl).	Nein
Quantifizierung	Führt eine Berechnung für eine Sequenz durch und liefert <code>true</code> oder <code>false</code> zurück.	Nein
Konvertierung: Import	Wandelt eine nichtgenerische Sequenz in eine (abfragbare) generische Sequenz um.	Ja
Konvertierung: Export	Wandelt eine Sequenz in ein Array, eine List, ein Dictionary oder einen Lookup um, wobei sofort ausgewertet wird.	Nein
Generieren	Erstellt eine einfache Sequenz.	Ja

Tabelle 2 bis Tabelle 13 bieten einen Überblick über alle Abfrageoperatoren. Für die fett gedruckten Operatoren bietet C# eine spezielle Unterstützung (siehe »Abfrageausdrücke« auf Seite 198).

Tabelle 2: Filteroperatoren

Methode	Beschreibung
Where	Liefert eine Untermenge mit Elementen zurück, die eine bestimmte Bedingung erfüllen.
Take	Liefert die ersten x Elemente zurück und verwirft den Rest.

Tabelle 2: Filteroperatoren (Fortsetzung)

Methode	Beschreibung
Skip	Ignoriert die ersten x Elemente und liefert den Rest zurück.
TakeLast	Gibt die letzten x Elemente zurück und verwirft den Rest.
SkipLast	Ignoriert die letzten x Elemente und gibt den Rest zurück.
TakeWhile	Gibt Elemente aus der Eingabesequenz zurück, bis das angegebene Prädikat true ist.
SkipWhile	Ignoriert Elemente aus der Eingabesequenz, bis das angegebene Prädikat true ist, und gibt dann den Rest zurück.
Distinct, DistinctBy	Liefert eine Collection ohne Duplikate zurück.

Tabelle 3: Projektionsoperatoren

Methode	Beschreibung
Select	Transformiert jedes Eingabeelement mit einem angegebenen Lambda-Ausdruck.
SelectMany	Transformiert jedes Eingabeelement, dann werden die Ergebnis-Untersequenzen flach geklopft und aneinandergehängt.

Tabelle 4: Verknüpfungsoperatoren

Methode	Beschreibung
Join	Wendet eine Lookup-Strategie an, um Elemente aus zwei Collections zu verbinden und eine flache Ergebnismenge zurückzugeben.
GroupJoin	Wie oben, aber es wird eine hierarchische Ergebnismenge zurückgegeben.
Zip	Enumeriert parallel zwei Sequenzen und liefert eine Sequenz, die eine Funktion auf jedes Elementpaar anwendet.

Tabelle 5: Ordnungsoperatoren

Methode	Beschreibung
OrderBy, ThenBy	Gibt die Elemente in aufsteigender Reihenfolge sortiert zurück.

Tabelle 5: Ordnungsoperatoren (Fortsetzung)

Methode	Beschreibung
OrderByDescending, ThenByDescending	Gibt die Elemente in absteigender Reihenfolge sortiert zurück.
Reverse	Liefert die Elemente in umgekehrter Reihenfolge zurück.

Tabelle 6: Gruppierungsoperatoren

Methode	Beschreibung
GroupBy	Gruppiert eine Sequenz in Untersequenzen.
Chunk	Gruppiert eine Sequenz in Bereiche vorgegebener Größe.

Tabelle 7: Mengenoperatoren

Methode	Beschreibung
Concat	Verbindet zwei Sequenzen.
Union, UnionBy	Verbindet zwei Sequenzen und entfernt doppelte Einträge.
Intersect, IntersectBy	Liefert die Elemente zurück, die in beiden Sequenzen vorhanden sind.
Except, ExceptBy	Liefert Elemente zurück, die in der ersten, aber nicht in der zweiten Sequenz vorhanden sind.

Tabelle 8: Elementoperatoren

Methode	Beschreibung
First, FirstOrDefault	Liefert das erste Element in der Sequenz bzw. das erste Element zurück, das eine angegebene Bedingung erfüllt.
Last, LastOrDefault	Liefert das letzte Element in der Sequenz bzw. das letzte Element zurück, das eine angegebene Bedingung erfüllt.
Single, SingleOrDefault	Entspricht First/FirstOrDefault, wirft aber eine Exception, wenn es mehr als einen Treffer gibt.
ElementAt, ElementAtOrDefault	Liefert das Element an der angegebenen Position zurück.
DefaultIfEmpty	Liefert eine Sequenz mit nur einem Wert, der null oder default(TSource) ist, wenn die Sequenz keine Elemente enthält.

Tabelle 9: Aggregationsoperatoren

Methode	Beschreibung
Count, LongCount	Liefert die Anzahl an Elementen in der Eingabesequenz zurück bzw. die Anzahl an Elementen, die ein Prädikat erfüllen.
Min, Max	Liefert das kleinste bzw. größte Element in der Sequenz zurück.
MinBy, MaxBy	Liefert das Element mit dem kleinsten oder größten Wert zurück, bestimmt über einen Schlüsselselektor.
Sum, Average	Berechnet die Summe bzw. den Mittelwert von Elementen in der Sequenz.
Aggregate	Führt eine benutzerdefinierte Zusammenfassungsoperation durch.

Tabelle 10: Quantifikatoren

Methode	Beschreibung
Contains	Liefert <code>true</code> zurück, wenn die Sequenz das angegebene Element enthält.
Any	Liefert <code>true</code> zurück, wenn mindestens ein Element das angegebene Prädikat erfüllt.
All	Liefert <code>true</code> zurück, wenn alle Elemente das angegebene Prädikat erfüllen.
SequenceEqual	Liefert <code>true</code> zurück, wenn die zweite Sequenz die gleichen Elemente enthält wie die Eingabesequenz.

Tabelle 11: Konvertierungsoperatoren (Import)

Methode	Beschreibung
OfType	Konvertiert <code>IEnumerable</code> nach <code>IEnumerable<T></code> und verwirft falsch typisierte Elemente.
Cast	Konvertiert <code>IEnumerable</code> nach <code>IEnumerable<T></code> und wirft eine <code>Exception</code> , wenn es ein falsch typisiertes Element gibt.

Tabelle 12: Konvertierungsoperatoren (Export)

Methode	Beschreibung
ToArray	Konvertiert <code>IEnumerable<T></code> nach <code>T[]</code> .

Tabelle 12: Konvertierungsoperatoren (Export) (Fortsetzung)

Methode	Beschreibung
ToList	Konvertiert IEnumerable<T> nach List<T>.
ToDictionary	Konvertiert IEnumerable<T> nach Dictionary<TKey, TValue>.
ToHashSet	Konvertiert IEnumerable<T> nach ILookup<TKey, TElement>.
ToLookup	Konvertiert IEnumerable<T> nach ILookup<TKey, TElement>.
AsEnumerable	Downcastet nach IEnumerable<T>.
AsQueryable	Castet oder konvertiert nach IQueryable<T>.

Tabelle 13: Generierungsoperatoren

Methode	Beschreibung
Empty	Erzeugt eine leere Sequenz.
Repeat	Erstellt eine Sequenz mit sich wiederholenden Elementen.
Range	Erstellt eine Sequenz mit Integer-Zahlen.

Abfrageoperatoren verketteten

Um komplexere Abfragen zu erstellen, verketteten Sie Abfrageoperatoren. So extrahiert zum Beispiel die folgende Abfrage alle Strings, die den Buchstaben *a* enthalten, sortiert sie nach der Länge und wandelt dann die Ergebnisse in Großbuchstaben um:

```
string[] names = { "Tom", "Dick", "Harry", "Mary", "Jay" };

IEnumerable<string> query = names
    .Where (n => n.Contains ("a"))
    .OrderBy (n => n.Length)
    .Select (n => n.ToUpper());

foreach (string name in query)
    Console.Write (name + "|");

// RESULT: JAY|MARY|HARRY|
```

Where, OrderBy und Select sind Standard-Abfrageoperatoren, die in Erweiterungsmethoden der Klasse `Enumerable` aufgelöst werden. Der Operator `Where` liefert eine gefilterte Version der Eingabesequenz; `OrderBy` gibt eine sortierte Version seiner Eingabesequenz aus; `Select` liefert eine Sequenz, in der jedes Eingabeelement mit einem angegebenen Lambda-Ausdruck (in diesem Fall `n.ToUpper()`) transformiert oder *projiziert* wird. Die Daten fließen von links nach rechts durch die Kette der Operatoren, sodass die Daten erst gefiltert, dann sortiert und dann projiziert werden. Das Endergebnis sieht aus wie eine Kette von Förderbändern (siehe Abbildung 6).

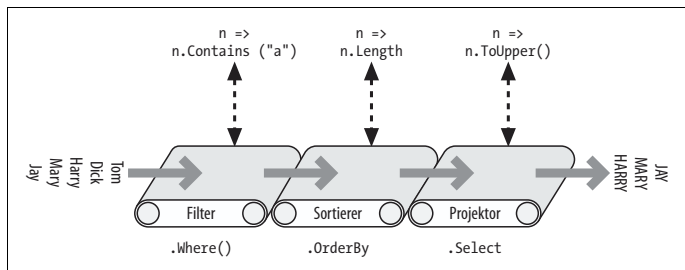


Abbildung 6: Verkettung von Abfrageoperatoren

Die verzögerte Ausführung wird von allen Operatoren unterstützt, daher wird nichts gefiltert, sortiert oder projiziert, bevor die Abfrage tatsächlich enumeriert wird.

Abfrageausdrücke

Bislang haben wir Abfragen geschrieben, indem wir die Erweiterungsmethoden aus der Klasse `Enumerable` aufgerufen haben. In diesem Buch bezeichnen wir das als *fließende Syntax*. C# bietet außerdem eine spezielle Sprachunterstützung zum Schreiben von Abfragen, die den Titel *Abfrageausdrücke* trägt. Hier ist die vorangegangene Abfrage in Form eines Abfrageausdrucks:

```
IEnumerable<string> query =
    from n in names
    where n.Contains ("a")
    orderby n.Length
    select n.ToUpper();
```


in der sie ausgeführt werden. Damit kann Visual Studio Ihnen beim Tippen sinnvolle IntelliSense-Hinweise geben und auch die Scoping-Regeln für Unterabfragen vereinfachen.

Der Compiler verarbeitet den Abfrageausdruck, indem er ihn in die fließende Syntax übersetzt. Das macht er auf eine recht mechanische Weise – ganz ähnlich, wie er foreach-Anweisungen in Aufrufe von GetEnumerator und MoveNext übersetzt:

```
IEnumerable<string> query = names
    .Where (n => n.Contains ("a"))
    .OrderBy (n => n.Length)
    .Select (n => n.ToUpper());
```

Die Operatoren Where, OrderBy und Select werden dann nach denselben Regeln aufgelöst, die gelten würden, wenn die Abfrage in Lambda-Syntax geschrieben wäre. In diesem Fall binden sie an Erweiterungsmethoden der Klasse Enumerable (vorausgesetzt, Sie haben den Namensraum System.Linq importiert), da names das Interface IEnumerable<string> implementiert. Der Compiler bevorzugt aber beim »Übersetzen« der Query-Syntax nicht unbedingt die Klasse Enumerable. Sie können es sich so vorstellen, dass der Compiler ganz mechanisch die Wörter Where, OrderBy und Select in die Anweisung einfügt und das Ganze dann so kompiliert, als hätten Sie die Methodennamen selbst eingetippt. Damit bleibt die Flexibilität beim Auflösen bestehen – die Operatoren in Entity-Framework-Abfragen zum Beispiel binden stattdessen an die Erweiterungsmethoden der Klasse Queryable.

Abfrageausdrücke vs. fließende Syntax

Abfrageausdrücke und die fließende Syntax haben jeweils ihre Vorteile.

Abfrageausdrücke unterstützen nur eine kleine Untermenge der Abfrageoperatoren, und zwar folgende:

```
Where, Select, SelectMany
OrderBy, ThenBy, OrderByDescending, ThenByDescending
GroupBy, Join, GroupJoin
```

Bei Abfragen, die andere Operatoren nutzen, müssen Sie sie entweder komplett in fließender Syntax schreiben oder eine Abfrage mit gemischter Syntax erstellen, zum Beispiel so:

```
string[] names = { "Tom", "Dick", "Harry", "Mary", "Jay" };

IEnumerable<string> query =
    from n in names
    where n.Length == names.Min (n2 => n2.Length)
    select n;
```

Diese Abfrage liefert Namen zurück, deren Länge der der kürzesten Namen entspricht (»Tom« und »Jay«). Die Unterabfrage (in Fettdruck) ermittelt die minimale Länge der verschiedenen Namen, also 3. Wir müssen für die Unterabfrage fließende Syntax benutzen, weil der Operator `Min` in der Abfragesyntax nicht unterstützt wird. Wir können aber immer noch für die äußere Abfrage die Abfragesyntax verwenden.

Der Hauptvorteil der Abfragesyntax ist, dass sie Abfragen radikal vereinfachen kann, die folgende Elemente enthalten:

- Eine `let`-Klausel, mit der neben der Bereichsvariablen eine neue Variable eingeführt wird.
- Mehrere Generatoren (`SelectMany`), gefolgt von einer äußeren Bereichsvariablenreferenz.
- Ein `Join`- oder `GroupJoin`-Äquivalent, gefolgt von einer äußeren Bereichsvariablenreferenz.

Das Schlüsselwort `let`

Das Schlüsselwort `let` führt neben der Bereichsvariablen eine neue Variable ein. Stellen Sie sich zum Beispiel vor, dass wir alle Namen ausgeben wollen, deren Länge ohne Vokale größer als zwei Zeichen ist:

```
string[] names = { "Tom", "Dick", "Harry", "Mary", "Jay" };

IEnumerable<string> query =
    from n in names
    let vowelless = Regex.Replace (n, "[aeiou]", "")
```

```
where vowelless.Length > 2
orderby vowelless
select n + " - " + vowelless;
```

Die Ausgabe beim Enumerieren dieser Abfrage sieht so aus:

```
Dick - Dck
Harry - Hrry
Mary - MrY
```

Die `let`-Klausel führt für jedes Element eine Berechnung durch, ohne das ursprüngliche Element zu verlieren. In unserer Abfrage haben die folgenden Klauseln (`where`, `orderby` und `select`) Zugriff sowohl auf `n` als auch auf `vowelless`. Abfragen können mehrere `let`-Klauseln nutzen und diese bei zusätzlichen `where`- und `join`-Klauseln einstreuen.

Der Compiler übersetzt das Schlüsselwort `let`, indem er in einen temporären, anonymen Typ projiziert, der das ursprüngliche und das transformierte Element enthält:

```
IEnumerable<string> query = names
    .Select (n => new
        {
            n = n,
            vowelless = Regex.Replace (n, "[aeiou]", "")
        }
    )
    .Where (temp0 => (temp0.vowelless.Length > 2))
    .OrderBy (temp0 => temp0.vowelless)
    .Select (temp0 => ((temp0.n + " - ") + temp0.vowelless))
```

Abfragefortsetzungen

Wenn Sie nach einer `select`- oder `group`-Klausel noch Klauseln anhängen wollen, müssen Sie das Schlüsselwort `into` nutzen, um die Abfrage »fortzusetzen«. Hier sehen Sie ein Beispiel dafür:

```
from c in "The quick brown tiger".Split()
select c.ToUpper() into upper
where upper.StartsWith ("T")
select upper

// Ergebnis: "THE", "TIGER"
```

Nach einer `into`-Klausel ist die vorherige Bereichsvariable nicht mehr gültig.

Der Compiler übersetzt Abfragen mit einem `into`-Schlüsselwort einfach in eine längere Kette von Operatoren:

```
"The quick brown tiger".Split()  
.Select (c => c.ToUpper())  
.Where (upper => upper.StartsWith ("T"))
```

(Er lässt das letzte `Select(upper=>upper)` weg, da es redundant ist.)

Mehrere Generatoren

Eine Abfrage kann mehrere Generatoren (`from`-Klauseln) enthalten:

```
int[] numbers = { 1, 2, 3 };  
string[] letters = { "a", "b" };  
  
IEnumerable<string> query = from n in numbers  
                           from l in letters  
                           select n.ToString() + l;
```

Das Ergebnis ist ein Kreuzprodukt, wie Sie es bei verschachtelten `foreach`-Schleifen erhalten würden:

```
"1a", "1b", "2a", "2b", "3a", "3b"
```

Wenn es in einer Abfrage mehr als eine `from`-Klausel gibt, erzeugt der Compiler einen Aufruf von `SelectMany`:

```
IEnumerable<string> query = numbers.SelectMany (  
    n => letters,  
    (n, l) => (n.ToString() + l));
```

`SelectMany` führt verschachtelte Schleifendurchläufe aus. Es enumeriert jedes Element in der Quell-Collection (`numbers`) und wandelt jedes Element mit dem ersten Ausdruck um (`letters`). Damit wird eine Sequenz aus *Subsequenzen* erzeugt, die dann enumeriert werden. Die endgültigen Ausgabeelemente werden durch den zweiten Ausdruck bestimmt (`n.ToString() + l`).

Wenn Sie anschließend eine `where`-Klausel anwenden, können Sie das Kreuzprodukt filtern und ein Ergebnis projizieren, das einem *Join* ähnelt:

```
string[] players = { "Tom", "Jay", "Mary" };
```

```
IEnumerable<string> query =  
    from name1 in players  
    from name2 in players  
    where name1.CompareTo (name2) < 0  
    orderby name1, name2  
    select name1 + " vs " + name2;
```

```
RESULT: { "Jay vs Mary", "Jay vs Tom", "Mary vs Tom" }
```

Das Übersetzen dieser Abfrage in die fließende Syntax ist natürlich deutlich komplexer, da eine temporäre anonyme Projektion erforderlich ist. Die Möglichkeit, diese Übersetzung automatisch durchführen zu lassen, ist einer der Hauptvorteile der Abfragesyntax.

Der Ausdruck im zweiten Generator darf die erste Bereichsvariable nutzen:

```
string[] fullNames =  
    { "Anne Williams", "John Fred Smith", "Sue Green" };
```

```
IEnumerable<string> query =  
    from fullName in fullNames  
    from name in fullName.Split()  
    select name + " kommt aus " + fullName;
```

```
Anne kommt aus Anne Williams  
Williams kommt aus Anne Williams  
John kommt aus John Fred Smith
```

Das funktioniert, weil der Ausdruck `fullName.Split` eine *Sequenz* ausgibt (ein Array mit Strings).

In Datenbankabfragen werden sehr häufig mehrere Generatoren eingesetzt, um Eltern-Kind-Beziehungen zu ebnen und manuelle Joins durchzuführen.

Verknüpfen

LINQ bietet drei *Verknüpfungsoperatoren*, von denen `Join` und `GroupJoin` am wichtigsten sind, die schlüsselbasierte Joins durchführen. `Join` und `GroupJoin` unterstützen nur eine Untermenge der Funktionalität, die Sie mit mehreren Generatoren/`SelectMany` erhalten können, sind bei lokalen Abfragen aber leistungsfähiger, weil

sie eine Hashtable-basierte Suchstrategie nutzen und auf geschachtelte Schleifen verzichten. (Bei Entity-Framework-Abfragen bieten die Verknüpfungsoperatoren keine Vorteile gegenüber mehreren Generatoren.)

Join und GroupJoin unterstützen nur *Equi-Joins* (das heißt, die Verknüpfungsbedingung muss den Gleichheitsoperator nutzen). Es gibt dabei zwei Methoden: Join und GroupJoin. Join erzeugt eine flache Ergebnismenge, während GroupJoin eine hierarchische Ergebnismenge liefert.

Die Abfrageausdrucksyntax für einen flachen Join sieht so aus:

```
from äußere-Var in äußere-Sequenz
join innere-Var in innere-Sequenz
    on äußerer-Schlüssel equals innerer-Schlüssel
```

Nehmen wir zum Beispiel die folgenden Collections:

```
var customers = new[]
{
    new { ID = 1, Name = "Tom" },
    new { ID = 2, Name = "Dick" },
    new { ID = 3, Name = "Harry" }
};
var purchases = new[]
{
    new { CustomerID = 1, Product = "Haus" },
    new { CustomerID = 2, Product = "Boot" },
    new { CustomerID = 2, Product = "Auto" },
    new { CustomerID = 3, Product = "Urlaub" }
};
```

Damit können wir auf folgende Weise eine Verknüpfung durchführen:

```
IEnumerable<string> query =
    from c in customers join p in purchases on c.ID equals p.
CustomerID
    select c.Name + " hat gekauft: " + p.Product;
```

Der Compiler übersetzt das in

```
customers.Join (                // äußere Collection
    purchases,                  // innere Collection
    c => c.ID,                  // äußerer Schlüssel
    p => p.CustomerID,         // innerer Schlüssel
```

```
(c, p) => // Ergebnisselektor
    c.Name + " hat gekauft: " + p.Product
);
```

Und so sieht das Ergebnis aus:

```
Tom hat gekauft: Haus
Dick hat gekauft: Boot
Dick hat gekauft: Auto
Harry hat gekauft: Urlaub
```

Bei lokalen Sequenzen sind `Join` und `GroupJoin` effizienter beim Verarbeiten großer Collections als `SelectMany`, weil sie zunächst die innere Sequenz in einen Schlüssel-Hashtable-basierten Lookup laden. Bei einer Datenbankabfrage können Sie das gleiche Ergebnis genauso effizient erreichen, wenn Sie so vorgehen:

```
from c in customers
from p in purchases
where c.ID == p.CustomerID
select c.Name + " bought a " + p.Product;
```

GroupJoin

`GroupJoin` macht dasselbe wie `Join`, liefert aber keine flache Ergebnismenge zurück, sondern ein hierarchisches Ergebnis, das nach den einzelnen äußeren Elementen gruppiert ist.

Die Syntax für `GroupJoin` ist die gleiche wie für `Join`, nur dass ihr das Schlüsselwort `into` folgt. Ein einfaches Beispiel mit den Collections `customers` und `purchases`, die wir im vorigen Abschnitt angelegt haben, sieht so aus:

```
var query =
    from c in customers
    join p in purchases on c.ID equals p.CustomerID
    into custPurchases
    select custPurchases; // custPurchases ist eine Sequenz
```



Eine `into`-Klausel wird nur dann in einen `GroupJoin` umgewandelt, wenn sie direkt auf eine `join`-Klausel folgt. Nach einer `select`- oder `group`-Klausel bedeutet sie eine *Abfragefortsetzung*. Die zwei Anwendungsarten

des Schlüsselworts `into` sind ziemlich verschieden, sie haben aber ein Feature gemeinsam: Beide führen eine neue Abfragevariable ein.

Das Ergebnis ist eine Sequenz mit Sequenzen – `IEnumerable<IEnumerable<T>>` –, die wir wie folgt enumerieren könnten:

```
foreach (var purchaseSequence in query)
    foreach (var purchase in purchaseSequence)
        Console.WriteLine (purchase.Product);
```

Das ist natürlich nicht sehr nützlich, da `outerSeq` keine Referenz auf den äußeren Kunden enthält. Im Allgemeinen werden Sie die äußere Bereichsvariable in der Projektion referenzieren:

```
from c in customers
join p in purchases on c.ID equals p.CustomerID
into custPurchases
select new { CustName = c.Name, custPurchases };
```

Wir könnten das gleiche Ergebnis (allerdings für lokale Abfragen weniger effektiv) erreichen, indem wir in einen anonymen Typ projizieren, der eine Unterabfrage enthält:

```
from c in customers
select new
{
    CustName = c.Name,
    custPurchases =
        purchases.Where (p => c.ID == p.CustomerID)
}
```

Zip

`Zip` ist der einfachste Verknüpfungsoperator. Er enumeriert zwei Sequenzen nebeneinander (wie ein Reißverschluss) und liefert eine Sequenz, die auf der Anwendung einer Funktion auf die jeweiligen Elementpaare basiert. Hier sehen Sie ein Beispiel dafür:

```
int[] numbers = { 3, 5, 7 };
string[] words = { "drei", "fünf", "sieben", "ignoriert" };
IEnumerable<string> zip =
    numbers.Zip (words, (n, w) => n + "=" + w);
```

Dieser Ausdruck erzeugt eine Sequenz mit den folgenden Elementen:

```
3=drei  
5=fünf  
7=sieben
```

Zusätzliche Elemente in einer der Eingabesequenzen werden ignoriert. Zip wird bei Abfragen von Datenbanken nicht unterstützt.

Ordnen

Das Schlüsselwort `orderby` sortiert eine Sequenz. Sie können beliebig viele Ausdrücke angeben, nach denen sortiert werden soll:

```
string[] names = { "Tom", "Dick", "Harry", "Mary", "Jay" };  
  
IEnumerable<string> query = from n in names  
                             orderby n.Length, n  
                             select n;
```

Damit wird zuerst nach der Länge und dann nach dem Namen sortiert, sodass das Ergebnis so aussieht:

```
Jay, Tom, Dick, Mary, Harry
```

Der Compiler übersetzt den ersten `orderby`-Ausdruck in einen Aufruf von `OrderBy`, die folgenden werden dann zu `ThenBy` umgewandelt:

```
IEnumerable<string> query = names  
    .OrderBy (n => n.Length)  
    .ThenBy (n => n)
```

Der Operator `ThenBy` *ergänzt* die vorige Sortierung, *ersetzt* sie aber nicht.

Sie können das Schlüsselwort `descending` in den `orderby`-Ausdrücken nutzen:

```
orderby n.Length descending, n
```

Das wird umgewandelt in:

```
.OrderByDescending (n => n.Length).ThenBy (n => n)
```



Die Ordnungsoperatoren liefern einen erweiterten Typ von `IEnumerable<T>` namens `IOrderedEnumerable<T>` zurück. Dieses Interface definiert die zusätzliche Funktionalität, die vom `ThenBy`-Operator benötigt wird.

Gruppieren

`GroupBy` arrangiert eine flache Eingabesequenz in Sequenzen aus *Gruppen*. So gruppiert zum Beispiel der folgende Code eine Sequenz aus Namen nach ihrer Länge:

```
string[] names = { "Tom", "Dick", "Harry", "Mary", "Jay" };

var query = from name in names
            group name by name.Length;
```

Der Compiler wandelt das um in:

```
IEnumerable<IGrouping<int,string>> query =
    names.GroupBy (name => name.Length);
```

So kann man das Ergebnis enumerieren:

```
foreach (IGrouping<int,string> grouping in query)
{
    Console.WriteLine ("\r\n Length=" + grouping.Key + ":");
    foreach (string name in grouping)
        Console.WriteLine (" " + name);
}
```

```
Length=3: Tom Jay
Length=4: Dick Mary
Length=5: Harry
```

`Enumerable.GroupBy` liest die Eingabeelemente in ein temporäres Dictionary mit Listen ein, sodass alle Elemente mit demselben Schlüssel in derselben Unterliste landen. Dann gibt die Methode eine Sequenz aus *Gruppierungen* aus. Eine Gruppierung ist eine Sequenz mit einer Eigenschaft `Key`:

```
public interface IGrouping <TKey,TElement>
: IEnumerable<TElement>, IEnumerable
{
    // Key gilt für die ganze Untersequenz.
    TKey Key { get; }
}
```

Standardmäßig handelt es sich bei den Elementen in jeder Gruppierung um die nicht transformierten Eingabeelemente, sofern Sie kein Argument `elementSelector` angeben. Durch den folgenden Code wird jedes Eingabeelement in Großbuchstaben umgewandelt:

```
from name in names
group name.ToUpper() by name.Length
```

Das wird umgewandelt in

```
names.GroupBy (
    name => name.Length,
    name => name.ToUpper() )
```

Die Untercollections werden nicht in der Reihenfolge ihrer Schlüssel ausgegeben. `GroupBy` führt keine Sortierung durch (es behält sogar die ursprüngliche Reihenfolge bei). Um sortieren zu können, müssen Sie einen Operator `OrderBy` hinzufügen (was bedeutet, dass Sie zunächst eine `into`-Klausel anhängen müssen, da `group by` normalerweise eine Abfrage abschließt):

```
from name in names
group name.ToUpper() by name.Length into grouping
orderby grouping.Key
select grouping
```

Abfragefortsetzungen werden häufig in einer Abfrage mit `group by` genutzt. Die nächste Abfrage filtert Gruppen heraus, die aus genau zwei Einträgen bestehen:

```
from name in names
group name.ToUpper() by name.Length into grouping
where grouping.Count() == 2
select grouping
```



Ein `where` nach einem `group by` entspricht `HAVING` in SQL. Es wird für jede Untersequenz bzw. Gruppierung als Ganzes angewendet, statt auf die einzelnen Elemente zu wirken.

OfType und Cast

`OfType` und `Cast` erwarten eine nichtgenerische Collection `IEnumerable` und geben eine generische Sequenz `IEnumerable<T>` zurück, die Sie dann im Weiteren abfragen können:

```
var classicList = new System.Collections.ArrayList();
classicList.AddRange ( new int[] { 3, 4, 5 } );
IEnumerable<int> sequence1 = classicList.Cast<int>();
```

Das ist nützlich, da Sie damit Collections abfragen können, die vor C# 2.0 geschrieben wurden (als `IEnumerable<T>` ins Spiel kam), zum Beispiel `ControlCollection` in `System.Windows.Forms`.

`Cast` und `OfType` unterscheiden sich in ihrem Verhalten, wenn sie auf ein Eingabeelement treffen, dessen Typ nicht kompatibel ist: `Cast` wirft eine Exception, während `OfType` das inkompatible Element ignoriert.

Die Regeln der Elementkompatibilität entsprechen denen des `is`-Operators von C#. So sieht die interne Implementierung von `Cast` aus:

```
public static IEnumerable<TSource> Cast <TSource>
    (IEnumerable source)
{
    foreach (object element in source)
        yield return (TSource)element;
}
```

C# unterstützt den Operator `Cast` in Abfrageausdrücken. Fügen Sie einfach direkt nach dem Schlüsselwort `from` den Elementtyp ein:

```
from int x in classicList ...
```

Das wird in Folgendes übersetzt:

```
from x in classicList.Cast <int>() ...
```

Die dynamische Bindung

Dynamische Bindung schiebt die *Bindung* – den Vorgang der Auflösung von Typen, Mitgliedern und Operatoren – von der Kompilierzeit bis zur Laufzeit hinaus. Dynamische Bindung ist praktisch, wenn Sie zur Kompilierzeit wissen, dass eine bestimmte Funktion, ein bestimmtes Member oder ein bestimmter Operator existiert, während der Compiler es nicht weiß. Das tritt beispielsweise bei der Interaktion mit dynamischen Sprachen (wie IronPython), bei COM und in anderen Szenarien ein, bei denen Sie ansonsten Reflection nutzen würden.

Ein dynamischer Typ wird mit dem kontextabhängigen Schlüsselwort `dynamic` deklariert:

```
dynamic d = GetSomeObject();  
d.Quack();
```

Ein dynamischer Typ sagt dem Compiler, dass er sich entspannen soll: »Wir gehen davon aus, dass der Laufzeittyp von `d` eine `Quack`-Methode bietet wird. Wir können das statisch einfach nur nicht belegen.« Da `d` dynamisch ist, schiebt der Compiler die Bindung von `Quack` an `d` bis zur Laufzeit hinaus. Um das zu begreifen, müssen Sie verstehen, welcher Unterschied zwischen *statischer* und *dynamischer* Bindung besteht.

Statische Bindung vs. dynamische Bindung

Das übliche Beispiel für die Bindung ist die Zuordnung eines Namens zu einer bestimmten Funktion bei der Kompilierung eines Ausdrucks. Wenn der Compiler den folgenden Ausdruck kompilieren soll, muss er die Implementierung einer Methode namens `Quack` finden:

```
d.Quack();
```

Nehmen wir an, der statische Typ von `d` ist `Duck`:

```
Duck d = ...  
d.Quack();
```

Im einfachsten Fall führt der Compiler die Bindung durch, indem er nach einer parameterlosen Methode mit dem Namen `Quack` auf `Duck` sucht. Kann er keine finden, erweitert er seine Suche auf Methoden aus, die optionale Parameter erwarten, Methoden in Basisklassen von `Duck` und Erweiterungsmethoden, die `Duck` als ersten Parameter erwarten. Wenn keine passende Methode gefunden wird, erhalten Sie einen Kompilierungsfehler. Ganz gleich, welche Methode gebunden wird – entscheidend ist, dass das Binden durch den Compiler erfolgt und vollständig von einer statischen Kenntnis der Typen (hier `d`) der Operanden abhängig ist. Das ist das, was diese Art der Bindung *statisch* macht.

Ändern wir nun den statischen Typ von `d` in `object`:

```
object d = ...  
d.Quack();
```

Dieser Aufruf von `Quack` führt zu einem Kompilierungsfehler, da der in `d` gespeicherte Wert zwar eine Methode namens `Quack` besitzen kann, der Compiler das aber nicht wissen kann, weil seine einzige Information der Typ der Variablen ist, der in diesem Fall `object` ist. Ändern wir jetzt einmal den statischen Typ von `d` in `dynamic`:

```
dynamic d = ...  
d.Quack();
```

Der Typ `dynamic` ist wie `object` – er sagt ebenso wenig über den (tatsächlichen) Typ aus. Der Unterschied ist, dass er Ihnen gestattet, ihn auf Weisen zu verwenden, die zur Kompilierungszeit nicht bekannt sind. Ein dynamisches Objekt bindet zur Laufzeit auf Basis der Laufzeittypen, nicht auf Basis des Kompilierzeittyps. Wenn der Compiler einen dynamisch gebundenen Ausdruck sieht (was in der Regel ein Ausdruck ist, der einen Wert des Typs `dynamic` einschließt), verpackt er diesen Ausdruck einfach so, dass die Bindung später zur Laufzeit erfolgen kann.

Wenn ein dynamisches Objekt `IDynamicMetaObjectProvider` implementiert, wird zur Laufzeit dieses Interface genutzt, um die Bindung durchzuführen. Ist das nicht der Fall, erfolgt die Bindung fast auf die gleiche Weise, auf die sie erfolgt wäre, wenn der Compiler den Laufzeittyp des dynamischen Objekts gekannt hätte. Diese beiden Optionen werden als *benutzerdefinierte Bindung* und *Sprachbindung* bezeichnet.

Benutzerdefinierte Bindung

Benutzerdefinierte Bindung erfolgt, wenn ein dynamisches Objekt `IDynamicMetaObjectProvider` (IDMOP) implementiert. Obwohl Sie IDMOP auf Typen implementieren können, die Sie in C# schreiben – und das auch sehr praktisch sein kann –, ist es dennoch am häufigsten der Fall, dass Sie ein IDMOP-Objekt von einer dynamischen Sprache erhalten haben, die in .NET auf der *Dynamic Language Runtime* (DLR) implementiert, ist wie IronPython oder IronRuby. Objekte von diesen Sprachen implementieren IDMOP implizit als

Mittel zur unmittelbaren Steuerung der Operationen, die von ihnen ausgeführt werden. Hier ist ein einfaches Beispiel dafür:

```
dynamic d = new Duck();
d.Quack();           // Quack wurde gerufen
d.Waddle();          // Waddle wurde gerufen

public class Duck : DynamicObject // in System.Dynamic
{
    public override bool TryInvokeMember (
        InvokeMemberBinder binder, object[] args,
        out object result)
    {
        Console.WriteLine (binder.Name + " wurde gerufen");
        result = null;
        return true;
    }
}
```

Die Klasse Duck hat eigentlich keine Quack-Methode. Stattdessen nutzt sie benutzerdefinierte Bindung, um alle Methodenaufrufe abzufangen und zu interpretieren.

Sprachbindung

Sprachbindung findet statt, wenn ein dynamisches Objekt `IDynamicMetaObjectProvider` nicht implementiert. Sprachbindung ist praktisch, wenn man mit schlecht entworfenen Typen arbeiten muss oder eine Möglichkeit sucht, die Grenzen des .NET-Typsystems auszuhebeln. Ein typisches Problem bei der Verwendung numerischer Typen ist, dass diese keine einheitliche Schnittstelle haben. Wir haben gesehen, dass Methoden dynamisch gebunden werden können – und das gilt auch für Operatoren:

```
int x = 3, y = 4;
Console.WriteLine (Mean (x, y));

dynamic Mean (dynamic x, dynamic y) => (x+y) / 2;
```

Der Vorteil ist offensichtlich: Sie müssen diesen Code nicht mehr einzeln für jeden numerischen Typ schreiben. Aber Sie verlieren damit die statische Typsicherheit und riskieren Laufzeitfehler anstelle von Kompilierungsfehlern.



Dynamische Bindung umgeht die statische Typsicherheit, aber nicht die Laufzeit-Typsicherheit. Anders als bei Reflection können Sie mit dynamischer Bindung die Zugriffsregeln für Member nicht aushebeln.

Die Sprachbindung zur Laufzeit wurde so gestaltet, dass sie sich der statischen Bindung so ähnlich wie möglich verhält, wenn die Laufzeittypen dynamischer Objekte bei der Kompilierung bekannt gewesen wären. In unserem vorangegangenen Beispiel hätte sich unser Programm auf genau die gleiche Weise verhalten, wenn wir `Mean` so definiert hätten, dass es mit dem Typ `int` funktioniert. Die auffälligste Ausnahme bei der Gleichheit von statischer und dynamischer Bindung taucht bei Erweiterungsmethoden auf, mit denen wir uns im Abschnitt »Nicht aufrufbare Funktionen« auf Seite 219 befassen werden.



Dynamische Bindung bringt Leistungseinbußen mit sich. Aufgrund der Caching-Mechanismen der DLR werden wiederholte Aufrufe dynamischer Ausdrücke jedoch optimiert – das ermöglicht Ihnen, dynamische Ausdrücke effizient in Schleifen aufzurufen. Diese Optimierung reduziert den üblichen Nachteil für einen einfachen dynamischen Ausdruck auf moderner Hardware auf weniger als 100 Nanosekunden.

RuntimeBinderException

Wenn ein Member nicht gebunden werden kann, wird eine `RuntimeBinderException` ausgelöst. Diese können Sie als eine Art Compilerfehler zur Laufzeit betrachten:

```
dynamic d = 5;  
d.Hello();           // löst RuntimeBinderException aus
```

Die Exception wird ausgelöst, weil der Typ `int` keine `Hello`-Methode hat.

Laufzeitdarstellung von dynamic

Es gibt eine tiefgehende Äquivalenz zwischen den Typen `dynamic` und `object`. Die Laufzeitumgebung behandelt den folgenden Ausdruck als `true`:

```
typeof (dynamic) == typeof (object)
```

Dieses Prinzip erstreckt sich auch auf Collection- und Array-Typen:

```
typeof (List<dynamic>) == typeof (List<object>)  
typeof (dynamic[]) == typeof (object[])
```

Wie eine Objektreferenz kann eine dynamische Referenz auf ein Objekt eines beliebigen Typs (außer Zeigertypen) zeigen:

```
dynamic x = "Hallo";  
Console.WriteLine (x.GetType().Name); // String  
  
x = 123; // kein Fehler (obwohl die gleiche Variable  
        // genutzt wird)  
Console.WriteLine (x.GetType().Name); // Int32
```

Strukturell gibt es keinen Unterschied zwischen einer Objektreferenz und einer dynamischen Referenz. Eine dynamische Referenz macht einfach nur dynamische Operationen auf dem Objekt möglich, auf das sie zeigt. Sie können von `object` in `dynamic` umwandeln, um beliebige dynamische Operationen an einem `object` auszuführen:

```
object o = new System.Text.StringBuilder();  
dynamic d = o;  
d.Append ("Hallo");  
Console.WriteLine (o); // Hallo
```

Dynamische Konvertierungen

Der Typ `dynamic` bietet implizite Konvertierungen in alle anderen Typen und zurück. Eine Konvertierung ist dann erfolgreich, wenn der Laufzeittyp des dynamischen Objekts implizit in den statischen Zieltyp konvertierbar ist.

Das folgende Beispiel löst eine `RuntimeBinderException` aus, weil ein `int` nicht implizit in einen `short` umgewandelt werden kann:

```
int i = 7;
dynamic d = i;
long l = d;           // Okay - implizite Konvertierung klappt
short j = d;          // RuntimeBinderException
```

var vs. dynamic

Die Typen `var` und `dynamic` weisen eine oberflächliche Ähnlichkeit auf, aber die Unterschiede gehen tief:

- `var` sagt: »Lass den *Compiler* den Typ herausfinden.«
- `dynamic` sagt: »Lass die *Laufzeit* den Typ herausfinden.«

Zur Illustration:

```
dynamic x = "Hallo"; // statischer Typ ist dynamic
var y = "Hallo";     // statischer Typ ist string
int i = x;           // Laufzeitfehler
int j = y;           // Kompilierungsfehler
```

Dynamische Ausdrücke

Felder, Eigenschaften, Methoden, Konstruktoren, Indexer, Operatoren und Konvertierungen können alle dynamisch aufgerufen werden.

Es ist verboten, das Ergebnis eines dynamischen Ausdrucks mit dem Rückgabetypp `void` weiterzuverarbeiten – genau wie bei statisch typisierten Ausdrücken. Der Unterschied ist, dass der Fehler erst zur Laufzeit auftritt.

Ausdrücke mit dynamischen Operanden sind üblicherweise selbst dynamisch, da die Wirkung der fehlenden Typinformationen im Ausdruck weitergereicht wird:

```
dynamic x = 2;
var y = x * 3; // Der statische Typ von y ist dynamisch.
```

Es gibt einige offensichtliche Ausnahmen von dieser Regel. Erstens erhält man einen statischen Ausdruck, wenn man einen dynamischen Ausdruck auf einen statischen Typ castet, und zweitens liefern Konstruktoraufrufe immer statische Ausdrücke, selbst wenn sie mit dynamischen Argumenten aufgerufen wurden.

Außerdem gibt es ein paar Randfälle, bei denen ein Ausdruck, der einen dynamischen Operanden enthält, statisch ist, unter anderem die Übergabe eines Index an ein Array und Ausdrücke zur Delegate-Erstellung.

Die dynamische Auflösung überladener Member

Der übliche Anwendungsfall für `dynamic` schließt einen dynamischen Empfänger ein. Das heißt, dass ein dynamisches Objekt der Empfänger eines dynamischen Funktionsaufrufs ist:

```
dynamic x = ...;  
x.Foo (123);           // x ist der Empfänger
```

Aber statische Bindung ist nicht auf Empfänger beschränkt: Methodenargumente können ebenfalls dynamisch gebunden werden. Der Aufruf einer Funktion mit dynamischen Argumenten bewirkt, dass die Auflösung von Überladungen von der Kompilierung bis zur Laufzeit aufgeschoben wird:

```
static void Foo (int x)    => Console.WriteLine ("int");  
static void Foo (string x) => Console.WriteLine ("str");  
  
static void Main()  
{  
    dynamic x = 5;  
    dynamic y = "Wassermelone";  
  
    Foo (x);    // int  
    Foo (y);    // str  
}
```

Die Überladungsauflösung zur Laufzeit wird auch als *Multiple Dispatch* bezeichnet und ist bei der Implementierung von Entwurfsmustern wie dem *Besuchermuster* hilfreich.

Wenn kein dynamischer Empfänger beteiligt ist, kann der Compiler statisch eine einfache Prüfung darauf durchführen, ob der dynamische Aufruf gelingen wird: Er prüft einfach, ob es eine Funktion mit dem richtigen Namen und der richtigen Anzahl von Argumenten gibt. Wird kein Kandidat gefunden, erhalten Sie einen Kompilierungsfehler.

Wenn eine Funktion mit einer Mischung aus dynamischen und statischen Argumenten aufgerufen wird, spiegelt die endgültige Entscheidung eine Mischung dynamischer und statischer Bindungsentscheidungen:

```
static void X(object x, object y) => Console.Write("oo");
static void X(object x, string y) => Console.Write("os");
static void X(string x, object y) => Console.Write("so");
static void X(string x, string y) => Console.Write("ss");

static void Main()
{
    object o = "Hello";
    dynamic d = "Goodbye";
    X(o, d);           // os
}
```

Der Aufruf von `X(o,d)` wird dynamisch gebunden, weil eines seiner Argumente, `d`, `dynamic` ist. Aber da `o` statisch bekannt ist, nutzt der Bindungsvorgang – obwohl er dynamisch erfolgt – diese Informationen. Hier wählt die Überladungsauflösung aufgrund des statischen Typs von `o` und des Laufzeittyps von `d` die zweite Implementierung von `X`. Anders formuliert: Der Compiler verhält sich so statisch, wie es eben geht .

Nicht aufrufbare Funktionen

Manche Funktionen können dynamisch nicht aufgerufen werden:

- Erweiterungsmethoden (über die Erweiterungsmethodensyntax)
- beliebige Member eines Interface (über das Interface)
- von einer Subklasse verborgene Basisklassen-Member

Das liegt daran, dass die dynamische Bindung zwei Informationselemente benötigt: den Namen der aufzurufenden Funktion und das Objekt, auf dem die Funktion aufgerufen werden soll. Aber bei den drei nicht aufrufbaren Konstrukten ist jeweils ein *zusätzlicher Typ* beteiligt, der nur zur Kompilierungszeit bekannt ist. Und es gibt keine Möglichkeit, diesen zusätzlichen Typ dynamisch anzugeben.

Wenn Sie Erweiterungsmethoden aufrufen, ist dieser zusätzliche Typ eine Erweiterungsklasse, die implizit auf Basis der `using`-Direk-

tiven in Ihrem Quellcode gewählt wird (die nach der Kompilierung verschwinden). Wenn Sie Member über ein Interface aufrufen, wird der zusätzliche Typ über einen impliziten oder expliziten Cast kommuniziert. (Bei expliziter Implementierung ist es in der Tat unmöglich, ein Member ohne einen Cast auf das Interface aufzurufen.) Ähnlich verhält es sich, wenn Sie versuchen, ein verborgenes Basis-klassen-Member aufzurufen: Sie müssen einen zusätzlichen Typ angeben, entweder über einen Cast oder über das Schlüsselwort `base` – und diese zusätzlichen Typinformationen sind zur Laufzeit verloren.

Überladen von Operatoren

Operatoren können überladen werden, um eine natürlichere Syntax für eigene Typen anzubieten. Das Überladen von Operatoren ist dann am sinnvollsten, wenn Sie eigene Structs implementieren, die halbwegs primitive Datentypen repräsentieren. So ist zum Beispiel ein eigener numerischer Typ ein exzellenter Kandidat für das Überladen von Operatoren.

Die symbolischen Operatoren, die sich überladen lassen, sind:

```
+  -  *  /  ++  --  !  ~  %  &  |  ^  
== != <  << >>  >
```

Implizite und explizite Konvertierungen können genauso überschrieben werden (mit den Schlüsselwörtern `implicit` und `explicit`) wie die Operatoren `true` und `false`.

Die zusammengesetzten Zuweisungsoperatoren (z. B. `+=` oder `/=`) werden automatisch überschrieben, wenn Sie die nicht zusammengesetzten Operatoren überschreiben (z. B. `+` oder `/`).

Operatorfunktion

Ein Operator wird überladen, indem man eine *Operatorfunktion* deklariert. Eine Operatorfunktion muss statisch sein, und mindestens einer der Operanden muss dem Typ entsprechen, in dem die Operatorfunktion deklariert wird.

Im folgenden Beispiel definieren wir ein Struct namens Note, das eine Musiknote repräsentiert, und überladen dann den Operator +:

```
public struct Note
{
    int value;

    public Note (int semitonesFromA)
        => value = semitonesFromA;

    public static Note operator + (Note x, int semitones)
    {
        return new Note (x.value + semitones);
    }
}
```

Durch dieses Überladen können wir nun ein int zu einer Note addieren:

```
Note B = new Note (2);
Note CSharp = B + 2;
```

Da wir + überschrieben haben, können wir auch += nutzen:

```
CSharp += 2;
```

Wie bei Methoden und Eigenschaften ist es erlaubt, Operatorfunktionen, die aus einem einzelnen Ausdruck bestehen, mit der Expression-bodied Syntax kompakter zu schreiben:

```
public static Note operator + (Note x, int semitones)
    => new Note (x.value + semitones);
```

Überladen von Gleichheits- und Vergleichsoperatoren

Gleichheits- und Vergleichsoperatoren werden häufig überschrieben, wenn man Structs erstellt, und in seltenen Fällen auch beim Schreiben von Klassen. Es gibt spezielle Regeln und Pflichten beim Überladen dieser Operatoren:

Paare

Der C#-Compiler erwartet, dass bei Operatoren, die logische Paare bilden, beide definiert sind. Dabei handelt es sich um (== !=), (< >) und (<= >=).

Equals und GetHashCode

Wenn Sie == und != überladen, werden Sie normalerweise auch die Methoden Equals und GetHashCode überschreiben müssen, sodass Collections und Hashtabellen zuverlässig mit dem Typ arbeiten können.

IComparable und IComparable<T>

Wenn Sie > und < überladen, werden Sie im Allgemeinen auch IComparable und IComparable<T> implementieren.

Wenn wir das vorige Beispiel erweitern, können wir den Gleichheitsoperator von Note überladen:

```
public static bool operator == (Note n1, Note n2)
    => n1.value == n2.value;
public static bool operator != (Note n1, Note n2)
    => !(n1.value == n2.value);

public override bool Equals (object otherNote)
{
    if (!(otherNote is Note)) return false;
    return this == (Note)otherNote;
}
// Den Hashcode des Werts nutzen.
public override int GetHashCode()
    => value.GetHashCode();
```

Eigene implizite und explizite Konvertierungen

Implizite und explizite Konvertierungen sind überladbare Operatoren. Diese Konvertierungen werden normalerweise überladen, um das Umwandeln eng verwandter Typen (wie zum Beispiel numerischer Typen) exakt und natürlich zu ermöglichen.

Wie bei der Betrachtung von Typen erläutert, ist das Prinzip bei impliziten Konvertierungen, dass sie immer erfolgreich sein sollten und während der Konvertierung keine Informationen verloren gehen sollen. Andernfalls sollten explizite Konvertierungen definiert werden.

Im folgenden Beispiel definieren wir Konvertierungen zwischen unserem musikalischen Typ Note und einem double (der die Frequenz der Note in Hertz repräsentiert):

```

...
// Konvertieren in Hertz
public static implicit operator double (Note x)
    => 440 * Math.Pow (2,(double) x.value / 12 );

// Konvertieren aus Hertz (in den nächsten Halbtton)
public static explicit operator Note (double x)
    => new Note ((int) (0.5 + 12 * (Math.Log(x/440)
        / Math.Log(2)) ));
...

Note n =(Note)554.37; // explizite Konvertierung
double x = n;        // implizite Konvertierung

```



Das ist ein etwas künstliches Beispiel: In der Praxis würden diese Konvertierungen wahrscheinlich besser über eine *ToFrequency*-Methode und eine (statische) *FromFrequency*-Methode implementiert.

Benutzerdefinierte Konvertierungen werden von den Operatoren *as* und *is* ignoriert.

Attribute

Sie sind schon darin geübt, Codeelemente eines Programms mit Modifikatoren zu versehen, zum Beispiel *virtual* oder *ref*. Diese Konstrukte sind in die Sprache eingebaut. *Attribute* sind ein Erweiterungsmechanismus, um Codeelementen (Assemblies, Typen, Membern, Rückgabewerten und Parametern) eigene Informationen hinzuzufügen. Diese Erweiterbarkeit ist für Services nützlich, die tief in das Typsystem eingebunden sind, ohne besondere Schlüsselwörter oder Konstrukte in C# erforderlich zu machen.

Ein gutes Szenario für Attribute ist die *Serialisierung* – der Prozess, bei dem eigene Objekte zum Speichern oder Übertragen in ein bestimmtes Format umgewandelt und wieder zurückgeholt werden. In diesem Szenario kann ein Attribut für ein Feld die Umsetzung zwischen der Repräsentation eines Felds in C# und der im Speicherformat definieren.

Attributklassen

Ein Attribut wird durch eine Klasse definiert, die (direkt oder indirekt) von der abstrakten Klasse `System.Attribute` erbt. Um einem Codeelement ein Attribut zuzuweisen, geben Sie den Typnamen des Attributs vor dem Codeelement in eckigen Klammern an. So wird zum Beispiel durch die folgende Zeile der Klasse `Foo` das `ObsoleteAttribute` zugewiesen:

```
[ObsoleteAttribute]
public class Foo {...}
```

Dieses Attribut wird vom Compiler erkannt und sorgt für Warnungen, wenn ein Typ oder Member, der oder das als *obsolet* gekennzeichnet ist, verwendet wird. Die Konvention ist, alle Attributtypen mit dem Wort `Attribute` enden zu lassen. Das weiß C# und erlaubt Ihnen daher, das Suffix beim Zuweisen eines Attributs wegzulassen:

```
[Obsolete]
public class Foo {...}
```

`ObsoleteAttribute` ist ein Typ, der im Namensraum `System` wie folgt definiert ist (aus Gründen der Übersichtlichkeit etwas vereinfacht):

```
public sealed class ObsoleteAttribute : Attribute {...}
```

Benannte und positionelle Attributparameter

Attribute können Parameter haben. Im folgenden Beispiel wenden wir `XmlElementAttribute` auf eine Klasse an. Dieses Attribut sagt `XmlSerializer` (in `System.Xml.Serialization`), wie ein Objekt in XML dargestellt wird, und akzeptiert verschiedene *Attributparameter*. Das folgende Attribut bildet die Klasse `CustomerEntity` auf ein XML-Element namens `Customer` ab, das zum Namensraum `http://oreilly.com` gehört:

```
[XmlElement ("Customer", Namespace="http://oreilly.com")]
public class CustomerEntity { ... }
```

Attributparameter lassen sich immer einer von zwei Kategorien zuordnen: *positioniert* und *benannt*. Im vorigen Beispiel ist das erste

Argument ein Positionsparameter, das zweite ein benannter Parameter. Positionsparameter entsprechen den Parametern der öffentlichen Konstruktoren des Attributtyps, benannte Parameter entsprechen öffentlichen Feldern oder öffentlichen Eigenschaften des Attributtyps.

Wenn Sie ein Attribut angeben, müssen Sie Positionsparameter angeben, die zu einem der Konstruktoren des Attributs passen. Benannte Parameter sind optional.

Attributziele

Das Ziel eines Attributs ist das Codeelement, vor dem es steht – normalerweise ein Typ oder ein Typ-Member. Sie können Attribute aber auch einer Assembly zuweisen. Dafür müssen Sie das Attributziel explizit angeben. Hier sehen Sie ein Beispiel für die Verwendung des Attributs `CLSCompliant`, um eine CLS-Konformität einer gesamten Assembly festzulegen:

```
[assembly:CLSCompliant(true)]
```

Seit C# 10 können Sie Attribute auf die Methode, die Parameter und den Rückgabewert eines Lambda-Ausdrucks anwenden:

```
Action<int> a =  
    [Description ("Method")]  
    [return: Description ("Return Value")]  
    ([Description ("Parameter")]int x) => Console.Write (x);
```

Angeben mehrerer Attribute

Es können mehrere Attribute für ein einzelnes Codeelement angegeben werden. Die Attribute können entweder in einem einzelnen Paar eckiger Klammern angegeben werden (getrennt durch Komma) oder jeweils extra in eckigen Klammern (oder als Kombination aus beidem). Die folgenden drei Beispiele sind semantisch identisch:

```
[Serializable, Obsolete, CLSCompliant(false)]  
public class Bar {...}  
  
[Serializable] [Obsolete] [CLSCompliant(false)]  
public class Bar {...}
```

Schreiben eigener Attribute

Sie definieren Ihre eigenen Attribute, indem Sie von `System.Attribute` ableiten. So könnten wir zum Beispiel das folgende eigene Attribut nutzen, um eine Methode zum Unit-Testing zu markieren:

```
[AttributeUsage (AttributeTargets.Method)]
public sealed class TestAttribute : Attribute
{
    public int    Repetitions;
    public string FailureMessage;

    public TestAttribute () : this (1) { }
    public TestAttribute (int repetitions)
        => Repetitions = repetitions;
}
```

Und so ließe sich das Attribut dann anwenden:

```
class Foo
{
    [Test]
    public void Method1() { ... }

    [Test(20)]
    public void Method2() { ... }

    [Test(20, FailureMessage="Debugging Time!")]
    public void Method3() { ... }
}
```

`AttributeUsage` ist selbst ein Attribut, das das Konstrukt (oder die Kombination von Konstrukten) kenntlich macht, auf das das eigene Attribut angewendet werden kann. Das Enum `AttributeTargets` enthält Member wie `Class`, `Method`, `Parameter` und `Constructor` (sowie `All`, das alle Ziele kombiniert).

Auslesen von Attributen zur Laufzeit

Es gibt zwei klassische Wege, Attribute zur Laufzeit auszulesen:

- Der Aufruf von `GetCustomAttributes` für ein Type- oder Member Info-Objekt.
- Der Aufruf von `Attribute.GetCustomAttribute` oder `Attribute.GetCustomAttributes`.

Diese beiden letzten Methoden sind überladen, damit ein beliebiges Reflection-Objekt übergeben werden kann, das zu einem gültigen Attributziel passt (Type, Assembly, Module, MemberInfo oder Parameter Info).

So können wir jede Methode in der Klasse Foo von gerade eben enumerieren, die ein TestAttribute hat:

```
foreach (MethodInfo mi in typeof (Foo).GetMethods())
{
    TestAttribute att = (TestAttribute)
        Attribute.GetCustomAttribute
            (mi, typeof (TestAttribute));

    if (att != null)
        Console.WriteLine (
            "{0} wird geprüft; reps={1}; msg={2}",
            mi.Name, att.Repetitions, att.FailureMessage);
}
```

Das ist die Ausgabe:

```
Method Method1 wird getestet; reps=1; msg=
Method Method2 wird getestet; reps=20; msg=
Method Method3 wird getestet; reps=20; msg=Debugging!
```

Aufrufer-Info-Attribute

Seit C# 5.0 können Sie optionale Parameter mit einem der drei *Aufrufer-Info-Attribute* versehen, die den Compiler anweisen, Daten, die aus dem Quellcode des Aufrufers entnommen werden, in den Standardwert für den Parameter aufzunehmen:

- [CallerMemberName] wendet den Member-Namen des Aufrufers an.
- [CallerFilePath] wendet den Pfad zur Quellcodedatei des Aufrufers an.
- [CallerLineNumber] wendet die Zeilennummer in der Quellcodedatei des Aufrufers an.

Die Foo-Methode im folgenden Programm stellt alle drei vor:

```
using System;
using System.Runtime.CompilerServices;

class Program
{
    static void Main() => Foo();

    static void Foo (
        [CallerMemberName] string memberName = null,
        [CallerFilePath] string filePath = null,
        [CallerLineNumber] int lineNumber = 0)
    {
        Console.WriteLine (memberName);
        Console.WriteLine (filePath);
        Console.WriteLine (lineNumber);
    }
}
```

Wenn unser Programm unter *c:\source\test\Program.cs* gespeichert wäre, würde die Ausgabe so aussehen:

```
Main
c:\source\test\Program.cs
6
```

Wie bei gewöhnlichen optionalen Parametern erfolgt die Substitution auf *Aufruferseite*. Unsere Main-Methode ist also nur syntaktischer Zucker für Folgendes:

```
static void Main()
=> Foo ("Main", @"c:\source\test\Program.cs", 6);
```

Aufrufer-Info-Attribute sind sowohl beim Schreiben von Logging-Funktionen als auch bei der Implementierung von Benachrichtigungsmustern für Veränderungen praktisch. Beispielsweise kann eine Methode wie die folgende aus dem set-Accessor einer Eigenschaft aufgerufen werden, ohne dass der Name der Eigenschaft angegeben werden muss:

```
void RaisePropertyChanged (
    [CallerMemberName] string propertyName = null)
{
    ...
}
```

CallerArgumentExpression (C# 10)

Ein Methodenparameter, auf den Sie das Attribut [CallerArgumentExpression] anwenden, übernimmt einen Argumentausdruck vom aufrufenden Code:

```
Print (Math.PI * 2);

void Print (double number,
    [CallerArgumentExpression("number")] string expr = null)
    => Console.WriteLine (expr);

// Ausgabe: Math.PI * 2
```

Das zentrale Anwendungsgebiet für dieses Feature liegt im Schreiben von Validation- und Assertion-Bibliotheken. Im folgenden Beispiel wird eine Exception geworfen, deren Message den Text »2 + 2 == 5« enthält. Das hilft beim Debuggen:

```
Assert (2 + 2 == 5);

void Assert (bool condition,
    [CallerArgumentExpression("condition")]
    string msg = null)
{
    if (!condition)
        throw new Exception ("Assert fehlgeschlagen: " + msg);
}
```

Sie können [CallerArgumentExpression] mehrfach verwenden, um auch mehrere Argumentausdrücke zu übernehmen.

Asynchrone Funktionen

Die Schlüsselwörter `await` und `async` unterstützen die *asynchrone Programmierung* – ein Programmierstil, bei dem Funktionen, die lange laufen, einen Großteil ihrer Arbeit verrichten, *nachdem* sie die Ausführungskontrolle an den Aufrufer zurückgegeben haben. Das steht im Gegensatz zur normalen *synchronen* Programmierung, bei der Funktionen, die lange laufen, den Aufrufer *blockieren*, bis die Operation abgeschlossen ist. Asynchrone Programmierung impliziert *Nebenläufigkeit*, da die lange laufenden Operationen *parallel*

zu den Operationen des Aufrufers ausgeführt werden. Der Implementierer einer asynchronen Funktion initiiert diese Nebenläufigkeit entweder durch Multithreading (für berechnungsgebundene Operationen) oder über einen Callback-Mechanismus (für Eingabe/Ausgabe-gebundene Operationen).



Multithreading, Nebenläufigkeit und asynchrone Programmierung sind umfangreiche Themen. Wir erörtern sie online unter <http://albahari.com/threading>.

Betrachten Sie zum Beispiel die folgende *synchrone* Methode, die lange läuft und berechnungsgebunden ist:

```
int ComplexCalculation()
{
    double x = 2;
    for (int i = 1; i < 100000000; i++)
        x += Math.Sqrt(x) / i;
    return (int)x;
}
```

Wenn diese Methode ausgeführt wird, blockiert sie den Aufrufer für einige Sekunden. Anschließend wird das Ergebnis der Berechnung an den Aufrufer zurückgeliefert:

```
int result = ComplexCalculation();
// Etwas später:
Console.WriteLine (result);    // 116
```

Die CLR definiert eine Klasse namens `Task<TResult>` (in `System.Threading.Tasks`), um das Konzept einer Operation zu kapseln, die irgendwann in der Zukunft abgeschlossen wird. Sie können einen `Task<TResult>` für eine berechnungsgebundene Operation erstellen, indem Sie `Task.Run` aufrufen, das die CLR anweist, das angegebene Delegate in einem separaten Thread auszuführen, der parallel zum Aufrufer ausgeführt wird:

```
Task<int> ComplexCalculationAsync()
=> Task.Run ( () => ComplexCalculation() );{
```

Diese Methode ist *asynchron*, weil sie die Ausführung unmittelbar an den Aufrufer zurückgibt, während sie nebenläufig ausgeführt

wird. Wir benötigen allerdings einen Mechanismus, über den der Aufrufer angeben kann, was passieren soll, wenn die Operation abgeschlossen und das Ergebnis verfügbar ist. `Task<TResult>` löst das, indem es eine `GetAwaiter`-Methode veröffentlicht, über die der Aufrufer eine *Continuation* (Fortsetzung) anbinden kann:

```
Task<int> task = ComplexCalculationAsync();
var awaiter = task.GetAwaiter();
awaiter.OnCompleted (() =>           // Continuation
{
    int result = awaiter.GetResult();
    Console.WriteLine (result);      // 116
});
```

Das sagt der Operation, dass sie, wenn sie fertig ist, das angegebene Delegate ausführen soll. Unsere Continuation ruft zuerst `GetResult`, das das Ergebnis der Berechnung liefert (wenn der Task *fehlgeschlagen* ist, also eine Exception ausgelöst hat, löst ein Aufruf von `GetResult` diese Exception neu aus). Die Continuation gibt das Ergebnis dann über `Console.WriteLine` aus.

Die Schlüsselwörter `await` und `async`

Das Schlüsselwort `await` vereinfacht das Anbinden von Continuations. Nehmen wir einen einfachen Fall als Beispiel. Der Compiler expandiert

```
var Ergebnis = await
Ausdruck;
Anweisung(en);
```

in etwas, das funktionell Folgendem entspricht:

```
var awaiter = Ausdruck.GetAwaiter();
awaiter.OnCompleted (() =>
{
    var Ergebnis = awaiter.GetResult();
    Anweisung(en);
});
```



Der Compiler gibt außerdem Code aus, um das Szenario zu optimieren, dass die Operation synchron (sofort) abgeschlossen wird. Dass eine asynchrone Operation

sofort abgeschlossen wird, tritt häufig dann ein, wenn sie einen internen Caching-Mechanismus implementiert und das Ergebnis bereits gecacht ist.

Also rufen wir die zuvor definierte `ComplexCalculationAsync`-Methode folgendermaßen auf:

```
int result = await ComplexCalculationAsync();  
Console.WriteLine (result);
```

Das lässt sich nur kompilieren, wenn wir die umschließende Methode mit dem Modifikator `async` versehen:

```
async void Test()  
{  
    int result = await ComplexCalculationAsync();  
    Console.WriteLine (result);  
}
```

Der Modifikator `async` sagt dem Compiler, dass er `await` als Schlüsselwort behandeln soll und nicht als Bezeichner, sollte in dieser Methode eine Mehrdeutigkeit auftauchen (das sorgt dafür, dass Code, der vor C# 5.0 geschrieben wurde und eventuell `await` als Bezeichner verwendet, weiterhin anstandslos kompiliert werden kann). Der Modifikator `async` kann nur auf Methoden (und Lambda-Ausdrücke) angewendet werden, die `void` oder (wie wir später sehen werden) ein `Task`- oder ein `Task<TResult>`-Objekt liefern.



Der Modifikator `async` ähnelt dem Modifikator `unsafe` darin, dass er keine Auswirkungen auf die Methodensignatur oder die öffentlichen Metadaten hat. Er wirkt sich nur auf das aus, was *in* der Methode passiert.

Methoden mit dem Modifikator `async` werden als *asynchrone Funktionen* bezeichnet, weil sie selbst üblicherweise asynchron sind. Schauen wir uns an, wie die Ausführung durch eine asynchrone Funktion verläuft, um zu sehen, warum.

Wenn die Ausführung auf einen `await`-Ausdruck stößt, gibt sie normalerweise an den Aufrufer zurück – ganz ähnlich wie ein `yield return` in einem Iterator. Aber bevor die Ausführung zurück-

kehrt, bindet die Runtime eine Continuation an den Task, dessen Abschluss erwartet wird, und sichert so, dass die Ausführung, wenn der Task fertig ist, wieder in die Methode springt und die Arbeit dort fortsetzt, wo sie ausgesetzt wurde. Wenn der Task fehlschlägt, wird seine Exception neu ausgelöst (was der `GetResult`-Aufruf bewirkt); andernfalls wird ihr Rückgabewert dem `await`-Ausdruck zugewiesen.



Die Implementierung, die die CLR für die `OnCompleted`-Methode für die Continuation eines Tasks bereitstellt, sorgt dafür, dass Continuations standardmäßig im gesamten aktuellen *Synchronisationskontext* bekannt gemacht werden, wenn ein solcher vorhanden ist. In der Praxis bedeutet das, dass in Rich-Client-UI-Umgebungen (WPF, UWP und Windows Forms) Ihr Code auf demselben Thread fortgesetzt wird, wenn Sie `await` auf einem UI-Thread setzen. Das vereinfacht die Threadsicherheit.

Der Ausdruck, auf dem Sie `await` nutzen, ist üblicherweise ein Task; aber den Compiler stellt jedes Objekt zufrieden, das eine `GetAwaiter`-Methode bietet, die ein Objekt liefert, das `await` unterstützt, also `INotifyCompletion.OnCompleted` implementiert und eine `GetResult`-Methode mit einem entsprechenden Rückgabebetyp bietet (und eine `bool IsCompleted`-Eigenschaft hat, die auf synchrone Vollständigkeit prüft).

Beachten Sie, dass unser `await`-Ausdruck zu einem `int` ausgewertet wird. Das liegt daran, dass der Ausdruck, auf den gewartet wird, ein `Task<int>` ist (dessen `GetAwaiter().GetResult()`-Methode einen `int` liefert).

Das Warten auf einen nichtgenerischen Task ist zulässig und erzeugt einen leeren Ausdruck (`void`):

```
await Task.Delay (5000);  
Console.WriteLine ("Fünf Sekunden verstrichen!");
```

`Task.Delay` ist eine statische Methode, die einen Task liefert, der in der angegebenen Anzahl von Millisekunden fertig ist. Das *synchrone* Äquivalent zu `Task.Delay` ist `Thread.Sleep`.

Task ist die nichtgenerische Basisklasse von Task<TResult> und entspricht funktionell Task<TResult>, davon abgesehen, dass es kein Ergebnis gibt.

Lokale Zustände einfangen

Die wahre Macht von await-Ausdrücken liegt darin, dass sie praktisch überall im Code erscheinen können. Genauer gesagt, kann ein await-Ausdruck (in einer asynchronen Funktion) überall dort stehen, wo ein Ausdruck stehen darf, außer innerhalb eines catch- oder finally-Blocks, eines lock-Ausdrucks oder eines unsafe-Kontexts.

Im folgenden Beispiel nutzen wir await in einer Schleife:

```
async void Test()
{
    for (int i = 0; i < 10; i++)
    {
        int result = await ComplexCalculationAsync();
        Console.WriteLine (result);
    }
}
```

Beim ersten Aufruf von ComplexCalculationAsync kehrt die Ausführung dank await-Ausdruck an den Aufrufer zurück. Wenn die Methode fertig ist (oder fehlschlägt), wird die Ausführung dort wieder aufgenommen, wo sie abgebrochen wurde, und die Werte der lokalen Variablen und Schleifenzähler bleiben dabei erhalten. Das erreicht der Compiler dadurch, dass er derartigen Code in eine Zustandsmaschine übersetzt, wie er es bei Iteratoren macht.

Ohne das Schlüsselwort await bringt es der manuelle Einsatz von Continuations mit sich, dass Sie etwas einer Zustandsmaschine Äquivalentes schreiben müssen. Das ist das, was die asynchrone Programmierung traditionell so kompliziert gemacht hat.

Asynchrone Funktionen schreiben

Bei jeder asynchronen Funktion können Sie den Rückgabetypp void durch Task ersetzen, um die Methode selbst *nützlich* asynchron

(und await-fähig) zu machen. Es sind keine weiteren Änderungen erforderlich:

```
async Task PrintAnswerToLife()
{
    await Task.Delay (5000);
    int answer = 21 * 2;
    Console.WriteLine (answer);
}
```

Beachten Sie, dass wir aus dem Methodenrumpf nicht explizit ein Task-Objekt zurückliefern. Der Compiler erzeugt das Task-Objekt, das er benachrichtigt, wenn die Methode abgearbeitet ist (oder eine unbehandelte Exception ausgelöst hat). Das macht es leicht, asynchrone Aufrufketten zu erstellen:

```
async Task Go()
{
    await PrintAnswerToLife();
    Console.WriteLine ("Fertig");
}
```

(Und weil Go ein Task-Objekt liefert, ist Go selbst await-fähig.) Der Compiler expandiert asynchrone Funktionen, die Tasks liefern, zu Code, der (indirekt) TaskCompletionSource nutzt, um einen Task zu erstellen, der dann benachrichtigt wird oder fehlschlägt.



TaskCompletionSource ist ein CLR-Typ, mit dem Sie Tasks erstellen können, die Sie manuell steuern und deren Abschluss Sie mit einem Ergebnis signalisieren (oder deren Fehlschlag mit einer Exception). Im Unterschied zu Task.Run bindet TaskCompletionSource keinen Thread für die Dauer der Operation. Die Klasse ist auch für Methoden geeignet, die Eingabe/Ausgabe-gebundene Tasks liefern (wie Task.Delay).

Das soll sicherstellen, dass die Ausführung immer über eine Continuation zum Wartenden zurückspringen kann, wenn die einen Task liefernde asynchrone Funktion fertig ist.

Task<TResult> zurückliefern

Sie können Task<TResult> liefern, wenn der Methodenrumpf TResult liefert:

```
async Task<int> GetAnswerToLife()
{
    await Task.Delay (5000);
    int answer = 21 * 2;
    // answer ist int, also nutzt die Methode Task<int>
    return answer;
}
```

Wir probieren GetAnswerToLife aus, indem wir es aus PrintAnswerToLife aufrufen (das seinerseits von Go aufgerufen wird):

```
async Task Go()
{
    await PrintAnswerToLife();
    Console.WriteLine ("Fertig");
}
async Task PrintAnswerToLife()
{
    int answer = await GetAnswerToLife();
    Console.WriteLine (answer);
}
async Task<int> GetAnswerToLife()
{
    await Task.Delay (5000);
    int answer = 21 * 2;
    return answer;
}
```

Asynchrone Funktionen gleichen die asynchrone Programmierung der synchronen Programmierung an. Hier ist das synchrone Äquivalent unseres Aufrufgrafen, für das ein Aufruf von Go() das gleiche Ergebnis liefert, nachdem die Ausführung für fünf Sekunden blockiert war:

```
void Go()
{
    PrintAnswerToLife();
    Console.WriteLine ("Fertig");
}
void PrintAnswerToLife()
{

```

```

        int answer = GetAnswerToLife();
        Console.WriteLine (answer);
    }
    int GetAnswerToLife()
    {
        Thread.Sleep (5000);
        int answer = 21 * 2;
        return answer;
    }

```

Das illustriert auch eines der Grundprinzipien beim Schreiben asynchroner Funktionen in C#, nämlich dass Sie Ihre Methoden synchron schreiben und dann die *synchronen* Methodenaufrufe durch *asynchrone* Methodenaufrufe ersetzen sollten, mit denen Sie *await* nutzen.

Parallelität

Wir haben gerade das Muster vorgestellt, das am weitesten verbreitet ist: unmittelbar nach dem Aufruf mit *await* auf Funktionen zu warten, die einen Task liefern. Das führt zu einem sequenziellen Programmablauf, der logisch dem synchronen Äquivalent ähnelt.

Der Aufruf einer asynchronen Methode ohne *await* gestattet die parallele Ausführung des nachfolgenden Codes. Beispielsweise führt Folgendes *PrintAnswerToLife* zwei Mal nebenläufig aus:

```

var task1 = PrintAnswerToLife();
var task2 = PrintAnswerToLife();
await task1; await task2;

```

Dadurch, dass wir danach auf beide Operationen warten, »beenden« wir die Parallelität an diesem Punkt (und lösen alle eventuellen Exceptions aus diesen Tasks neu aus). Die Klasse *Task* bietet eine statische Methode namens *WhenAll*, mit der sich das gleiche Ergebnis etwas effizienter erreichen lässt. *WhenAll* liefert einen Task, der fertig wird, wenn alle Tasks, die Sie ihm übergeben, fertig sind:

```

await Task.WhenAll (PrintAnswerToLife(),
                    PrintAnswerToLife());

```

WhenAll bezeichnet man als einen *Task-Verknüpfer*. (Die Klasse *Task* bietet außerdem einen Task-Verknüpfer namens *WhenAny*, der fertig ist, wenn *einer* der ihm übergebenen Tasks fertig ist.)

Asynchrone Lambda-Ausdrücke

Ebenso wie gewöhnliche *benannte* Methoden asynchron sein können:

```
async Task NamedMethod()  
{  
    await Task.Delay (1000);  
    Console.WriteLine ("Foo");  
}
```

können auch *namenlose* Methoden (Lambda-Ausdrücke und anonyme Methoden) asynchron sein, wenn ihnen das Schlüsselwort `async` vorangestellt wird:

```
Func<Task> unnamed = async () =>  
{  
    await Task.Delay (1000);  
    Console.WriteLine ("Foo");  
};
```

Wir rufen diese Methoden auf die gleiche Weise auf und warten auf sie:

```
await NamedMethod();  
await unnamed();
```

Asynchrone Lambda-Ausdrücke können beim Anbinden von Event-Handlern genutzt werden:

```
myButton.Click += async (sender, args) =>  
{  
    await Task.Delay (1000);  
    myButton.Content = "Fertig";  
};
```

Das ist kompakter als folgende Formulierung, die die gleiche Wirkung hat:

```
myButton.Click += ButtonHandler;  
...  
async void ButtonHandler (object sender, EventArgs args)  
{  
    await Task.Delay (1000);  
    myButton.Content = "Done";  
};
```

Asynchrone Lambda-Ausdrücke können auch `Task<TResult>` liefern:

```
Func<Task<int>> unnamed = async () =>
{
    await Task.Delay (1000);
    return 123;
};
int answer = await unnamed();
```

Asynchrone Streams

Mit `yield return` können Sie einen Iterator schreiben, mit `await` eine asynchrone Funktion. *Asynchrone Streams* kombinieren diese Konzepte (seit C# 8) und ermöglichen Ihnen, einen Iterator zu schreiben, der wartet und Elemente asynchron zurückgibt. Diese baut auf dem folgenden Interface-Paar auf, das das asynchrone Gegenstück zu den Enumeration-Interfaces aus dem Abschnitt »Enumeration und Iteratoren« auf Seite 157:

```
public interface IAsyncEnumerable<out T>
{
    IAsyncEnumerator<T> GetAsyncEnumerator (...);
}

public interface IAsyncEnumerator<out T>: IAsyncDisposable
{
    T Current { get; }
    ValueTask<bool> MoveNextAsync();
}
```

`ValueTask<T>` ist ein Struct, das `Task<T>` verpackt und sich entsprechend wie `Task<T>` verhält, nur dass es eine effizientere Ausführung bei einem synchronen Abschluss ermöglicht (was beim Enumerieren einer Sequenz häufig vorkommt). `IAsyncDisposable` ist eine asynchrone Version von `IDisposable` und bietet die Möglichkeit des Aufräumens, wenn Sie sich dazu entschließen, die Schnittstellen manuell zu implementieren:

```
public interface IAsyncDisposable
{
    ValueTask DisposeAsync();
}
```



Das Auslesen jedes Elements aus der Sequenz (`MoveNextAsync`) ist eine asynchrone Operation, daher sind asynchrone Streams dafür passend, wenn die Elemente nach und nach eintreffen (wie beim Verarbeiten von Daten aus einem Videostream). Im Gegensatz dazu passt der folgende Typ besser, wenn die Sequenz im Ganzen verzögert eintrifft, die Elemente dann aber alle zusammen da sind:

`Task<IEnumerable<T>>`

Um einen asynchronen Stream zu erzeugen, schreiben Sie eine Methode, die die Prinzipien von Iteratoren und asynchronen Methoden kombiniert. Mit anderen Worten: Ihre Methode sollte sowohl `yield return` wie auch `await` enthalten und `IAsyncEnumerable<T>` zurückgeben:

```
async IAsyncEnumerable<int> RangeAsync (  
    int start, int count, int delay)  
{  
    for (int i = start; i < start + count; i++)  
    {  
        await Task.Delay (delay);  
        yield return i;  
    }  
}
```

Um einen asynchronen Stream zu konsumieren, verwenden Sie die Anweisung `await foreach`:

```
await foreach (var number in RangeAsync (0, 10, 100))  
    Console.WriteLine (number);
```

Unsicherer Code und Zeiger

C# unterstützt die direkte Veränderung von Speicherinhalten durch Zeiger, die in als unsicher gekennzeichneten Codeblöcken genutzt werden und mit der Compileroption `/unsafe` kompiliert wurden. Zeigertypen werden vor allem aus Gründen der Interoperabilität mit C-APIs genutzt, sie können aber auch für den Zugriff auf Speicher außerhalb des verwalteten Heap oder für performance-kritische Hotspots verwendet werden.

Zeigergrundlagen

Zu jedem Werttyp bzw. Referenztyp V gibt es einen korrespondierenden Zeigertyp V^* . Eine Zeigerinstanz hält die Adresse einer Variablen. Zeigertypen können (unsicher) auf jeden anderen Zeigertyp gecastet werden. Die wichtigsten Zeigeroperatoren sind folgende:

Operator	Bedeutung
<code>&</code>	Der <i>Adressoperator</i> liefert einen Zeiger auf die Adresse des Werts zurück.
<code>*</code>	Der <i>Dereferenzierungsoperator</i> liefert den Wert an der Adresse eines Zeigers zurück.
<code>-></code>	Der <i>Zeiger-auf-Member-Operator</i> ist eine syntaktische Abkürzung: $x \rightarrow y$ ist äquivalent zu $(*x).y$.

Unsicherer Code

Dadurch, dass Sie einen Typ, ein Typ-Member oder einen Anweisungsblock mit dem Schlüsselwort `unsafe` kennzeichnen, dürfen Sie innerhalb dieses Geltungsbereichs Zeigertypen nutzen und Zeigeroperationen im C++-Stil auf Speicherbereichen durchführen. Hier sehen Sie ein Beispiel für die Verwendung von Zeigern, um eine Bitmap schnell zu verarbeiten:

```
unsafe void BlueFilter (int[, ] bitmap)
{
    int length = bitmap.Length;
    fixed (int* b = bitmap)
    {
        int* p = b;
        for (int i = 0; i < length; i++)
            *p++ &= 0xFF;
    }
}
```

Unsicherer Code kann schneller laufen als die entsprechende sichere Implementierung. In diesem Fall wäre eine verschachtelte Schleife mit Array-Indexierung und Prüfung der Array-Grenzen notwendig. Eine unsichere C#-Methode kann auch schneller sein als das Aufrufen einer externen C-Funktion, da es keinen Overhead gibt, der durch das Verlassen der verwalteten Ausführungsumgebung entsteht.

Die Anweisung fixed

Die Anweisung `fixed` ist notwendig, um ein verwaltetes Objekt »festzunageln«, zum Beispiel die Bitmap im vorigen Beispiel. Während der Ausführung eines Programms werden viele Objekte auf dem Heap angefordert und wieder freigegeben. Um eine unnötige Verschwendung und Fragmentierung von Speicher zu vermeiden, verschiebt der Garbage Collector Objekte. Das Zeigen auf ein Objekt ist aber nicht sinnvoll, wenn sich seine Adresse während des Referenzierens ändern kann, daher weist die Anweisung `fixed` den Garbage Collector an, ein Objekt nicht zu verschieben. Das kann einen Einfluss auf die Laufzeiteffizienz haben, daher sollten feste Blöcke nur verwendet werden, wenn es notwendig ist. Zudem sollte man das Anfordern von Objekten auf dem Heap innerhalb eines festen Blocks vermeiden.

Innerhalb einer `fixed`-Anweisung können Sie einen Zeiger auf einen beliebigen Werttyp, ein Array mit Werttypen oder einen String erhalten. Bei Arrays und Strings wird der Zeiger auf das erste Element zeigen, einen Werttyp.

Bei Werttypen, die innerhalb von Referenztypen deklariert wurden, muss der Referenztyp »festgenagelt« werden:

```
Test test = new Test();
unsafe
{
    fixed (int* p = &test.X)    // fixiert test
    {
        *p = 9;
    }
    Console.WriteLine (test.X);
}

class Test { public int X; }
```

Der Zeiger-auf-Member-Operator

Neben den Operatoren `&` und `*` bietet C# auch den Operator `->` im C++-Stil, der für Structs genutzt werden kann:

```

Test test = new Test();
unsafe
{
    Test* p = &test;
    p->X = 9;
    System.Console.WriteLine (test.X);
}

struct Test { public int X; }

```

Das Schlüsselwort `stackalloc` keyword

Speicher kann als Block auf dem Stack explizit mit dem Schlüsselwort `stackalloc` angefordert werden. Da er auf dem Stack angefordert wird, ist seine Lebenszeit auf die Ausführung der Methode beschränkt, so wie jede andere lokale Variable. Der Block kann den Operator `[]` nutzen, um auf Speicher indexiert zuzugreifen:

```

int* a = stackalloc int [10];
for (int i = 0; i < 10; ++i)
    Console.WriteLine (a[i]); // in den nackten Speicher
                                // ausgeben

```

Puffer fester Größe

Speicher kann in einem Block innerhalb eines Struct mithilfe des Schlüsselworts `fixed` angefordert werden:

```

unsafe struct UnsafeUnicodeString
{
    public short Length;
    public fixed byte Buffer[30];
}

unsafe class UnsafeClass
{
    UnsafeUnicodeString uus;

    public UnsafeClass (string s)
    {
        uus.Length = (short)s.Length;
        fixed (byte* p = uus.Buffer)
            for (int i = 0; i < s.Length; i++)

```

```

        p[i] = (byte) s[i];
    }
}

```

Puffer fester Größe sind keine Arrays: Würde es sich bei Buffer um ein Array handeln, bestünde es aus einer Referenz auf ein Objekt, das auf dem (gemanagten) Heap abgelegt ist, und nicht aus 30 Bytes im Struct selbst.

Das Schlüsselwort `fixed` wird in diesem Beispiel auch genutzt, um das Objekt, das den Puffer enthält, auf dem Heap festzuhalten (dabei handelt es sich um die Instanz von `UnsafeClass`).

void*

Ein *void-Zeiger* (`void*`) macht keine Annahmen in Bezug auf den Typ der zugrunde liegenden Daten und ist für Funktionen geeignet, die direkt mit dem Speicher arbeiten. Es existiert eine implizite Konvertierung von jedem Zeigertyp auf `void*`. Ein `void*` kann nicht dereferenziert werden, und es sind keine arithmetischen Operationen möglich:

```

short[] a = {1,1,2,3,5,8,13,21,34,55};
fixed (short* p = a)
{
    // sizeof liefert die Größe von Werttypen
    // in Byte zurück
    Zap (p, a.Length * sizeof (short));
}
foreach (short x in a)
    System.Console.WriteLine (x); // gibt Nullen aus

unsafe void Zap (void* memory, int byteCount)
{
    byte* b = (byte*) memory;
    for (int i = 0; i < byteCount; i++)
        *b++ = 0;
}

```

Funktionszeiger

Ein Funktionszeiger (seit C# 9) ähnelt einem Delegate, allerdings ohne die Indirektion einer Delegate-Instanz – stattdessen zeigt er

direkt auf eine Methode. Ein Funktionszeiger kann nur auf statische Methoden zeigen, ihm fehlt die Möglichkeit des Multicastings, und er braucht einen unsafe-Kontext (weil er die Typsicherheit zur Laufzeit umgeht). Sein Hauptzweck ist das Vereinfachen und Optimieren der Zusammenarbeit mit ungemanagten APIs.

Ein Funktionszeigertyp wird wie folgt deklariert (wobei der Rückgabewerttyp am Ende steht):

```
delegate*<int, char, string, void>
```

Das passt zu einer Funktion mit dieser Signatur:

```
void SomeFunction (int x, char y, string z)
```

Der &-Operator erzeugt einen Funktionszeiger aus einer Methoden-Gruppe. Hier ein vollständiges Beispiel:

```
unsafe
{
    delegate*<string, int> functionPointer = &GetLength;
    int length = functionPointer ("Hallo Welt");

    static int GetLength (string s) => s.Length;
}
```

In diesem Beispiel ist `functionPointer` kein *Objekt*, für das Sie eine Methode wie `Invoke` aufrufen können (oder mit einer Referenz auf ein `Target`-Objekt). Stattdessen handelt es sich um eine Variable, die direkt auf die Adresse der Zielmethode im Speicher zeigt:

```
Console.WriteLine ((IntPtr)functionPointer);
```

Präprozessordirektiven

Präprozessordirektiven versorgen den Compiler mit zusätzlichen Informationen über Codebereiche. Die am häufigsten verwendeten Präprozessordirektiven sind die bedingten Direktiven, die eine Möglichkeit bieten, Codebereiche beim Kompilieren auszuschließen oder mit zu berücksichtigen. Hier sehen Sie ein Beispiel dafür:

```
#define DEBUG
class MyClass
{
```

```

int x;
void Foo()
{
    # if DEBUG
    Console.WriteLine ("Test: x = {0}", x);
    # endif
    ...
}

```

In dieser Klasse wird die Anweisung in `Foo` in Abhängigkeit davon kompiliert, ob das Symbol `DEBUG` vorhanden ist. Wenn wir es entfernen, wird die Anweisung nicht berücksichtigt. Präprozessorsymbole können in einer Quelldatei definiert (wie wir es getan haben), dem Compiler über die Befehlszeilenoption `/define:symbol` mitgegeben oder beim Einsatz von Visual Studio oder MSBuild in der Projektdatei abgelegt werden.

Bei den Direktiven `#if` und `#elif` können Sie die Operatoren `||`, `&&` und `!` nutzen, um *oder*-, *und*- bzw. *nicht*-Operationen mit mehreren Symbolen durchzuführen. Die folgende Direktive weist den Compiler an, den Code mit zu berücksichtigen, wenn das Symbol `TESTMODE` definiert ist, `DEBUG` aber nicht:

```

#if TESTMODE && !DEBUG
    ...

```

Denken Sie jedoch daran, dass Sie hier keinen normalen C#-Ausdruck verwenden und die Symbole, mit denen Sie umgehen, keinerlei Verbindung zu Variablen haben – weder zu statischen noch zu anderen.

Die Symbole `#error` und `#warning` verhindern unbeabsichtigte Fehlanwendungen von bedingten Direktiven, indem sie dafür sorgen, dass der Compiler eine Warnung oder einen Fehler ausgibt, falls es zu unerwünschten Kombinationen von Kompilierungssymbolen kommt.

Hier ist eine vollständige Liste der Präprozessordirektiven:

Präprozessordirektive	Aktion
<code>#define <i>Symbol</i></code>	Definiert <i>Symbol</i> .
<code>#undef <i>Symbol</i></code>	Hebt Definition von <i>Symbol</i> auf.

Präprozessordirektive	Aktion
<code>#if Symbol</code> <code>[Operator Symbol2] ...</code>	Bedingte Kompilierung (<i>Operator</i> kann <code>==</code> , <code>!=</code> , <code>&&</code> und <code> </code> sein).
<code>#else</code>	Führt den Code bis zum nächsten <code>#endif</code> aus.
<code>#elif Symbol</code> <code>[Operator Symbol2]</code>	Kombiniert <code>#else</code> -Zweig und <code>#if</code> -Test.
<code>#endif</code>	Beendet Bedingungsdirektiven.
<code>#warning Text</code>	<i>Text</i> der Warnung, die in der Compilerausgabe erscheinen soll.
<code>#error Text</code>	<i>Text</i> des Fehlers, der in der Compilerausgabe erscheinen soll.
<code>#line [Zahl ["Datei "]</code> <code> hidden]</code>	<i>Zahl</i> legt die Zeile im Quellcode fest; <i>Datei</i> ist der Dateiname, der bei der Ausgabe erscheinen soll; <code>hidden</code> weist Debugger an, den Code zwischen dieser Zeile und der nächsten <code>#line</code> -Direktive zu überspringen.
<code>#region name</code>	Markiert den Beginn eines Abschnitts.
<code>#endregion</code>	Beendet einen Abschnitt.
<code>#pragma warning</code>	Siehe den nächsten Abschnitt.
<code>#nullable option</code>	Siehe Abschnitt »Nullbare Referenztypen« auf Seite 167.

Warnungen

Der Compiler erzeugt eine Warnung, wenn er in Ihrem Code auf etwas stößt, das nicht gewollt aussieht. Anders als bei Fehlern halten Warnungen Ihre Anwendung normalerweise nicht davon ab, kompiliert zu werden.

Compilerwarnungen können außerordentlich nützlich sein, wenn es darum geht, Fehler zu finden. Ihr Nutzen wird allerdings ausgehöhlt, wenn es *falsche* Warnungen gibt. In einer großen Anwendung ist es wichtig, ein gutes Signal-Rausch-Verhältnis zu haben, damit die *echten* Warnungen nicht übersehen werden.

Daher erlaubt der Compiler Ihnen, ausgewählte Warnungen mit der Direktive `#pragma warning` zu unterdrücken. In diesem Beispiel weisen wir den Compiler an, uns nicht davor zu warnen, dass das Feld `Message` nicht verwendet wird:

```
public class Foo
{
    #pragma warning disable 414
    static string Message = "Hallo";
    #pragma warning restore 414
}
```

Lässt man die Zahl in der Direktive `#pragma warning` weg, werden alle Warnmeldungen deaktiviert bzw. wieder aktiviert.

Wenn Sie bei der Anwendung dieser Direktive sehr exakt vorgehen, können Sie mit dem Schalter `/warnaserror` kompilieren – damit wird dem Compiler mitgeteilt, dass er jede verbleibende Warnung als Fehler ansehen soll.

XML-Dokumentation

Ein *Dokumentationskommentar* ist ein Stück eingebettetes XML, das einen Typ oder ein Member dokumentiert. Ein Dokumentationskommentar steht direkt vor einer Deklaration des Typs oder Members und beginnt mit drei Schrägstrichen (Slashes):

```
/// <summary>Beendet eine laufende Abfrage.</summary>
public void Cancel() { ... }
```

Kommentare über mehrere Zeilen können entweder so geschrieben werden:

```
/// <summary>
/// Beendet eine laufende Abfrage.
/// </summary>
public void Cancel() { ... }
```

oder so (beachten Sie das zusätzliche Sternchen am Anfang):

```
/**
    <summary> Beendet eine laufende Abfrage. </summary>
*/
public void Cancel() { ... }
```

Wenn Sie mit der Direktive `/doc` kompilieren (oder die XML-Dokumentation in der Projektdatei aktivieren), extrahiert der Compiler die Dokumentationskommentare und sammelt sie in einer einzelnen XML-Datei. Dafür gibt es zwei Hauptanwendungsfälle:

- Wenn sich die Datei im selben Ordner wie die kompilierte Assembly befindet, liest Visual Studio die XML-Datei automatisch ein und verwendet die Informationen, um Nutzern der Assembly gleichen Namens per IntelliSense die Member anzuzeigen.
- Tools von Fremdherstellern (wie Sandcastle und NDoc) können die XML-Datei in eine HTML-Hilfsdatei umwandeln.

Standard-XML-Dokumentations-Tags

Das hier sind die Standard-XML-Tags, die Visual Studio und Dokumentationsgeneratoren erkennen:

`<summary>`

`<summary>...</summary>`

Definiert den Tooltipp, den IntelliSense für den Typ oder das Member anzeigen soll, meist ein einzelner Satz oder eine Phrase.

`<remarks>`

`<remarks>...</remarks>`

Zusätzlicher Text, der den Typ oder das Member beschreibt. Dokumentationsgeneratoren nehmen dieses Tag und nutzen den Text für die Beschreibung eines Typs oder Members.

`<param>`

`<param name="Name">...</param>`

Erläutert einen Parameter einer Methode.

`<returns>`

`<returns>...</returns>`

Erläutert den Rückgabewert einer Methode.

`<exception>`

`<exception [cref="Typ"]>...</exception>`

Führt eine Exception auf, die eine Methode eventuell wirft (cref bezieht sich auf den Exception-Typ).

`<permission>`

`<permission [cref="Typ"]>...</permission>`

Gibt einen IPermission-Typ an, der vom dokumentierten Typ oder Member gebraucht wird.

`<example>`

`<example>...</example>`

Beschreibt ein Beispiel (das von Dokumentationsgeneratoren verwendet wird). Dazu gehören normalerweise sowohl beschreibender Text als auch Quellcode (der dann in den Tags `<c>` oder `<code>` steht).

`<c>`

`<c>...</c>`

Steht für einen Codeschnipsel innerhalb von Text. Dieses Tag wird meist in einem `<example>`-Block genutzt.

`<code>`

`<code>...</code>`

Steht für ein mehrzeiliges Codebeispiel. Dieses Tag wird meist in einem `<example>`-Block genutzt.

`<see>`

`<see cref="Member">...</see>`

Fügt einen Verweis auf einen anderen Typ oder ein anderes Member ein. HTML-Dokumentationsgeneratoren wandeln dies normalerweise in einen Hyperlink um. Der Compiler gibt eine Warnung aus, wenn der Name des Typs oder Members ungültig ist.

`<seealso>`

`<seealso cref="Member">...</seealso>`

Verweis auf einen anderen Typ oder ein anderes Member. Dokumentationsgeneratoren fügen den Text im Allgemeinen in einen separaten Abschnitt »See also« am Ende der Seite ein.

`<paramref>`

`<paramref name="Name"/>`

Referenziert einen Parameter innerhalb eines `<summary>`- oder `<remarks>`-Tags.

`<list>`

`<list type=[ bullet | number | table ]>`

`<listheader>`

`<term>...</term>`

`<description>...</description>`

`</listheader>`

`<item>`

`<term>...</term>`

`<description>...</description>`

`</item>`

`</list>`

Weist Dokumentationsgeneratoren an, eine unnummerierte, nummerierte oder tabellenförmige Liste auszugeben.

`<para>`

`<para>...</para>`

Weist Dokumentationsgeneratoren an, den Inhalt in einem eigenen Absatz zu formatieren.

`<include>`

`<include file='Dateiname' path='Tag-Pfad[@name="id"]'>`

`...`

`</include>`

Bindet eine externe XML-Datei ein. Das `path`-Attribut definiert eine XPath-Abfrage, um auf ein bestimmtes Element in dieser Datei zu verweisen.

Symbole

- ^ (Caret)
 - ^=-XOR-Zuweisungsoperator 56
 - ^ -Bit-XORER-Operator 30
- (Minuszeichen)
 - Negationsoperator 56
 - Subtraktionsoperator 28
 - Dekrementoperator 28
 - , Delegate-Instanzen entfernen 132
 - ==, Delegate-Instanzen entfernen 132
 - ==, Event-Accessor-Implementierung 141
 - ==, Subtraktionszuweisungsoperator 56
 - > Zeiger-auf-Member-Operator 241, 242
- ; (Semikolon)
 - ;, Beendigung von Anweisungen 13
- ! (Ausrufezeichen)
 - != -Nicht-gleich-Operator 33
 - ! -Nicht-Operator 33, 246
- ? (Fragezeichen)
 - ?, bei nullbaren Typen 162
 - ?:-Bedingungsoperator 34
 - ?? Null-Verbindungsoperator 60
 - ?. Null-Bedingungsoperator 60
 - ??=-Null-Verbindungszuweisungsoperator 59
- . (Punkt)
 - .-Member-Zugriffsoperator 13, 71
- ' (Anführungszeichen, einfache)
 - Escape-Sequenzen für 35
- " (Anführungszeichen, doppelte)
 - Escape-Sequenz für 35
 - um String-Literale 35
- () (Klammern)
 - () , Methodenaufruf oder -deklaration 13
 - ()-Cast-Operator 20, 96, 105, 164
- [] (eckige Klammern)
 - [] um Attributnamen 224
 - []-Array-Deklaration oder -indizierung 39
 - []-Indexerdekларation 89
- { } (geschweifte Klammern)
 - { }, um Anweisungsblöcke 13, 63
- @ (At-Zeichen)
 - @, vor Verbatim-String-Literalen 36

- * (Asterisk)
 - *=-Multiplikationszuweisungsoperator 56
 - *-Dereferenzierungsoperator 241
 - *-Multiplikationsoperator 9, 28
- / (Schrägstrich)
 - //, vor Kommentaren 7
 - /=-Divisionszuweisungsoperator 56
 - /-Divisionsoperator 28
- /doc-Direktive 248
- \ (Backslash)
 - \ vor Escape-Sequenzen 35
- \' (einfache Anführungszeichen) 35
- \'' (doppelte Anführungszeichen) 35
- \\ (Backslash) 35
- \0 (Null) 35
- \a (Alert) 35
- \b (Backspace) 35
- \f (Seitenvorschub) 35
- \n (Zeilenumbruch) 35
- \r (Carriage-Return) 35
- \t (horizontaler Tabulator) 35
- \v (vertikaler Tabulator) 35
- & (Ampersand)
 - &&-Bedingungs-UND-Operator 33, 246
 - &=-UND-Zuweisungsoperator 56
 - &-Adressoperator 241
 - &-Bit-UND-Operator 30
- # (Hash-Zeichen)
 - # vor Präprozessordirektiven 245
- % (Prozentzeichen)
 - %-Modulo-Operator 28
- + (Pluszeichen)
 - +, Delegate-Instanzen kombinieren 132
 - ++-Inkrementoperator 28
 - +=, Delegate-Instanzen kombinieren 132
 - +=-Additionszuweisungsoperator 56
 - +=-Event-Accessor-Implementierung 141
 - +-Additionsoperator 28
 - +-String-Verkettungsoperator 36
 - unäres Plus 56
- < (linke eckige Klammern)
 - <-Kleiner-als-Operator 33
- < (linke spitze Klammer)
 - <<=-Linksverschiebungszuweisungsoperator 56
 - <<-Linksverschiebungsoperator 30
 - <=-Kleiner-gleich-Operator 33
- = (Gleichheitszeichen)
 - ==-Gleichheitsoperator 31, 33
 - => Expression-bodied Eigenschaft 85
 - =>-Lambda-Operator 143
 - =-Zuweisungsoperator 13, 54
- > (rechte spitze Klammer)
 - >=-Größer-gleich-Operator 33
 - >>=-Rechtsverschiebungszuweisungsoperator 56
 - >>-Rechtsverschiebungsoperator 30
 - >-Größer-als-Operator 33
- | (senkrechter Strich)
 - |=-ODER-Zuweisungsoperator 56
 - ||-Bedingungs-ODER-Operator 33, 246

- | -Bit-ODER-Operatoren 30
- ~ (Tilde)
 - ~, vor Finalizer 91
 - ~-Bit-Komplement-Operator 30
- \$ (Dollarzeichen), String-Interpolation 37
- 16-Bit-Ganzzahltypen 30
- 8-Bit-Ganzzahltypen 30

A

- Abfrageausdrücke, LINQ 198–201
 - versus fließende Syntax 198
- Abfragefortsetzungen, LINQ 202
- Abfragen, LINQ 186–187
- Abfrageoperatoren, LINQ 186, 187–190, 192–197
- abgeleitete Klassen
 - siehe* Subklassen
- Abkürzen der Auswertung 34
- abstrakte Klassen 99
- Accessors 87, 141
 - Eigenschafts- 87
- Action-Delegate 133
- Additionsoperator (+) 28
- Additionszuweisungsoperator (+=) 56
- Adressoperator (&) 241
- Aggregate-Operator, LINQ 196
- Alert, Escape-Sequenz für 35
- Aliase 75
- All-Operator, LINQ 190, 196
- anonyme Methoden 148
- anonyme Typen 171
- Anweisungen 61–71
 - auf oberster Ebene 10, 19
 - Ausdrucksanweisungen 62

- Auswahanweisungen 63–65
- Deklarationsanweisungen 61
- Iterationsanweisungen 67–69
- Sprunganweisungen 69–71
- switch-Anweisung 64–65
- switch-Ausdrücke 67
- Anweisungsblöcke 9, 13, 61–71
- Any-Operator, LINQ 190, 196
- Argumente 9, 45, 46–52
 - benannte Argumente 51
 - CallerArgumentExpression 229
 - Typargumente 122
- arithmetische Operatoren 28
- Array-Klasse 40
- Arrays 38–44
 - auf dem Stack allozieren 243
 - automatische Initialisierung von 45
 - durchlaufen 39
 - in einem Struct allozieren 243
 - indizieren 41
 - Initialisierung 39, 40, 43
 - kopieren 40
 - mehrdimensionale 42
 - rechteckige 42
 - sortieren 40
 - Suchen in 40
 - ungleichförmige 43
- AsEnumerable-Operator, LINQ 197
- as-Operator 97
- AsQueryable-Operator, LINQ 197
- Assemblies 10, 112
- Assoziativität von Operatoren 55
- asynchrone Funktionen 229–239
 - einen Task liefern 236
 - Lambda-Ausdrücke 238–239

- lokalen Zustand einfangen 234
- Parallelität mit 237
- schreiben 234–237
- Streams 239
- async-Schlüsselwort 229, 231–234
- Attribute 223–228
 - Attributklassen 224
 - Attributparameter 224
 - Aufrufer-Info-Attribute 227–228
 - eigene Attribute 226
 - mehrere angeben 225
 - Ziele von Attributen 225
 - zur Laufzeit abrufen 226
- Ausdrücke 53–55
 - Abfrageausdrücke, LINQ 198–201
 - dynamische Ausdrücke 217
 - Lambda 142–147
 - new, mit dem Ziel typisierte 53
 - werfen 155
- Ausdrucksanweisungen 62
- Auslösen von Exceptions 155–156
- Auswahanweisungen 63–65
- äußere Variablen einfangen 144
- automatische Eigenschaften 86
- Average-Operator, LINQ 189, 196
- await-Schlüsselwort 229, 231–234

B

- Backspace, Escape-Sequenz für 35
- base-Schlüsselwort 101, 102

- Basisklassen 92
 - erben von 95, 100
 - object-Typ als 104
 - Upcasting zu 96
- Basisklassen-Constraint 125
- Basisklassenkonstruktor 101
- Bedingungsoperatoren 33
- benannte Argumente 51
- benannte Attributparameter 224
- benutzerdefinierte Typen 15
- Bereiche
 - mit Bereichen und Indizes arbeiten 40
 - und Indizes mit Indexern verwenden 90
- Beschneidung der Sichtbarkeit 113
- Bezeichner 11
- binäre Operatoren 54
- BinarySearch-Methode 40
- Bindung 213
 - benutzerdefinierte Bindung 213
 - dynamische Bindung 211–220
 - statische Bindung 212–213
- Bitoperatoren 30
- bool-Typ 15, 46, 166
- Boxing 105, 164
- break-Anweisung 65, 69
- Broadcaster 136
- byte-Typ 25, 30

C

- <c>-XML-Tag 250
- Callbacks
 - siehe* Delegates
- CallerArgumentExpression 229
- CallerFilePath-Attribut 227

- Caller-Info-Attribute 227–228
- CallerLineNumber-Attribut 227
- CallerMemberName-Attribut 227
- Camel-Case 11
- Carriage-Return, Escape-Sequenz für 35
- case-Klausel, switch-Anweisung 65
- Casting 20, 96–98
 - auf object-Typ 104
 - Enums 119
 - implizit, auf Interfaces 114
 - siehe auch* Boxing
- Cast-Operator (()) 20, 96, 105, 164
- Cast-Operator, LINQ 196, 210–211
- catch-Block 149, 151–152
- char-Literale 34
- char-Typ 34, 46
- checked-Operator 29
- class-Constraint 126
- class-Schlüsselwort 76
- Closures 144
- <code>-XML-Tag 250
- CompareTo-Methode 37
- Concat-Operator, LINQ 190, 195
- Constraints 125
- Contains-Methode 38
- Contains-Operator, LINQ 190, 196
- continue-Anweisung 70
- Copy-Konstruktor 177
- Copy-Methode 40
- Count-Operator, LINQ 189, 196
- CreateInstance-Methode 40
- csc.exe-Datei (Compiler) 10
- .cs-Dateiendung 10

D

- dateibezogene Namensräume 72
- Daten-Member 16, 17
- Datensätze 174, 180
 - definieren 174
 - Gleichheit 180
 - Gleichheitsprüfung 181
 - nicht destruktive Mutation 177
 - Parameterlisten 176
 - primäre Konstruktoren 179
- decimal-Typ 25, 32
- DefaultIfEmpty-Operator, LINQ 195
- Default-Interface-Member 117
- default-Schlüsselwort 46, 125
- #define-Direktive 246
- Deklarationsanweisungen 61
- Dekonstruktoren 81
 - Tupel 173
- Dekrementoperator (--) 28
- Delegates 130–136
 - Action-Delegate 133
 - Func-Delegate 133
 - generische Delegate-Typen 133, 136
 - Instanzmethode zuweisen an 132
 - Kompatibilität von Typen 134–136
 - Multicast-Delegates 132
 - Parametervarianz 135
 - Plugin-Methoden mit 131
 - Rückgabetyppvarianz 135
- delegate-Schlüsselwort 148
- Dereferenzierungsoperator (*) 241
- Direktive global using 73
- Direktiven, Präprozessor 245–248
- Discards 49

- Dispose-Methode 153
- Distinct-Operator, LINQ 194
- DivideByZeroException 150
- Division 29, 31, 150
- Divisionsoperator (/) 28
- Divisionszuweisungsoperator
 (/=) 56
- .dll-Dateiendung (Bibliothek) 10
- Dokumentationskommentare
 248–251
- double-Typ 25, 31, 32
- do-while-Schleifen 68
- Downcasting 96, 97
- dynamische Bindung 211–220
 - dynamische Ausdrücke 217
 - im Vergleich zu statischer Bindung 212–213
 - im Vergleich zum Typ object 216
 - Konvertierungen 216
 - nicht aufrufbare Funktionen 219
 - RuntimeBinderException 215
 - Sprachbindung 214–215
 - Überladungsauflösung 218–219
 - vs. var-Typ 217
- dynamische überladene Member 218

E

- eigene Attribute 226
- Eigenschaften 84–87
- Eigenschafts-Accessor 87
- Eigenschaftsinitialisierer 86
- Eigenschaftsmuster 184
- eingebaute Typen
 - siehe* vordefinierte Typen
- eingebettete Typen 121

- elementare Typen 24
- ElementAt-Operator, LINQ 189, 195
- ElementAtOrDefault-Operator, LINQ 195
- Elemente, LINQ 186
- Elementoperatoren, LINQ 189
- #elif-Direktive 246, 247
- #else-Direktive 247
- else-Klausel, if-Anweisung 63–64
- Empty-Operator, LINQ 197
- #endif-Direktive 247
- Endlosschleifen 69
- #endregion-Direktive 247
- EndsWith-Methode 38
- Enumeration von Abfrageoperatoren, LINQ 191
- Enumeratoren 157–162
 - initialisieren 158
 - iterieren über 157
 - mit Iteratoren erzeugen 159–162
 - nach der Iteration entsorgen 158
- enum-Typ 33, 46, 118–120
- Equals-Methode 31, 107
- #error-Direktive 246, 247
- Erweitern von Interfaces 114
- erweiterte partielle Methoden 93
- Erweiterungsmethoden 169–171
- Escape-Sequenzen 35, 36
- Events 136–142
 - Delegate-Parameter-Kontravarianz mit 135
 - virtuelle 98
 - zusammengesetzte Zuweisungsoperatoren bei 55
- event-Schlüsselwort 137
- <example>-XML-Tag 250
- <exception>-XML-Tag 249

- Exception-Filter 152–154
- Exception-Klasse 149, 151, 156
- Exceptions 149–156
 - abfangen 150, 151–152
 - aufräumen nach 152–154
 - Ausdrücke werfen 155
 - auslösen 155–156
- Except-Operator, LINQ 190, 195
- .exe-Dateiendung (Anwendung)
 - 10
- explizite Interface-Implementierung 114
- explizite Konvertierungen 222
- explizite Umwandlungen 20, 27
- Expression Trees 142
- Expression-bodied Eigenschaften 85
- Expression-bodied Methoden 78

F

- Felder 76–77
 - automatische Initialisierung von 45
 - Initialisierungsabfolge 103
 - static readonly-Felder 77
 - statische Feldinitialisierer 91
- Finalizer 91
- finally-Block 149, 152
- FindIndex-Methode 40
- FindLastIndex-Methode 40
- Find-Methode, Arrays 40
- First-Operator, LINQ 189, 195
- FirstOrDefault-Operator, LINQ 195
- fixed-Anweisung 242
- fixed-Schlüsselwort 243
- fixieren 242
- Flags-Attribut 120
- fließende Syntax 198

- float-Typ 25, 31, 32
- foreach-Schleifen 39, 69, 157
- for-Schleifen 39, 68
- Friend Assemblies 112
- from-Klausel, Abfrageausdruck 199, 203
- Func-Delegate 133
- Funktionen
 - asynchrone 229–239
 - dynamisch nicht aufrufbare 219
 - Operatorfunktionen 220
 - siehe auch* Methoden
 - versiegeln 101
- Funktions-Member 16, 17
- Funktionszeiger 244

G

- ganzzahlige Literale 26
- ganzzahlige Typen 25
 - Division von 29
 - Umwandlungen zwischen 27
- Ganzzahltypen, 8- und 16-Bit 30
- Geltung in Namensräumen 74
- Geltung lokaler Variablen 62
- Generics 121–130
 - generische Typen 122
 - Klassen ableiten 127
 - Kontravarianz 130
 - Kovarianz 128–129
 - selbstreferentielle 127
 - statische Daten 128
 - ungebundene generische Typen 124
- generische Constraints 125
- generische Delegate-Typen 133, 136
- generische Methoden 123
- geschlossene Typen 123

- get-Accessor 84, 87
- GetCustomAttribute-Methode 226
- GetHashCode-Methode 108
- GetType-Methode 106
- GetValue-Methode 40
- Gleichheitsoperator (==) 31, 33, 165, 221–222
 - bei Datensätzen 180
- Gleitkommatypen 26
- global using, Direktive 73
- global:: (Qualifizierer) 75
- globaler Namensraum 72
- goto-Anweisung 65, 70
- Groß- und Kleinschreibung von Bezeichnern 11
- Größer-als-Operator (>) 33
- Größer-gleich-Operator (>=) 33
- GroupBy-Operator, LINQ 195, 209–210
- GroupJoin-Operator, LINQ 194, 204, 206
- group-Klausel, Abfrageausdruck 199

H

- Heap 45
- horizontaler Tabulator, Escape-Sequenz für 35

I

- IDE 10
- IDynamicMetaObjectProvider (IDMOP) 213
- IEnumerable-Interface 160, 186
- IEnumerator-Interface 114, 157, 160

- if-Anweisung 63
- #if-Direktive 246, 247
- implizite Konvertierungen 20, 27, 222
- implizite parameterlose Konstrukturen 81
- implizite Typisierung 52
- implizites Casting auf ein Interface 114
- <include>-XML-Tag 251
- Indexer 88–89
 - Array 39
 - verwenden mit Indizes und Bereichen 88
 - virtuelle 98
- IndexOf-Methode 38, 40
- IndexOutOfRangeException 39
- Indizes
 - mit Elementen arbeiten 40
 - Typ 41
 - und Bereiche mit Indexern verwenden 90
- Infinity-Werte 31
- Infix-Notation 54
- Initialisierung
 - anonymer Typen 171
 - statischer Felder 91
 - von Arrays 39, 40, 43
 - von Enumeratoren 158
 - von Feldern 77
 - von Objekten 82
- Init-Only-Setter 87
- Inkrementoperator (++) 28
- InnerException-Eigenschaft, Exception-Klasse 156
- Insert-Methode 38
- Instanzen von Typen 14, 16
- Instanzkonstruktoren
 - siehe* Konstruktoren

- Instanz-Member 17
- Instanzmethoden 132, 170
- Integrated Development Environment 10
- Interface-Constraint 125
- Interfaces 113–116
- internal-Zugriffsmodifikator 87, 111
- Intersect-Operator, LINQ 190, 195
- into-Keyword, LINQ 202
- int-Typ 25
- InvalidCastException 97
- IQueryable<>-Interface 186
- is-Operator 97, 98
- Iterationsanweisungen 67–69
- Iterationsvariablen 68, 146
- Iteratoren 159–162

J

- Join-Methode 38
- Join-Operator, LINQ 194, 204
- Joins, LINQ 194, 204–208

K

- Kapselung
 - private Member durch öffentliche Member 18
 - Zugriffsmodifikatoren für 111
- Klassen 76–93
 - abstrakte Klassen 99
 - Basisklassen
 - siehe* Basisklassen
 - deklarieren 76
 - Dekonstruktoren 81
 - statische Klassen 17, 91
 - Subklassen

- siehe* Subklassen
 - versiegeln 101
- Kleiner-als-Operator (<) 33
- Kleiner-gleich-Operator (<=) 33
- kombinierbare Enums 119
- Kommentare 7, 13, 248–251
- Kompilieren 10
- Komplementoperator, Bit (~) 30
- konstante interpolierte Strings 38
- Konstanten 14, 45, 77
 - als Ausdrücke 54
 - deklarieren 62
 - Gruppen von
 - siehe* enum-Typ
- Konstantenmuster 182
- Konstruktoren 16, 80–81
 - Feldinitialisierungsabfolge mit 103
 - implizite parameterlose Konstruktoren 81
 - nicht öffentliche 81
 - parameterlose 102
 - primäre 179
 - statische Konstruktoren 90
 - überladen 80
 - von Subklassen 102
- kontextbezogene Schlüsselwörter 12
- Kontravarianz 130, 135
- Konvertierungen 20
 - benutzerdefinierte 222
 - Boxing 105
 - char-Typ 35
 - dynamische Typen 216
 - Enums 119
 - explizite 20, 27, 222
 - implizite 20, 27, 222
 - LINQ 192, 193, 196, 210
 - nullbare Typen 163, 166

- numerische 27
- Referenzumwandlung 96
- kovariante Rückgabetypen 99
- Kovarianz 128–129, 135

L

- Lambda-Ausdrücke 142–147,
238–239
 - statische 145
 - vs. lokale Methoden 147
- Lambda-Operator (=>) 143
- Language Integrated Query
 - siehe LINQ
- LastIndexOf-Methode 38, 40
- Last-Operator, LINQ 189, 195
- LastOrDefault-Operator, LINQ 195
- Laufzeit, Typprüfung zur 97, 106
- leere Ausdrücke 54
- Length-Eigenschaft, Arrays 39
- let-Schlüsselwort, LINQ 201–202
- #line-Direktive 247
- linksassoziative Operatoren 55
- LINQ (Language Integrated Query) 185–211
 - Abfrageausdrücke 198–201
 - Abfragefortsetzungen 202
 - Abfrageoperatoren 186,
187–190, 192–197
 - verketten 197–198
 - Abfragesyntax 186–187
 - Elemente 186
 - extrahieren 189
 - Gruppierung 209–210
 - into-Keyword 202
 - Joins 194, 204–208
 - Konvertierungen 192, 193,
196, 210–211

- let-Schlüsselwort 201–202
- mehrere Generatoren 203–204
- Ordnen von Ergebnissen 208
- Projektion 187, 193, 194
- Quantifizierer 190, 193, 196
- Reihenfolge der Eingabesequenz 188
- Sequenzen 186
- Sets 190, 193, 195
- Unterabfragen 192
- verzögerte Ausführung 191–192
- Zusammenfassung von Elementen 189, 193, 196

- LINQPad 186
- <list>-XML-Tag 251
- Literale 13
 - als Argumente 9
 - ganzzahlige 26
 - String 35
 - Zeichen 34
- logische Muster 183
- lokale Methoden 78
 - vs. Lambda-Ausdrücke 147
- lokale Variablen 11, 44, 45, 52, 62
- LongCount-Operator, LINQ 196
- long-Typ 25

M

- Main-Methode 10
 - definieren 18
- Max-Operator, LINQ 189, 196
- mehrdimensionale Arrays 42
- Member eines Typs 16, 17
 - Instanz-Member 17
 - statische Member 17

- Member-Zugriffsoperator (.) 13, 71
- Message-Eigenschaft, Exception-Klasse 156
- Methoden 9, 78
 - anonyme Methoden 148
 - Argumente 45, 46–52
 - erweiterte partielle 93
 - Erweiterungsmethoden 169–171
 - Expression-bodied 78
 - Finalizer 91
 - generische Methoden 123
 - Instanzmethode 132, 170
 - lokale 78
 - Main-, definieren 18
 - Parameter 44, 46–52
 - partielle Methoden 92
 - Plugin, mit Delegates schreiben 131
 - Signatur von 78
 - statische lokale Methoden 79
 - statische Member 17
 - überladen 79, 103
 - überschreiben 98
 - virtuelle 98
- Min-Operator, LINQ 189, 196
- minus unendlich 31
- Modulinitialisierer 91
- Modulo-Operator (%) 28
- Multicast-Delegates 132
- Multiplikationsoperator (*) 28
- Multiplikationszuweisungsoperator (*=) 56
- Multithreading
 - siehe* asynchrone Funktionen
- Muster 181
 - Eigenschafts- 184
 - Konstanten 182

- logische 183
- relationale 183
- Tupel- und Positions- 183
- var 182

- Muster für Events 138
- Mutation, nicht destruktive 177

N

- Namensräume 71–76
 - dateibezogene 72
- nameof-Operator 94
- namespace-Schlüsselwort 71
- NaN-Wert (Not a Number) 31
- Nebenläufigkeit
 - siehe* asynchrone Funktionen
- Negationsoperator (-) 56
- new-Ausdrücke, mit dem Ziel typisierte 53
- new-Modifikator 100
- new-Operator 16
- nicht destruktive Mutation 177
- nicht öffentliche Konstrukteure 81
- Nicht-gleich-Operator (!=) 33
- Nicht-Operator (!) 33, 246
- Not a Number-Wert (NaN) 31
- Null, Division durch 31
- Null, Escape-Sequenz für 35
- Nullable<T>-Struct 163
- Nullbare Referenztypen 126, 167–169
- nullbare Typen 162–167
 - boolesche Operatoren mit 166
 - Boxing und Unboxing 164
 - Null-Bedingungsoperator (?)
 - für 60
 - Null-Verbindungsoperator (??)
 - für 60

- Operatoren 164–166
 - Umwandlung von 163, 166
- Null-Bedingungsoperator 167
- Null-Literal 23
- Null-Operatoren 166
- Null-Verbindungsoperator (??)
 - 60, 166
- Null-Verbindungszuweisungsoperator (??=) 59
- numerische Literale 26
- numerische Suffixe 26
- numerische Typen 24, 25–32, 46
- nur schreibbare Eigenschaften 85

O

- object-Typ 104–108
- ODER-Operator, Bedingungen
 - (||) 33, 246
- ODER-Operator, Bits (|) 30
- ODER-Zuweisungsoperator (|=)
 - 56
- offene Typen 123
- öffentliche Member 18
- OfType-Operator, LINQ 196,
 - 210–211
- Operatoren 13, 54
 - Abfrageoperatoren, LINQ 186,
 - 187–190, 192–197
 - arithmetische Operatoren 28
 - Assoziativität von 55
 - Bedingungsoperatoren 33
 - Bitoperatoren 30
 - checked und unchecked 29
 - Enums, arbeiten mit 120
 - für nullbare Typen 164–166
 - Gleichheitsoperatoren 33, 165,
 - 221–222
 - Inkrement und Dekrement 28

- Priorität von 55–59
- relationale Operatoren 165
- überladen 220–223
- Vergleichsoperatoren 33, 37,
 - 221–222
- Zeiger-auf-Member-Operator
 - (->) 241, 242
- Zuweisungsoperatoren 13
- Operatorfunktionen 220
- optionale Parameter 50
- OrderByDescending-Operator,
 - LINQ 195
- OrderBy-Operator, LINQ 194
- orderby-Schlüsselwort, LINQ
 - 208
- out-Modifikator 47, 49
- OverflowException 29
- override-Modifikator 98

P

- PadLeft-Methode 38
- PadRight-Methode 38
- <para>-XML-Tag 251
- Parallelität bei asynchronen
 - Funktionen 237
- <param>-XML-Tag 249
- Parameter 44, 46–52
 - Attributparameter 224
 - CallerArgumentExpression
 - 229
 - out-Variablen und Discards 49
 - Typparameter 122, 123, 124,
 - 125
- Parameterlisten 176
- parameterlose Konstruktoren 81,
 - 102
- parameterloser Konstruktor-
 - Constraint 125, 126

- <paramref>-XML-Tag 251
- params-Modifikator 50
- partielle Methoden 92
- partielle Typen 92
- Pascal-Schreibweise 11
- <permission>-XML-Tag 250
- Plugin-Methoden 131
- Polymorphie 95
- positionelle Attributparameter 224
- Positionsmuster 183
- #pragma warning-Direktive 247
- Präprozessordirektiven 245–248
- primäre Konstruktoren 179
- primäre Operatoren 54
- Priorität von Operatoren 55–59
- private Member 18
- private-Zugriffsmodifikator 87, 111
- Projektion, LINQ 187, 193, 194
- protected internal-Zugriffsmodifikator 112
- protected-Zugriffsmodifikator 112
- public-Zugriffsmodifikator 18, 111

Q

- Quantifizierer, LINQ 190, 193, 196

R

- Range-Operator, LINQ 197
- Rank-Eigenschaft, Arrays 40
- readonly, Structs und Funktionen 110
- readonly-Modifikator, Felder 76

- rechteckige Arrays 42
- rechtsassoziative Operatoren 55
- Records Siehe Datensätze
- reelle Literale 26
- reelle Zahlen 27
 - Typen 26
- Refaktorisieren 8
- ReferenceEquals-Methode 108
- Referenz, Argumente übergeben
 - als 48, 49
- Referenztyp-Constraint 125
- Referenztypen 22
- Referenzumwandlung 96
- ref-Modifikator 47, 48
- #region-Direktive 247
- relationale Muster 183
- relationale Operatoren 165
- <remarks>-XML-Tag 249
- Remove-Methode 38
- Repeat-Operator, LINQ 197
- Return, Escape-Sequenz für 35
- return-Anweisung 70
- <returns>-XML-Tag 249
- Reverse-Operator, LINQ 188, 195
- Rückgabetypen, kovariante 99
- Rundungsfehler 32
- RuntimeBinderException 215

S

- Satzzeichen 13
- sbyte-Typ 25, 30
- Schleifenanweisungen 67–69
- Schlüsselwörter 11–13
- schreibgeschützte Eigenschaften 85
- schreibgeschützte Indexer 89
- Schreibung von Bezeichnern 11

- ul style="list-style-type: none; padding-left: 0;">
- sealed-Schlüsselwort 101
- <see>-XML-Tag 250
- <seealso>-XML-Tag 250
- Seitenvorschub, Escape-Sequenz für 35
- select-Klausel, Abfrageausdruck 199
- SelectMany-Operator, LINQ 194, 203
- Select-Operator, LINQ 187, 194
- SequenceEquals-Operator, LINQ 190, 196
- Sequenzen, LINQ 186
- Serialisierung 223
- set-Accessor 84, 87
- Sets, LINQ 190, 193, 195
- SetValue-Methode 40
- short-Typ 25, 30
- sichere Zuweisung 45
- Sichtbarkeit von Accessors 87
- Signatur von Methoden 78
- Single-Operator, LINQ 189, 195
- SingleOrDefault-Operator, LINQ 195
- Skip-Operator, LINQ 188, 194
- SkipWhile-Operator, LINQ 194
- Sort-Methode 40
- Speicher
 - Arrays in 38
 - Heap 45
 - Manipulation mit unsicherem Code 240
 - Referenztypen in 22
 - Stack 44
 - Werttypen in 21
- Speicherung
 - siehe* Speicher
- Split-Methode 38
- Sprachbindung 214–215
- Sprunganweisungen 65, 69–71
- Stack 44
- stackalloc-Schlüsselwort 243
- StackTrace-Eigenschaft, Exception-Klasse 156
- StartsWith-Methode 38
- static readonly-Felder 77
- static-Schlüsselwort 145
- statische Bindung 212–213
- statische Daten 128
- statische Felder 45
- statische Feldinitialisierer 91
- statische Klassen 17, 91
- statische Konstruktoren 90
- statische Lambdas 145
- statische lokale Methoden 79
- statische Member 17
- statische Typen 52
- statische Typprüfung 106
- String-Interpolation 37
- String-Literal 35
- Strings, konstante interpolierte 38
- string-Typ 14, 35–38, 88
- String-Verkettungsoperator (+) 36
- struct-Constraint 126
- Structs 108–109
- struct-Schlüsselwort 21
- Subklassen 95, 100
 - Downcasting auf 97
 - generische Typen 127
 - Reimplementierung von Interface-Membern in 116
- Subscriber 136
- Substring-Methode 38
- Subtraktionsoperator (-) 28
- Subtraktionszuweisungsoperator (==) 56

Suchen in Strings 38
Suffixe, numerische 26
<summary>-XML-Tag 249
Sum-Operator, LINQ 196
switch-Anweisung 64–65
 abhängig vom Typ 66
switch-Ausdrücke 67
Syntax 11–14

T

Take-Operator, LINQ 188, 193
TakeWhile-Operator, LINQ 194
Task<TResult>-Klasse 230
ternärer Bedingungsoperator (?:)
 34
Ternäroperator 54
ThenByDescending-Operator,
 LINQ 195
ThenBy-Operator, LINQ 194
this-Eigenschaft, Indexer 89
this-Modifizier, Erweiterungsme-
 thoden 169
this-Referenz 83
this-Schlüsselwort 80
throw-Anweisung 155
ToArray-Operator, LINQ 192,
 196
ToDictionary-Operator, LINQ
 192, 197
ToList-Operator, LINQ 192, 197
ToLookup-Operator, LINQ 192,
 197
ToLower-Methode 38
ToString-Methode 108
ToUpper-Methode 38
TrimEnd-Methode 38
Trim-Methode 38
TrimStart-Methode 38

try-Anweisung 149–156
Tupel 172–174
 benannte Elemente 173
 dekonstruieren 173
Tupelmuster 183
Typargumente 122
Typ-Constraint 125
Typen 14–25
 Aliase 75
 anonyme Typen 171
 Arrays 38–44
 benutzerdefinierte Typen 15
 bool-Typ 166
 eingebettete 121
 elementare Typen 24
 generische Typen 122, 124
 geschlossene Typen 123
 Instanziierung 16
 Kompatibilität von, mit Dele-
 gates 134–136
 nullbare Typen 162–167
 numerische Typen 24, 25–32
 offene Typen 123
 partielle Typen 92
 Referenztypen 22, 24
 string-Typ 35–38
 Umwandlung
 siehe Konvertierungen
 vordefinierte Typen 14, 16, 24
 Vorgabewerte 46
 Werttypen 21, 24
Typen-Klasse 106
typeof-Operator 106, 124
Typparameter 122, 123, 124, 125
Typplatzhalter 122
Typprüfung 97, 106, 186
Typsicherheit 121, 129, 214

U

- Übergeben als Referenz 48, 49
- Übergeben als Wert 47
- Überladen
 - von Konstruktoren 80
 - von Methoden 79, 103
 - von Operatoren 220–223
- Überlauf 29, 56
- übernommene Variablen 144
- überschriebene Funktions-Member, Zugriff auf 101
- uint-Typ 25
- ulong-Typ 25
- unäre Operatoren 54
- unäres Plus (+) 56
- Unboxing 105, 164
- unchecked-Operator 29
- #undef-Direktive 246
- UND-Operator, Bedingungen
 - (&&) 33, 246
- UND-Operator, Bits (&) 30
- UND-Zuweisungsoperator (&=) 56
- ungebundene generische Typen 124
- ungleichförmige Arrays 43
- Union-Operator, LINQ 190, 195
- unsafe-Schlüsselwort 241
- unsicherer Code 240–244
- Unterabfragen, LINQ 192
- Upcasting 96, 104
- ushort-Typ 25, 30
- using static 73
- using-Anweisung 153, 158
- using-Deklarationen 154
- using-Direktive 8, 73

V

- Variablen 14, 44
 - als Ausdrücke 54
 - deklarieren 61
 - Geltung von lokalen Variablen 62
 - implizite Typisierung von 52
 - lokale Variablen 11, 44, 45, 52
 - sichere Zuweisung 45
 - Speicherung von 44
- var-Muster 182
- var-Schlüsselwort 52
- Verbatim-String-Literale 36
- Verbergen geerbter Member 100
- Verdecken von Namen 75
- Vererbung 94–103
- Vergleichsoperatoren 33, 37, 221–222
- Verketteten
 - von Abfrageoperatoren, LINQ 197–198
 - von Erweiterungsmethoden 170
 - von Strings 36
- Verschiebungsoperatoren 30
- vertikaler Tabulator, Escape-Sequenz für 35
- verzögerte Ausführung, LINQ 191–192
- virtual-Schlüsselwort 98
- virtuelle Funktions-Member 98–99
- virtuelle Interface-Implementierung 115
- void*-Zeiger 244
- void-Rückgabety, Multicast-Delegates 133
- vordefinierte Typen 14, 16, 24

W

- #warning-Direktive 246, 247
- Wert, Argument übergeben als 47
- Werttyp-Constraint 125
- Werttypen 24
- Where-Operator, LINQ 186, 193
- while-Schleifen 68
- WriteLine-Methode 8

X

- XML-Dokumentation 248–251
- XODER-Operator, Bits (^) 30
- XOR-Zuweisungsoperator (^=) 56

Y

- yield break-Anweisung 161
- yield-Anweisung 159, 160

Z

- Zeiger 240–244
- Zeiger-auf-Member-Operator (->) 241, 242
- Zeilenumbruch, Escape-Sequenz für 35
- Zip-Operator, LINQ 194, 207
- Zugriffsmethoden 84
- Zugriffsmodifikatoren 111–113
- Zusammenfassung von Elementen, LINQ 189, 193, 196
- Zuweisung, sichere 45
- Zuweisungsoperator (=) 13, 54
- Zuweisungsoperatoren 13
 - Referenztypen 22
 - Werttypen 21
 - zusammengesetzte 55, 220
- Zuweisungsausdrücke 54

Über die Autoren

Joseph Albahari entwirft Enterprise-Anwendungen auf .NET und auf anderen Plattformen und hat in der Telekommunikation sowie im Gesundheits- und im Bildungswesen gearbeitet. Neben einer Reihe von O'Reilly-Büchern hat er auch LINQPad, das beliebte Query-Tool für LINQ, geschrieben.

Ben Albahari ist Mitbegründer von *Auditionist*, einer Casting-Website für Schauspieler in Großbritannien. Er war fünf Jahre lang Programm-Manager bei Microsoft, wo er an mehreren Projekten arbeitete, darunter am Entity Framework und am .NET Compact Framework. Er ist Mitbegründer von *Genamics*, einem Tools-Anbieter für C#- und J++-Programmierer, der Software für DNA- und Proteinsequenz-Analysen entwickelt. Bei einer Reihe von O'Reilly-Büchern zu C# war Co-Autor.

Kolophon

Das Tier auf dem Cover von *C# 10 – kurz & gut* ist ein Südafrika-Kronenkranich (*Balearica regulorum*). Das Verbreitungsgebiet dieses Vogels erstreckt sich von Kenia und Uganda im Norden bis in den Osten Südafrikas. Er bevorzugt Lebensräume wie offene Sümpfe und Grasland.

Ausgewachsene Vögel sind 90 bis 120 Zentimeter groß und wiegen etwa dreieinhalb Kilogramm. Sie sind auffällige Tiere mit einem grauen Körper und einem hellgrauen Hals, weiß und goldenen Flügeln, einem weißen Gesicht (oben mit einem roten Fleck), einer schwarzen Kappe, einem leuchtend roten Kehlläppchen und blauen Augen. Die ausgeprägte goldfarbene Federkrone gibt ihnen ihren Namen.

Kronenkraniche können in freier Wildbahn bis zu 20 Jahre alt werden. Sie verbringen ihre Zeit damit, durch das Gras zu stolzieren und nach kleinen Tieren und Insekten sowie nach Samen und Körnern zu suchen. Sie sind eine von nur zwei Arten von Kranichen, die

nachts in Bäumen schläft, was durch eine zum Greifen geeignete Hinterzehe ermöglicht wird, mit der sie Äste umklammern können. Sie legen bis zu vier Eier; einige Stunden nach dem Schlüpfen können die Küken ihren Eltern folgen, und die Familie sucht gemeinsam nach Nahrung.

Als gesellige und gesprächige Vögel gruppieren sich Kronenkräniche zu Paaren oder Familien, die sich zuweilen auch zu Schwärmen von mehr als 100 Tieren zusammenschließen. Wie andere Kraniche sind sie für ihren aufwendigen Paarungstanz bekannt, der Elemente wie kurze Aufwärtsflüge, Flügelschlag und tiefe Verbeugungen umfasst.

Trotz ihres großen Verbreitungsgebiets gelten diese Vögel derzeit als gefährdet, bedroht durch Lebensraumverlust, Wilderei und Pestizideinsatz.

Viele der Tiere auf den O'Reilly-Covern sind vom Aussterben bedroht, doch jedes einzelne von ihnen ist für den Erhalt unserer Erde wichtig. Wie man bedrohten Arten helfen kann, erfahren Sie auf animals.oreilly.com.

Der Umschlagsentwurf dieses Buchs stammt von Karen Montgomery und Michael Oreal, die hierfür einen Stich aus *Cassell's Natural History* von 1896 verwendet haben. Auf dem Cover nutzen wir die Schriften Gilroy und Guardian Sans, als Textschrift die Linotype Birka, die Überschriftenschrift ist die Adobe Myriad Condensed, und die Nichtproportionalschrift für Codes ist LucasFonts TheSans Mono Condensed.